

Mục lục

1	Giới thiệu	17
2	Đề Sở Hồ Chí Minh	19
2.1	Bài 1: CASE	19
2.1.1	Tóm tắt đề bài	19
2.1.2	Subtask 1: $1 \leq P \leq 10$	19
2.1.3	Subtask 2: $1 \leq P \leq 10^{12}$	19
2.1.4	Subtask 3: $1 \leq P \leq 10^{18}$	20
2.1.5	Code mẫu:	20
2.2	Bài 2: LUCKYNUM	20
2.2.1	Subtask 1: $1 \leq L \leq R \leq 1000$ và $0 \leq P \leq 9$	21
2.2.2	Subtask 2: $1 \leq L \leq R \leq 10^6$ và $P = 5$	21
2.2.3	Subtask 3 và 4: $1 \leq L \leq R \leq 10^9$	21
2.2.4	Code mẫu:	22
2.3	Bài 3: HOSTING	22
2.3.1	Subtask 1: $1 \leq N \leq 100$ và $1 \leq x_i + d_i \leq 100$	23
2.3.2	Subtask 2 + 3: $1 \leq N \leq 8 \cdot 10^5$ và $1 \leq x_i + d_i \leq 10^5$	23
2.3.3	Code mẫu:	23
3	Đề Hồ Chí Minh trường Phổ Thông Năng Khiếu	24
3.1	Bài 1: CHUỖI ONLINE BẠN BÈ:	24
3.1.1	Tóm tắt đề bài	24
3.1.2	Lời giải	24
3.1.3	Code mẫu tham khảo:	25
3.2	Bài 2: HÀNH TRÌNH XE ĐIỆN:	25
3.2.1	Tóm tắt đề bài	25
3.2.2	Lời giải	26
3.2.3	Code mẫu tham khảo:	26
3.3	Bài 3: TRÒ CHƠI XẾP CHỮ:	27
3.3.1	Tóm tắt đề bài	27
3.3.2	Lời giải	27
3.3.3	Code mẫu tham khảo:	27
3.4	Bài 4: Mật khẩu:	28
3.4.1	Tóm tắt đề bài	28
3.4.2	Lời giải	29
3.4.3	Code mẫu tham khảo:	29
4	An Giang	31
4.1	Bài 1: Tách số	31

4.1.1	Lời giải:	31
4.1.2	Code mẫu:	31
4.2	Bài 2: Chia hết cho 3	31
4.2.1	Lời giải:	31
4.2.2	Code mẫu:	32
4.3	Bài 3: Cân đĩa	32
4.3.1	Lời giải:	32
4.3.2	Code mẫu:	33
4.4	Bài 4: Tính giá trị biểu thức	33
4.4.1	Lời giải:	33
4.4.2	Code mẫu:	34
4.5	Bài 5: Tam giác	36
4.5.1	Lời giải:	36
4.5.2	Code mẫu:	36
5	Bà Rịa - Vũng Tàu	36
5.1	Bài 1: Đếm ước	36
5.1.1	Subtask 1 ($O(n^2)$)	37
5.1.2	Subtask 2 ($O(n \log n)$)	37
5.2	Bài 2: Số siêu nguyên tố	38
5.2.1	Subtask 1 ($O(N\sqrt{N})$)	38
5.2.2	Subtask 2 (Sinh số tối ưu)	39
5.3	Bài 3: Số đặc biệt	40
5.3.1	Subtask 1 & 2 (Q nhỏ hoặc chỉ có 1 truy vấn)	41
5.3.2	Subtask 3 (Full test - Tối ưu bằng mảng cộng dồn)	42
5.4	Bài 4: Ôn tập	42
5.4.1	Subtask 1 ($n, m \leq 10^3$)	43
5.4.2	Subtask 2 (Full test - Tối ưu bằng Tìm kiếm nhị phân)	44
6	Phú Yên	45
6.1	Bài 1: TÌM CẶP SỐ	45
6.1.1	Subtask 1: Cách duyệt trâu $O(n^2)$	45
6.1.2	Subtask 2: Tối ưu hoá $O(n)$	46
6.2	Bài 2: CHUẨN HÓA XÂU	47
6.2.1	Code mẫu	47
6.3	Bài 3: CHI PHÍ NHỎ NHẤT	48
6.3.1	Code mẫu	49
6.4	Bài 4: PHÂN TÍCH SỐ	49
6.4.1	Subtask 1: $N \leq 100$	49
6.4.2	Subtask 2: $100 < N \leq 10^4$	50

7	Bắc Giang	51
7.1	Bài 1: Chữ số tận cùng	51
7.1.1	Phân tích và Thuật toán	51
7.1.2	Code mẫu (C++):	52
7.2	Bài 2: Số chính phương	52
7.2.1	Phân tích thuật toán	53
7.2.2	Code mẫu (C++):	53
7.2.3	Giải thích ví dụ:	54
7.3	Bài 3: Quản lý nhiệm vụ	54
7.3.1	Subtask 1: $M \leq 50, N \leq 50, a_{i,j} \leq 10^2$	54
7.3.2	Subtask 2: $M \leq 5, N \leq 100, a_{i,j} \leq 10^8$	55
7.3.3	Subtask 3: $M \leq 10^3, N \leq 500, a_{i,j} \leq 10^4$	56
7.4	Bài 4: Trò chơi xoá số	58
7.4.1	Subtask 1 & 2: $ N \leq 10^4$	58
7.4.2	Subtask 3: $ N \leq 10^5$	59
8	Bạc Liêu	60
8.1	Bài 1: CHỈ SỐ THÀNH CÔNG	60
8.1.1	Code mẫu	60
8.2	Bài 2: NGUYÊN VỌNG TỐI ƯU	61
8.2.1	Code mẫu (Sử dụng hàm tối ưu)	61
8.3	Bài 3: SỐ CHÍNH PHƯƠNG	61
8.3.1	Code mẫu (Sử dụng hàm căn bậc hai)	62
8.4	Bài 4: ƯỚC CHUNG LỚN NHẤT CỦA DÃY SỐ	63
8.4.1	Code mẫu (Sử dụng hàm <code>__gcd</code> có sẵn)	63
9	Bến Tre	64
9.1	Bài 1: Hình vuông	64
9.1.1	Lời giải	64
9.1.2	Code mẫu	64
9.2	Bài 2: Tổng lớn nhất	65
9.2.1	Lời giải	65
9.2.2	Code mẫu	65
9.3	Bài 3: Chia giấy	66
9.3.1	Lời giải	66
9.3.2	Code mẫu	66
9.4	Bài 4: Xâu đối xứng	66
9.4.1	Lời giải	66
9.4.2	Code mẫu	66

10 Bình Dương	67
10.1 Bài 1: Số lượng nguyên tố	67
10.1.1 Lời giải:	67
10.1.2 Code mẫu:	67
10.2 Bài 2: Mật khẩu	68
10.2.1 Lời giải:	68
10.2.2 Code mẫu:	69
10.3 Bài 3: Trò chơi ghép gỗ	69
10.3.1 Lời giải:	69
10.3.2 Code mẫu:	70
10.4 Bài 4: Biến đổi	70
10.4.1 Lời giải:	70
10.4.2 Code mẫu:	70
11 Cần Thơ	71
11.1 Bài 1: Tổng tiền	71
11.1.1 Lời giải:	71
11.1.2 Code mẫu:	71
11.2 Bài 2: Tồn kho	72
11.2.1 Lời giải:	72
11.2.2 Code mẫu:	72
11.3 Bài 3: Số đảo nguyên tố	72
11.3.1 Lời giải:	72
11.3.2 Code mẫu:	72
11.4 Bài 4: Tìm số	73
11.4.1 Lời giải:	73
11.4.2 Code mẫu:	74
11.5 Bài 5: Di chuyển trên bản đồ	74
11.5.1 Lời giải:	74
11.5.2 Code mẫu:	74
12 Đà Nẵng	75
12.1 Bài 1: Lọc tín hiệu	75
12.1.1 Lời giải:	75
12.1.2 Code mẫu:	75
12.2 Bài 2: Mã hóa	76
12.2.1 Lời giải:	76
12.2.2 Code mẫu:	76
12.3 Bài 3: Năng lượng	77
12.3.1 Lời giải:	77
12.3.2 Code mẫu:	77

12.4 Bài 4: Lựa chọn cấu hình	78
12.4.1 Lời giải:	78
12.4.2 Code mẫu:	78
13 Đắc Lắc	79
13.1 Bài 1: ĐẾM SỐ PHƯƠNG TRÌNH CÓ NGHIỆM	79
13.1.1 Code mẫu (Xử lý toàn bộ dữ liệu)	79
13.2 Bài 2: ALIBABA VÀ CÁC TÚI VÀNG	80
13.2.1 Subtask 1: $a_1 + a_2 + \dots + a_n$ là số chẵn (90% số điểm)	80
13.2.2 Subtask 2: Không có ràng buộc gì thêm (100% số điểm)	81
13.3 Bài 3: XÓA TẬP TIN	81
13.3.1 Subtask 1: $N, Q \leq 100$	82
13.3.2 Subtask 2: $N, Q \leq 10^5$ (Full Test)	82
13.4 Bài 4: ĐOẠN CON ĐẸP	84
13.4.1 Subtask 1 & 2: $N \leq 3000$	84
13.4.2 Subtask 3: $N \leq 10^6$	85
13.5 Bài 5: CHƯƠNG TRÌNH LUYỆN TẬP	87
13.5.1 Subtask 1: $N, M \leq 10^3$	87
13.5.2 Subtask 2 & 3: $N, M \leq 5 \times 10^5$ (Full Test)	88
14 Đắc Nông	89
14.1 Bài 1: Số bộ bàn ghế	89
14.1.1 Code mẫu:	90
14.2 Bài 2: Dây bóng đèn	90
14.2.1 Subtask 1 ($b \leq 10^9$)	91
14.2.2 Subtask 2 ($b < 10^{18}$)	92
14.3 Bài 3: Tam giác	92
14.3.1 Subtask 1 ($n \leq 500$)	93
14.3.2 Subtask 2 ($n \leq 1000$)	93
14.4 Bài 4: Ghép hình	94
14.4.1 Subtask 1 và 2 ($n \leq 1000$)	95
14.4.2 Subtask 3 ($n \leq 10^6$)	96
15 Điện Biên	97
15.1 Bài 1: Tính biểu thức	97
15.1.1 Lời giải:	97
15.1.2 Code mẫu:	97
15.2 Bài 2: Tổng ước số	97
15.2.1 Lời giải:	97
15.2.2 Code mẫu:	97
15.3 Bài 3: Đoạn con nhỏ nhất	98

15.3.1 Lời giải:	98
15.3.2 Code mẫu:	98
15.4 Bài 4: Chọn đồ chơi	99
15.4.1 Lời giải	99
15.4.2 Code mẫu:	99
16 Đồng Tháp	100
16.1 Bài 1: Ghép số	100
16.1.1 Lời giải:	100
16.1.2 Code mẫu:	100
16.2 Bài 2: Cắt gỗ	100
16.2.1 Lời giải:	100
16.2.2 Code mẫu:	101
16.3 Bài 3: Đo tốc độ	101
16.3.1 Lời giải:	101
16.3.2 Code mẫu:	101
16.4 Bài 4: Đổi quà	101
16.4.1 Lời giải:	101
16.4.2 Code mẫu:	102
17 Hà Nội KHTN	102
17.1 Bài 1: DIEM	102
17.1.1 Subtask 1: $N \leq 100$	102
17.1.2 Subtask 2: $N \leq 10^5$ (Full Test)	103
17.2 Bài 2: SODU	104
17.2.1 Subtask 1: $N \leq 5$	104
17.2.2 Subtask 2: $N \leq 10^9$ (Full Test)	105
17.3 Bài 3: KCNN	106
17.3.1 Subtask 1: $N \leq 10^3$	106
17.3.2 Subtask 2: $N \leq 10^5$ (Full Test)	107
17.4 Bài 4: HCN	108
17.4.1 Subtask 1: $N \leq 10^3$	108
17.4.2 Subtask 2: $N \leq 10^5$ (Full Test)	109
18 Hà Nội - Chuyên Sư Phạm	111
18.1 Bài 1: Ước nguyên tố	111
18.1.1 Tóm tắt đề bài	111
18.1.2 Subtask 1: $n \leq 10^4$; $q \leq 10$	111
18.1.3 Subtask 2: $n \leq 10^6$; $q \leq 10^6$	111
18.1.4 Code mẫu:	111
18.2 Bài 2: Chạy tiếp sức	112

18.2.1 Tóm tắt đề bài:	112
18.2.2 Subtask 1: $n \leq 100$	112
18.2.3 Subtask 2: $n \leq 5000$	112
18.2.4 Subtask 3: $n \leq 10^6$	113
18.2.5 Code mẫu:	113
18.3 Bài 3: Những gói kẹo	114
18.3.1 Tóm tắt đề bài:	114
18.3.2 Subtask 1: $n \leq 1000$; $a_1, a_2, \dots, a_n \leq 1000$	114
18.3.3 Subtask 2 + 3: $n \leq 10^6$; $a_1, a_2, \dots, a_n \leq 10^6$	114
18.3.4 Code mẫu:	115
18.4 Bài 4: Lời chào	116
18.4.1 Tóm tắt đề bài:	116
18.4.2 Subtask 1: $n, d_1, d_2, \dots, d_n \leq 100$	116
18.4.3 Subtask 2: $n \leq 5000$	117
18.4.4 Subtask 3: $d_1 = d_2 = \dots = d_n$	118
18.4.5 Subtask 4: $n \leq 2 \cdot 10^5, d_i \leq 10^9$	118
18.4.6 Code mẫu:	119
19 Hà Tĩnh	123
19.1 Bài 1: Tìm số	123
19.1.1 Subtask 1 ($n \leq 10^6$)	123
19.1.2 Subtask 2 (Full test $n \leq 10^{12}$)	124
19.2 Bài 2: Không chia hết	124
19.2.1 Subtask 1 ($L, R \leq 10^6$)	125
19.2.2 Subtask 2 (Full test $L, R \leq 10^9$)	125
19.3 Bài 3: Cặp số chia hết	126
19.3.1 Subtask 1 ($n \leq 10^3$)	127
19.3.2 Subtask 2 (Full test $n, a_i \leq 10^6$)	127
19.4 Bài 4: Đoạn con lớn nhất	128
19.4.1 Subtask 1 & 2 ($n \leq 10^3$)	129
19.4.2 Subtask 4 ($n > 10^5, p = q$)	129
19.4.3 Subtask 3 (Full test $n = 10^5$ hoặc 10^6 dùng Deque)	130
20 Hải Dương	130
20.1 Bài 1: Hành trình ngắn nhất	130
20.1.1 Code mẫu:	131
20.2 Bài 2: Máy rút tiền	131
20.2.1 Code mẫu:	132
20.3 Bài 3: Dây con lớn nhất	133
20.3.1 Subtask 1 & 2 ($n \leq 5000$)	133
20.3.2 Subtask 3 ($n \leq 10^6$)	134

20.4 Bài 4: Lát gạch	135
20.4.1 Subtask 1 ($m, n \leq 100$)	135
20.4.2 Subtask full ($T \leq 2 \times 10^5, m, n \leq 5 \times 10^6$)	136
20.5 Bài 5: Số đẹp	137
20.5.1 Subtask 1 ($n \leq 5000$)	138
20.5.2 Subtask 2 & 3 ($M \leq 10^6$)	139
20.5.3 Subtask 4 ($M \leq 10^9$)	139
21 Hải Phòng	140
21.1 Bài 1: Hình vành khăn	140
21.1.1 Subtask 1 ($R_2 \leq 10^3$)	141
21.1.2 Subtask 2 ($R_2 \leq 10^6$)	141
21.1.3 Subtask 3 (Full - Tối ưu đối xứng)	142
21.2 Bài 2: Xâu con	143
21.2.1 Subtask 1 & 2 ($n \leq 10^3$)	144
21.2.2 Subtask 3 ($n \leq 2 \times 10^5$)	144
21.3 Bài 3: Dãy số	145
21.3.1 Subtask 1, 2, 3 ($O(q \times n)$)	146
21.3.2 Subtask 4 ($O((n + q) \log n)$)	147
21.4 Bài 4: Robot di chuyển	149
21.4.1 Subtask 1, 2, 3, 4 ($a_{i,j} \leq 10^6$)	149
21.4.2 Subtask 5 ($a_{i,j}$ lớn, dùng tối ưu tính tổng ước bằng sàng trên đoạn) . .	150
22 Hưng Yên	151
22.1 Bài 1	151
22.1.1 Subtask 1+2+3	151
22.1.2 Code mẫu tham khảo	152
22.2 Bài 2	152
22.2.1 Subtask 1	152
22.2.2 Subtask 2+3	152
22.2.3 Code mẫu tham khảo	153
22.3 Bài 3	153
22.3.1 Subtask 1+2	153
22.3.2 Subtask 3	153
22.3.3 Code mẫu tham khảo	153
22.4 Bài 4	154
22.4.1 Subtask 1	154
22.4.2 Subtask 2+3	154
22.4.3 Code mẫu tham khảo	155
22.5 Bài 5	156
22.5.1 Subtask 1	156

22.5.2 Subtask 2	156
22.5.3 Subtask 3	156
22.5.4 Code mẫu tham khảo	156
23 Khánh Hòa	158
23.1 Câu 1	158
23.1.1 Subtask 1 (80% số điểm: $x_2 \leq 10^6$)	158
23.1.2 Subtask 2 (20% số điểm: Không có ràng buộc gì thêm, $x_2 \leq 10^{12}$)	159
23.2 Câu 2	159
23.2.1 Subtask 1 (50% số điểm: $m = 1$)	160
23.2.2 Subtask 2 (50% số điểm: Không có ràng buộc gì thêm)	160
23.3 Câu 3	161
23.3.1 Subtask 1 (50% số điểm: $n \leq 10^3, a_i \leq 10^3$)	162
23.3.2 Subtask 2 (30% số điểm: $n \leq 10^4, a_i \leq 10^6$)	162
23.3.3 Subtask 3 (20% số điểm: $n \leq 10^6, a_i \leq 10^{18}$)	162
23.4 Câu 4	163
23.4.1 Subtask 1 (60% số điểm: $n, x_i, m_i, k_i \leq 10^3$)	164
23.4.2 Subtask 2 (40% số điểm: Không có ràng buộc gì thêm)	164
23.5 Câu 5	165
23.5.1 Subtask 1 & 2 ($n \leq 10^6$)	166
23.5.2 Subtask 3 ($n \leq 10^9$)	167
24 Kiên Giang	168
24.1 Bài 1: Tính tiền taxi	168
24.1.1 Code mẫu	169
24.2 Bài 2: Bộ ba số	170
24.2.1 Code mẫu:	171
24.3 Bài 3: Bội số nguyên tố	172
24.3.1 Code mẫu:	172
24.4 Bài 4: Căn hộ	173
24.4.1 Code mẫu:	174
25 Kon Tum	175
25.1 Bài 1: Trả tiền	175
25.1.1 Code mẫu:	176
25.2 Bài 2: Chia kẹo	176
25.2.1 Subtask 1 ($n \leq 10^6$)	177
25.2.2 Subtask 2 ($n \leq 10^9$)	177
25.3 Bài 3: Số nguyên tố lớn nhất	178
25.3.1 Subtask 1 ($ S \leq 255$, số ≤ 5 chữ số)	178
25.3.2 Subtask 2 ($ S \leq 10^5$, số ≤ 9 chữ số)	179

25.4 Bài 4: Sơn tường	180
25.4.1 Subtask 1 & 2 ($M \leq 10^6, N \leq 10^3$)	181
25.4.2 Subtask 3 (Tối ưu cho $M \leq 10^9, N \leq 10^5$)	181
26 Lâm Đồng	182
26.1 Bài 1: Chia hết	182
26.1.1 Code mẫu	183
26.2 Bài 2: Chi phí	183
26.2.1 Code mẫu	184
26.3 Bài 3: Mã số	184
26.3.1 Subtask ($n \leq 10^3$ - Duyệt $O(n^2)$)	185
26.3.2 Code tối ưu ($n \leq 10^6$ - Độ phức tạp $O(n)$)	186
26.4 Bài 4: Rô bốt	187
26.4.1 Code mẫu	187
27 Lạng Sơn	188
27.1 Bài 1: TÍNH TỔNG (SUM)	188
27.1.1 Code mẫu (Toán học tối ưu $O(1)$)	189
27.2 Bài 2: ƯỚC SỐ (UOCSO)	189
27.2.1 Code mẫu	190
27.3 Bài 3: QUAY THUỖNG (THUONG)	191
27.3.1 Code mẫu (Tối ưu $O(n + m)$)	191
28 Lào Cai	192
28.1 Bài 1: CHIA HẾT CHO 3 HOẶC 5	192
28.1.1 Code mẫu C++	193
28.2 Bài 2: KÝ TỰ GIỐNG NHAU	193
28.2.1 Subtask 1: $Q = 1$	194
28.2.2 Subtask 2: $Q \leq 10^5$ (Full Test)	194
28.3 Bài 3: TỔNG NHỎ NHẤT	195
28.3.1 Phân tích Toán học	195
28.3.2 Subtask 1: $c \times d \leq 10^9$	196
28.3.3 Subtask 2: $c \times d \leq 10^{18}$ (Full Test)	196
28.4 Bài 4: MUA HÀNG TIẾT KIỆM	197
28.4.1 Subtask 1: $N \leq 10^3$	197
28.4.2 Subtask 2: $N \leq 10^5$	198
28.5 Bài 5: LUYỆN TẬP	199
28.5.1 Subtask 1: $n, m \leq 10^4$	199
28.5.2 Subtask 2: $n, m \leq 10^5$	200

29 Nam Định	202
29.1 Bài 1. BẰNG NHAU	202
29.1.1 Phân tích và Thuật toán	202
29.1.2 Code mẫu (C++):	202
29.2 Bài 2. KHÁC NHAU	203
29.2.1 Phân tích thuật toán	203
29.2.2 Code mẫu (C++):	204
29.3 Bài 3. CẮT DÂY	205
29.3.1 Subtask 1: $n, k \leq 10^4$	205
29.3.2 Subtask 2: $n \leq 10^{18}$	205
29.3.3 Code mẫu (C++):	206
29.4 Bài 4. CHÍNH PHƯƠNG	207
29.4.1 Phân tích thuật toán	207
29.4.2 Code mẫu (C++):	208
29.5 Bài 5. CHỤP ẢNH	208
29.5.1 Phân tích thuật toán	209
29.5.2 Code mẫu (C++):	209
30 Nghệ An	210
30.1 Bài 1: SỐ THÚ VỊ	210
30.1.1 Subtask 1: $T = 1, 10^2 \leq a \leq b \leq 10^6$	210
30.1.2 Subtask 2: $T \leq 10^5, 10^2 \leq a \leq b \leq 10^6$	211
30.1.3 Subtask 3: $T \leq 10^5, 10^2 \leq a \leq b \leq 10^{18}$	212
30.2 Bài 2: HỘP QUÀ MAY MẮN	213
30.2.1 Subtask 1 & 2: $ S \leq 10^3$	213
30.2.2 Subtask 3: $ S \leq 10^5$	214
30.3 Bài 3: QUY HOẠCH THÀNH PHỐ	215
30.3.1 Subtask 1: $ S \leq 10^3$	215
30.3.2 Subtask 2: $ S \leq 5 \times 10^5$ (Full Test)	216
30.4 Bài 4: ĐẦU TƯ TÀI CHÍNH	217
30.4.1 Subtask 1: $n \leq 10^2, k = 1$	218
30.4.2 Subtask 2 & 3: $n \leq 10^5, k \leq 100$	218
31 Chuyên ĐH Vinh	220
31.1 Bài 1: TAM GIÁC VUÔNG	220
31.1.1 Subtask 1	220
31.1.2 Subtask 2	220
31.1.3 Code mẫu tham khảo	221
31.2 Bài 2: XÂU TƯƠNG TỰ	221
31.2.1 Subtask 1+2+3	221
31.2.2 Code mẫu tham khảo	222

31.3 Bài 3: QUÀ CỨU TRỢ	223
31.3.1 Subtask 1	223
31.3.2 Subtask 2+3	223
31.3.3 Code mẫu tham khảo	224
31.4 Bài 4: VÙNG SÁNG ẮNH	225
31.4.1 Subtask 1	225
31.4.2 Subtask 2	225
31.4.3 Subtask 3	225
31.4.4 Code mẫu tham khảo	226
32 Ninh Bình	227
32.1 Bài 1	227
32.1.1 Subtask 1	227
32.1.2 Subtask 2	227
32.1.3 Code mẫu tham khảo	228
32.2 Bài 2	229
32.2.1 Subtask 1	229
32.2.2 Subtask 2+3	229
32.2.3 Code mẫu tham khảo	229
32.3 Bài 3	230
32.3.1 Subtask 1+2	230
32.3.2 Subtask 3	230
32.3.3 Code mẫu tham khảo	231
32.4 Bài 4	232
32.4.1 Subtask 1+2	232
32.4.2 Subtask 3	232
32.4.3 Code mẫu tham khảo	232
33 Phú Thọ	233
33.1 Bài 1: Tính nhanh	233
33.1.1 Subtask 1 ($a, b \leq 10^6$)	234
33.1.2 Subtask 2 (Full điểm - $a, b \leq 2 \times 10^9$)	234
33.2 Bài 2: Trò chơi	235
33.2.1 Subtask 1 & 2 ($n \times q \leq 10^4$)	235
33.2.2 Subtask 3	236
33.3 Bài 3: Xuất bản	237
33.3.1 Subtask 1 & 2 - Quy hoạch động	237
33.3.2 Subtask 3 & 4 - Chặt nhị phân	238
33.4 Bài 4: Chia kẹo	239
33.4.1 Subtask 1 ($N \leq 20$)	240
33.4.2 Subtask 2 $\rightarrow$ 4	240

34 Quảng Bình	241
34.1 Bài 1: Tính tổng	241
34.1.1 Cách 1: duyệt vòng lặp (Dễ hiểu)	242
34.1.2 Cách 2: sử dụng công thức Toán học (Tối ưu)	242
34.2 Bài 2: Đếm ký tự	243
34.2.1 Code mẫu	243
34.3 Bài 3: Đoạn đường đẹp nhất	244
34.3.1 Subtask 1 ($n \leq 10^3$ - Duyệt $O(n^2)$ cơ bản)	244
34.3.2 Subtask 2 ($10^3 < n \leq 10^4$ - Tối ưu hóa hoặc dùng $O(n^2)$ nhanh)	245
34.4 Bài 4: Robot tìm đường đi	246
34.4.1 Subtask 1 & 2 ($k \leq 10^4$)	246
34.4.2 Subtask 3 ($k \leq 10^{10}$ - Tối ưu $\sqrt{k}$)	247
35 Quảng Nam	247
35.1 Bài 1: Bàn học sinh	247
35.1.1 Subtask 1 ($x = y$)	248
35.1.2 Subtask 2 ($x = 2, y = 1$)	248
35.1.3 Subtask 3 (Full - Không ràng buộc)	249
35.2 Bài 2: Mật thư	249
35.2.1 Subtask 1 (S chỉ gồm chữ cái)	250
35.2.2 Subtask 2 (S có số 1 chữ số, n nhỏ)	250
35.2.3 Subtask 3 (Full - Số nguyên nhiều chữ số, n lớn)	251
35.3 Bài 3: Khai thác gỗ	251
35.3.1 Subtask 1 & 2 (Duyệt đơn giản)	252
35.3.2 Subtask 3 (Full - Tối ưu bằng mảng cộng dồn ngược)	253
35.4 Bài 4: Số đẹp	253
35.4.1 Subtask 1 ($N \leq 15$)	254
35.4.2 Subtask 2 & 3 (Full $N \leq 10^9$)	254
36 Quảng Ngãi	255
36.1 Bài 1: Số chính phương	255
36.1.1 Code mẫu	256
36.2 Bài 2: Diện tích	256
36.2.1 Code mẫu	257
36.3 Bài 3: Người may mắn	258
36.3.1 Subtask 1 ($O(n^2)$)	258
36.3.2 Subtask 2 (Tối ưu $O(n)$ với unordered_map và Sàng Eratosthenes)	259
36.4 Bài 4: Hộp quà	260
36.4.1 Subtask 1 (Sử dụng mảng cộng dồn cơ bản)	261
36.4.2 Subtask 2 (Tương tự Subtask 1 nhưng tối ưu tốc độ nhập xuất)	261

37 Quảng Ninh	262
37.1 Bài 1: Ước số	262
37.1.1 Code theo chiến thuật duyệt ngược ($O(n)$)	263
37.1.2 Code theo chiến thuật tối ưu ($O(\sqrt{n})$)	263
37.2 Bài 2: Tổng dãy	264
37.2.1 Subtask 1 & 2 ($O(n^2)$)	265
37.2.2 Subtask 3 ($O(n)$)	265
37.3 Bài 3: Trung vị	266
37.3.1 Subtask 3 ($O(n^3)$)	267
37.3.2 Subtask 4 ($O(n \log n)$)	267
37.4 Bài 4: Dãy con tăng	268
37.4.1 Subtask 3 ($O(n \times m)$ hoặc $O(n^2)$)	269
37.4.2 Subtask 4 ($O(n + m)$)	270
38 Quảng Trị	271
38.1 Bài 1: BƠM NƯỚC	271
38.1.1 Subtask 1: $X, Y \leq 10^8, K \leq 10^5$	272
38.1.2 Subtask 2: $X, Y \leq 10^{12}, K \leq 10^4$	272
38.2 Bài 2: TAM GIÁC	273
38.2.1 Subtask 1: $a, b \leq 10^5$	273
38.2.2 Subtask 2: $a, b \leq 10^9$ (Full Test)	274
38.3 Bài 3: CHIA KẸO	274
38.3.1 Code Full	275
38.4 Bài 4: SỐ CAMERAS	278
38.4.1 Subtask 1, 2 & 3: $n, q \leq 7000$ hoặc $r = 0$	278
38.4.2 Subtask 4: Không có ràng buộc gì thêm ($n, q \leq 10^5$)	279
39 Sóc Trăng	280
39.1 Bài 1: PHẦN THƯỜNG	280
39.1.1 Code mẫu	280
39.2 Bài 2: BÁNH PÍA	281
39.2.1 Code mẫu	281
39.3 Bài 3: MUA SÁCH	282
39.3.1 Subtask 1, 2 & 3: $N \leq 1000$ hoặc mảng đặc biệt	282
39.3.2 Subtask 4: Không có ràng buộc gì thêm ($N \leq 10^5$)	283
39.4 Bài 4: ROBOT	284
39.4.1 Code mẫu cho toàn bộ Subtasks	284
40 Tây Ninh	285
40.1 Bài 1: Counting	285
40.1.1 Lời giải:	285

40.1.2 Code mẫu:	285
40.2 Bài 2: Min3	286
40.2.1 Lời giải:	286
40.2.2 Code mẫu:	286
40.3 Bài 3: Primes	286
40.3.1 Subtask 1, 2, 3: $1 \leq n \leq 10^4, 1 \leq A_i \leq 10^6$	286
40.3.2 Subtask 4: $1 \leq n \leq 10^6, 1 \leq A_i \leq 10^6$	286
40.3.3 Code mẫu:	287
40.4 Bài 4: Pair	287
40.4.1 Lời giải:	287
40.4.2 Code mẫu:	287
41 Thái Nguyên	288
41.1 Bài 1: CẶP SỐ NGUYÊN TỔ CÙNG NHAU	288
41.1.1 Code mẫu (Không comment)	289
41.2 Bài 2: TÌM XÂU KÍ TỰ	290
41.2.1 Subtask 1: Độ dài xâu A bằng 1	290
41.2.2 Subtask 2: Xâu A chỉ chứa các ký tự giống nhau	290
41.2.3 Subtask 3: Không có ràng buộc thêm (Full Test)	291
41.3 Bài 3: TRÒ CHƠI DÂY SỐ	292
41.3.1 Subtask 1: $1 \leq l \leq r \leq 9$	293
41.3.2 Subtask 2: $r - l \leq 10^3$	293
41.3.3 Subtask 3: $1 \leq l \leq r \leq 10^{18}$ (Full Test)	294
42 Thanh Hóa	294
42.1 Bài 1: THỐNG KÊ	294
42.1.1 Subtask 1 & 2: $A \leq B \leq 10^6$	295
42.1.2 Subtask 3: $A \leq B \leq 10^{18}$ (Full Test)	295
42.1.3 Code tối ưu	296
42.2 Bài 2: THÁP ĐẦY ĐỦ	296
42.2.1 Subtask 1: $a < b \leq 4a \leq 10^{18}$	297
42.2.2 Subtask 2: $1 \leq a < b \leq 10^5$	297
42.2.3 Subtask 3: $1 \leq a < b \leq 10^{12}$	298
42.3 Bài 3: TÍNH CĂN	300
42.3.1 Subtask 1: $1 < N \leq 20$	300
42.3.2 Subtask 2: $N \leq 10^5$	301
42.4 Bài 4: TẦN SỐ	303
42.4.1 Subtask 1 & 2: Độ dài $S \leq 2 \cdot 10^3$	303
42.4.2 Subtask 3: Độ dài $S \leq 2 \cdot 10^5$ (Full Test)	304

43 Thừa Thiên Huế - ĐH Khoa Học	306
43.1 Bài 1: Kim tự tháp	306
43.1.1 Tóm tắt đề bài	306
43.1.2 Lời giải	306
43.1.3 Code mẫu tham khảo:	306
43.2 Bài 2: Đếm số từ	306
43.2.1 Tóm tắt đề bài	306
43.2.2 Lời giải	307
43.2.3 Code mẫu tham khảo:	307
43.3 Bài 3: Diện tích tam giác	307
43.3.1 Tóm tắt đề bài	307
43.3.2 Lời giải	308
43.3.3 Code mẫu tham khảo:	308
43.4 Bài 4: Mã hoá thẻ	309
43.4.1 Tóm tắt đề bài	309
43.4.2 Lời giải	309
43.4.3 Code mẫu tham khảo:	309
44 Tiền Giang	310
44.1 Bài 1: Tính tổng:	310
44.1.1 Subtask 1: $1 \leq n \leq 10^6$	310
44.1.2 Subtask 2: $10^6 < n \leq 10^9$	310
44.1.3 Code mẫu:	311
44.2 Bài 2: Số Niven cân bằng	311
44.2.1 Lời giải:	311
44.2.2 Code mẫu:	311
44.3 Bài 3: Tìm số lớn nhất	312
44.3.1 Subtask 1: $1 \leq n, Q \leq 10^3$	312
44.3.2 Subtask 2: $10^3 < n, Q \leq 10^6$	312
44.3.3 Code mẫu:	313
44.4 Bài 4: Từ đa dạng	313
44.4.1 Lời giải:	313
44.4.2 Code mẫu:	313
44.5 Bài 5: Miền nguyên tố	314
44.5.1 Subtask 1: $n \leq 10^3, A_i \leq 10^3$	314
44.5.2 Subtask 2: $n \leq 10^4, A_i \leq 10^5$	314
44.5.3 Subtask 2: $n \leq 10^5, A_i \leq 10^9$	315
44.5.4 Code mẫu:	315

45 Tuyên Quang	316
45.1 Bài 1: THU DỌN ĐỒ CHƠI	316
45.1.1 Code mẫu	316
45.2 Bài 2: SIÊU LỄ	317
45.2.1 Subtask 1: $M = 1$	317
45.2.2 Subtask 2: $M \leq 10^6, a, b \leq 10^6$	318
45.3 Bài 3: NGUYÊN TỐ	319
45.3.1 Phân tích thuật toán	319
45.3.2 Subtask 1: $N \leq 10^3, A_i \leq 1000$	319
45.3.3 Subtask 2: $N \leq 10^6, A_i \leq 10^6$ (Full Test)	320
45.4 Bài 4: ĐOẠN CON CHUNG	322
45.4.1 Subtask 1 & 2: $N \leq 1000$	322
45.4.2 Subtask 3: $N \leq 10^6$	323
46 Vĩnh Phúc	324
46.1 Bài 1: QUÀ [GIFT]	324
46.1.1 Code mẫu	325
46.2 Bài 2: TÀU ĐIỆN [METRO]	325
46.2.1 Code mẫu	326
46.3 Bài 3: TRÒ CHƠI [GAME]	327
46.3.1 Subtask 1: $N \leq 100$	327
46.3.2 Subtask 2: $N \leq 50000$ (Full Test)	328
46.4 Bài 4: CHỌN [SELECT]	330
46.4.1 Code mẫu	330
47 Tổng biên tập	332
48 Ban tạo bài	332
49 Ban viết đề + solution	332

1 Giới thiệu

- Nhân dịp tuyển sinh sắp tới chúng mình đã sưu tầm đề tuyển sinh toàn quốc năm học 2025-2026 bao gồm tầm 45 đề từ các tỉnh để viết lời giải cũng như làm trình chấm cho các bạn nộp. Vì có rất nhiều bài nên sẽ được tách thành 2 link chấm, link đầu sẽ gồm 22 tỉnh và link sau sẽ gồm phần còn lại.

- Link contest 22 tỉnh đầu: <https://codeforces.com/contestInvitation/08fc806c37fc291c9b24b23>

- Link contest các tỉnh còn lại: <https://codeforces.com/contestInvitation/d7e3c798b6d8236d95f3>

- Nếu có thắc mắc hay cần hỗ trợ các bạn có thể trao đổi trên group facebook :<https://www.facebook.com/groups/7038269849574622>

- Nếu bạn nào quan tâm tới việc học thuật toán để thi học sinh giỏi, đội tuyển quốc gia các cấp cho hè này thì có thể liên hệ mình qua:

- Facebook: <https://www.facebook.com/Ton2808/>

- Zalo: 0762310005

Đây là bảng thành tích của các bạn học mình trong kỳ thi quốc gia năm 2025–2026

VINH DANH HSG Quốc Gia Tin Học 2026			
	Tên	Giải	THPT
1	Bùi Tuấn Anh	Nhì	Chuyên Trần Văn Giàu - LA
2	Trần Đại Hào	Nhì	Chuyên Lương Văn Chánh - PY
3	Mã Lê Hoàng Phát	Nhì	Chuyên Thủ Khoa Nghĩa - AG
4	Ôn Huỳnh Hiếu Ngân	Nhì	FPT Cần Thơ
5	Nguyễn Đức Thành Đạt	Nhì	Chuyên Bảo Lộc
6	Nguyễn Xuân Thái	Nhì	Phổ thông Năng Khiếu - HCM
7	Lê Hồng Anh	Nhì	Chuyên Lê Hồng Phong - HCM
8	Nguyễn Đức Dũng	Nhì	Chuyên Nguyễn Du - ĐL
9	Nguyễn Hoàng Long	Nhì	Chuyên Hà Tĩnh
10	Nguyễn Nhật Anh	Nhì	Chuyên Hà Tĩnh
11	Trần Kim Nhật	Nhì	Hương Khê - Hà Tĩnh
12	Bùi Quốc Cường	Nhì	Chuyên Lê Khiết - QN
13	Hồ Chánh Nam	Nhì	Chuyên Huỳnh Mẫn Đạt - KG
14	Lê Thiện Khôi	Nhì	Chuyên Lê Thánh Tông - QN
15	Đoàn Nhật Anh	Ba	Chuyên Hùng Vương - GL
16	Hoàng Gia Bảo	Ba	Trần Biên - ĐN
17	Lê Viết Thành Nhân	Ba	Chuyên Lê Quý Đôn - VT
18	Nguyễn Viết Thông	Ba	Chuyên Lê Quý Đôn - VT
19	Trần Bình Nguyên	Ba	Chuyên Lê Quý Đôn - VT
20	Nguyễn Thanh	Ba	Chuyên Hà Tĩnh
21	Phạm Ngọc Thơ	Ba	Chuyên Hà Tĩnh
22	Nguyễn Thanh Tú	Ba	Chuyên Hà Tĩnh
23	Phạm Vũ Phương	Ba	Chuyên Hà Tĩnh
24	Nguyễn Hoàng	Ba	Chuyên Trần Đại Nghĩa - HCM
25	Nguyễn Hoàng	Ba	Chuyên Trần Đại Nghĩa - HCM
26	Nguyễn Lê Minh	Ba	Chuyên Trần Đại Nghĩa - HCM
27	Hoàng Minh Quân	Ba	Gia Định - HCM
28	Phạm Quốc Anh	Ba	Chuyên Lê Hồng Phong - HCM
29	Nguyễn Thiện Nhân	Ba	Chuyên Bắc Ninh
30	Mai Thế Dũng	Ba	Chuyên Lê Quý Đôn - KH

VINH DANH HSG Quốc Gia Tin Học 2026			
	Tên	Giải	THPT
31	Ngô Duy Trọng	Ba	Chuyên Tiền Giang
32	Nguyễn Chấn Hưng	Ba	Chuyên Vĩnh Phúc
33	Nguyễn Trung Hiếu	KK	Chuyên Lê Hồng Phong - HCM
34	Lê Mạnh Anh Khoa	KK	Chuyên Lê Quý Đôn - VT
35	Nguyễn Công Minh	KK	Chuyên Lê Quý Đôn - VT
36	Lê Trần Thiên Phúc	KK	Chuyên Lê Quý Đôn - VT
37	Võ Đình Bảo	KK	Chuyên Lê Quý Đôn - VT
38	Phạm Minh Quân	KK	Chuyên Hùng Vương - GL
39	Nguyễn Lê Anh Minh	KK	Chuyên Nguyễn Du - ĐL
40	Nguyễn Nhật Anh	KK	Chuyên Lương Văn Chánh - PY
41	Huỳnh Trần Bảo Đại	KK	Chuyên Bảo Lộc
42	Nguyễn Kiến Hào	KK	Chuyên Bến Tre
43	Nguyễn Hải Đăng	KK	Chuyên Hùng Vương - PT
44	Kiều Gia Huy	KK	Chuyên Hùng Vương - PT
45	Nguyễn Huy Bảo	KK	Chuyên Vĩnh Phúc
46	Huỳnh Đặng Thức	KK	Chuyên Thoại Ngọc Hầu
47	Nghiêm Đình An	KK	Chuyên Thoại Ngọc Hầu
48	Nguyễn Minh Hiến	KK	Chu Văn An - AG
49	Đỗ Phú Quang	KK	Chuyên Lê Thánh Tông - QN
50	Chu Chính Hoàng	KK	Chuyên Hà Giang
51	Nguyễn Thái Hưng	KK	Chuyên Lê Quý Đôn - KH
52	Hoàng Long	KK	Chuyên Lê Quý Đôn - KH
53	Nguyễn Hoàng	KK	Chuyên Lê Quý Đôn - KH
54	Nguyễn Đăng Huy	KK	Chuyên Lê Quý Đôn - BD
55	Kiều Văn Đạt	KK	Chuyên Lê Quý Đôn - BD
56	Nguyễn Hoàng	KK	Chuyên Tiền Giang
57	Trần Bá Hiền	KK	Chuyên Tiền Giang
58	Quang Chương	KK	Phổ thông Năng Khiếu - HCM
59	Nguyễn Gia Minh	KK	Trần Biên - ĐN
60	Vũ Văn Mạnh	KK	Bạch Đằng - Hải Phòng
61	Đặng Kỳ Anh	KK	Lê Lợi - PY
62	Nguyễn Nam Khánh	KK	Chuyên Quốc Học - Huế

2 Đề Sở Hồ Chí Minh

2.1 Bài 1: CASE

2.1.1 Tóm tắt đề bài

Cho bảng số có dạng hình tam giác, các dòng được đánh số từ 1, ở dòng thứ i có i cột được đánh số từ 1 đến i . Mỗi ô mang một giá trị được đánh số từ 1 đến hết với quy luật như sau:

```

1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
⋮
    
```

Yêu cầu: Cho P , tìm hàng và cột của P trong bảng trên.

2.1.2 Subtask 1: $1 \leq P \leq 10$

- Ta duy trì một biến `col` (cột) và `row` (hàng). Với số $i \in [1, P]$, ta xác định xem i đang ở cột nào.
- Vì hàng thứ k chỉ có đúng k cột, nên nếu `col` > `row`, ta hiểu rằng ta phải xuống hàng và đặt lại cột từ 1
- Độ phức tạp: $\mathcal{O}(P)$.

2.1.3 Subtask 2: $1 \leq P \leq 10^{12}$

- Ta có nhận xét rằng, tổng số phần tử ở hàng 1 đến hàng i là:

$$S_i = 1 + 2 + \dots + i = \frac{i \cdot (i + 1)}{2}$$

$\Rightarrow$ Số P nằm ở hàng i nếu $S_{i-1} < P \leq S_i$

- Gọi `start` là số đầu tiên của hàng i , ta có:

$$\text{start} = \frac{i \cdot (i - 1)}{2} + 1$$

- Vị trí cột j của P nằm ở $P - \text{start} + 1$.
- Vậy, ta chỉ cần xác định hàng i chứa P và suy ra cột j từ công thức trên.
- Ta biết rằng P nằm ở hàng i nếu: $P \leq \frac{i \cdot (i + 1)}{2}$, vì thế với ta có thể duyệt để tìm i phù hợp với $i_{\max} \approx \sqrt{2 \cdot P} \approx 1.5 \cdot 10^6$
- Độ phức tạp: $\mathcal{O}(\sqrt{P})$.

2.1.4 Subtask 3: $1 \leq P \leq 10^{18}$

- Ta dùng lại tư tưởng của Subtask 2, tuy nhiên ta nhận thấy rằng hàm S_i tuyến tính, nên có thể áp dụng tư tưởng chặt nhị phân được.
- Sau khi tìm được hàng cho P , ta làm lại như Subtask 2.
- Độ phức tạp: $\mathcal{O}(\log n)$

2.1.5 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

long long p;

int main() {
    ios_base::sync_with_stdio(0); cin.tie(0);

    freopen("CASE.INP", "r", stdin);
    freopen("CASE.OUT", "w", stdout);

    cin >> p;

    long long left = 1, right = 2e9 + 3, result = -1;

    while(left <= right) {
        long long mid = (left + right) / 2;
        if(mid * (mid + 1) / 2 >= p) {
            result = mid;
            right = mid - 1;
        } else left = mid + 1;
    }

    cout << result << " " << (p - result * (result - 1) / 2) << "\n";

    return 0;
}
```

2.2 Bài 2: LUCKYNUM**Tóm tắt đề bài:**

Cho ba số nguyên dương L , R và P . Cặp số (i, j) được coi là may mắn nếu $i \cdot j$ có tận cùng là P và $L \leq i, j \leq R$.

Yêu cầu: Hãy đếm số lượng cặp số may mắn.

2.2.1 Subtask 1: $1 \leq L \leq R \leq 1000$ và $0 \leq P \leq 9$

- Trước khi đến với lời giải, ta có nhận xét quan trọng rằng $(i \cdot j) \bmod 10 = ((i \bmod 10) \cdot (j \bmod 10)) \bmod 10$, vì thế ta chỉ cần quan tâm tới chữ số tận cùng của các số. Nhận xét này sẽ đi theo ta xuyên suốt bài này.
- Vì giới hạn L và R nhỏ, ta có thể duyệt qua được bằng 2 vòng lặp và kiểm tra điều kiện đề bài.
- Độ phức tạp: $\mathcal{O}(L \cdot R)$

2.2.2 Subtask 2: $1 \leq L \leq R \leq 10^6$ và $P = 5$

- Để thoả mãn biểu thức $(i \cdot j) \bmod 10 = 5$. Ta phải đồng thời thoả mãn 2 điều kiện:
 1. 1 số có tận cùng là 5
 2. Số còn lại có tận cùng là số lẻ.
- Từ đây, ta có ý tưởng sử dụng mảng đếm để đếm lại các số có tận cùng là các số lẻ và dùng tổ hợp để tính toán.
- Độ phức tạp: $\mathcal{O}(R - L)$

2.2.3 Subtask 3 và 4: $1 \leq L \leq R \leq 10^9$

- Vì cả 2 Subtask cuối đều có giới hạn L và R lớn, ta cần phải có một cách tiếp cận khác thay vì duyệt từ L tới R .
- Gọi $f(x)^\dagger$ là số lượng số trong đoạn $[L, R]$ có tận cùng là x .
- Với mỗi cặp $(i, j) \in [0, 9]$. Nếu $(i \cdot j) \bmod 10 = P$. Cặp (i, j) đóng góp vào đáp án một lượng là $f(i) \times f(j)$.

† Để tính được hàm f :

- Với một số x , ta có nhận xét rằng các số có tận cùng bằng x tạo thành một cấp số cộng:

$$x, x + 10, x + 20, x + 30, \dots$$

- Từ đó để tính được hàm f , ta cần xác định số bé nhất có tận cùng là x mà không bé hơn L .
- Sau khi xác định được số đó, ta cần đếm thêm mỗi lần cộng 10 thì còn bao nhiêu số không lớn hơn R .

Cài đặt:

2.2.4 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

int l, r, p;

long long f(int x) {
    int i = 1;
    while(i <= r) {
        if ((i % 10) == x) break;
        ++i;
    }
    if (i > r) return 0;
    return (r - i) / 10 + 1;
}

int main() {
    ios_base::sync_with_stdio(0); cin.tie(0);

    freopen("LUCKYNUM.INP", "r", stdin);
    freopen("LUCKYNUM.OUT", "w", stdout);

    cin >> l >> r >> p;
    long long ans = 0;

    for(int i = 0; i <= 9; ++i) {
        for(int j = 0; j <= 9; ++j) {
            if ((i * j) % 10 == p) {
                ans += f(i) * f(j);
            }
        }
    }

    cout << ans;

    return 0;
}
```

2.3 Bài 3. HOSTING

Tóm tắt đề bài:

Cho N khách hàng. Khách hàng thứ i sẽ dùng một lượng dung lượng k_i từ ngày x_i đến ngày $x_i + d_i - 1$.

Công ty cần theo dõi biến động của tổng dung lượng các khách hàng sử dụng theo từng ngày, kể từ ngày bắt đầu (giá trị x_i nhỏ nhất) đến ngày kết thúc (giá trị $x_i + d_i - 1$ lớn nhất).

Yêu cầu: Hãy in ra các biến động, nói cách khác là những lần mà dung lượng của công ty có sự thay đổi so với ngày hôm trước.

2.3.1 Subtask 1: $1 \leq N \leq 100$ và $1 \leq x_i + d_i \leq 100$

- Để ý thấy giới hạn của subtask này rất bé, vì thế ta chỉ cần mô phỏng lại những gì đề bài yêu cầu:
 1. Ta xét từng ngày một, ở mỗi ngày, ta duyệt qua toàn bộ danh sách khách hàng để kiểm tra:
 - khách hàng đó có đang sử dụng dịch vụ vào ngày này hay không
 - nếu có thì cộng thêm dung lượng mà khách hàng đó sử dụng
 2. Sau khi tính được tổng dung lượng của ngày hiện tại, ta so sánh với tổng dung lượng của ngày hôm trước.
 3. Nếu hai giá trị khác nhau, tức là có biến động, ta in ra giá trị mới.

2.3.2 Subtask 2 + 3: $1 \leq N \leq 8 \cdot 10^5$ và $1 \leq x_i + d_i \leq 10^5$

- Để giải quyết 2 subtask này, ta dùng kĩ thuật mảng hiệu (difference array).
- Gọi mảng `pref` để xác định tổng dung lượng đang được sử dụng ở mỗi ngày.
- Vì khách hàng bắt đầu dùng tại ngày x_i và kết thúc sau ngày $x_i + d_i - 1$, nên:

$$\begin{cases} \text{pref}[x_i] \leftarrow \text{pref}[x_i] + k_i \\ \text{pref}[x_i + d_i] \leftarrow \text{pref}[x_i + d_i] - k_i \end{cases}$$
- Từ đây, ta tính prefix sum để có tổng dung lượng từng ngày.
- Sau đó, in ra các ngày mà tổng dung lượng khác ngày trước.
- Độ phức tạp: $\mathcal{O}(N + D)$

Cài đặt:

2.3.3 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

const int N = 1e6;

int n, m;
long long pref[N];

int main() {
```

```

ios_base::sync_with_stdio(0); cin.tie(0);

freopen("HOSTING.INP", "r", stdin);
freopen("HOSTING.OUT", "w", stdout);

cin >> n;
for(int i = 1; i <= n; ++i) {
    int x, d, k; cin >> x >> d >> k;

    pref[x] += k;
    pref[x + d] -= k;
    m = max(m, x + d - 1);
}

for(int i = 1; i <= m; ++i) {
    pref[i] += pref[i - 1];
}

for(int i = 1; i <= m; ++i) {
    if (pref[i] != pref[i - 1]) {
        cout << pref[i] << " ";
    }
}

return 0;
}

```

3 Đề Hồ Chí Minh trường Phổ Thông Năng Khiếu

3.1 Bài 1: CHUỖI ONLINE BẠN BÈ:

3.1.1 Tóm tắt đề bài

Cho dãy gồm n xâu, mỗi xâu chỉ có dạng là **ONLINE** hoặc **OFFLINE** hoặc **IDLE**. Hãy tìm dãy các xâu liên tiếp dài nhất, mà các xâu trong đó đều là **ONLINE**.

3.1.2 Lời giải

- Gọi cur là độ dài dãy con kết thúc tại vị trí i mà ta đang xét sao cho toàn bộ là xâu **ONLINE**.
- Nếu xâu thứ i không phải **ONLINE** thì $cur = 0$
- Gọi res là đáp án hiện tại.
- Với mỗi xâu i :

- Nếu xâu i là **ONLINE**, ta tăng cur một lượng là 1, ngược lại ta gán cur thành 0.
- Sau mỗi lượt, ta cập nhật $res \leftarrow \max(res, cur)$.

- Độ phức tạp: $\mathcal{O}(n)$

Cài đặt:

3.1.3 Code mẫu tham khảo:

```
#include<bits/stdc++.h>

using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);

    int n;
    cin >> n;

    int cur = 0, res = 0;

    for(int i = 1; i <= n; ++i) {
        string s; cin >> s;

        if (s == "ONLINE") ++cur;
        else cur = 0;

        res = max(res, cur);
    }

    cout << res << '\n';

    return 0;
}
```

3.2 Bài 2: HÀNH TRÌNH XE ĐIỆN:

3.2.1 Tóm tắt đề bài

Có một chiếc xe điện với dung lượng pin là P đơn vị, Để di chuyển 1 km tốn 1 đơn vị pin. Xe hết pin sẽ không thể di chuyển. Có N trạm sạc pin cho xe nằm tại các vị trí $a_1, a_2, \dots, a_n$. Mỗi lần sạc thì pin sẽ về dung lượng tối đa là $P_{\max}$. Bạn sẽ bắt đầu tại vị trí 0 km với quả pin đầy. Hãy tìm cách di chuyển vị trí D mà sạc ít lần nhất

3.2.2 Lời giải

- Ta coi như vị trí bắt đầu là trạm sạc đầu tiên $a_0 = 0$.
- Ta sắp xếp vị trí các trạm sạc theo thứ tự tăng dần.
- Gọi $dp[i]$ là số lần sạc tối thiểu khi ta có đầy pin ở vị trí a_i , đồng thời ta có sạc tại a_i .
- Để có thể tới trạm a_i , ta phải sạc đầy pin ở trạm a_j trước đó ($j < i$), sao cho $a_i - a_j \leq P$.
- Từ đây, ta rút ra được công thức truy hồi:

$$dp[i] = \min_{\substack{1 \leq j < i \\ a_i - a_j \leq P}} (dp[j] + 1)$$

- Ta khởi tạo $dp[i] = \infty \forall i \in [1, n]$ và $dp[0] = 0$.
- Đáp án của ta sẽ là min của mọi $dp[i]$ thỏa mãn: $D - a_i \leq P$.
- Lưu ý: ta xét cả a_0 . (trường hợp không cần sạc lần nào)
- Độ phức tạp $\mathcal{O}(n^2)$

Cài đặt:

3.2.3 Code mẫu tham khảo:

```
#include <bits/stdc++.h>

using namespace std;

int d[1005], dp[1005];

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);

    int n, P, D; cin >> n >> P >> D;

    for(int i = 1; i <= n; ++i) cin >> d[i];

    sort(d + 1, d + 1 + n);

    for(int i = 1; i <= n; ++i) dp[i] = 1e9;
    dp[0] = 0;

    for(int i = 1; i <= n; ++i) {
        for(int j = 0; j < i; ++j) {
            if (d[i] - d[j] <= P) {
```

```

        dp[i] = min(dp[i], dp[j] + 1);
    }
}

int ans = 1e9;
for(int i = 0; i <= n; ++i) {
    if (D - d[i] <= P) ans = min(ans, dp[i]);
}

cout << (ans == 1e9 ? -1 : ans) << '\n';

return 0;
}

```

3.3 Bài 3: TRÒ CHƠI XẾP CHỮ:

3.3.1 Tóm tắt đề bài

Cho một chuỗi s chỉ gồm các kí tự in thường. Hãy loại bỏ ít kí tự nhất sao cho chuỗi s là chuỗi đối xứng.

3.3.2 Lời giải

- Ta sẽ giải quyết bài toán ngược của bài toán này, ta tìm chuỗi con đối xứng dài nhất trong s , vậy đáp án của ta là $n - \text{res}$ với res là độ dài chuỗi con đối xứng dài nhất trong s .
- Gọi $\text{dp}[l][r]$ là độ dài chuỗi con dài nhất từ vị trí l tới vị trí r trong s sao cho chuỗi đó là chuỗi đối xứng.
- Ta có công thức truy hồi

$$\text{dp}[l][r] = \begin{cases} 1, & l = r \\ 0, & l > r \\ \text{dp}[l+1][r-1] + 2, & s_l = s_r \\ \max(\text{dp}[l+1][r], \text{dp}[l][r-1]), & s_l \neq s_r \end{cases}$$

- Độ phức tạp: $\mathcal{O}(n^2)$

Cài đặt:

3.3.3 Code mẫu tham khảo:

```

#include<bits/stdc++.h>

using namespace std;

int dp[2005][2005];

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);

    string s; cin >> s;
    int n = (int)s.size();
    s = ' ' + s;

    for(int i = 1; i <= n; ++i) {
        dp[i][i] = 1;
    }

    for(int r = 1; r <= n; ++r) {
        for(int l = r - 1; l >= 1; --l) {
            if (s[l] == s[r]) {
                dp[l][r] = max(dp[l][r], dp[l + 1][r - 1] + 2);
            }
            else {
                dp[l][r] = max({dp[l][r], dp[l + 1][r], dp[l][r - 1]});
            }
        }
    }

    cout << n - dp[1][n] << '\n';

    return 0;
}

```

3.4 Bài 4: Mật khẩu:

3.4.1 Tóm tắt đề bài

Cho N giao dịch, giao dịch thứ i có phí F_i và kích thước S_i . Một khối blockchain có kích thước tối đa $S_{\max}$.

Cần chọn một tập con các giao dịch sao cho tổng kích thước không vượt quá $S_{\max}$ và tổng phí là lớn nhất.

Ngoài ra, có D ràng buộc phụ thuộc: nếu chọn giao dịch A thì bắt buộc phải chọn giao dịch B . Các ràng buộc này không có chu trình.

Hãy tính tổng phí lớn nhất có thể đạt được.

3.4.2 Lời giải

Lời giải được tham khảo từ code của chemthan: <https://ideone.com/zEa0yF>

- Bài toán này là bài toán thuộc NP-hard, vì vậy không có lời giải chính xác trong thời gian đa thức.
- Dưới đây, mình xin trình bày lời giải cho $N \leq 25$ và đưa ra vài hướng tiếp cận heuristic cho $N \leq 50$.
- $N \leq 25$
 - Tư tưởng của chúng ta là ta sẽ duyệt mọi tập con của n phần tử. Sau đó, ta kiểm tra xem tập con này có thoả mãn hay không.
 - Với mỗi điều kiện i , ta lưu lại các vật phẩm phải mua dưới dạng bitmask.
 - Để kiểm tra hợp lệ, ta sử dụng toán tử AND.
 - Gọi mask là tập con hiện tại ta đang xét, must là tập hợp các vật phẩm khi mua vật phẩm thứ i , sao cho vật phẩm thứ i thuộc mask.
 - Tập này thoả yêu cầu đề bài khi và chỉ khi $\text{mask} \& \text{must} = \text{must}$.
 - Sau đó, ta tính tổng S , nếu $S \leq S_{\max}$ thì cập nhật đáp án.
 - Độ phức tạp: $\mathcal{O}(2^n \times n)$.
- $N \leq 50$
 - Tham lam (ví dụ: theo tỉ lệ $\frac{F_i}{S_i}$)
 - Hill climb
 - Simulated Annealing

Cài đặt:

3.4.3 Code mẫu tham khảo:

```
#include<bits/stdc++.h>

using namespace std;

int f[51], s[51];
long long must[51];

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);

    int n, m;
```

```
cin >> n >> m;

assert(n <= 25);

for (int i = 0; i < n; i++) {
    cin >> f[i] >> s[i];
    must[i] = 0;
}

int k;
cin >> k;

for (int i = 0; i < k; i++) {
    int a, b;
    cin >> a >> b;
    —a; —b;
    must[a] |= (1LL << b);
}

for (int mid = 0; mid < n; mid++) {
    for (int i = 0; i < n; i++) {
        if (must[i] & (1 << mid)) {
            must[i] |= must[mid];
        }
    }
}

int ans = 0;

for (int mask = 0; mask < (1 << n); mask++) {
    int cost = 0, value = 0;
    bool ok = true;

    for (int i = 0; i < n; i++) {
        if (mask & (1 << i)) {
            if ((mask & must[i]) != must[i]) {
                ok = false;
                break;
            }
            cost += s[i];
            if (cost > m) {
                ok = false;
                break;
            }
            value += f[i];
        }
    }
}
```

```
        if (ok) ans = max(ans, value);
    }

    cout << ans << '\n';

    return 0;
}
```

4 An Giang

4.1 Bài 1: Tách số

4.1.1 Lời giải:

Sử dụng vòng lặp for để duyệt xâu, nếu ký tự đang xét hiện tại là ký tự chữ số thì in ra.

4.1.2 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    string s;
    getline(cin, s);
    int found = 0;
    for (int i = 0; i < (int)s.size(); i++)
        if ('0' <= s[i] && s[i] <= '9') {
            cout << s[i];
            found = 1;
        }
    if (!found) cout << 0;
    return 0;
}
```

4.2 Bài 2: Chia hết cho 3

4.2.1 Lời giải:

Đầu tiên, kiểm tra xem N có chia hết cho 3 hay không.

Nếu N không chia hết cho 3, duyệt qua các chữ số của N , giả sử loại bỏ chữ số đang xét khỏi N , kiểm tra các chữ số còn lại (vẫn giữ nguyên thứ tự ban đầu) có tạo thành một số chia hết cho 3 hay không.

4.2.2 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    long long n;
    cin >> n;
    if (n % 3 == 0) cout << n << "\n";
    else {
        vector<int> digit;
        int ans = 0;
        while (n) {
            digit.push_back(n % 10);
            n /= 10;
        }
        for (int i = 0; i < (int)digit.size(); i++) {
            int num = 0;
            for (int j = (int)digit.size() - 1; j >= 0; j--)
                if (j != i) num = num * 10 + digit[j];
            if (num % 3 == 0) ans = max(ans, num);
        }
        cout << ans << "\n";
    }
    return 0;
}
```

4.3 Bài 3: Cân đĩa

4.3.1 Lời giải:

Vì các quả cân có dạng 3^k , nên ta sẽ quan tâm đến biểu diễn N ở hệ cơ số 3.

Xử lý từ bit thấp đến bit cao, với mỗi bit thứ i đang xét:

- Nếu bit thứ i có hệ số là 1: Ta cần đặt thêm một quả cân 3^i ở đĩa cân bên phải để cân bằng với đĩa cân bên trái hiện tại.
- Nếu bit thứ i có hệ số là 2: Vì mỗi quả cân chỉ được sử dụng một lần, nên ta không thể đặt 2 quả cân 3^i vào đĩa cân bên phải được. Chính vì vậy, ta sẽ đặt thêm 1 quả cân 3^i vào đĩa cân bên trái, khi đó sẽ thành $3 \times 3^i = 3^{i+1}$, nên ta sẽ tăng hệ số của bit thứ $i + 1$ lên 1 đơn vị.
- Nếu bit thứ i có hệ số là 3: Ta có $3 \times 3^i = 3^{i+1}$, nên ta sẽ tăng hệ số của bit thứ $i + 1$ lên 1 đơn vị.

4.3.2 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

long long pw3[45];

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    long long n;
    cin >> n;
    pw3[0] = 1;
    for (int i = 1; i <= 39; i++) pw3[i] = pw3[i - 1] * 3;
    vector<int> bit, left, right;
    while (n) {
        bit.push_back(n % 3);
        n /= 3;
    }
    int i = 0;
    while (i < (int)bit.size()) {
        if (bit[i] == 1) right.push_back(pw3[i]);
        else if (bit[i] == 2) {
            if (i + 1 == (int)bit.size()) bit.push_back(1);
            else bit[i + 1]++;
            left.push_back(pw3[i]);
        }
        else if (bit[i] == 3) {
            if (i + 1 == (int)bit.size()) bit.push_back(1);
            else bit[i + 1]++;
        }
        i++;
    }
    cout << left.size() << " ";
    for (int i = 0; i < (int)left.size(); i++) cout << left[i] << " ";
    cout << "\n";
    cout << right.size() << " ";
    for (int i = 0; i < (int)right.size(); i++) cout << right[i] << " ";
    cout << "\n";
}
```

4.4 Bài 4: Tính giá trị biểu thức

4.4.1 Lời giải:

Duyệt từ đầu xâu đến cuối xâu:

- Nếu ký tự hiện tại là ký tự chữ số, tiến hành tạo thành một số hoàn chỉnh rồi đẩy vào một vector chứa giá trị.

- Nếu ký tự hiện tại là dấu ngoặc mở '(' thì đẩy vào vector chứa dấu.
- Nếu ký tự hiện tại là dấu ngoặc đóng ')' thì tiến hành tính toán biểu thức trong dấu ngoặc.
- Nếu ký tự hiện tại là một dấu '+', '-', '*', '/' hay '^' thì thực hiện trước những phép toán có thứ tự ưu tiên lớn hoặc bằng phép toán này, rồi mới đẩy ký tự hiện tại vào vector chứa dấu.

4.4.2 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

#define int long long

int level(char op) {
    if (op == '+' || op == '-') return 1;
    else if (op == '*' || op == '/') return 2;
    else if (op == '^') return 3;
    return 0;
}

int power(int a, int b) {
    int ans = 1;
    while (b) {
        if (b % 2) ans = ans * a;
        a = a * a;
        b /= 2;
    }
    return ans;
}

int calc(int x, int y, char op) {
    if (op == '+') return x + y;
    else if (op == '-') return x - y;
    else if (op == '*') return x * y;
    else if (op == '/') return x / y;
    else return power(x, y);
}

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    string s;
    cin >> s;
    vector<int> val;
    vector<char> op;
```

```

int i = 0;
while (i < (int)s.size()) {
    if ('0' <= s[i] && s[i] <= '9') {
        int j = i, num = 0;
        while (j < (int)s.size() && '0' <= s[j] && s[j] <= '9') {
            num = num * 10 + s[j] - '0';
            j++;
        }
        val.push_back(num);
        i = j;
    }
    else {
        if (s[i] == '(') op.push_back(s[i]);
        else if (s[i] == ')') {
            while (!op.empty() && op.back() != '(') {
                int y = val.back(); val.pop_back();
                int x = val.back(); val.pop_back();
                val.push_back(calc(x,y,op.back()));
                op.pop_back();
            }
            if (!op.empty() && op.back() == '(') op.pop_back();
        }
        else {
            while (!op.empty() && level(s[i]) <= level(op.back())) {
                int y = val.back(); val.pop_back();
                int x = val.back(); val.pop_back();
                val.push_back(calc(x,y,op.back()));
                op.pop_back();
            }
            op.push_back(s[i]);
        }
        i++;
    }
}

while (!op.empty()) {
    int y = val.back(); val.pop_back();
    int x = val.back(); val.pop_back();
    val.push_back(calc(x,y,op.back()));
    op.pop_back();
}

cout << val.back() << "\n";
}

```

4.5 Bài 5: Tam giác

4.5.1 Lời giải:

Sử dụng bất đẳng thức tam giác để kiểm tra xem a, b, c có tạo thành một tam giác hay không.

a, b, c tạo thành một tam giác khi và chỉ khi: $a + b > c$ và $a + c > b$ và $b + c > a$.

Nếu a, b, c thỏa mãn điều kiện trên thì in ra chu vi của tam giác là $a + b + c$, ngược lại, in ra "KHONG PHAI TAM GIAC".

4.5.2 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    int a, b, c;
    cin >> a >> b >> c;
    if (a + b > c && a + c > b && b + c > a) cout << a + b + c << "\n";
    else cout << "KHONG PHAI TAM GIAC\n";
    return 0;
}
```

5 Bà Rịa - Vũng Tàu

5.1 Bài 1: Đếm ước

Tóm tắt bài toán:

- Cho hai số n, k .
- Cần xét các số i trong khoảng $[1, n - 1]$.
- Với mỗi i , đếm số lượng ước của nó. Nếu số lượng ước đúng bằng k thì tăng biến đếm kết quả.

Phân tích bài toán:

- **Chiến thuật ban đầu (Dành cho Subtask 1):**
 - Duyệt từng số i từ 1 đến $n - 1$.
 - Với mỗi số i , ta đếm ước bằng cách duyệt từ 1 đến i .
 - Độ phức tạp: $O(n^2)$. Với $n = 10^3$, số phép tính là 10^6 , hoàn toàn khả thi.

• **Tối ưu hóa (Dành cho Subtask 2):**

- Để xử lý $n = 10^6$, ta không thể dùng $O(n^2)$.
- **Cách 1: Duyệt đến $\sqrt{i}$.** Với mỗi i , ta chỉ cần duyệt ước đến $\sqrt{i}$. Độ phức tạp $O(n\sqrt{n})$. Với $n = 10^6$, số phép tính khoảng 10^9 , có thể bị quá thời gian (thường giới hạn là 10^8 phép tính/giây).
- **Cách 2: Sử dụng Sàng nguyên tố biến thể.** Tương tự như sàng Eratosthenes, ta dùng một mảng $d[1\dots n]$ để lưu số lượng ước.
- Duyệt mỗi số j từ 1 đến n . Với mỗi j , tất cả các bội của nó là $j, 2j, 3j, \dots$ đều nhận j làm một ước. Ta cộng thêm 1 vào các vị trí $d[j], d[2j], d[3j], \dots$.
- Độ phức tạp: $O(n \log n)$. Đây là cách tối ưu nhất cho bài toán này.

5.1.1 Subtask 1 ($O(n^2)$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, k;
    cin >> n >> k;
    int cnt = 0;
    for (int i = 1; i < n; i++) {
        int uoc = 0;
        for (int j = 1; j <= i; j++) {
            if (i % j == 0) uoc++;
        }
        if (uoc == k) cnt++;
    }
    cout << cnt;
    return 0;
}
```

5.1.2 Subtask 2 ($O(n \log n)$)

```
#include <bits/stdc++.h>
using namespace std;

int d[1000005];

int main() {
    int n, k;
    cin >> n >> k;

    for (int i = 1; i < n; i++) {
        for (int j = i; j < n; j += i) {
```

```

        d[j]++;
    }
}

int ans = 0;
for (int i = 1; i < n; i++) {
    if (d[i] == k) ans++;
}

cout << ans;
return 0;
}

```

5.2 Bài 2: Số siêu nguyên tố

Tóm tắt bài toán:

- Tìm các số $X \in [a, b]$ thỏa mãn 3 điều kiện: X là số nguyên tố, tổng chữ số của X là số nguyên tố, và mọi chữ số của $X \in \{2, 3, 5, 7\}$.

Phân tích bài toán:

- **Chiến thuật ban đầu (Dành cho Subtask 1):**

- Duyệt mọi số X từ a đến b .
- Viết hàm kiểm tra số nguyên tố và hàm kiểm tra các điều kiện về chữ số.
- Độ phức tạp: $O((b - a) \times \sqrt{X})$. Với $b = 10^5$, thuật toán này chạy ổn định.

- **Tối ưu hóa (Dành cho Subtask 2):**

- Với $b = 10^8$, việc duyệt từng số sẽ rất chậm (10^8 vòng lặp).
- Nhận xét: Các chữ số của X chỉ có thể là $\{2, 3, 5, 7\}$. Thay vì duyệt mọi số, ta chỉ sinh ra các số được cấu tạo từ 4 chữ số này.
- Số lượng số có tối đa 8 chữ số mà chỉ gồm $\{2, 3, 5, 7\}$ là $4^1 + 4^2 + \dots + 4^8 = 87380$ số. Đây là một con số rất nhỏ.
- Với mỗi số được sinh ra, ta kiểm tra xem nó có nằm trong $[a, b]$ không, sau đó kiểm tra tính nguyên tố của bản thân nó và tổng các chữ số của nó.

5.2.1 Subtask 1 ($O(N\sqrt{N})$)

```

#include <bits/stdc++.h>
using namespace std;

bool isPrime(int n) {
    if (n < 2) return false;

```

```

    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}

bool checkDigits(int n) {
    int sum = 0;
    while (n > 0) {
        int d = n % 10;
        if (d != 2 && d != 3 && d != 5 && d != 7) return false;
        sum += d;
        n /= 10;
    }
    return isPrime(sum);
}

int main() {
    int a, b;
    cin >> a >> b;
    bool found = false;
    for (int i = a; i <= b; i++) {
        if (checkDigits(i) && isPrime(i)) {
            cout << i << " ";
            found = true;
        }
    }
    if (!found) cout << 0;
    return 0;
}

```

5.2.2 Subtask 2 (Sinh số tối ưu)

```

#include <bits/stdc++.h>
using namespace std;

long long a, b;
vector<long long> res;
int digits[] = {2, 3, 5, 7};

bool isPrime(long long n) {
    if (n < 2) return false;
    for (long long i = 2; i * i <= n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}

```

```

void backtrack(long long cur, int sumDigits) {
    if (cur > b) return;
    if (cur >= a) {
        if (isPrime(cur) && isPrime(sumDigits)) {
            res.push_back(cur);
        }
    }
    if (cur > 100000000) return;
    for (int i = 0; i < 4; i++) {
        backtrack(cur * 10 + digits[i], sumDigits + digits[i]);
    }
}

int main() {
    cin >> a >> b;
    for (int i = 0; i < 4; i++) {
        backtrack(digits[i], digits[i]);
    }
    sort(res.begin(), res.end());
    if (res.empty()) {
        cout << 0;
    } else {
        for (int i = 0; i < res.size(); i++) {
            cout << res[i] << " ";
        }
    }
    return 0;
}

```

5.3 Bài 3: Số đặc biệt

Tóm tắt bài toán

- Cho một dãy số a có n phần tử và Q truy vấn.
- Mỗi truy vấn yêu cầu đếm các số a_i trong đoạn $[L, R]$ sao cho a_i chia hết cho tổng các chữ số của chính nó.
- Giới hạn n, Q lên đến 10^5 , giá trị phần tử lên đến 10^9 .

Phân tích bài toán

- **Kiểm tra số đặc biệt:** Với mỗi số x , ta tính tổng các chữ số bằng cách dùng vòng lặp lấy $x \bmod 10$ và $x/10$. Sau đó kiểm tra $x \bmod \text{tổng} == 0$.
- **Chiến thuật ban đầu (Vét cạn):** Với mỗi truy vấn $[L, R]$, ta duyệt từ L đến R , kiểm tra từng số xem có phải số đặc biệt không và tăng biến đếm. Thuật toán này có độ phức tạp $O(Q \times n)$, chỉ phù hợp với các subtask có Q nhỏ hoặc tổng n nhỏ.

• **Tối ưu hóa (Tiền xử lý mảng cộng dồn):**

- Nhận xét rằng việc kiểm tra một số có là số đặc biệt hay không là độc lập và cố định.
- Ta tạo một mảng phụ b trong đó $b_i = 1$ nếu a_i là số đặc biệt, ngược lại $b_i = 0$.
- Xây dựng mảng cộng dồn P với $P_i = P_{i-1} + b_i$.
- Khi đó, kết quả của truy vấn $[L, R]$ sẽ là $P_R - P_{L-1}$ với độ phức tạp $O(1)$ mỗi truy vấn.
- Tổng độ phức tạp: $O(n \log_{10}(\max a_i) + Q)$.

5.3.1 Subtask 1 & 2 (Q nhỏ hoặc chỉ có 1 truy vấn)

```
#include <bits/stdc++.h>
using namespace std;

bool isSpecial(long long x) {
    long long sum = 0;
    long long temp = x;
    while (temp > 0) {
        sum += temp % 10;
        temp /= 10;
    }
    return (x % sum == 0);
}

int main() {
    int n, q;
    cin >> n >> q;
    vector<long long> a(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    while (q--) {
        int l, r;
        cin >> l >> r;
        int cnt = 0;
        for (int i = l; i <= r; i++) {
            if (isSpecial(a[i])) {
                cnt++;
            }
        }
        cout << cnt << endl;
    }
    return 0;
}
```

5.3.2 Subtask 3 (Full test - Tối ưu bằng mảng cộng dồn)

```
#include <bits/stdc++.h>
using namespace std;

bool check(long long x) {
    if (x == 0) return false;
    long long s = 0;
    long long t = x;
    while (t > 0) {
        s += t % 10;
        t /= 10;
    }
    return (x % s == 0);
}

int p[100005];
long long a[100005];

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    int n, q;
    cin >> n >> q;
    p[0] = 0;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        p[i] = p[i - 1];
        if (check(a[i])) {
            p[i]++;
        }
    }
    while (q--) {
        int l, r;
        cin >> l >> r;
        cout << p[r] - p[l - 1] << "\n";
    }
    return 0;
}
```

5.4 Bài 4: Ôn tập

Tóm tắt bài toán

- Có n học sinh với năng lực ban đầu a_i và m bài tập với độ khó b_j .
- Điều kiện giải bài: Năng lực hiện tại $\geq$ độ khó bài tập.
- Phần thưởng: Giải được bài b_j thì năng lực tăng thêm đúng b_j .

- Mỗi học sinh có thể giải các bài tập theo thứ tự bất kỳ để tối ưu hóa năng lực.
- Mục tiêu: Tìm năng lực cực đại của từng học sinh sau khi giải hết các bài có thể.

Phân tích bài toán

- **Nhận xét:** Để giải được nhiều bài nhất và tăng năng lực nhanh nhất, học sinh nên ưu tiên giải các bài có độ khó từ thấp đến cao. Nếu không giải được bài dễ nhất hiện có, học sinh đó cũng không thể giải được các bài khó hơn.
- **Chiến thuật ban đầu (Vét cạn):**
 - Sắp xếp mảng b tăng dần.
 - Với mỗi học sinh, duyệt qua danh sách bài tập đã sắp xếp. Nếu năng lực hiện tại $\geq b_j$, cộng b_j vào năng lực và tiếp tục xét bài tiếp theo.
 - Độ phức tạp: $O(m \log m + n \times m)$. Với $n, m = 5 \cdot 10^5$, cách này sẽ bị quá thời gian.
- **Tối ưu hóa:**
 - Sau khi sắp xếp b tăng dần, ta nhận thấy việc giải bài tập là một quá trình "ăn" các phần tử liên tiếp trên mảng b tính từ vị trí 1.
 - Gọi $S_k = \sum_{j=1}^k b_j$ là tổng độ khó của k bài tập dễ nhất. Năng lực của học sinh sau khi giải k bài đầu tiên là $a_i + S_k$.
 - Học sinh giải được bài thứ $k+1$ nếu: $a_i + S_k \geq b_{k+1}$.
 - Vì $a_i + S_k$ tăng dần theo k , ta có thể sử dụng tìm kiếm nhị phân để tìm chỉ số k lớn nhất thỏa mãn điều kiện giải được bài.
 - Tuy nhiên, điều kiện $a_i + S_k \geq b_{k+1}$ không phải lúc nào cũng duy trì tính đơn điệu trực tiếp với k nếu xét a_i . Nhưng thực tế, vì b_j tăng dần và S_k cũng tăng, nếu đã giải được k bài thì khả năng giải bài $k+1$ chỉ phụ thuộc vào tổng năng lực tích lũy. Ta tìm k tối đa sao cho mọi bước từ $1 \dots k$ đều hợp lệ.
 - Cách hiệu quả nhất: Xây dựng mảng $C_k = b_k - S_{k-1}$. Điều kiện để giải được bài k là $a_i \geq C_k$. Để giải được đến bài k , học sinh cần $a_i \geq \max(C_1, C_2, \dots, C_k)$.
 - Đặt $M_k = \max_{j=1 \dots k} (C_j)$. Khi đó học sinh giải được k bài nếu $a_i \geq M_k$. Vì M_k là dãy không giảm, ta tìm kiếm nhị phân k lớn nhất sao cho $M_k \leq a_i$.

5.4.1 Subtask 1 ($n, m \leq 10^3$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
```

```

vector<long long> a(n);
for (int i = 0; i < n; i++) cin >> a[i];
vector<long long> b(m);
for (int i = 0; i < m; i++) cin >> b[i];

sort(b.begin(), b.end());

for (int i = 0; i < n; i++) {
    long long cur = a[i];
    for (int j = 0; j < m; j++) {
        if (current >= b[j]) {
            cur += b[j];
        } else {
            break;
        }
    }
    cout << cur << " ";
}
return 0;
}

```

5.4.2 Subtask 2 (Full test - Tối ưu bằng Tìm kiếm nhị phân)

```

#include <bits/stdc++.h>
using namespace std;

long long a[500005];
long long b[500005];
long long s[500005];
long long m_val[500005];

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= n; i++) cin >> a[i];
    for (int i = 1; i <= m; i++) cin >> b[i];

    sort(b + 1, b + m + 1);

    s[0] = 0;
    m_val[0] = 0;
    for (int i = 1; i <= m; i++) {
        s[i] = s[i - 1] + b[i];
        m_val[i] = max(m_val[i - 1], b[i] - s[i - 1]);
    }
}

```

```

    for (int i = 1; i <= n; i++) {
        int low = 1, high = m, ans = 0;
        while (low <= high) {
            int mid = (low + high) / 2;
            if (a[i] >= m_val[mid]) {
                ans = mid;
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        cout << a[i] + s[ans] << " ";
    }

    return 0;
}

```

6 Phú Yên

6.1 Bài 1: TÌM CẶP SỐ

Phân tích thuật toán: Bài toán yêu cầu tìm số lượng cặp (a_i, a_j) sao cho $a_i + a_j = T$ với $i < j$. Với giới hạn $n \leq 10^3$, ta có thể duyệt trau $O(n^2)$. Tuy nhiên, giới hạn giá trị $a_i \leq 10^5$ cho phép ta tối ưu bằng mảng đếm phân phối để đạt độ phức tạp $O(n)$.

6.1.1 Subtask 1: Cách duyệt trau $O(n^2)$

Phân tích: Dùng hai vòng lặp lồng nhau duyệt qua tất cả các cặp i và j (với $i < j$) để kiểm tra xem tổng $a_i + a_j$ có bằng T hay không. Thuật toán này chạy khoảng 10^6 phép tính, hoàn toàn qua được thời gian 1 giây.

```

#include <bits/stdc++.h>
#define int long long
using namespace std;

int n, t;
int a[1005];
int ans = 0;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n >> t;

```

```

    for(int i = 1; i <= n; i++) {
        cin >> a[i];
    }

    for(int i = 1; i < n; i++) {
        for(int j = i + 1; j <= n; j++) {
            if(a[i] + a[j] == t) {
                ans++;
            }
        }
    }

    cout << ans << "\n";
}

```

6.1.2 Subtask 2: Tối ưu hoá $O(n)$

Phân tích: Dùng một mảng cnt để lưu số lần xuất hiện của các phần tử. Khi duyệt đến phần tử a_i , ta chỉ cần xem giá trị $req = t - a[i]$ đã xuất hiện bao nhiêu lần trước đó và cộng thẳng vào biến ans. Sau đó tăng biến đếm $cnt[a[i]]$ lên.

```

#include <bits/stdc++.h>
#define int long long
using namespace std;

int n, t;
int a[1005];
int cnt[100005];
int ans = 0;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n >> t;
    for(int i = 1; i <= n; i++) {
        cin >> a[i];

        int req = t - a[i];
        if(req >= 0 && req <= 100000) {
            ans += cnt[req];
        }

        if(a[i] <= 100000) {
            cnt[a[i]]++;
        }
    }
}

```

```
cout << ans << "\n";
}
```

6.2 Bài 2: CHUẨN HÓA XÂU

Phân tích thuật toán: Chúng ta sẽ đọc từng dòng dữ liệu đầu vào cho đến khi kết thúc file. Với mỗi dòng, ta sử dụng một xâu kết quả `res` và duyệt qua từng ký tự của xâu ban đầu:

- Nếu là dấu cách (' '): Ta chỉ thêm vào res khi res không rỗng và ký tự cuối cùng của res không phải là dấu cách (đảm bảo không có dấu cách thừa ở đầu và giữa các từ chỉ có 1 dấu cách).
- Nếu là dấu câu (' , ' , ' . ' , ' ? ' , ' ! '): Ta kiểm tra xem ký tự cuối của res có phải là dấu cách hay không. Nếu có, ta xóa nó đi (đảm bảo trước dấu câu không có dấu cách). Sau đó, ta thêm dấu câu này vào res và tự động thêm một dấu cách ngay sau nó.
- Nếu là các ký tự chữ cái/số bình thường: Ta thêm trực tiếp vào res.
- Cuối cùng, sau khi duyệt xong, xâu res có thể bị dư một dấu cách ở cuối (do quy tắc thêm dấu cách sau dấu câu). Ta thực hiện xóa các dấu cách thừa ở cuối xâu và in ra kết quả.

Thuật toán chạy với độ phức tạp $O(L)$ cho mỗi dòng (với L là độ dài xâu), vô cùng tối ưu và vượt qua mọi bộ test.

6.2.1 Code mẫu

```
#include <bits/stdc++.h>
#define int long long

using namespace std;

string s;
string res;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    while (getline(cin, s)) {
        res = "";
        for (int i = 0; i < s.length(); i++) {
            if (s[i] == '\\r') continue;

```

```

        if (s[i] == ' ') {
            if (!res.empty() && res.back() != ' ') {
                res += ' ';
            }
        } else if (s[i] == ',' || s[i] == '.' || s[i] == '?' || s[i] == '!') {
            if (!res.empty() && res.back() == ' ') {
                res.pop_back();
            }
            res += s[i];
            res += ' ';
        } else {
            res += s[i];
        }
    }

    while (!res.empty() && res.back() == ' ') {
        res.pop_back();
    }

    cout << res << "\n";
}
}

```

6.3 Bài 3: CHI PHÍ NHỎ NHẤT

Phân tích thuật toán: Bài toán yêu cầu ghép cặp N khoảng cách D_i với N chiếc xe (được chọn từ M chiếc xe có sẵn) sao cho tổng chi phí xăng $\sum(D_i \times V_j)$ là nhỏ nhất.

Theo nguyên lý của **Bất đẳng thức sắp xếp (Rearrangement Inequality)**, để tổng các tích của hai dãy số đạt giá trị nhỏ nhất, ta cần ghép phần tử lớn nhất của dãy này với phần tử nhỏ nhất của dãy kia, phần tử lớn thứ hai của dãy này với phần tử nhỏ thứ hai của dãy kia, và tiếp tục như vậy.

Áp dụng vào bài toán:

1. Những đoàn đi khoảng cách càng xa thì càng tốn nhiều xăng, do đó ta phải ưu tiên bố trí cho họ những chiếc xe tiêu thụ ít xăng nhất.
2. Sắp xếp mảng khoảng cách D theo thứ tự giảm dần.
3. Sắp xếp mảng mức tiêu thụ xăng V theo thứ tự tăng dần.
4. Chọn N chiếc xe đầu tiên trong mảng V (chính là N xe tiết kiệm xăng nhất) và lần lượt ghép cặp với N khoảng cách trong mảng D đã sắp xếp.

Với $N, M \leq 200$, thao tác sắp xếp có độ phức tạp $O(M \log M)$, thuật toán chạy cực kỳ tối ưu và nhẹ nhàng vượt qua mọi giới hạn thời gian.

6.3.1 Code mẫu

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int n, m;
int d[205];
int v[205];
int ans = 0;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n >> m;

    for(int i = 1; i <= n; i++)
        cin >> d[i];

    for(int i = 1; i <= m; i++)
        cin >> v[i];

    sort(d + 1, d + n + 1, greater<int>());
    sort(v + 1, v + m + 1);

    for(int i = 1; i <= n; i++)
        ans += d[i] * v[i];

    cout << ans << "\n";
}
```

6.4 Bài 4: PHÂN TÍCH SỐ

Phân tích thuật toán: Bài toán yêu cầu tìm số cách biểu diễn một số tự nhiên N thành tổng của ít nhất 2 số tự nhiên liên tiếp. Với giới hạn $N \leq 10^4$, ta có thể giải quyết bằng các vòng lặp duyệt cơ bản hoặc tối ưu hóa bằng kỹ thuật Hai con trỏ (Sliding Window).

6.4.1 Subtask 1: $N \leq 100$

Phân tích: Với N rất nhỏ, ta dùng hai vòng lặp lồng nhau. Vòng lặp ngoài cố định điểm bắt đầu i , vòng lặp trong chạy từ i trở đi để cộng dồn các giá trị. Nếu tổng bằng N , ta tăng biến đếm và dừng vòng lặp trong. Nếu tổng vượt quá N , ta cũng dừng vòng lặp trong để chuyển sang điểm bắt đầu mới.

```
#include <bits/stdc++.h>
```

```

#define int long long
using namespace std;

int n;
int ans = 0;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n;

    for (int i = 1; i <= n / 2; i++) {
        int sum = 0;
        for (int j = i; j < n; j++) {
            sum += j;
            if (sum == n) {
                ans++;
                break;
            }
            if (sum > n) {
                break;
            }
        }
    }

    cout << ans;
}

```

6.4.2 Subtask 2: $100 < N \leq 10^4$

Phân tích: Để tối ưu, ta sử dụng kỹ thuật **Hai con trỏ (Two Pointers)**. Duy trì một cửa sổ trượt từ vị trí l đến r .

- Mở rộng cửa sổ sang phải bằng cách tăng r và cộng r vào tổng.
- Nếu tổng vượt quá N , ta thu hẹp cửa sổ từ bên trái bằng cách trừ l khỏi tổng và tăng l .
- Nếu tổng đúng bằng N và cửa sổ có độ dài lớn hơn 1 ($r > l$), ta tăng biến đếm kết quả.

Độ phức tạp của thuật toán này là $O(N)$, xử lý cực kỳ mượt mà giới hạn $N = 10^4$ (thậm chí có thể chạy tốt với $N = 10^7$).

```

#include <bits/stdc++.h>
#define int long long
using namespace std;

```

```

int n, ans = 0, l = 1, sum = 0;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n;

    for (int r = 1; r < n; r++) {
        sum += r;

        while (sum > n) {
            sum -= l;
            l++;
        }

        if (sum == n && r > 1) {
            ans++;
        }
    }

    cout << ans;
}

```

7 Bắc Giang

7.1 Bài 1: Chữ số tận cùng

Tóm tắt đề bài: Cho hai số nguyên dương a và b . Tìm chữ số tận cùng của biểu thức a^b .

Ví dụ: $3^4 = 81$, chữ số tận cùng là 1.

7.1.1 Phân tích và Thuật toán

Chữ số tận cùng của một số chính là số dư của số đó khi chia cho 10. Do đó, yêu cầu bài toán là tính:

$$res = a^b \pmod{10}$$

Với bài toán này, nếu b lớn, ta không thể tính trực tiếp a^b vì sẽ gây tràn số ngay lập tức. Ta cần sử dụng thuật toán **Lũy thừa nhanh (Binary Exponentiation)** để tính toán trong độ phức tạp $O(\log b)$.

Các bước thực hiện:

1. Chỉ quan tâm đến chữ số tận cùng của cơ số a (tức $a = a \pmod{10}$).
2. Sử dụng vòng lặp để chia đôi số mũ b ở mỗi bước:
 - Nếu b lẻ: $res = (res \cdot a) \pmod{10}$.

- $a = (a \cdot a) \pmod{10}$.
- $b = b/2$.

7.1.2 Code mẫu (C++):

```
#include<bits/stdc++.h>
using namespace std;

#define int long long

int pw(int a, int b, int mod) {
    int res = 1;
    a %= mod;
    while (b) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    int a, b;
    cin >> a >> b;

    cout << pw(a, b, 10);

    return 0;
}
```

Độ phức tạp: $O(\log b)$, cho phép xử lý b lên đến 10^{18} trong thời gian cực ngắn.

7.2 Bài 2: Số chính phương

Tóm tắt đề bài: Cho dãy số gồm n số nguyên không âm $a_1, a_2, \dots, a_n$. Hãy tìm số chính phương nhỏ nhất $(0, 1, 4, 9, 16, 25, \dots)$ không xuất hiện trong dãy số đã cho.

Giới hạn:

- $1 \leq n \leq 10^6$.
- $0 \leq a_i \leq 10^{12}$.

7.2.1 Phân tích thuật toán

Nhận xét quan trọng: Trong một dãy có n phần tử, số chính phương nhỏ nhất không xuất hiện chắc chắn phải nằm trong tập hợp $n + 1$ số chính phương đầu tiên: $\{0^2, 1^2, 2^2, \dots, n^2\}$. Lý do: Nếu tất cả $n + 1$ số chính phương này đều xuất hiện, thì dãy phải có ít nhất $n + 1$ phần tử, điều này mâu thuẫn với giả thiết dãy chỉ có n phần tử.

Chiến thuật giải:

1. Ta sử dụng một mảng đánh dấu (hoặc vector bool) có kích thước $n + 1$ để lưu trạng thái xuất hiện của các căn bậc hai.
2. Với mỗi số a_i nhập vào:
 - Kiểm tra xem a_i có phải là số chính phương hay không bằng cách tính $r = \text{round}(\sqrt{a_i})$.
 - Nếu $r \times r = a_i$ và $r \leq n$, ta đánh dấu căn bậc hai r đã xuất hiện.
3. Sau khi xử lý hết n số, duyệt từ $i = 0$ đến n . Giá trị i đầu tiên chưa được đánh dấu sẽ cho kết quả là i^2 .

Độ phức tạp:

- **Thời gian:** $O(n)$ – duyệt qua n phần tử một lần.
- **Bộ nhớ:** $O(n)$ – sử dụng mảng đánh dấu kích thước 10^6 .

7.2.2 Code mẫu (C++):

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

const int N = 1000005;

int n, a, r;
int mark[N];

main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    cin >> n;

    for (int i = 1; i <= n; i++) {
        cin >> a;

        r = sqrt(a);
        if (r * r == a && r <= n) mark[r] = 1;
    }
```

```

    }

    for (int i = 0; i <= n; i++) {
        if (!mark[i]) {
            cout << i * i;
            return;
        }
    }
}

```

7.2.3 Giải thích ví dụ:

Với Input: 8

0 3 4 2 1 4 16 25

- Các số chính phương có trong dãy: 0, 4, 1, 16, 25.
- Các căn bậc hai tương ứng: 0, 2, 1, 4, 5.
- Danh sách các căn bậc hai đã xuất hiện: {0, 1, 2, 4, 5}.
- Căn bậc hai nhỏ nhất còn thiếu là 3.
- Kết quả: $3^2 = 9$.

7.3 Bài 3: Quản lý nhiệm vụ

Tóm tắt đề bài: Cho sạ bàn kích thước $M \times N$. Khu vực nhiệm vụ là tập hợp các số khác 0 đứng liên tiếp nhau trên cùng một hàng. Độ khó của nhiệm vụ là tổng các số trong khu vực đó. Cần đếm số lượng khu vực có độ khó là một số nguyên tố.

Giới hạn chung: $1 \leq M, N \leq 10^5$; $0 \leq a_{i,j} \leq 10^8$.

—

7.3.1 Subtask 1: $M \leq 50, N \leq 50, a_{i,j} \leq 10^2$

Phân tích: Tổng độ khó lớn nhất có thể đạt được trên một hàng là $50 \times 100 = 5000$. Với giá trị này, kiểu dữ liệu int là đủ để lưu trữ. Việc kiểm tra số nguyên tố có thể thực hiện bằng vòng lặp cơ bản duyệt từ 2 đến căn bậc hai của tổng. Thuật toán xử lý bằng cách đọc trực tiếp từng phần tử, nếu gặp số khác 0 thì cộng dồn, gặp số 0 thì kiểm tra và reset tổng.

Code mẫu Subtask 1:

```

#include<bits/stdc++.h>
#define int long long
using namespace std;

int m, n, ans, sum, x;

```

```

int nt(int n) {
    if (n < 2) return 0;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) return 0;
    }
    return 1;
}

main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    cin >> m >> n;

    for (int i = 1; i <= m; i++) {
        sum = 0;

        for (int j = 1; j <= n; j++) {
            cin >> x;

            if (x) sum += x;
            else {
                if (sum > 0 && nt(sum)) ans++;
                sum = 0;
            }
        }

        if (sum > 0 && nt(sum)) ans++;
    }

    cout << ans;
}

```

7.3.2 Subtask 2: $M \leq 5, N \leq 100, a_{i,j} \leq 10^8$

Phân tích: Tổng độ khó tối đa là $100 \times 10^8 = 10^{10}$. Giá trị này vượt quá khả năng lưu trữ của int 32-bit, bắt buộc phải sử dụng long long cho biến tính tổng. Đồng thời, hàm kiểm tra số nguyên tố cần được tối ưu hóa bằng cách xử lý riêng các bội của 2, 3 và tăng bước nhảy lên 6 để giảm số lần lặp, đảm bảo chương trình chạy nhanh với các số lên tới 10^{10} .

Code mẫu Subtask 2:

```

#include<bits/stdc++.h>
#define int long long
using namespace std;

int m, n, ans, sum, x;

```

```

int nt(int n) {
    if (n < 2) return 0;
    if (n == 2 || n == 3) return 1;
    if (n % 2 == 0 || n % 3 == 0) return 0;

    for (int i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) return 0;
    }

    return 1;
}

main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    cin >> m >> n;

    for (int i = 1; i <= m; i++) {
        sum = 0;

        for (int j = 1; j <= n; j++) {
            cin >> x;

            if (x) sum += x;
            else {
                if (sum > 0 && nt(sum)) ans++;
                sum = 0;
            }
        }

        if (sum > 0 && nt(sum)) ans++;
    }

    cout << ans;
}

```

7.3.3 Subtask 3: $M \leq 10^3, N \leq 500, a_{i,j} \leq 10^4$

Phân tích: Tổng độ khó tối đa là $500 \times 10^4 = 5 \cdot 10^6$. Số lượng khu vực nhiệm vụ cần kiểm tra có thể rất lớn do kích thước ma trận. Thay vì tính lại từ đầu mỗi lần, ta dùng thuật toán Sàng nguyên tố (Sieve of Eratosthenes) chuẩn bị sẵn mảng `is_prime` tới $5 \cdot 10^6$. Sau đó, mỗi thao tác kiểm tra chỉ tốn $O(1)$.

Code mẫu Subtask 3:

```

#include<bits/stdc++.h>
#define int long long
using namespace std;

```



```

const int N = 5000005;

int m, n, ans, sum, x;
int nt[N];

void sang() {
    for (int i = 0; i < N; i++) nt[i] = 1;
    nt[0] = nt[1] = 0;

    for (int i = 2; i * i < N; i++) {
        if (nt[i]) {
            for (int j = i * i; j < N; j += i) nt[j] = 0;
        }
    }
}

main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    sang();

    cin >> m >> n;

    for (int i = 1; i <= m; i++) {
        sum = 0;

        for (int j = 1; j <= n; j++) {
            cin >> x;

            if (x) sum += x;
            else {
                if (sum > 0 && sum < N && nt[sum]) ans++;
                sum = 0;
            }
        }

        if (sum > 0 && sum < N && nt[sum]) ans++;
    }

    cout << ans;
}

```

Phân tích tổng hợp: Để vượt qua toàn bộ giới hạn lớn nhất ($10^5 \times 10^5$), ta không thể dùng mảng tĩnh hay vector lớn (tránh Memory Limit Exceeded). Thuật toán đọc đến đâu cộng dồn đến đó kết hợp hàm kiểm tra số nguyên tố $O(\sqrt{N})$ tối ưu (như Subtask 2) chính là lời giải tổng quát, xử lý mượt mà cả tốc độ và bộ nhớ.

7.4 Bài 4: Trò chơi xóa số

Tóm tắt đề bài: Cho số tự nhiên N có độ dài $|N| \leq 10^5$. Một thao tác biến đổi là xóa đi một đoạn chữ số liên tiếp của N (nhưng không được xóa toàn bộ chữ số của N), hoặc giữ nguyên N . Yêu cầu đếm số lượng cách biến đổi sao cho số mới tạo thành chia hết cho 3. Các vị trí xóa khác nhau được tính là các cách khác nhau.

Phân tích bài toán:

- **Dấu hiệu chia hết cho 3:** Một số chia hết cho 3 khi và chỉ khi tổng các chữ số của nó chia hết cho 3. Do đó, việc giữ nguyên thứ tự các phần còn lại khi xóa không làm ảnh hưởng đến tính chất chia hết cho 3. Ta chỉ cần quan tâm đến tổng các chữ số.
- **Điều kiện xóa:** Giả sử tổng tất cả các chữ số của N là S_{total} . Nếu ta xóa đi một đoạn từ vị trí L đến R , tổng các chữ số bị xóa là $S_{deleted}$. Số mới chia hết cho 3 khi và chỉ khi:

$$(S_{total} - S_{deleted}) \equiv 0 \pmod{3} \implies S_{deleted} \equiv S_{total} \pmod{3}$$

- **Mảng cộng dồn (Prefix Sum):** Gọi $P[i]$ là tổng các chữ số từ vị trí 1 đến i lấy dư cho 3. Tổng đoạn xóa từ L đến R là:

$$S_{deleted} = (P[R] - P[L - 1]) \pmod{3}$$

Để thỏa mãn điều kiện, ta cần:

$$P[R] - P[L - 1] \equiv S_{total} \pmod{3} \implies P[L - 1] \equiv P[R] - S_{total} \pmod{3}$$

- **Trường hợp đặc biệt:**
 - Không xóa gì cả: Thêm 1 cách nếu $S_{total} \equiv 0 \pmod{3}$.
 - Không được xóa toàn bộ số N : Điều này tương đương với $L = 1$ và $R = |N|$, tức là $L - 1 = 0$ và $R = |N|$. Ta cần phải loại trừ trường hợp này nếu nó vô tình thỏa mãn điều kiện đếm.

7.4.1 Subtask 1 & 2: $|N| \leq 10^4$

Thuật toán: Tạo mảng cộng dồn $P[i]$ lưu tổng các chữ số mod 3. Dùng 2 vòng lặp lồng nhau cho L và R (với $1 \leq L \leq R \leq |N|$). Với mỗi cặp (L, R) , kiểm tra xem $(P[R] - P[L - 1]) \pmod{3} == S_{total}$ hay không. Nhó bỏ qua cặp $L = 1$ và $R = |N|$. Độ phức tạp $O(|N|^2)$ sẽ dư sức vượt qua Subtask 1 (300) và có thể lọt qua Subtask 2 (10^4) nếu thời gian cho phép.

7.4.2 Subtask 3: $|N| \leq 10^5$

Thuật toán tối ưu $O(|N|)$: Thay vì duyệt L , ta duyệt R từ 1 đến $|N|$. Tại mỗi R , ta cần biết có bao nhiêu giá trị $L - 1 < R$ thỏa mãn $P[L - 1] \equiv (P[R] - S_{total}) \pmod{3}$. Ta có thể dùng một mảng `count[3]` để đếm tần số xuất hiện của các giá trị $P[0 \dots R - 1]$. Tại mỗi bước R , target cần tìm là $(P[R] - S_{total} + 3) \pmod{3}$. Số lượng L thỏa mãn chính là `count[target]`.

Sau khi đếm, ta loại bỏ trường hợp xóa toàn bộ số ($R = |N|$ và $target == 0$ vì $P[0]$ luôn bằng 0). Cuối cùng cộng kết quả với 1 nếu $S_{total} == 0$ (ứng với trường hợp không xóa gì).

Code Full (Đạt 100% Test):

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

const int MAXN = 1e5 + 7;
string s;
int n, ans, tong;
int p[MAXN], cnt[3];

main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    cin >> s;
    n = s.size();

    for (int i = 0; i < n; i++) {
        p[i + 1] = (p[i] + s[i] - '0') % 3;
    }

    tong = p[n];
    ans = (tong == 0);

    cnt[0] = 1;

    for (int i = 1; i <= n; i++) {
        int need = (p[i] - tong + 3) % 3;
        int add = cnt[need];

        if (i == n && need == 0) add--;

        ans += add;
        cnt[p[i]]++;
    }

    cout << ans;

    return 0;
}
```

}

8 Bạc Liêu

8.1 Bài 1: CHỈ SỐ THÀNH CÔNG

Tóm tắt đề bài: Cho ba số nguyên a, b, c lần lượt là chỉ số Nỗ lực, Kiên trì và Niềm tin. Công thức tính chỉ số Thành công là:

$$\text{Thành công} = (a + b) \times \sqrt{c}$$

Yêu cầu in ra phần nguyên của kết quả thu được. Giới hạn: $1 \leq a, b, c \leq 1000$.

Phân tích thuật toán:

- Bài toán yêu cầu tính toán có liên quan đến căn bậc hai ($\sqrt{c}$), kết quả này thường là một số thực. Do đó, ta cần sử dụng kiểu dữ liệu số thực (double) để tính toán nhằm đảm bảo độ chính xác.
- Hàm `sqrt(c)` trong thư viện `cmath` sẽ giúp tính căn bậc hai của một số.
- Để lấy phần nguyên của một số thực, trong C++ ta có thể thực hiện ép kiểu trực tiếp về kiểu số nguyên (`long long`). Phép ép kiểu này sẽ tự động loại bỏ phần thập phân (lấy phần nguyên dưới).
- Với $a, b, c \leq 1000$, giá trị lớn nhất của Thành công là $(1000 + 1000) \times \sqrt{1000} \approx 63245$. Giá trị này nằm gọn trong kiểu dữ liệu `int`, tuy nhiên để đồng nhất với các bài tập trước, ta vẫn sử dụng `long long`.

8.1.1 Code mẫu

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int a, b, c;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> a >> b >> c;

    double success = (double)(a + b) * sqrt(c);
    int result = (int)success;
```

```
cout << result;
}
```

8.2 Bài 2: NGUYÊN VỌNG TỐI ƯU

Tóm tắt đề bài: Cho hai số nguyên a và b lần lượt đại diện cho điểm xét tuyển của lớp chuyên Toán và lớp chuyên Tin. Yêu cầu tìm và in ra giá trị nhỏ hơn trong hai số đó để xác định nguyện vọng ưu tiên. Nếu hai giá trị bằng nhau, in ra chính giá trị đó. Giới hạn: $20 \leq a, b \leq 100$.

Phân tích thuật toán:

- Đây là bài toán so sánh hai số cơ bản.
- Ta có thể sử dụng cấu trúc điều kiện if-else để so sánh: nếu $a < b$ thì kết quả là a , ngược lại kết quả là b .
- Cách nhanh hơn là sử dụng hàm có sẵn $\min(a, b)$ trong thư viện algorithm của C++.
- Với giới hạn điểm từ 20 đến 100, kiểu dữ liệu `int` là hoàn toàn đủ để lưu trữ và xử lý.

8.2.1 Code mẫu (Sử dụng hàm tối ưu)

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int a, b, ans;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> a >> b;

    ans = min(a, b);

    cout << ans;
}
```

8.3 Bài 3: SỐ CHÍNH PHƯƠNG

Tóm tắt đề bài: Cho dãy gồm n số tự nhiên $a_1, a_2, \dots, a_n$. Hãy đếm xem trong dãy có bao nhiêu số là số chính phương. Một số a là số chính phương nếu tồn tại số tự nhiên k sao cho $a = k^2$. Giới hạn: $n \leq 10^5, a_i \leq 10^9$.

Phân tích thuật toán:

- Để kiểm tra một số a_i có phải là số chính phương hay không, phương pháp phổ biến nhất là lấy căn bậc hai của nó: $k = \sqrt{a_i}$.
- Nếu a_i là số chính phương, thì k phải là một số nguyên. Sau khi tính $k = \text{round}(\sqrt{a_i})$, ta kiểm tra lại bằng điều kiện $k \times k = a_i$.
- **Lưu ý về số thực:** Khi sử dụng hàm `sqrt()` trong C++, kết quả trả về là số thực (double). Do sai số dấu phẩy động, ta nên ép kiểu hoặc làm tròn về số nguyên rồi mới nhân ngược lại để kiểm tra.
- **Độ phức tạp:** Việc kiểm tra mỗi số tốn $O(1)$, tổng độ phức tạp cho cả dãy là $O(n)$. Với $n = 10^5$, thuật toán này hoàn toàn đáp ứng được giới hạn thời gian.
- **Kiểu dữ liệu:** Với $a_i \leq 10^9$, ta có thể sử dụng kiểu `int`, tuy nhiên để an toàn khi thực hiện phép nhân $k \times k$, ta nên sử dụng `long long`.

8.3.1 Code mẫu (Sử dụng hàm căn bậc hai)

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int n, a, cnt;

bool check(int x) {
    if (x < 0) return false;
    int k = sqrt(x);
    return k * k == x;
}

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n;
    cnt = 0;
    for (int i = 0; i < n; i++) {
        cin >> a;
        if (check(a)) cnt++;
    }
    cout << cnt;
}
```

Giải thích Sample 1:

- $49 = 7^2$ (Số chính phương)
- 6 (Không phải)
- $9 = 3^2$ (Số chính phương)
- 5 (Không phải)
- 2 (Không phải)
- **Kết quả:** 2 số.

8.4 Bài 4: ƯỚC CHUNG LỚN NHẤT CỦA DÃY SỐ

Tóm tắt đề bài: Cho dãy gồm n số nguyên $a_1, a_2, \dots, a_n$. Tìm ước chung lớn nhất (UCLN) của toàn bộ dãy số này. Lưu ý dãy có thể chứa số âm. Giới hạn: $n \leq 10^5, |a_i| \leq 10^6$.

Phân tích thuật toán:

- **Định nghĩa UCLN với số âm:** Theo quy ước toán học, UCLN của một tập hợp số luôn là một số nguyên dương. Do đó, trước khi tính toán, ta nên lấy giá trị tuyệt đối của các phần tử trong dãy: $\gcd(a, b) = \gcd(|a|, |b|)$.
- **Tính chất kết hợp:** UCLN của một dãy số có tính chất kết hợp:

$$\gcd(a_1, a_2, a_3, \dots, a_n) = \gcd(a_1, \gcd(a_2, \gcd(a_3, \dots, a_n)))$$

Điều này cho phép ta tính UCLN của cả dãy bằng cách duyệt từ trái sang phải và cập nhật giá trị UCLN hiện tại.

- **Thuật toán Euclid:** Để tính $\gcd(a, b)$ hiệu quả, ta sử dụng thuật toán Euclid với độ phức tạp $O(\log(\min(a, b)))$.
- **Độ phức tạp tổng quát:** $O(n + \log(\max |a_i|))$, hoàn toàn đáp ứng tốt giới hạn $n = 10^5$.

8.4.1 Code mẫu (Sử dụng hàm `__gcd` có sẵn)

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int n, a[100005], res;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n;
    for (int i = 0; i < n; i++) {
```

```

        cin >> a[i];
        a[i] = abs(a[i]);
    }

    res = a[0];
    for (int i = 1; i < n; i++) {
        res = __gcd(res, a[i]);
        if (res == 1) break;
    }

    cout << res;
}

```

Giải thích Sample:

- Dãy ban đầu: {12, -24, 36, -60, 48}.
- Lấy giá trị tuyệt đối: {12, 24, 36, 60, 48}.
- $\gcd(12, 24) = 12$.
- $\gcd(12, 36) = 12$.
- $\gcd(12, 60) = 12$.
- $\gcd(12, 48) = 12$.
- **Kết quả:** 12.

9 Bền Tre

9.1 Bài 1: Hình vuông

9.1.1 Lời giải

Vì hình vuông song song với hai trục tọa độ, do đó tất cả các tọa độ (x, y) của 4 đỉnh đều xuất hiện chính xác hai lần. Đáp án góc còn thiếu sẽ là tọa độ x, y xuất hiện duy nhất một lần trong 3 đỉnh đã cho.

9.1.2 Code mẫu

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    vector<int> x(3), y(3);
    for (int i = 0; i < 3; i++) cin >> x[i] >> y[i];
}

```



```

int xr, yr;

if (x[0] == x[1]) xr = x[2];
else if (x[0] == x[2]) xr = x[1];
else xr = x[0];

if (y[0] == y[1]) yr = y[2];
else if (y[0] == y[2]) yr = y[1];
else yr = y[0];

cout << xr << " " << yr;
}

```

9.2 Bài 2: Tổng lớn nhất

9.2.1 Lời giải

Ta sẽ cố định j , việc còn lại chỉ là tìm i, k sao cho tổng lớn nhất. Để tổng lớn nhất, ta sẽ chọn A_i, A_k lớn nhất. Gọi $p_i = \max(A_j) \forall j = 1 \rightarrow i$ và $s_i = \max(A_j) \forall j = i \rightarrow n$. Đáp án là $\max(3 \times p_{j-1} + 2 \times A_j + 5 \times s_{j+1})$

9.2.2 Code mẫu

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    int n; cin >> n;
    vector<int> a(n);
    for (int &x : a) cin >> x;

    vector<int> maxL(n), maxR(n);

    maxL[0] = a[0];
    for (int i = 1; i < n; i++)
        maxL[i] = max(maxL[i-1], a[i]);

    maxR[n-1] = a[n-1];
    for (int i = n-2; i >= 0; i--)
        maxR[i] = max(maxR[i+1], a[i]);

    long long res = 0;

    for (int j = 1; j < n-1; j++) {
        res = max(res, 3LL * maxL[j-1] + 2LL * a[j] + 5LL * maxR[j+1]);
    }
}

```

```
cout << res;
}
```

9.3 Bài 3: Chia giấy

9.3.1 Lời giải

Ta sẽ cắt hình vuông theo độ dài cạnh lớn nhất có thể, sau đó giảm dần xuống. Độ dài cạnh lớn nhất có thể để diện tích không vượt quá n là $k = \sqrt{n}$.

9.3.2 Code mẫu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n; cin >> n;
    vector<int> res;

    while (n > 0) {
        int k = sqrt(n);
        res.push_back(k);
        n -= k * k;
        cout << k << ' ';
    }
}
```

9.4 Bài 4: Xâu đối xứng

9.4.1 Lời giải

Gọi $f(l, r)$ là số kí tự ít nhất cần chèn để xâu $s[l...r]$ là xâu đối xứng. Ta có công thức quy hoạch động như sau (các kí tự đánh số từ 1):

- $f(i, i) = 0$
- $f(i, i + 1) = 0$ nếu $s_i = s_{i+1}$, ngược lại thì $f(i, i + 1) = 1$.
- $f(i, j) = \min(f(i, j - 1), f(i, j + 1)) + 1$ nếu $s_i \neq s_j$ (vì ta phải chèn một kí tự vào đầu i hoặc đầu j).

9.4.2 Code mẫu

```
#include <bits/stdc++.h>
using namespace std;
```

```

int main() {
    freopen("palin.inp", "r", stdin);
    freopen("palin.out", "w", stdout);
    string s;
    cin >> s;
    int n = (int)s.size();
    vector<vector<int>> f(n + 5, vector<int>(n + 5, 10000));
    for(int i = 1; i <= n; i++) f[i][i] = 0;
    for(int i = 1; i < n; i++) f[i][i + 1] = (s[i] != s[i + 1]);
    for(int l = n - 2; l > 0; l--) for(int r = l + 2; r <= n; r++)
        if (s[l + 1] == s[r + 1]) f[l][r] = f[l + 1][r + 1];
        else f[l][r] = min(f[l + 1][r], f[l][r + 1]) + 1;
    cout << f[1][n] + n;
    return 0;
}

```

10 Bình Dương

10.1 Bài 1: Số lượng nguyên tố

10.1.1 Lời giải:

Thử từng độ dài từ 1 đến độ dài của N . Với mỗi độ dài, sử dụng *next_permutation* để tạo ra các hoán vị, mỗi hoán vị chỉ lấy i ký tự đầu tiên để tạo thành một số, rồi kiểm tra xem số vừa tạo thành có là một số nguyên tố hay không. Vì có thể có nhiều số thỏa mãn trùng nhau, nên phải đẩy các số này vào một mảng và xóa trùng.

10.1.2 Code mẫu:

```

#include <bits/stdc++.h>
using namespace std;

bool isPrime(int n) {
    if (n == 2 || n == 3) return true;
    if (n < 2 || n % 2 == 0 || n % 3 == 0) return false;
    for (int i = 5; i <= (int)sqrt(n); i += 6)
        if (n % i == 0 || n % (i + 2) == 0) return false;
    return true;
}

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    string n;
    cin >> n;
    int len = n.size();

```

```

vector<int>res;
for (int i = 1; i <= len; i++) {
    sort(n.begin(), n.end());
    do {
        string tmp = n.substr(0, i);
        int num = 0;
        for (int j = 0; j < (int)tmp.size(); j++)
            num = num * 10 + tmp[j] - '0';
        if (isPrime(num)) res.push_back(num);
    } while (next_permutation(n.begin(), n.end()));
}
sort(res.begin(), res.end());
res.erase(unique(res.begin(), res.end()), res.end());
cout << res.size() << "\n";
}

```

10.2 Bài 2: Mật khẩu

10.2.1 Lời giải:

Dựa vào bảng mã ASCII:

- Ký tự chữ số '0' đến '9' có mã ASCII từ 48 đến 57.
- Ký tự chữ cái in hoa 'A' đến 'Z' có mã ASCII từ 65 đến 90.
- Ký tự chữ cái in thường 'a' đến 'z' có mã ASCII từ 97 đến 122.

Ta thấy, theo thứ tự từ điển: ký tự chữ số < ký tự chữ cái in hoa < ký tự chữ cái in thường.
Để tạo ra được mật khẩu có thứ tự từ điển nhỏ nhất:

- Ưu tiên ký tự chữ số, rồi đến chữ cái in hoa, và cuối cùng là ký tự in thường.
- Hai ký tự liên tiếp phải khác nhau, nên ở phần ký tự chữ số, ta xen kẽ '0' và '1'; phần chữ cái in hoa thì xen kẽ 'A' và 'B'; phần chữ cái in thường thì xen kẽ 'a' và 'b'.
- Nếu $a + b + c < n$ thì tăng thêm số lượng ký tự chữ số phải là $n - a - b$.

Để tạo ra được mật khẩu có thứ tự từ điển lớn nhất:

- Ưu tiên chữ cái in thường, rồi đến chữ cái in hoa, và cuối cùng là ký tự chữ số.
- Hai ký tự liên tiếp phải khác nhau, nên ở phần chữ cái in thường thì xen kẽ 'z' và 'y'; phần chữ cái in hoa thì xen kẽ 'Z' và 'Y'; phần ký tự chữ số thì xen kẽ '9' và '8'.
- Nếu $a + b + c < n$ thì tăng thêm số lượng ký tự chữ in thường phải là $n - a - c$.

10.2.2 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    int n,a,b,c;
    cin >> n >> a >> b >> c;
    int nw_c = c;
    if (a + b + c < n) nw_c = n - a - b;
    for (int i = 1; i <= nw_c; i++)
        if (i % 2 != 0) cout << '0';
        else cout << '1';
    for (int i = 1; i <= a; i++)
        if (i % 2 != 0) cout << 'A';
        else cout << 'B';
    for (int i = 1; i <= b; i++)
        if (i % 2 != 0) cout << 'a';
        else cout << 'b';
    cout << "\n";
    int nw_b = b;
    if (a + b + c < n) nw_b = n - a - c;
    for (int i = 1; i <= nw_b; i++)
        if (i % 2 != 0) cout << 'z';
        else cout << 'y';
    for (int i = 1; i <= a; i++)
        if (i % 2 != 0) cout << 'Z';
        else cout << 'Y';
    for (int i = 1; i <= c; i++)
        if (i % 2 != 0) cout << '9';
        else cout << '8';
    cout << "\n";
}
```

10.3 Bài 3: Trò chơi ghép gỗ

10.3.1 Lời giải:

Đây là một bài toán tham lam.

Trước hết, sắp xếp giảm dần các khối gỗ theo chiều dài.

Giả sử chọn i khối gỗ đầu tiên của dãy sau khi đã sắp xếp, ta được một hình vuông có cạnh lớn nhất là $\min(i, a_i)$.

Vậy đáp án của bài toán là $\max(\min(i, a_i))$ với $1 \leq i \leq n$.

10.3.2 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

const int N = 1e3 + 5;
int a[N];

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) cin >> a[i];
    sort(a + 1, a + n + 1, greater<long long>());
    int ans = 0;
    for (int i = 1; i <= n; i++) {
        int x = min(a[i], i);
        ans = max(ans, x);
    }
    cout << ans << "\n";
}
```

10.4 Bài 4: Biến đổi

10.4.1 Lời giải:

Đây là một bài toán tham lam.

Mỗi phép biến đổi chữ số x ở hàng 10^d thành chữ số 9 sẽ đóng góp vào độ chênh lệch giữa tổng sau khi biến đổi và tổng ban đầu một lượng là:

$$\Delta = (9 - x) \times 10^d$$

Để chênh lệch cuối cùng là lớn nhất có thể, ta sẽ ưu tiên chọn d lớn nhất, và x ($x < 9$) nhỏ nhất có thể.

10.4.2 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

vector<int> digit[15];

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    int n, k;
```

```

cin >> n >> k;
long long sum = 0;
for (int i = 1; i <= n; i++) {
    long long x;
    cin >> x;
    sum += x;
    int j = 0;
    while (x) {
        digit[j].push_back(x % 10);
        x /= 10;
        j++;
    }
}
for (int i = 9; i >= 0; i--) {
    if (k == 0) break;
    if (digit[i].empty()) continue;
    sort(digit[i].begin(), digit[i].end());
    for (int j = 0; j < (int)digit[i].size(); j++)
        if (digit[i][j] < 9) {
            digit[i][j] = 9;
            k--;
            if (k == 0) break;
        }
    else break;
}
long long cur = 1, sum2 = 0;
for (int i = 0; i <= 9; i++) {
    for (int j = 0; j < (int)digit[i].size(); j++)
        sum2 += digit[i][j] * cur;
    cur *= 10;
}
cout << sum2 - sum << "\n";
}

```

11 Cần Thơ

11.1 Bài 1: Tổng tiền

11.1.1 Lời giải:

Ta có tổng số tiền của đơn hàng sẽ là $x \times v + y \times t$. Nếu số tiền này lớn hơn s thì in ra $(x-1) \times v + y \times t$, ngược lại, in ra $x \times v + y \times t$.

11.1.2 Code mẫu:

```

#include <bits/stdc++.h>
using namespace std;

```

```

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    long long x,y,v,t,s;
    cin >> x >> y >> v >> t >> s;
    if (x * v + y * t > s) cout << (x - 1) * v + y * t << "\n";
    else cout << x * v + y * t << "\n";
    return 0;
}

```

11.2 Bài 2: Tồn kho

11.2.1 Lời giải:

Tổng số hàng tồn kho sau n ngày chính là $\sum_{i=1}^n a_i - b_i$.

11.2.2 Code mẫu:

```

#include <bits/stdc++.h>
using namespace std;

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    int n;
    cin >> n;
    int sum = 0;
    for (int i = 1; i <= n; i++) {
        int a,b;
        cin >> a >> b;
        sum += a - b;
    }
    cout << sum << "\n";
    return 0;
}

```

11.3 Bài 3: Số đảo nguyên tố

11.3.1 Lời giải:

Với mỗi số trong dãy, tiến hành đảo ngược rồi kiểm tra xem số đảo ngược vừa tìm được có phải số nguyên tố hay không.

11.3.2 Code mẫu:


```

#include <bits/stdc++.h>
using namespace std;

int daonguoc(int n) {
    int ans = 0;
    while (n) {
        ans = ans * 10 + n % 10;
        n /= 10;
    }
    return ans;
}

bool isPrime(int n) {
    if (n == 2 || n == 3) return true;
    if (n < 2 || n % 2 == 0 || n % 3 == 0) return false;
    for (int i = 5; i <= (int)sqrt(n); i += 6)
        if (n % i == 0 || n % (i + 2) == 0) return false;
    return true;
}

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    int n;
    cin >> n;
    int ans = 0;
    for (int i = 1; i <= n; i++) {
        int x;
        cin >> x;
        x = daonguoc(x);
        if (isPrime(x)) ans++;
    }
    cout << ans << "\n";
    return 0;
}

```

11.4 Bài 4: Tìm số

11.4.1 Lời giải:

Trước tiên, sắp xếp mảng a theo thứ tự tăng dần.

Ta có: $a_i \bmod d = a_j \bmod d$ ($i < j$) $\Leftrightarrow (a_j - a_i) \bmod d = 0$.

Suy ra d thỏa mãn yêu cầu đề bài khi và chỉ khi d là ước của $a_i - a_{i-1}$ với mọi $i = 1..n$.

Vậy đáp án của bài toán là ước chung lớn nhất của $a_i - a_{i-1}$ với $i = 1..n$.

Lưu ý: Nếu tất cả các phần tử trong dãy đều bằng nhau, thì mọi d trong đoạn $[1, m]$ đều thỏa mãn, nên đáp án sẽ là m .

11.4.2 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

#define int long long
const int N = 1e5 + 5;
int a[N];

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= n; i++) cin >> a[i];
    sort(a + 1, a + n + 1);
    if (a[n] == a[1]) {
        cout << m << "\n";
        return 0;
    }
    int ans = a[2] - a[1];
    for (int i = 3; i <= n; i++) ans = __gcd(ans, a[i] - a[i - 1]);
    cout << ans << "\n";
    return 0;
}
```

11.5 Bài 5: Di chuyển trên bản đồ

11.5.1 Lời giải:

Gọi x là số bước cần thiết, n là độ dài của chuỗi.

x luôn có thể được tách thành $x = a * n + b$ với $b < n$. Hay nói cách khác, ta thực hiện hoàn chỉnh a lần chuỗi s và thêm b kí tự đầu của chuỗi s sao cho thỏa mãn.

- $a * s[1...n] + s[1...b] = X$
- $a * s[1...n] + s[1...b] = Y$

Ta có thể duyệt b , và giải phương trình để tìm a , sau đó kiểm tra a có thỏa mãn hay không và lấy min $a * n + b$.

11.5.2 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

signed main() {
    int X, Y;
```

```

string s;
cin >> X >> Y >> s;
int n = (int)s.size();
vector<int> x(n + 5, 0), y(n + 5, 0);
for(int i = 1; i <= n; i++) {
    x[i] = x[i - 1];
    y[i] = y[i - 1];
    if (s[i - 1] == 'U') y[i]++;
    if (s[i - 1] == 'D') y[i]--;
    if (s[i - 1] == 'R') x[i]++;
    if (s[i - 1] == 'L') x[i]--;
}

long long ans = 1e18;
for(int b = 0; b < n; b++) {
    long long a = (x[n] == 0 ?
        (y[n] == 0 ? 0 : (Y - y[b]) / y[n]) : (X - x[b]) / x[n]);
    if (a >= 0 && a * x[n] + x[b] == X && a * y[n] + y[b] == Y)
        ans = min(ans, a * n + b);
}
cout << (ans == (long long)1e18 ? -1 : ans);
return 0;
}

```

12 Đà Nẵng

12.1 Bài 1: Lọc tín hiệu

12.1.1 Lời giải:

Nhận xét: Sau khi thực hiện xong bước 1, sẽ không còn 3 tín hiệu liên tiếp nào giống nhau. Xét lần lượt các phần tử của dãy số từ trái sang phải, nếu phần hiện tại đang xét và hai phần tử liền trước bằng nhau thì sẽ đẩy ra khỏi mảng. Ngược lại, đẩy vào cuối mảng.

Sau khi duyệt hết dãy số và thực hiện thao tác như trên, ta được dãy số cuối cùng đảm bảo không có 3 tín hiệu liên tiếp nào giống nhau.

12.1.2 Code mẫu:

```

#include <bits/stdc++.h>
using namespace std;

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    int n;
    cin >> n;

```

```

vector<int>a;
for (int i = 1; i <= n; i++) {
    int x;
    cin >> x;
    if (a.size() >= 2 && a[a.size() - 2] == x && a.back() == x) {
        a.pop_back();
        a.pop_back();
    }
    else a.push_back(x);
}
for (int i = 0; i < (int)a.size(); i++) cout << a[i] << " ";
return 0;
}

```

12.2 Bài 2: Mã hóa

12.2.1 Lời giải:

Đặt N là độ dài chuỗi X . Giả sử các ký tự trong chuỗi được đánh số từ 0 đến $N - 1$.

- **Trường hợp 1:** Nếu N là số lẻ, Chuỗi kết quả sẽ có dạng: các ký tự ở vị trí chẵn theo thứ tự từ phải sang trái rồi đến các ký tự ở vị trí lẻ theo thứ tự từ trái sang phải. Ví dụ: $X = "abcde" \rightarrow Y = "ecabd"$.
- **Trường hợp 2:** Nếu N là số chẵn, Chuỗi kết quả sẽ có dạng: các ký tự ở vị trí lẻ theo thứ tự từ phải sang trái rồi đến các ký tự ở vị trí chẵn theo thứ tự từ trái sang phải. Ví dụ: $X = "abcd" \rightarrow Y = "dbac"$.

12.2.2 Code mẫu:

```

#include <bits/stdc++.h>
using namespace std;

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    string s;
    cin >> s;
    int n = s.size();
    if (n % 2 == 0) {
        for (int i = n - 1; i >= 1; i -= 2) cout << s[i];
        for (int i = 0; i < n; i += 2) cout << s[i];
    }
    else {
        for (int i = n - 1; i >= 0; i -= 2) cout << s[i];
        for (int i = 1; i < n; i += 2) cout << s[i];
    }
}

```

```

    return 0;
}

```

12.3 Bài 3: Năng lượng

12.3.1 Lời giải:

Xét x, y, z bất kì, ta có:

$$Q = xy - y^2 - xz + yz = y(x - y) - z(x - y) = (x - y)(y - z)$$

Với mỗi i từ 2 đến $n - 1$, giả sử $y = a_i$, ta cần tìm $x = a_j$ và $z = a_k$ sao cho $1 \leq j < i < k \leq n$ và $(x - y)(y - z)$ lớn nhất có thể.

Để biểu thức $(x - y)(y - z)$ lớn nhất có thể thì $x - y$ và $y - z$ phải lớn nhất có thể. Vậy cặp (x, z) tối ưu nhất với $y = a_i$ chính là $(\min_{1 \leq j \leq i-1} a_j, \max_{i+1 \leq k \leq n} a_k)$ hoặc $(\max_{1 \leq j \leq i-1} a_j, \min_{i+1 \leq k \leq n} a_k)$.

12.3.2 Code mẫu:

```

#include <bits/stdc++.h>
using namespace std;

#define int long long
const int N = 1e6 + 5;
const int inf = 1e18;
int a[N], preMax[N], preMin[N], sufMax[N], sufMin[N];

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) cin >> a[i];
    preMax[1] = preMin[1] = a[1];
    for (int i = 2; i <= n; i++) {
        preMax[i] = max(preMax[i - 1], a[i]);
        preMin[i] = min(preMin[i - 1], a[i]);
    }
    sufMin[n] = sufMax[n] = a[n];
    for (int i = n - 1; i >= 1; i--) {
        sufMin[i] = min(sufMin[i + 1], a[i]);
        sufMax[i] = max(sufMax[i + 1], a[i]);
    }
    int ans = -inf;
    for (int i = 2; i <= n - 1; i++) {
        ans = max(ans, (preMax[i - 1] - a[i]) * (a[i] - sufMin[i + 1]));
        ans = max(ans, (preMin[i - 1] - a[i]) * (a[i] - sufMax[i + 1]));
    }
    cout << ans << "\n";
    return 0;
}

```

}

12.4 Bài 4: Lựa chọn cấu hình

12.4.1 Lời giải:

Chia thành hai nhóm:

- Nhóm 1 gồm các cặp nhà cung cấp của bộ phận 1 và bộ phận 2.
- Nhóm 2 gồm các cặp nhà cung cấp của bộ phận 3 và bộ phận 4.

Sắp xếp hai nhóm tăng dần theo chi phí.

Duyệt từng cặp trong nhóm 1. Giả sử ta chọn cặp đó làm nhà cung cấp cho bộ phận 1 và bộ phận 2, cần tìm cặp cung cấp bộ phận 3 và 4 tối ưu nhất. Sử dụng kỹ thuật hai con trỏ kết hợp với prefix max để tìm ra cặp tối ưu đó.

12.4.2 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

#define int long long
const int N = 1e3 + 5;
int c[5][N], v[5][N], preMax[N * N];

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= 4; i++)
        for (int j = 1; j <= n; j++) cin >> c[i][j] >> v[i][j];
    vector<pair<int, int>> A, B;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            A.push_back({c[1][i] + c[2][j], v[1][i] + v[2][j]});
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            B.push_back({c[3][i] + c[4][j], v[3][i] + v[4][j]});
    sort(A.begin(), A.end());
    sort(B.begin(), B.end());
    preMax[0] = B[0].second;
    for (int i = 1; i < (int)B.size(); i++)
        preMax[i] = max(preMax[i - 1], B[i].second);
    int j = (int)B.size() - 1, ans = 0;
    for (int i = 0; i < (int)A.size(); i++) {
        while (j >= 0 && B[j].first > m - A[i].first) j--;
```

```

        if (j >= 0) ans = max(ans, A[i].second + preMax[j]);
    }
    cout << ans << "\n";
    return 0;
}

```

13 Đắc Lắc

13.1 Bài 1: ĐẾM SỐ PHƯƠNG TRÌNH CÓ NGHIỆM

Tóm tắt đề bài: Cho N phương trình bậc hai dạng $ax^2 + bx + c = 0$ ($a \neq 0$). Yêu cầu đếm xem trong N phương trình đó có bao nhiêu phương trình có nghiệm (bao gồm cả nghiệm phân biệt và nghiệm kép). Giới hạn: $N \leq 100$; a, b, c là các số nguyên trong khoảng $[-10^6, 10^6]$.

Phân tích thuật toán: Để xác định một phương trình bậc hai $ax^2 + bx + c = 0$ ($a \neq 0$) có nghiệm hay không, ta dựa vào giá trị của biệt thức Delta (Δ):

- Công thức tính Delta: $\Delta = b^2 - 4ac$.
- Điều kiện để phương trình có nghiệm: $\Delta \geq 0$.
- Nếu $\Delta < 0$, phương trình vô nghiệm trên tập số thực.

Lưu ý về kiểu dữ liệu: Các hệ số a, b, c có thể lên đến 10^6 . Khi tính b^2 hoặc $4ac$, giá trị có thể đạt tới 10^{12} . Do đó, biến lưu giá trị Delta phải sử dụng kiểu dữ liệu số nguyên 64-bit (long long) để tránh hiện tượng tràn số.

13.1.1 Code mẫu (Xử lý toàn bộ dữ liệu)

```

#include <bits/stdc++.h>
#define int long long
using namespace std;

int n, a, b, c, delta, cnt;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n;
    cnt = 0;

    for (int i = 0; i < n; i++) {
        cin >> a >> b >> c;
        delta = b * b - 4 * a * c;
        if (delta >= 0) {

```

```

        cnt++;
    }
}

cout << cnt;
}

```

13.2 Bài 2: ALIBABA VÀ CÁC TÚI VÀNG

Tóm tắt đề bài: Cho N túi vàng, túi thứ i có a_i đồng tiền. Alibaba cần chọn các túi vàng sao cho tổng số tiền là lớn nhất và phải là một số chẵn. Giới hạn: $N \leq 10^5, a_i \leq 10^9$.

Phân tích thuật toán: Gọi S là tổng của tất cả N túi vàng trong hàng.

- **Trường hợp 1: S là số chẵn.** Đây là giá trị lớn nhất có thể đạt được. Alibaba chỉ cần lấy toàn bộ các túi vàng. Kết quả là S . (Trường hợp này chiếm 90% số điểm theo mô tả Subtask).
- **Trường hợp 2: S là số lẻ.** Để biến một tổng từ lẻ thành chẵn mà vẫn giữ được giá trị lớn nhất, ta phải loại bỏ đi một túi vàng có số lượng đồng tiền là **số lẻ nhỏ nhất** trong tất cả các túi vàng.
 - Nếu trong hàng có ít nhất một túi vàng lẻ, kết quả sẽ là $S - \min(a_{l_i})$.
 - Nếu trong hàng không có túi vàng lẻ nào (điều này mâu thuẫn với việc S lẻ), nhưng theo logic toán học, nếu không có túi lẻ thì S không bao giờ lẻ.

13.2.1 Subtask 1: $a_1 + a_2 + \dots + a_n$ là số chẵn (90% số điểm)

Phân tích: Ta chỉ cần tính tổng của tất cả các phần tử trong mảng. Vì mỗi phần tử lên đến 10^9 và $N = 10^5$, tổng có thể đạt 10^{14} , nên sử dụng `long long`.

```

#include <bits/stdc++.h>
#define int long long
using namespace std;

int n, sum;
main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n;
    for (int i = 1; i <= n; i++) {
        int a;
        cin >> a;
        sum += a;
    }
}

```



```

    cout << sum << "\n";
    return 0;
}

```

13.2.2 Subtask 2: Không có ràng buộc gì thêm (100% số điểm)

Phân tích: Ta tính tổng toàn bộ S và đồng thời tìm số lẻ nhỏ nhất (min_odd). Nếu S lẻ, ta lấy $S - \text{min_odd}$. Nếu S chẵn, ta lấy S .

```

#include <bits/stdc++.h>
#define int long long
using namespace std;

int n, a, s, mn_odd;
bool ok;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n;
    s = 0;
    mn_odd = 2e18;
    ok = false;

    for (int i = 0; i < n; i++) {
        cin >> a;
        s += a;
        if (a % 2 != 0) {
            mn_odd = min(mn_odd, a);
            ok = true;
        }
    }

    if (s % 2 == 0) cout << s;
    else {
        if (ok) cout << s - mn_odd;
        else cout << 0;
    }
}

```

13.3 Bài 3: XÓA TẬP TIN

Tóm tắt đề bài: Hệ thống có N tập tin với tên gọi S_i . Thực hiện Q thao tác xóa, mỗi thao tác cho một ký tự c . Thao tác này sẽ xóa toàn bộ các tập tin hiện còn trong hệ thống mà

trong tên có chứa ít nhất một ký tự c . Yêu cầu in ra số lượng tập tin còn lại sau mỗi thao tác. Giới hạn: $N, Q \leq 10^5$; Độ dài tên tập tin ≤ 8 .

13.3.1 Subtask 1: $N, Q \leq 100$

Phân tích: Với giới hạn nhỏ, ta có thể mô phỏng trực tiếp quá trình xóa. Sử dụng một mảng `bool` để đánh dấu tập tin nào đã bị xóa. Với mỗi thao tác xóa ký tự c , ta duyệt qua toàn bộ N tập tin, nếu tập tin chưa bị xóa và chứa ký tự c thì đánh dấu đã xóa. Sau đó đếm số lượng tập tin chưa bị xóa để in ra.

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int n, q, rem;
string s[200005];
bool del[200005];
char c;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n >> q;
    for (int i = 0; i < n; i++) cin >> s[i];

    rem = n;
    while (q--) {
        cin >> c;
        for (int i = 0; i < n; i++) {
            if (!del[i]) {
                if (s[i].find(c) != string::npos) {
                    del[i] = true;
                    rem--;
                }
            }
        }
        cout << rem << "\n";
    }
}
```

13.3.2 Subtask 2: $N, Q \leq 10^5$ (Full Test)

Phân tích thuật toán tối ưu: Độ dài mỗi tên tập tin rất ngắn (tối đa 8). Số lượng ký tự Latin in thường chỉ có 26. Ta có thể tối ưu bằng cách quản lý danh sách tập tin theo các ký

tự mà chúng chứa.

- Sử dụng một mảng các vector `adj[26]`, trong đó `adj[c - 'a']` lưu chỉ số (ID) của các tập tin có chứa ký tự `c`.
- Vì một tập tin có thể chứa nhiều ký tự (ví dụ "shoes" chứa 's', 'h', 'o', 'e'), ID của nó sẽ xuất hiện trong nhiều vector. Ta cần một mảng `is_deleted[N]` để đảm bảo mỗi tập tin chỉ bị xóa một lần.
- Với mỗi thao tác xóa ký tự `x`:
 1. Truy cập vào `adj[x - 'a']`.
 2. Duyệt qua danh sách ID trong đó. Nếu tập tin nào chưa bị xóa, ta xóa nó và giảm biến đếm `remaining`.
 3. Để tránh duyệt lại những tập tin đã xóa ở các thao tác sau của cùng một ký tự `x`, sau khi xử lý xong ta có thể xóa trắng (clear) vector `adj[x - 'a']`.

Độ phức tạp: $O(N \times 8 + Q + \sum |S_i|)$. Do mỗi ID tập tin chỉ được "xóa" một lần duy nhất trong suốt quá trình, tổng số lần thực hiện lệnh giảm `remaining` tối đa là N . Thuật toán chạy cực nhanh với 10^5 .

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int n, q, rem;
bool del[100005], ex[26];
vector<int> adj[26];
string s;
char c;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n >> q;

    for (int i = 0; i < n; i++) {
        cin >> s;
        memset(ex, 0, sizeof(ex));
        for (char ch : s) ex[ch - 'a'] = 1;
        for (int j = 0; j < 26; j++) {
            if (ex[j]) adj[j].push_back(i);
        }
    }

    rem = n;
    while (q--) {
```

```

    cin >> c;
    int idx = c - 'a';
    for (int id : adj[idx]) {
        if (!del[id]) {
            del[id] = 1;
            rem--;
        }
    }
    adj[idx].clear();
    cout << rem << "\n";
}
}

```

13.4 Bài 4: ĐOẠN CON ĐẸP

Tóm tắt đề bài: Cho dãy số nguyên $a_1, a_2, \dots, a_N$. Một đoạn con từ chỉ số L đến R được gọi là "đẹp" nếu cả L và R đều là số nguyên tố. Yêu cầu tìm tổng lớn nhất của một đoạn con đẹp như vậy. Giới hạn: $N \leq 10^6, |a_i| \leq 10^6$.

Phân tích thuật toán:

- **Điều kiện đoạn con đẹp:** $L, R \in \mathbb{P}$ (tập số nguyên tố) và $L \leq R$.
- **Tính tổng đoạn con:** Sử dụng mảng cộng dồn $P[i] = a_1 + a_2 + \dots + a_i$. Khi đó tổng đoạn con từ L đến R là $P[R] - P[L - 1]$.
- **Sàng số nguyên tố:** Với $N = 10^6$, ta dùng Sàng Eratosthenes để xác định nhanh các số nguyên tố trong đoạn $[1, N]$.

13.4.1 Subtask 1 & 2: $N \leq 3000$

Phân tích: Với $N \leq 3000$, số lượng cặp (L, R) thỏa mãn L, R là số nguyên tố không quá lớn. Ta có thể duyệt qua tất cả các cặp số nguyên tố L, R bằng hai vòng lặp lồng nhau, tính tổng và cập nhật giá trị lớn nhất. Độ phức tạp: $O(N^2)$ (thực tế ít hơn vì chỉ duyệt các số nguyên tố).

```

#include <bits/stdc++.h>
#define int long long
using namespace std;

bool is_prime[1000005];
int a[1000005], p[1000005];
int ans = -2e18, n;

void sangnt() {
    for(int i = 0; i <= 1000000; i++)

```

```

        is_prime[i] = 1;
    is_prime[0] = is_prime[1] = 0;
    for (int i = 2; i * i <= n; i++) {
        if (is_prime[i]) {
            for (int j = i * i; j <= n; j += i)
                is_prime[j] = 0;
        }
    }
}

signed main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    cin >> n;
    sangnt();
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        p[i] = p[i - 1] + a[i];
    }

    vector<int> primes;
    for (int i = 2; i <= n; i++) if (is_prime[i]) primes.push_back(i);
    for (int i = 0; i < primes.size(); i++) {
        for (int j = i; j < primes.size(); j++) {
            int L = primes[i], R = primes[j];
            ans = max(ans, p[R] - p[L - 1]);
        }
    }
    cout << ans << "\n";
    return 0;
}

```

13.4.2 Subtask 3: $N \leq 10^6$

Phân tích thuật toán tối ưu: Để đạt được độ phức tạp $O(N)$, ta cố định chỉ số R và tìm L sao cho $P[R] - P[L - 1]$ lớn nhất.

- Biểu thức cần cực đại hóa: $P[R] - P[L - 1]$. Với R cố định, ta cần tìm L là số nguyên tố sao cho $P[L - 1]$ là **nhỏ nhất** và $L \leq R$.
- Ta duyệt i từ 2 đến N :
 1. Nếu i là số nguyên tố, nó có thể đóng vai trò là L . Ta cập nhật giá trị nhỏ nhất của các $P[L - 1]$ đã gặp: $\text{min_prefix} = \min(\text{min_prefix}, P[i - 1])$.
 2. Nếu i là số nguyên tố, nó cũng có thể đóng vai trò là R . Lúc này, tổng lớn nhất kết thúc tại i là $P[i] - \text{min_prefix}$.

- **Lưu ý:** Phải đảm bảo cập nhật *min_prefix* trước khi tính kết quả cho $R = i$ vì đoạn con đẹp có thể chỉ gồm 1 phần tử ($L = R$).

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

bool nt[1000005];
int p[1000005];
int n, mx, mn;

void sangnt() {
    for(int i = 0; i <= 1000000; i++)
        ntk[i] = 1;
    nt[0] = nt[1] = 0;
    for (int i = 2; i * i <= n; i++) {
        if (nt[i]) {
            for (int j = i * i; j <= n; j += i)
                nt[j] = 0;
        }
    }
}

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n;
    sangnt();

    for (int i = 1; i <= n; i++) {
        int x; cin >> x;
        p[i] = p[i - 1] + val;
    }

    mx = -2e18;
    mn = 2e18;

    for (int i = 2; i <= n; i++) {
        if (nt[i]) {
            mn = min(mn, p[i - 1]);
            mx = max(mx, p[i] - mn);
        }
    }

    cout << mx;
}
```

13.5 Bài 5: CHƯƠNG TRÌNH LUYỆN TẬP

Tóm tắt đề bài: Có N học sinh với kỹ năng ban đầu a_i và M bài tập với độ khó b_j . Một học sinh có kỹ năng S chỉ giải được bài tập có độ khó x nếu $S \geq x$. Sau khi giải, kỹ năng tăng thêm x . Mỗi bài tập làm tối đa một lần. Yêu cầu tìm kỹ năng tối đa mỗi học sinh có thể đạt được.

Phân tích thuật toán:

- **Chiến thuật Tham lam (Greedy):** Để đạt kỹ năng cao nhất, học sinh nên ưu tiên giải các bài tập có độ khó từ thấp đến cao. Việc giải một bài tập dễ sẽ giúp tăng kỹ năng, từ đó có khả năng giải được các bài tập khó hơn. Nếu bỏ qua bài dễ để làm bài khó trước, học sinh có thể bị "kẹt" không đủ kỹ năng để làm bất kỳ bài nào.
- **Tối ưu hóa:**
 - Nếu xét từng học sinh và duyệt qua toàn bộ bài tập, độ phức tạp là $O(N \times M)$, sẽ bị TLE với giới hạn 5×10^5 .
 - Nhận xét: Nếu học sinh A có kỹ năng ban đầu thấp hơn học sinh B ($a_A < a_B$), thì học sinh B chắc chắn sẽ giải được tất cả các bài mà học sinh A giải được và thậm chí nhiều hơn.
 - Ta sắp xếp mảng bài tập b tăng dần. Sắp xếp các học sinh theo kỹ năng a tăng dần (vẫn giữ chỉ số cũ để in kết quả).
 - Sử dụng kỹ thuật **Hai con trỏ (Two Pointers)**: Khi chuyển sang học sinh tiếp theo (có kỹ năng cao hơn), ta kế thừa tổng độ khó các bài tập mà học sinh trước đã làm được và chỉ cần kiểm tra các bài tập tiếp theo trong danh sách đã sắp xếp.

13.5.1 Subtask 1: $N, M \leq 10^3$

Phân tích: Với giới hạn nhỏ, ta có thể dùng thuật toán $O(N \times M)$. Với mỗi học sinh, ta sắp xếp mảng b và duyệt từ đầu đến cuối để cộng dồn kỹ năng.

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int n, m;
int a[200005], b[200005];

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n >> m;
```

```

    for (int i = 0; i < n; i++) cin >> a[i];
    for (int i = 0; i < m; i++) cin >> b[i];

    sort(b, b + m);

    for (int i = 0; i < n; i++) {
        int cur = a[i];
        for (int j = 0; j < m; j++) {
            if (cur >= b[j]) cur += b[j];
            else break;
        }
        cout << cur << " ";
    }
}

```

13.5.2 Subtask 2 & 3: $N, M \leq 5 \times 10^5$ (Full Test)

Phân tích: Sử dụng kỹ thuật hai con trỏ phối hợp với sắp xếp học sinh để đạt độ phức tạp $O(N \log N + M \log M)$.

```

#include <bits/stdc++.h>
#define int long long
using namespace std;

struct Student {
    int skill, id;
};

int n, m, p, sum_val;
Student st[200005];
int b[200005], res[200005];

bool cmp(Student x, Student y) {
    return x.skill < y.skill;
}

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n >> m;
    for (int i = 0; i < n; i++) {
        cin >> st[i].skill;
        st[i].id = i;
    }
    for (int i = 0; i < m; i++) cin >> b[i];
}

```



```

    sort(b, b + m);
    sort(st, st + n, cmp);

    p = 0;
    sum_val = 0;
    for (int i = 0; i < n; i++) {
        int cur = st[i].skill + sum_val;
        while (p < m && cur >= b[p]) {
            cur += b[p];
            sum_val += b[p];
            p++;
        }
        res[st[i].id] = cur;
    }

    for (int i = 0; i < n; i++) {
        cout << res[i] << " ";
    }
}

```

14 Đắc Nông

14.1 Bài 1: Số bộ bàn ghế

Tóm tắt đề bài:

- Bối cảnh: Nhà trường cần mua bàn ghế cho 2 lớp mới với số lượng học sinh lần lượt là x và y .
- Điều kiện: Mỗi bộ bàn ghế chỉ chứa tối đa 2 học sinh.
- Chi phí: Giá của mỗi bộ bàn ghế là z đồng.
- Yêu cầu:
 - Tính tổng số lượng bộ bàn ghế ít nhất cần mua cho cả hai lớp.
 - Tính tổng kinh phí tối thiểu để mua số bàn ghế đó.
- Ràng buộc: $1 \leq x, y \leq 50$ và $1 \leq z \leq 10^4$.

Phân tích bài toán:

- Chiến thuật ban đầu:
 - Vì mỗi bộ bàn ghế ngồi được tối đa 2 người, nên với mỗi lớp, ta cần tính số bàn bằng cách lấy số học sinh chia cho 2.
 - Nếu số học sinh là số chẵn (ví dụ 20), số bàn cần là $x/2$.

- Nếu số học sinh là số lẻ (ví dụ 21), số bàn cần là $x/2 + 1$ (phần dư cần thêm 1 bàn riêng).
- Công thức tổng quát cho một lớp có k học sinh là: $\lceil k/2 \rceil$ hoặc dùng phép chia nguyên trong lập trình: $(k + 1)/2$.
- Tối ưu hóa:
 - Do giới hạn dữ liệu rất nhỏ ($x, y \leq 50$), ta chỉ cần sử dụng các kiểu dữ liệu số nguyên cơ bản (int hoặc long long).
 - Tổng số bàn $S = \lfloor (x + 1)/2 \rfloor + \lfloor (y + 1)/2 \rfloor$.
 - Tổng kinh phí $T = S \times z$.
 - Thuật toán có độ phức tạp là $O(1)$.

14.1.1 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int x, y, z;
    cin >> x >> y >> z;

    int ban_lop_1 = (x + 1) / 2;
    int ban_lop_2 = (y + 1) / 2;

    int tong_ban = ban_lop_1 + ban_lop_2;
    long long tong_tien = (long long)tong_ban * z;

    cout << tong_ban << endl;
    cout << tong_tien << endl;

    return 0;
}
```

14.2 Bài 2: Dây bóng đèn

Tóm tắt bài toán:

- Bối cảnh: Một dây bóng đèn có mã số từ 1 đến n . Ta xét các bóng đèn trong đoạn từ $[a, b]$.
- Quy tắc hiển thị màu:
 - Số chẵn: Màu xanh.
 - Số lẻ: Màu đỏ.

- Chia hết cho 3: Màu vàng (quy tắc này ưu tiên đặc biệt).
- **Yêu cầu:** Đếm số lượng bóng đèn hiển thị màu xanh trong đoạn $[a, b]$.
- Điều kiện màu xanh: Một bóng đèn hiển thị màu xanh nếu nó là số chẵn và không chia hết cho 3.
- Ràng buộc: $1 \leq a \leq b < 10^{18}$.

Phân tích bài toán:

- Chiến thuật ban đầu:
 - Duyệt từ a đến b , với mỗi số i , kiểm tra điều kiện: $(i \bmod 2 == 0)$ và $(i \bmod 3) \neq 0$.
 - Nếu thỏa mãn thì tăng biến đếm.
 - Đánh giá: Chỉ phù hợp với $b \leq 10^7$. Với b lên đến 10^{18} (Subtask 2), cách này sẽ bị quá thời gian (*TLE*).
- Tối ưu hóa:
 - Sử dụng tính toán trên đoạn: Số lượng giá trị thỏa mãn trong đoạn $[a, b]$ bằng số lượng trong đoạn $[1, b]$ trừ đi số lượng trong đoạn $[1, a - 1]$.
 - Gọi $f(x)$ là số lượng số chẵn và không chia hết cho 3 trong đoạn $[1, x]$.
 - Các số chẵn trong $[1, x]$ là các số chia hết cho 2. Số lượng là: $count_2 = \lfloor x/2 \rfloor$.
 - Trong các số chẵn này, ta phải loại bỏ các số chia hết cho 3. Một số vừa chia hết cho 2 vừa chia hết cho 3 thì chia hết cho 6. Số lượng là: $count_6 = \lfloor x/6 \rfloor$.
 - Vậy $f(x) = \lfloor x/2 \rfloor - \lfloor x/6 \rfloor$.
 - Kết quả bài toán là: $f(b) - f(a - 1)$.
 - Độ phức tạp: $O(1)$, đáp ứng hoàn toàn giới hạn 10^{18} .

14.2.1 Subtask 1 ($b \leq 10^9$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    long long a, b;
    cin >> a >> b;
    long long dem = 0;
    for (long long i = a; i <= b; i++) {
        if (i % 2 == 0 && i % 3 != 0) {
            dem++;
        }
    }
}
```

```

    }
    cout << dem;
    return 0;
}

```

14.2.2 Subtask 2 ($b < 10^{18}$)

```

#include <bits/stdc++.h>
using namespace std;

long long f(long long x) {
    if (x < 1) return 0;
    return (x / 2) - (x / 6);
}

int main() {
    long long a, b;
    cin >> a >> b;
    cout << f(b) - f(a - 1);
    return 0;
}

```

14.3 Bài 3: Tam giác

Tóm tắt bài toán:

- Định nghĩa: Một bộ ba số (a_i, a_j, a_k) được gọi là bộ số tam giác nếu chúng thỏa mãn bất đẳng thức tam giác: tổng hai cạnh bất kỳ luôn lớn hơn cạnh còn lại.
- Dữ liệu: Cho dãy a gồm n số nguyên dương $a_1, a_2, \dots, a_n$.
- Yêu cầu: Đếm số lượng bộ ba chỉ số (i, j, k) đôi một khác nhau sao cho (a_i, a_j, a_k) tạo thành một tam giác.
- Ràng buộc:
 - $3 \leq n \leq 1000$.
 - $a_i \leq 10^9$.

Phân tích bài toán:

- Chiến thuật ban đầu:
 - Sử dụng 3 vòng lặp lồng nhau để duyệt qua tất cả các bộ ba (i, j, k) với $1 \leq i < j < k \leq n$.
 - Kiểm tra điều kiện: $a_i + a_j > a_k$ và $a_i + a_k > a_j$ và $a_j + a_k > a_i$.

- Đánh giá: Độ phức tạp $O(n^3)$. Với $n = 1000$, số phép tính khoảng $10^9/6$, có thể bị quá thời gian (*TLE*) tùy vào giới hạn thời gian của hệ thống. Tuy nhiên, với $n \leq 500$ (Subtask 1), cách này hoạt động tốt.

- Tối ưu hóa:

- Bước 1: Sắp xếp dãy số tăng dần: $a_1 \leq a_2 \leq \dots \leq a_n$.
- Khi dãy đã sắp xếp, với một bộ ba a_i, a_j, a_k ($i < j < k$), ta chỉ cần kiểm tra một điều kiện duy nhất: $a_i + a_j > a_k$.
- Bước 2 (Hai con trỏ): Cố định i và j , ta cần tìm giá trị k lớn nhất sao cho $a_i + a_j > a_k$. Tất cả các chỉ số từ $j + 1$ đến k đều thỏa mãn.
- Tuy nhiên, với $n = 1000$, thuật toán $O(n^2)$ bằng cách dùng 2 vòng lặp cố định i, j và dùng tìm kiếm nhị phân (hoặc hai con trỏ) để tìm k là tối ưu nhất.
- Độ phức tạp: $O(n^2 \log n)$ hoặc $O(n^2)$.

14.3.1 Subtask 1 ($n \leq 500$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    long long dem = 0;
    for (int i = 0; i < n - 2; i++) {
        for (int j = i + 1; j < n - 1; j++) {
            for (int k = j + 1; k < n; k++) {
                if (a[i] + a[j] > a[k] && a[i] + a[k] > a[j] && a[j] + a[k] > a[i]) {
                    dem++;
                }
            }
        }
    }
    cout << dem;
    return 0;
}
```

14.3.2 Subtask 2 ($n \leq 1000$)

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    sort(a.begin(), a.end());

    long long dem = 0;
    for (int i = 0; i < n - 2; i++) {
        int k = i + 2;
        for (int j = i + 1; j < n - 1; j++) {
            while (k < n && a[i] + a[j] > a[k]) {
                k++;
            }
            dem += (k - 1 - j);
            if (k <= j) k = j + 1;
        }
    }
    cout << dem;
    return 0;
}

```

14.4 Bài 4: Ghép hình

Tóm tắt bài toán:

- Bối cảnh: Cho n thẻ tre với độ dài lần lượt là $a_1, a_2, \dots, a_n$.
- Điều kiện ghép hình: Một hình chữ nhật được ghép từ 4 thẻ tre. Để tạo thành hình chữ nhật, cần có 2 cặp thẻ tre mà trong mỗi cặp, 2 thẻ phải có độ dài bằng nhau. Hình vuông cũng được tính là một hình chữ nhật.
- Yêu cầu: Tìm hình chữ nhật có kích thước lớn nhất (tổng độ dài hai cạnh lớn nhất) có thể ghép được từ các thẻ tre đã cho.
- Dữ liệu xuất:
 - Dòng 1: Tổng độ dài hai cạnh (chiều dài + chiều rộng).
 - Dòng 2: Chu vi lớn nhất của hình chữ nhật đó.
 - Nếu không ghép được hình nào, in ra số 0.

- Ràng buộc: $1 \leq n \leq 10^6$ và $1 \leq a_i \leq 10^6$.

Phân tích bài toán:

- Chiến thuật ban đầu:
 - Để tạo được một hình chữ nhật, ta cần ít nhất hai cặp cạnh, mỗi cặp gồm hai thẻ tre có độ dài bằng nhau.
 - Ta có thể đếm số lần xuất hiện của mỗi độ dài thẻ tre. Nếu một độ dài xuất hiện từ 2 lần trở lên, nó có thể trở thành một cặp cạnh của hình chữ nhật.
 - Nếu một độ dài xuất hiện từ 4 lần trở lên, nó có thể tạo thành cả 4 cạnh của một hình chữ nhật (hình vuông).
 - Cách làm đơn giản là liệt kê tất cả các cặp cạnh có thể tạo ra, sau đó chọn ra hai cặp có độ dài lớn nhất.
- Tối ưu hóa:
 - Sử dụng mảng phân phối hoặc kỹ thuật đếm để thống kê số lượng thẻ tre của từng loại độ dài vì $a_i \leq 10^6$.
 - Duyệt qua các độ dài từ lớn đến bé để tìm các cặp cạnh.
 - Gọi L_1 là độ dài của cặp cạnh lớn nhất tìm được và L_2 là độ dài của cặp cạnh lớn thứ hai.
 - Nếu một độ dài x có số lượng ≥ 4 , ta có thể coi như tìm được cả L_1 và L_2 đều bằng x . Nếu số lượng chỉ ≥ 2 , ta lấy một cặp cạnh độ dài x .
 - Độ phức tạp: $O(n + \max(a_i))$, đảm bảo chạy tốt với $n = 10^6$.

14.4.1 Subtask 1 và 2 ($n \leq 1000$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }
    sort(a.begin(), a.end(), greater<int>());
    vector<int> pairs;
    for (int i = 0; i < n - 1; i++) {
        if (a[i] == a[i+1]) {
            pairs.push_back(a[i]);
            i++;
        }
    }
}
```

```

    }
}
if (pairs.size() < 2) {
    cout << 0;
} else {
    long long canh1 = pairs[0];
    long long canh2 = pairs[1];
    cout << canh1 + canh2 << endl;
    cout << (canh1 + canh2) * 2;
}
return 0;
}

```

14.4.2 Subtask 3 ($n \leq 10^6$)

```

#include <bits/stdc++.h>
using namespace std;

int counts[1000001];

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    int n;
    cin >> n;
    int max_val = 0;
    for (int i = 0; i < n; i++) {
        int x;
        cin >> x;
        counts[x]++;
        if (x > max_val) max_val = x;
    }
    long long L1 = 0, L2 = 0;
    for (int i = max_val; i >= 1; i--) {
        while (counts[i] >= 2) {
            if (L1 == 0) {
                L1 = i;
                counts[i] -= 2;
            } else if (L2 == 0) {
                L2 = i;
                counts[i] -= 2;
                break;
            } else {
                break;
            }
        }
        if (L1 > 0 && L2 > 0) break;
    }
}

```



```

    if (L1 == 0 || L2 == 0) {
        cout << 0;
    } else {
        cout << L1 + L2 << endl;
        cout << (L1 + L2) * 2;
    }
    return 0;
}

```

15 Điện Biên

15.1 Bài 1: Tính biểu thức

15.1.1 Lời giải:

Vì $n \leq 10^5$ nên có thể tính giá trị biểu thức bằng cách sử dụng vòng lặp để duyệt từ 1 đến n .

15.1.2 Code mẫu:

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    long long ans = 0;
    for (int i = 1; i <= n; i++) ans += i * i;
    cout << ans * 2025 << '\n';
    return 0;
}

```

15.2 Bài 2: Tổng ước số

15.2.1 Lời giải:

Với một số nguyên dương N bất kỳ, ta luôn có thể biểu diễn thành các cặp tích (a, b) sao cho $a \leq b$ và $N = a \times b$.

Dễ dàng thấy được: giá trị của a không bao giờ vượt quá $\sqrt{N}$, và từ a có thể suy ra được $b = \frac{N}{a}$

Chính vì vậy, chỉ cần duyệt đến $\sqrt{n}$ đã có thể tính được tổng các ước nguyên dương của n .

Độ phức tạp thuật toán: $O(\sqrt{n})$.

15.2.2 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    long long n;
    cin >> n;
    long long ans = 0;
    for (long long i = 1; i * i <= n; i++)
        if (n%i == 0) {
            ans += i;
            if (n / i != i) ans += n / i;
        }
    cout << ans << '\n';
    return 0;
}
```

15.3 Bài 3: Đoạn con nhỏ nhất

15.3.1 Lời giải:

Duyệt lần lượt các vị trí i ($3 \leq i \leq n$). Với mỗi vị trí, tính tổng 3 phần tử liên tiếp $a[i-2], a[i-1]$ và $a[i]$, rồi so sánh với kết quả để tìm ra tổng nhỏ nhất.

15.3.2 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

const int N = 1e5 + 5;
long long a[N];

int main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) cin >> a[i];
    long long ans = 3e9;
    for (int i = 3; i <= n; i++)
        ans = min(ans, a[i - 2] + a[i - 1] + a[i]);
    cout << ans << "\n";
    return 0;
}
```

15.4 Bài 4: Chọn đồ chơi

15.4.1 Lời giải

Đây là một bài toán quy hoạch động cái túi.

Đặt $dp[i][j]$ là tổng giá trị lớn nhất của các món đồ đã chọn khi xét đến món đồ chơi thứ i và tổng trọng lượng đã chọn không quá j .

Khi xét đến món đồ chơi thứ i và tổng trọng lượng là j , có hai trường hợp xảy ra:

- Trường hợp 1: Chọn món đồ chơi thứ i (chỉ chọn được khi $j \geq w[i]$).
Khi đó, tổng giá trị đã chọn sẽ là $dp[i-1][j-w[i]] + v[i]$.
- Trường hợp 2: Không chọn món đồ chơi thứ i . Khi đó, tổng giá trị đã chọn sẽ là $dp[i][j]$.

$dp[i][j]$ sẽ nhận giá trị lớn nhất trong hai trường hợp trên.

Đối với phần truy vết các món đồ đã chọn, ta bắt đầu xét ngược từ $dp[n][m]$ về. Nếu $dp[i][j] \neq dp[i-1][j]$, thì nghĩa là món đồ chơi thứ i đã được chọn, nên thêm i vào danh sách các món đồ chơi đã chọn.

15.4.2 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<int> w(n + 1), v(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> w[i] >> v[i];
    }
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int j = 0; j <= m; j++) {
            dp[i][j] = dp[i - 1][j];
            if (j >= w[i]) {
                dp[i][j] = max(dp[i][j], dp[i - 1][j - w[i]] + v[i]);
            }
        }
    }
    vector<int> chosen;
    int j = m;
    for (int i = n; i >= 1; i--) {
        if (j >= w[i] && dp[i][j] == dp[i - 1][j - w[i]] + v[i]) {
            chosen.push_back(i);
        }
    }
}
```

```

        j -= w[i];
    }
}
reverse(chosen.begin(), chosen.end());
cout << dp[n][m] << '\n';
for (int i = 0; i < (int)chosen.size(); i++) {
    if (i) cout << ' ';
    cout << chosen[i];
}
cout << '\n';
return 0;
}

```

16 Đồng Tháp

16.1 Bài 1: Ghép số

16.1.1 Lời giải:

Để được số lớn nhất có thể, ta sẽ ưu tiên đặt chữ số lớn hơn trong hai chữ số ở hàng chục.

16.1.2 Code mẫu:

```

#include <bits/stdc++.h>
using namespace std;

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    int a, b;
    cin >> a >> b;
    if (b > a) swap(a, b);
    cout << a << b << "\n";
    return 0;
}

```

16.2 Bài 2: Cắt gỗ

16.2.1 Lời giải:

Để không tồn tại một mẫu gỗ thừa thì đáp án cần tìm phải là một ước của a và b .

Vậy đáp án của bài toán chính là ước chung lớn nhất của a và b , ta có thể tìm đáp án bằng hàm `__gcd(a, b)`.

16.2.2 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    long long a,b;
    cin >> a >> b;
    cout << __gcd(a,b) << "\n";
    return 0;
}
```

16.3 Bài 3: Đo tốc độ

16.3.1 Lời giải:

Sử dụng vòng for để duyệt dãy số, nếu số hiện tại đang xét lớn hơn V thì tăng đáp án lên 1 đơn vị.

16.3.2 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    int n,v;
    cin >> n >> v;
    int ans = 0;
    for (int i = 1; i <= n; i++) {
        int x;
        cin >> x;
        if (x > v) ans++;
    }
    cout << ans << "\n";
    return 0;
}
```

16.4 Bài 4: Đổi quà

16.4.1 Lời giải:

Với mỗi phần tử a_i ($1 \leq i \leq n-1$), sử dụng kỹ thuật hai con trỏ hoặc tìm kiếm nhị phân để tìm phần tử a_j ($i < j \leq n$) lớn nhất sao cho $a_j \leq m - a_i$.

16.4.2 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

const int N = 1e5 + 5;
int a[N];

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    int n,m;
    cin >> n >> m;
    for (int i = 1; i <= n; i++) cin >> a[i];
    sort(a + 1, a + n + 1);
    int j = n, ans = -1;
    for (int i = 1; i <= n; i++) {
        while (j > i && a[j] > m - a[i]) j--;
        if (j > i) ans = max(ans, a[i] + a[j]);
    }
    cout << ans << "\n";
    return 0;
}
```

17 Hà Nội KHTN

17.1 Bài 1: DIEM

Phân tích thuật toán chung: Bài toán yêu cầu tính tổng sức $T_i = m_i + v_i$ của mỗi đội tuyển, sau đó tìm sự chênh lệch (hiệu số) giữa giá trị T_i lớn nhất và nhỏ nhất trong tất cả các đội. Vì $m_i, v_i \leq 10^9$, tổng T_i có thể lên tới 2×10^9 , hoàn toàn có thể vừa trong giới hạn số nguyên 32-bit. Tuy nhiên, để tránh mọi rủi ro tràn số và chuẩn hóa phong cách lập trình, ta luôn sử dụng kiểu số nguyên 64-bit (long long).

17.1.1 Subtask 1: $N \leq 100$

Phân tích: Với N rất nhỏ, ta có thể lưu toàn bộ tổng sức của các đội vào một mảng A . Sau khi nhập và tính xong, ta dùng hàm sort để sắp xếp mảng tăng dần. Giá trị nhỏ nhất sẽ nằm ở $A[1]$ và lớn nhất nằm ở $A[N]$. Kết quả là $A[N] - A[1]$. Cách này dùng bộ nhớ $O(N)$ và thời gian $O(N \log N)$, hoàn toàn dư sức qua subtask 1.

```
#include <bits/stdc++.h>
#define int long long
using namespace std;
```

```

int n;
int m, v;
int a[105];
int res;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n;
    for (int i = 1; i <= n; i++) {
        cin >> m >> v;
        a[i] = m + v;
    }

    sort(a + 1, a + n + 1);

    res = a[n] - a[1];

    cout << res << "\n";
}

```

17.1.2 Subtask 2: $N \leq 10^5$ (Full Test)

Phân tích: Với $N \leq 10^5$, việc lưu mảng và sắp xếp $O(N \log N)$ vẫn có thể qua được. Tuy nhiên, ta có thể tối ưu hơn nữa về cả bộ nhớ lẫn thời gian. Thay vì lưu tất cả các giá trị vào mảng, ta chỉ cần hai biến mx (lưu giá trị lớn nhất) và mn (lưu giá trị nhỏ nhất). Khởi tạo mx là một số rất nhỏ và mn là một số rất lớn. Khi đọc vào từng cặp m_i và v_i , ta tính tổng và cập nhật trực tiếp cho mx và mn . Thuật toán này giảm bộ nhớ xuống $O(1)$ và thời gian xuống đúng $O(N)$ - xử lý online, chỉ đọc qua dữ liệu một lần.

```

#include <bits/stdc++.h>
#define int long long
using namespace std;

int n;
int m, v, sum;
int mn = 3e18, mx = -1, res;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n;
    for (int i = 1; i <= n; i++) {
        cin >> m >> v;

```

```

sum = m + v;

if (sum > mx) {
    mx = sum;
}
if (sum < mn) {
    mn = sum;
}

res = mx - mn;

cout << res << "\n";
}

```

17.2 Bài 2: SODU

Phân tích thuật toán chung: Bài toán yêu cầu tính số dư của 2^{3^N} khi chia cho 5. Bản chất đây là bài toán tính lũy thừa tăng tầng với modulo. Theo **Định lý Fermat nhỏ** (Fermat's Little Theorem), với một số nguyên tố p và số nguyên a không chia hết cho p , ta có:

$$a^{p-1} \equiv 1 \pmod{p}$$

Áp dụng vào bài toán với $a = 2$ và $p = 5$, ta có:

$$2^4 \equiv 1 \pmod{5}$$

Điều này có nghĩa là số dư của lũy thừa cơ số 2 khi chia cho 5 sẽ lặp lại theo chu kỳ 4. Do đó:

$$2^K \pmod{5} = 2^{K \pmod{4}} \pmod{5}$$

Với $K = 3^N$, bài toán được rút gọn về việc tìm phần dư của 3^N khi chia cho 4.

—

17.2.1 Subtask 1: $N \leq 5$

Phân tích: Với $N \leq 5$, ta có $3^5 = 243$. Giá trị này khá nhỏ nên ta có thể tính trực tiếp $K = 3^N$, sau đó dùng thuật toán nhân modulo (hoặc vòng lặp) để tính $2^K \pmod{5}$.

```

#include <bits/stdc++.h>
#define int long long
using namespace std;

int n, k, res;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
}

```



```

cin >> n;

k = 1;
for (int i = 1; i <= n; i++) {
    k *= 3;
}

res = 1;
for (int i = 1; i <= k; i++) {
    res = (res * 2) % 5;
}

cout << res << "\n";
}

```

17.2.2 Subtask 2: $N \leq 10^9$ (Full Test)

Phân tích: Với $N \leq 10^9$, việc tính trực tiếp 3^N là bất khả thi vì sẽ gây tràn số khổng lồ. Áp dụng tính chất chu kỳ đã chứng minh ở trên, ta cần tìm $3^N \pmod{4}$. Ta thấy rằng:

$$3 \equiv -1 \pmod{4}$$

Nâng lên lũy thừa N hai vế:

$$3^N \equiv (-1)^N \pmod{4}$$

Từ đây ta chia làm hai trường hợp dựa vào tính chẵn lẻ của N :

- Nếu N là số **chẵn**: $(-1)^N = 1 \Rightarrow 3^N \equiv 1 \pmod{4}$. Khi đó kết quả bài toán là $2^1 \pmod{5} = 2$.
- Nếu N là số **lẻ**: $(-1)^N = -1 \equiv 3 \pmod{4}$. Khi đó kết quả bài toán là $2^3 \pmod{5} = 8 \pmod{5} = 3$.

Thuật toán rút gọn lại chỉ còn đúng $O(1)$ phép toán kiểm tra tính chẵn lẻ của N , cực kỳ ngắn gọn và hiệu quả tuyệt đối.

```

#include <bits/stdc++.h>
#define int long long
using namespace std;

int n;
int res;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n;

```

```

    if (n % 2 == 0) {
        res = 2;
    } else {
        res = 3;
    }

    cout << res << "\n";
}

```

17.3 Bài 3: KCNN

Phân tích thuật toán chung: Bài toán yêu cầu tìm khoảng cách nhỏ nhất giữa hai phần tử có cùng giá trị trong dãy số. Khoảng cách giữa hai phần tử ở vị trí i và j là $|i - j|$. Nếu không có phần tử nào lặp lại, kết quả là -1. Ta cần tìm $\min(|i - j|)$ với mọi cặp $a_i = a_j$.

17.3.1 Subtask 1: $N \leq 10^3$

Phân tích: Với giới hạn $N \leq 1000$, ta có thể sử dụng hai vòng lặp lồng nhau để duyệt qua tất cả các cặp vị trí (i, j) với $i < j$. Nếu $a_i = a_j$, ta tính khoảng cách $j - i$ và cập nhật vào biến kết quả nhỏ nhất. Độ phức tạp thời gian là $O(N^2)$, khoảng 10^6 phép tính, hoàn toàn vượt qua dễ dàng.

```

#include <bits/stdc++.h>
#define int long long
using namespace std;

int n;
int a[100005];
int res = 1e15;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n;
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (a[i] == a[j]) {
                res = min(res, j - i);
            }
        }
    }
}

```

```

    }

    if (res == 1e15) {
        cout << -1 << "\n";
    } else {
        cout << res << "\n";
    }
}

```

17.3.2 Subtask 2: $N \leq 10^5$ (Full Test)

Phân tích: Với $N \leq 10^5$, thuật toán $O(N^2)$ sẽ gây quá thời gian (TLE). Ta cần tối ưu độ phức tạp xuống $O(N)$ hoặc $O(N \log N)$. Một cách giải quyết cực kỳ thanh lịch là sử dụng cấu trúc dữ liệu map để lưu lại **vị trí xuất hiện gần nhất** của mỗi giá trị.

- Duyệt i từ 0 đến $N - 1$. Tại mỗi vị trí, ta đọc vào giá trị x .
- Kiểm tra xem x đã từng xuất hiện trước đó chưa (bằng hàm count của map).
- Nếu x đã tồn tại trong map, ta tính khoảng cách từ vị trí hiện tại i đến vị trí gần nhất của nó (lưu trong map) và cập nhật vào kết quả.
- Cuối cùng, cập nhật lại vị trí mới nhất của x vào map: $mp[x] = i$.

Vì ta duyệt từ trái sang phải, mỗi khi gặp lại một số, khoảng cách tính được luôn là khoảng cách ngắn nhất so với lần xuất hiện ngay liền trước đó của nó. Độ phức tạp chỉ còn $O(N \log N)$, kết hợp không gian nhớ $O(N)$, tối ưu hoàn toàn cho giới hạn bài toán.

```

#include <bits/stdc++.h>
#define int long long
using namespace std;

int n, x;
int res = 1e15;
map<int, int> mp;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n;
    for (int i = 0; i < n; i++) {
        cin >> x;

        if (mp.count(x)) {
            res = min(res, i - mp[x]);
        }
    }
}

```

```

        mp[x] = i;
    }

    if (res == 1e15) {
        cout << -1 << "\n";
    } else {
        cout << res << "\n";
    }
}

```

17.4 Bài 4: HCN

Phân tích thuật toán chung: Bài toán yêu cầu tìm "Dãy giảm dần dài nhất" (Longest Decreasing Subsequence - LDS) của một mảng các hình chữ nhật. Hai tiêu chí để so sánh hình chữ nhật $A > B$ là:

- Diện tích A lớn hơn diện tích B ($S_A > S_B$).
- Nếu $S_A = S_B$, thì chiều dài A phải lớn hơn chiều dài B ($D_A > D_B$).

Vì $D_i, R_i \leq 10^9$, diện tích có thể lên tới 10^{18} , ta bắt buộc phải dùng kiểu số nguyên 64-bit (long long) cho toàn bộ các phép tính. Ta sẽ tạo một struct Rect và định nghĩa hàm so sánh cmp(a, b) để code được gọn gàng và dễ hiểu.

—

17.4.1 Subtask 1: $N \leq 10^3$

Phân tích: Với $N \leq 1000$, ta có thể sử dụng thuật toán Quy hoạch động (Dynamic Programming) cơ bản với độ phức tạp $O(N^2)$. Gọi $dp[i]$ là độ dài của dãy giảm dần dài nhất kết thúc tại hình chữ nhật thứ i . Tại mỗi vị trí i , ta duyệt lại tất cả các vị trí $j < i$ trước đó. Nếu hình chữ nhật j lớn hơn hình chữ nhật i ($\text{cmp}(a[j], a[i]) == \text{true}$), ta cập nhật $dp[i] = \max(dp[i], dp[j] + 1)$. Kết quả bài toán là giá trị lớn nhất trong mảng dp .

```

#include <bits/stdc++.h>
#define int long long
using namespace std;

struct Rect {
    int s, d;
};

bool cmp(Rect a, Rect b) {
    return (a.s > b.s) || (a.s == b.s && a.d > b.d);
}

int n;

```

```

Rect a[1005];
int dp[1005];
int res = 0;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n;
    for (int i = 1; i <= n; i++) {
        int d, r;
        cin >> d >> r;
        a[i].s = d * r;
        a[i].d = d;
    }

    for (int i = 1; i <= n; i++) {
        dp[i] = 1;
        for (int j = 1; j < i; j++) {
            if (cmp(a[j], a[i])) {
                dp[i] = max(dp[i], dp[j] + 1);
            }
        }
        res = max(res, dp[i]);
    }

    cout << res << "\n";
}

```

17.4.2 Subtask 2: $N \leq 10^5$ (Full Test)

Phân tích: Với $N \leq 10^5$, thuật toán $O(N^2)$ sẽ gây quá thời gian (TLE). Ta cần tối ưu bài toán dãy con giảm dần dài nhất xuống $O(N \log N)$ bằng **Tìm kiếm nhị phân (Binary Search)**. Ta duy trì một mảng *tail*, trong đó *tail*[*k*] lưu trữ "hình chữ nhật nhỏ nhất có thể đứng ở cuối của một dãy giảm dần có độ dài *k*". Mảng *tail* này sẽ luôn có tính chất giảm dần ngặt. Khi xét đến hình chữ nhật $a[i]$:

- Ta dùng tìm kiếm nhị phân để tìm vị trí *pos* đầu tiên trong mảng *tail* mà hình chữ nhật tại đó **không lớn hơn** $a[i]$ (tức là $\leq a[i]$ theo định nghĩa hàm *cmp*).
- Nếu tìm thấy, ta thay thế *tail*[*pos*] bằng $a[i]$ để tạo cơ hội nối thêm phần tử vào sau.
- Nếu $a[i]$ nhỏ hơn tất cả các phần tử trong *tail*, nó sẽ được nối vào cuối mảng *tail*, đồng thời độ dài của dãy *len* tăng thêm 1.

Thuật toán kết hợp Quy hoạch động và Tìm kiếm nhị phân này xử lý mượt mà 10^5 phần tử với độ phức tạp tối ưu tuyệt đối $O(N \log N)$.

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

struct Rect {
    int s, d;
};

bool cmp(Rect a, Rect b) {
    return (a.s > b.s) || (a.s == b.s && a.d > b.d);
}

int n;
Rect a[100005];
Rect tail[100005];
int len = 0;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n;
    for (int i = 1; i <= n; i++) {
        int d, r;
        cin >> d >> r;
        a[i].s = d * r;
        a[i].d = d;
    }

    for (int i = 1; i <= n; i++) {
        int l = 1, r = len, pos = len + 1;

        while (l <= r) {
            int mid = (l + r) / 2;
            if (!cmp(tail[mid], a[i])) {
                pos = mid;
                r = mid - 1;
            } else {
                l = mid + 1;
            }
        }

        tail[pos] = a[i];
        if (pos > len) {
            len++;
        }
    }
}
```

```
cout << len << "\n";
}
```

18 Hà Nội - Chuyên Sư Phạm

18.1 Bài 1: Ước nguyên tố

18.1.1 Tóm tắt đề bài

Với một số nguyên dương $x \geq 2$, gọi $f(x)$ là ước số nguyên tố lớn nhất của x .

Yêu cầu: Cho số n và q câu hỏi, mỗi câu hỏi cho bởi số nguyên dương k . Bạn hãy cho biết trong phạm vi từ 2 tới n có bao nhiêu giá trị x mà $f(x) \leq k$.

18.1.2 Subtask 1: $n \leq 10^4$; $q \leq 10$

- Với giới hạn n và q đủ bé, ta hoàn toàn có thể duyệt qua từng k trong q và tính hàm $f(x) \forall x \in [2, n]$, dễ thấy hàm f có thể tính trong $\sqrt{x}$.
- Độ phức tạp: $\mathcal{O}(q \times n \times \sqrt{n})$.

18.1.3 Subtask 2: $n \leq 10^6$; $q \leq 10^6$

- Để giải quyết subtask có n và q lớn, ta có thể sử dụng sàng nguyên tố Eratosthenes để tính nhanh ước lớn nhất của từng x .
- $\forall x \in [2, n]$ ta có thể tính $f(2) + f(3) + f(4) + \dots + f(x)$ bằng mảng cộng dồn.
- Độ phức tạp: $\mathcal{O}(n \log n + q)$

18.1.4 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

const int N = 1e6 + 1;

int largest[N], f[N];
bool is_prime[N];

int main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);

    memset(is_prime, true, sizeof(is_prime));
    for(int i = 2; i * i < N; i++) {
        if(is_prime[i]) {
```

```

        for(int j = i * i; j < N; j += i) {
            is_prime[j] = false;
        }
    }
    for(int i = 2; i < N; i++) {
        if(is_prime[i]) {
            largest[i] = i;
            for(int j = i * 2; j < N; j += i) largest[j] = i;
        }
    }

    int n, q; cin >> n >> q;

    for(int i = 2; i <= n; i++) f[largest[i]] += 1;
    for(int i = 2; i <= n; i++)
        f[i] += f[i - 1];

    while(q--) {
        int k; cin >> k;
        cout << f[k] << '\n';
    }

    return 0;
}

```

18.2 Bài 2: Chạy tiếp sức

18.2.1 Tóm tắt đề bài:

Cho n chặng chạy liên tiếp, chặng thứ i có độ dài a_i . Ba vận động viên A, B, C lần lượt chạy các chặng từ đầu đến cuối đường đua.

Cần chia dãy $a_1, a_2, \dots, a_n$ thành ba đoạn liên tiếp (có thể rỗng), lần lượt giao cho A, B, C .

Mỗi vận động viên chỉ chạy các chặng liên tiếp và tổng quãng đường thỏa mãn $S_A \leq S_B \leq S_C$.

Tối đa hóa tổng quãng đường mà A chạy.

18.2.2 Subtask 1: $n \leq 100$

- Ta có thể sử dụng lần lượt 3 vòng for để tính tổng quãng đường của 3 vận động viên.
- Sau khi cố định được quãng đường, ta chỉ cần kiểm tra điều kiện và đưa ra đáp án.

18.2.3 Subtask 2: $n \leq 5000$

- Ta có thể sử dụng 2 vòng for để tính tổng quãng đường của vận động viên A và B sau đó lấy tổng $a_1 + a_2 + \dots + a_n$ trừ đi cho A và B để tính ra tổng quãng đường của C .

- Sau đó, ta làm tương tự Subtask 2.

18.2.4 Subtask 3: $n \leq 10^6$

- Ta thấy được rằng, tổng quãng đường của A là một hàm tuyến tính, vì thế ta có thể áp dụng chặt nhị phân để tính quãng đường của A .
- Với mỗi lần chặt nhị phân mà ta cố định được tổng quãng đường của A , ta chỉ cần kiểm tra xem có tồn tại một cách xếp sao cho quãng đường của B và C sao cho $S_A \leq S_B \leq S_C$.
- Để thực hiện điều đó, ta sử dụng kỹ thuật mảng cộng dồn, gọi $\text{pref}[i]$ là tổng $a_1 + a_2 + \dots + a_i$.

$$S_A \leq S_B \leq S_C$$

$$\text{pref}[\text{mid}] \leq \text{pref}[i] - \text{pref}[\text{mid}] \text{ và } \text{pref}[i] - \text{pref}[\text{mid}] \leq \text{pref}[n] - \text{pref}[i]$$

$$2 \times \text{pref}[\text{mid}] \leq \text{pref}[i] \text{ và } 2 \times \text{pref}[i] \leq \text{pref}[n] + \text{pref}[\text{mid}]$$

18.2.5 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

const int N = 1e6 + 3;

int a[N];
ll prefix[N];

int main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);

    int n; cin >> n;

    for(int i = 1; i <= n; i++) {
        cin >> a[i];
        prefix[i] = prefix[i - 1] + a[i];
    }

    int best = 0, low = 1, high = n;
    while(low <= high) {
        int mid = (low + high) / 2;
        bool passed = false;
        for(int i = mid + 1; i <= n; i++) {
            if(prefix[mid] * 2 <= prefix[i] &&
               prefix[i] * 2 <= prefix[n] + prefix[mid]) {
```

```

        passed = true;
        break;
    }
}
if(passed) {
    best = mid;
    low = mid + 1;
} else high = mid - 1;
}

cout << prefix[best] << endl;

return 0;
}

```

18.3 Bài 3: Những gói kẹo

18.3.1 Tóm tắt đề bài:

Có n gói kẹo, gói thứ i chứa a_i viên. Giáo sư cần chọn ra m gói ($m < n$) sao cho số kẹo trong m gói này bằng nhau.

Trong mỗi giây, giáo sư có thể lấy đúng một viên kẹo từ một gói và chuyển sang một gói khác.

Tính thời gian tối thiểu (tức số thao tác ít nhất) để từ n gói ban đầu thu được m gói có số kẹo bằng nhau.

18.3.2 Subtask 1: $n \leq 1000$; $a_1, a_2, \dots, a_n \leq 1000$

- Ta có nhận xét rằng, để tạo ra m gói kẹo có giá trị bằng nhau, ta chỉ nên sử dụng các gói kẹo có số lượng viên "sàn sát" nhau.
- Vì thế, ta có thể sắp xếp lại mảng a và sau đó duy trì một cửa sổ gồm m phần tử liên tiếp nhau, với mỗi lần duyệt qua cửa sổ mới ta phải tính toán chi phí để di chuyển kẹo.
- Để tính hàm cost, ta có thể duyệt từ $a_{\min}$ tới $a_{\max}$ và dùng mảng cộng dồn để tính chi phí.

18.3.3 Subtask 2 + 3: $n \leq 10^6$; $a_1, a_2, \dots, a_n \leq 10^6$

- Ta sử dụng lại tư tưởng của Subtask 1, tuy nhiên ta cần đổi cách tính chi phí để tối ưu kết quả.
- Gọi X là giá trị sau khi sắp xếp của m gói kẹo trong cửa sổ đang xét, để tối ưu hoá X , thì X phải là trung bình cộng của cửa sổ.

- Ta cần xét 2 giá trị "khả thi":
 - Làm tròn lên của trung bình cộng (ceil_X)
 - Làm tròn xuống của trung bình cộng (floor_X)
- Gọi $\text{cost}(l, r, X)$ là chi phí để biến nguyên đoạn từ l tới r thành giá trị X .
 - Ta chia đoạn thành 3 phần: bé hơn X , bằng X và lớn hơn X
 - Sau đó ta tính chi phí để đưa hết phần lớn hơn X qua bé hơn X
 - Việc xác định đoạn có thể thực hiện bằng chặt nhị phân vì mảng đã được sắp xếp
- Sau đó ta chỉ cần xét xem ceil_X hay floor_X có chi phí nhỏ hơn (lưu ý kiểm tra ceil_X còn nằm trong tổng n hay không)

18.3.4 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

const int N = 1e6 + 3;

int n, m;
int a[N];
ll prefix[N];

ll sum(int l, int r) {
    if(l > r) return 0;
    return prefix[r] - prefix[l - 1];
}

ll cost(int l, int r, ll x) {
    int u = lower_bound(a + l, a + l + r, x) - a;
    int v = upper_bound(a + l, a + l + r, x) - a;

    ll lack = x * (u - l) - sum(l, u - 1);
    ll extra = sum(v, r) - x * (r - v + 1);

    return max(lack, extra);
}

int main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);

    cin >> n >> m;
```

```

    for(int i = 1; i <= n; i++) {
        cin >> a[i];
    }

    sort(a + 1, a + 1 + n);
    for(int i = 1; i <= n; i++) {
        prefix[i] = prefix[i - 1] + a[i];
    }

    ll best = LLONG_MAX;
    for(int i = 1; i <= n - m + 1; i++) {
        int j = i + m - 1;

        ll sum = prefix[j] - prefix[i - 1];
        ll floor_x = sum / m;
        ll ceil_x = (sum + m - 1) / m;

        best = min(best, cost(i, j, floor_x));
        if(ceil_x * m <= prefix[n])
            best = min(best, cost(i, j, ceil_x));
    }

    cout << best << endl;

    return 0;
}

```

18.4 Bài 4: Lời chào

18.4.1 Tóm tắt đề bài:

Có n chú kiến đứng đều quanh một vòng tròn chu vi n mét, đánh số từ 1 đến n theo chiều kim đồng hồ. Mỗi chú kiến i bắt đầu tại vị trí của mình, di chuyển với cùng tốc độ, theo hướng c_i (thuận chiều nếu $c_i = +$, ngược chiều nếu $c_i = -$) và đi tổng quãng đường d_i .

Hai chú kiến đi ngược chiều khi gặp nhau sẽ chào nhau rồi tiếp tục di chuyển, thời gian chào không đáng kể. Một cặp kiến có thể chào nhau nhiều lần. Khi một chú kiến đi hết quãng đường d_i , nó rời khỏi vòng tròn và không chào nữa.

Yêu cầu: Với mỗi chú kiến, hãy xác định tổng số lần chào mà chú kiến đó thực hiện trong suốt hành trình.

18.4.2 Subtask 1: $n, d_1, d_2, \dots, d_n \leq 100$

- Ta duyệt mọi cặp kiến đi ngược chiều nhau.

- Xét cặp i, j sao cho $c_i = +$ và $c_j = -$, đặt

$$s = (p_j - p_i) \bmod n.$$

- Hai chú kiến này gặp nhau tại các thời điểm t thỏa

$$2t = s + kn \quad (k \geq 0).$$

- Vì mỗi chú kiến chỉ đi trong thời gian tương ứng với quãng đường của mình, nên số lần chào của cặp này là số k sao cho

$$s + kn \leq 2\min(d_i, d_j).$$

- Với n, d_i rất nhỏ, ta có thể kiểm tra trực tiếp từng k hoặc tính thẳng số nghiệm của bất đẳng thức trên.
- Cộng kết quả của từng cặp vào đáp án của cả hai chú kiến.

18.4.3 Subtask 2: $n \leq 5000$

- Từ công thức ở Subtask 1, ta không cần mô phỏng theo thời gian nữa mà chỉ cần đếm số nghiệm của bất đẳng thức.
- Với cặp i, j đi ngược chiều, vẫn đặt

$$s = (p_j - p_i) \bmod n.$$

- Khi đó số lần chào của cặp là

$$\{k \geq 0 : s + kn \leq 2\min(d_i, d_j)\}.$$

- Viết

$$2\min(d_i, d_j) = qn + r \quad (0 \leq r < n),$$

thì số lần chào bằng

$$q + \mathbf{1}[s \leq r].$$

- Vì thế chỉ cần duyệt mọi cặp kiến khác hướng và tính đóng góp trong $O(1)$ cho mỗi cặp.
- Tổng độ phức tạp là $O(n^2)$, phù hợp với $n \leq 5000$.

18.4.4 Subtask 3: $d_1 = d_2 = \dots = d_n$

- Gọi giá trị chung là d , và viết

$$2d = qn + r \quad (0 \leq r < n).$$

- Khi đó, với mọi cặp kiến đi ngược chiều, phần đóng góp cơ bản luôn là q .
- Phần cộng thêm 1 chỉ xảy ra khi vị trí ban đầu của chú kiến kia nằm trong một đoạn dài r trên vòng tròn.
- Cụ thể:

- Nếu $c_i = +$ và $c_j = -$ thì cần kiểm tra xem

$$p_j \in [p_i, p_i + r] \pmod{n}$$

hay không.

- Nếu $c_i = -$ và $c_j = +$ thì cần kiểm tra xem

$$p_j \in [p_i - r, p_i] \pmod{n}$$

hay không.

- Vì vậy, ta chỉ cần đếm số chú kiến đổi hướng nằm trong các đoạn trên.
- Dùng Fenwick Tree để lưu số chú kiến của một hướng trên vòng tròn, ta truy vấn được số lượng trong mỗi đoạn trong $\mathcal{O}(\log n)$.
- Độ phức tạp là $\mathcal{O}(n \log n)$.

18.4.5 Subtask 4: $n \leq 2 \cdot 10^5$, $d_i \leq 10^9$

- Đây là tổng quát của Subtask 3, nhưng mỗi chú kiến có một giá trị d_i riêng.
- Với mỗi chú kiến i , đặt

$$q_i = \left\lfloor \frac{2d_i}{n} \right\rfloor, \quad r_i = (2d_i) \bmod n.$$

- Xét một cặp kiến đi ngược chiều, giả sử $c_i = +$ và $c_j = -$. Đặt

$$s = (p_j - p_i) \bmod n.$$

- Khi đó số lần chào của cặp (i, j) là số nghiệm của

$$s + kn \leq 2 \min(d_i, d_j).$$

- Có hai trường hợp:

- Nếu $d_i \leq d_j$ thì đóng góp của cặp này cho i là

$$q_i + \mathbf{1}[s \leq r_i].$$

- Nếu $d_i > d_j$ thì đóng góp của cặp này cho i là

$$q_j + \mathbf{1}[s \leq r_j].$$

- Từ đây ta thấy mỗi cặp gồm hai phần:
 - phần nguyên q ,
 - phần dư r , chỉ quyết định có được cộng thêm 1 hay không.
- Ta tách theo hướng và theo thứ tự của d :
 - (+) với (-) và $d_i \leq d_j$,
 - (+) với (-) và $d_i > d_j$,
 - (-) với (+) và $d_i \leq d_j$,
 - (-) với (+) và $d_i > d_j$.
- Với hai trường hợp có $d_i \leq d_j$, ta duy trì Fenwick Tree đếm số vị trí của các chú kiến đối hướng có d lớn hơn hoặc bằng hiện tại, rồi cộng thêm số chú kiến nằm trong đoạn dài r_i .
- Với hai trường hợp có $d_i > d_j$, ta quét theo thứ tự tăng của d , cộng dồn các giá trị q của những chú kiến đã gặp, đồng thời dùng Fenwick Tree kiểu cộng đoạn – truy vấn điểm để đánh dấu các đoạn gây thêm 1.
- Mỗi cặp được xử lý đúng một lần, nên không bị đếm trùng.
- Độ phức tạp: $\mathcal{O}(n \log n)$.

18.4.6 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

const int N = 2e5 + 3;

struct Ant {
    int p;
    ll d, q, r;

    bool operator < (const Ant &other) const {
```

```

        return d < other.d;
    }
};

struct FenwickTree {
    int n;
    vector<int> tree;
    FenwickTree (int n) : n(n), tree(n + 3) {}

    void update(int p, int v) {
        for(++p; p <= n; p += p & -p)
            tree[p] += v;
    }

    int get(int p) {
        int total = 0;
        for(++p; p > 0; p -= p & -p)
            total += tree[p];
        return total;
    }

    int get(int l, int r) {
        if(l <= r) return get(r) - get(l - 1);
        return get(n - 1) - (get(l - 1) - get(r));
    }

    void reload() {
        fill(tree.begin(), tree.end(), 0);
    }
};

struct RangedFenwickTree {
    int n;
    vector<int> tree;
    RangedFenwickTree (int n) : n(n), tree(n + 3) {}

    void update(int p, int v) {
        for(++p; p <= n; p += p & -p)
            tree[p] += v;
    }

    void update(int l, int r, int v) {
        if(l <= r) {
            update(l, v);
            update(r + 1, -v);
        } else {
            update(0, v);
            update(r + 1, -v);
        }
    }
};

```



```

        update(l, v);
        update(n, -v);
    }
}

int get(int p) {
    int total = 0;
    for(++p; p > 0; p -= p & -p)
        total += tree[p];
    return total;
}

void reload() {
    fill(tree.begin(), tree.end(), 0);
}
};

ll answer[N];

int main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);

    int n; cin >> n;

    vector<Ant> plus, minus;
    for(int i = 0; i < n; i++) {
        char type; ll d; cin >> type >> d;
        ll q = (2 * d) / n;
        ll r = (2 * d) % n;
        if(type == '+')
            plus.push_back({i, d, q, r});
        else
            minus.push_back({i, d, q, r});
    }

    sort(plus.begin(), plus.end());
    sort(minus.begin(), minus.end());
    int size_p = plus.size() - 1, size_m = minus.size() - 1;

    // Case 1: (+)i and (-)j with d_i <= d_j
    FenwickTree fen(n);
    for(int i = size_p, j = size_m; i >= 0; i--) {
        while(j >= 0 && plus[i].d <= minus[j].d) {
            fen.update(minus[j].p, 1);
            j--;
        }
        answer[plus[i].p] += plus[i].q * (size_m - j);
    }
}

```

```

        answer[plus[i].p] += fen.get(plus[i].p, (plus[i].p + plus[i].r) % n);
    }

    // Case 2: (+)i and (-)j with d_i > d_j
    RangedFenwickTree rfen(n);
    ll sum_q = 0;
    for(int i = 0, j = 0; i <= size_p; i++) {
        while(j <= size_m && plus[i].d > minus[j].d) {
            sum_q += minus[j].q;
            rfen.update((minus[j].p - minus[j].r + n) % n, minus[j].p, 1);
            j++;
        }
        answer[plus[i].p] += sum_q;
        answer[plus[i].p] += rfen.get(plus[i].p);
    }

    // Case 3: (-)i and (+)j with d_i <= d_j
    fen.reload();
    for(int i = size_m, j = size_p; i >= 0; i--) {
        while(j >= 0 && minus[i].d <= plus[j].d) {
            fen.update(plus[j].p, 1);
            j--;
        }
        answer[minus[i].p] += minus[i].q * (size_p - j);
        answer[minus[i].p] += fen.get((minus[i].p - minus[i].r + n) % n, minus[i].p);
    }

    // Case 4: (-)i and (+)j with d_i > d_j
    rfen.reload();
    sum_q = 0;
    for(int i = 0, j = 0; i <= size_m; i++) {
        while(j <= size_p && minus[i].d > plus[j].d) {
            sum_q += plus[j].q;
            rfen.update(plus[j].p, (plus[j].p + plus[j].r) % n, 1);
            j++;
        }
        answer[minus[i].p] += sum_q;
        answer[minus[i].p] += rfen.get(minus[i].p);
    }

    for(int i = 0; i < n; i++) {
        cout << answer[i] << endl;
    }

    return 0;
}

```

19 Hà Tĩnh

19.1 Bài 1: Tìm số

Tóm tắt bài toán

- Tìm số d lớn nhất sao cho $1 \leq d \leq n$.
- d phải đồng thời chia hết cho a và chia hết cho b .
- Nếu không tìm được d nào trong khoảng $[1, n]$, in ra -1 .

Phân tích bài toán

- **Phân tích tính chất:** Một số chia hết cho cả a và b thì nó phải chia hết cho Bội chung nhỏ nhất của a và b , ký hiệu là $LCM(a, b)$.
- Công thức tính $LCM(a, b) = \frac{a \times b}{GCD(a, b)}$, trong đó GCD là ước chung lớn nhất.
- **Chiến thuật ban đầu (Dành cho subtask 1):** Duyệt ngược từ n về 1. Số đầu tiên chia hết cho cả a và b chính là đáp án. Cách này có độ phức tạp $O(n)$, chỉ chạy được với $n \leq 10^6$.
- **Tối ưu hóa (Dành cho subtask 2):**
 - Ta cần tìm bội lớn nhất của $L = LCM(a, b)$ mà không vượt quá n .
 - Số đó có dạng: $d = \lfloor \frac{n}{L} \rfloor \times L$.
 - Nếu $d = 0$ (nghĩa là $n < L$), theo yêu cầu tìm số nguyên dương nên kết quả là -1 .
 - Lưu ý: Vì $n \leq 10^{12}$ nên cần dùng kiểu dữ liệu long long trong C++.

19.1.1 Subtask 1 ($n \leq 10^6$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    long long a, b, n;
    cin >> a >> b >> n;
    for (long long i = n; i >= 1; i--) {
        if (i % a == 0 && i % b == 0) {
            cout << i;
            return 0;
        }
    }
    cout << -1;
    return 0;
}
```

19.1.2 Subtask 2 (Full test $n \leq 10^{12}$)

```
#include <bits/stdc++.h>
using namespace std;

long long gcd(long long a, long long b) {
    while (b != 0) {
        a %= b;
        swap(a, b);
    }
    return a;
}

long long lcm(long long a, long long b) {
    return (a / gcd(a, b)) * b;
}

int main() {
    long long a, b, n;
    cin >> a >> b >> n;

    long long L = lcm(a, b);
    long long res = (n / L) * L;

    if (res == 0) {
        cout << -1;
    } else {
        cout << res;
    }

    return 0;
}
```

19.2 Bài 2: Không chia hết**Tóm tắt bài toán**

- Cần tính tổng các số x sao cho $L \leq x \leq R$ và $x \pmod k \neq 0$.
- Giới hạn L, R lên đến 10^9 , yêu cầu thuật toán tối ưu hơn việc duyệt từng số.

Phân tích bài toán

- **Chiến thuật ban đầu (Vét cạn):**
 - Duyệt vòng lặp từ L đến R .
 - Nếu số hiện tại không chia hết cho k thì cộng vào biến tổng.
 - Độ phức tạp $O(R - L + 1)$, chỉ chạy được với subtask 1 ($R \leq 10^6$).

• **Tối ưu hóa (Sử dụng công thức toán học):**

- Gọi $S(n)$ là tổng các số nguyên dương từ 1 đến n : $S(n) = \frac{n(n+1)}{2}$.
- Tổng các số từ L đến R là: $T_{all} = S(R) - S(L - 1)$.
- Gọi $G(n, k)$ là tổng các số từ 1 đến n mà chia hết cho k . Các số này là: $k, 2k, 3k, \dots, mk$ với $m = \lfloor \frac{n}{k} \rfloor$.
- $G(n, k) = k + 2k + \dots + mk = k \times (1 + 2 + \dots + m) = k \times \frac{m(m+1)}{2}$.
- Tổng các số chia hết cho k trong đoạn $[L, R]$ là: $T_{div} = G(R, k) - G(L - 1, k)$.
- Đáp án cuối cùng: $Ans = T_{all} - T_{div}$.
- Độ phức tạp $O(1)$, đáp ứng hoàn toàn giới hạn 10^9 .

19.2.1 Subtask 1 ($L, R \leq 10^6$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    long long l, r, k;
    cin >> l >> r >> k;
    long long sum = 0;
    for (long long i = l; i <= r; i++) {
        if (i % k != 0) {
            sum += i;
        }
    }
    cout << sum;
    return 0;
}
```

19.2.2 Subtask 2 (Full test $L, R \leq 10^9$)

```
#include <bits/stdc++.h>
using namespace std;

long long sum(long long n) {
    if (n <= 0) return 0;
    return n * (n + 1) / 2;
}

long long sumdiv(long long n, long long k) {
    if (n < k) return 0;
    long long m = n / k;
    return k * m * (m + 1) / 2;
}
```

```

int main() {
    long long l, r, k;
    cin >> l >> r >> k;

    long long totAll = sum(r) - sum(l - 1);
    long long totDiv = sumdiv(r, k) - sumdiv(l - 1, k);

    cout << totAll - totDiv;

    return 0;
}

```

19.3 Bài 3: Cặp số chia hết

Tóm tắt bài toán

- Đếm số lượng cặp chỉ số (i, j) với $i < j$ sao cho $a_i : a_j$ hoặc $a_j : a_i$.
- Giới hạn n và giá trị a_i đều lên đến 10^6 .

Phân tích bài toán

- **Chiến thuật ban đầu (Vét cạn):**
 - Sử dụng hai vòng lặp lồng nhau để duyệt qua mọi cặp (i, j) .
 - Kiểm tra điều kiện chia hết trong $O(1)$.
 - Độ phức tạp $O(n^2)$, chỉ phù hợp với $n \leq 10^3$.
- **Tối ưu hóa (Sử dụng mảng tần suất và kỹ thuật sàng):**
 - Vì giá trị $a_i \leq 10^6$, ta dùng mảng $cnt[x]$ để đếm số lần xuất hiện của giá trị x trong dãy.
 - Cặp chia hết có hai loại:
 - * Hai số bằng nhau ($a_i = a_j$): Với mỗi giá trị x xuất hiện $cnt[x]$ lần, số cặp là $\frac{cnt[x] \times (cnt[x] - 1)}{2}$.
 - * Một số là bội của số kia ($a_i = k \cdot a_j$ với $k > 1$): Duyệt x từ 1 đến 10^6 . Với mỗi x , duyệt các bội $y = 2x, 3x, \dots \leq 10^6$. Số cặp tạo bởi giá trị x và giá trị y là $cnt[x] \times cnt[y]$.
 - Độ phức tạp: $O(\max(a_i) \log(\max(a_i)) + n)$, đủ nhanh để chạy trong thời gian cho phép.

19.3.1 Subtask 1 ($n \leq 10^3$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; i++) cin >> a[i];

    long long count = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (a[i] % a[j] == 0 || a[j] % a[i] == 0) {
                count++;
            }
        }
    }
    cout << count;
    return 0;
}
```

19.3.2 Subtask 2 (Full test) $n, a_i \leq 10^6$

```
#include <bits/stdc++.h>
using namespace std;

int cnt[1000005];

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;
    int max_val = 0;
    for (int i = 0; i < n; i++) {
        int x;
        cin >> x;
        cnt[x]++;
        if (x > max_val) max_val = x;
    }

    long long ans = 0;

    for (int i = 1; i <= max_val; i++) {
        if (cnt[i] == 0) continue;

```

```

    ans += (long long)cnt[i] * (cnt[i] - 1) / 2;

    for (int j = 2 * i; j <= max_val; j += i) {
        if (cnt[j] > 0) {
            ans += (long long)cnt[i] * cnt[j];
        }
    }
}

cout << ans;

return 0;
}

```

19.4 Bài 4: Đoạn con lớn nhất

Tóm tắt bài toán

- Cho dãy A gồm n phần tử ($n \leq 10^6$).
- Cần tìm một đoạn con liên tiếp có độ dài len thỏa mãn $p \leq len \leq q$ sao cho tổng các phần tử trong đoạn đó là lớn nhất.

Phân tích bài toán → Chiến thuật ban đầu → Tối ưu hóa

- **Phân tích:** Tổng đoạn con từ i đến j có thể tính nhanh bằng mảng cộng dồn $S[j] - S[i - 1]$ với $S[k] = \sum_{t=1}^k a_t$.
- **Chiến thuật ban đầu (Duyệt cạn - Subtask 1, 2):** Duyệt qua mọi cặp (i, j) sao cho $p \leq j - i + 1 \leq q$ và tính tổng. Độ phức tạp $O(n^2)$, chỉ chạy được với $n \leq 10^3$.
- **Tối ưu hóa 1 (Trường hợp $p = q$ - Subtask 4):** Sử dụng kỹ thuật cửa sổ trượt (sliding window) hoặc mảng cộng dồn để tính tổng mọi đoạn độ dài cố định p trong $O(n)$.
- **Tối ưu hóa 2 (Trường hợp tổng quát $n \leq 10^5, 10^6$):**
 - Cần tìm $\max(S[j] - S[i - 1])$ với $p \leq j - i + 1 \leq q$, tương đương $j - q + 1 \leq i \leq j - p + 1$.
 - Với mỗi j cố định, ta cần tìm $\min(S[i - 1])$ trong khoảng $i - 1 \in [j - q, j - p]$.
 - Đây là bài toán tìm giá trị nhỏ nhất trên đoạn trượt. Ta có thể sử dụng hàng đợi hai đầu (Deque) hoặc Segment Tree để duy trì giá trị nhỏ nhất của S trong cửa sổ có độ dài từ $j - q$ đến $j - p$.
 - Độ phức tạp đạt $O(n)$ khi dùng Deque.

19.4.1 Subtask 1 & 2 ($n \leq 10^3$)

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, p, q;
    cin >> n >> p >> q;
    vector<long long> a(n + 1);
    vector<long long> s(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        s[i] = s[i - 1] + a[i];
    }
    long long ans = -2e18;
    for (int i = 1; i <= n; i++) {
        for (int j = i + p - 1; j <= min(n, i + q - 1); j++) {
            ans = max(ans, s[j] - s[i - 1]);
        }
    }
    cout << ans;
    return 0;
}

```

19.4.2 Subtask 4 ($n > 10^5, p = q$)

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    int n, p, q;
    cin >> n >> p >> q;
    vector<long long> a(n + 1);
    vector<long long> s(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        s[i] = s[i - 1] + a[i];
    }
    long long ans = -2e18;
    for (int j = p; j <= n; j++) {
        ans = max(ans, s[j] - s[j - p]);
    }
    cout << ans;
    return 0;
}

```

19.4.3 Subtask 3 (Full test $n = 10^5$ hoặc 10^6 dùng Deque)

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    int n, p, q;
    cin >> n >> p >> q;
    vector<long long> s(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        long long x;
        cin >> x;
        s[i] = s[i - 1] + x;
    }
    long long ans = -2e18;
    deque<int> dq;
    for (int j = p; j <= n; j++) {
        while (!dq.empty() && dq.front() < j - q) {
            dq.pop_front();
        }
        int idx = j - p;
        while (!dq.empty() && s[dq.back()] >= s[idx]) {
            dq.pop_back();
        }
        dq.push_back(idx);
        ans = max(ans, s[j] - s[dq.front()]);
    }
    cout << ans;
    return 0;
}

```

20 Hải Dương**20.1 Bài 1: Hành trình ngắn nhất****Tóm tắt bài toán:**

- Các địa điểm: Nhà (N), Trường (T), Thư viện (V).
- Các tuyến đường: $N \leftrightarrow T$ (độ dài a), $N \leftrightarrow V$ (độ dài b), $T \leftrightarrow V$ (độ dài c).
- Lộ trình bắt buộc: Nhà $\rightarrow$ Trường $\rightarrow$ Thư viện $\rightarrow$ Nhà.
- Yêu cầu: Tìm tổng độ dài nhỏ nhất để đi qua đủ các địa điểm theo đúng thứ tự đó, sử dụng các con đường có sẵn.

Phân tích bài toán:

- Chiến thuật ban đầu:
 - Để hoàn thành hành trình $Nhà \rightarrow Tr\ddot{o}ng \rightarrow Th\ddot{o}v\ddot{i}n \rightarrow Nh\ddot{a}$, An cần thực hiện 3 chặng:
 - Chặng 1 (Nhà $\rightarrow$ Trường): Có thể đi trực tiếp đường a , hoặc đi vòng qua thư viện ($b + c$). Độ dài ngắn nhất là $\min(a, b + c)$.
 - Chặng 2 (Trường $\rightarrow$ Thư viện): Có thể đi trực tiếp đường c , hoặc đi vòng qua nhà ($a + b$). Độ dài ngắn nhất là $\min(c, a + b)$.
 - Chặng 3 (Thư viện $\rightarrow$ Nhà): Có thể đi trực tiếp đường b , hoặc đi vòng qua trường ($c + a$). Độ dài ngắn nhất là $\min(b, c + a)$.
- Tối ưu hóa:
 - Tổng hành trình ngắn nhất sẽ là: $S = \min(a, b + c) + \min(c, a + b) + \min(b, c + a)$.
 - Do giới hạn $a, b, c \leq 10^6$, kết quả tối đa có thể đạt 3×10^6 , hoàn toàn nằm trong kiểu dữ liệu int hoặc long long.
 - Độ phức tạp: $O(1)$.

20.1.1 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    long long a, b, c;
    cin >> a >> b >> c;

    long long chang1 = min(a, b + c);
    long long chang2 = min(c, a + b);
    long long chang3 = min(b, c + a);

    cout << chang1 + chang2 + chang3;

    return 0;
}
```

20.2 Bài 2: Máy rút tiền**Tóm tắt bài toán:**

- Dữ liệu ban đầu: Máy ATM có tổng số tiền là T và có n người xếp hàng.
- Cơ chế: Duyệt lần lượt từng người i với số tiền muốn rút là a_i .

- Điều kiện:
 - Nếu $a_i \leq$ số tiền còn lại trong máy: Rút thành công, số tiền trong máy giảm đi a_i .
 - Nếu $a_i >$ số tiền còn lại trong máy: Rút thất bại, người đó "buồn bã bỏ đi", số tiền trong máy giữ nguyên.
- Yêu cầu: Đếm tổng số người rút thất bại.

Phân tích bài toán:

- Chiến thuật: Đây là một bài toán mô phỏng đơn giản. Ta cần một biến để lưu trữ số dư hiện tại trong máy (khởi tạo bằng T) và một biến đếm số người thất bại.
- Phân tích: Duyệt một vòng lặp từ người đầu tiên đến người cuối cùng. Tại mỗi bước, so sánh yêu cầu rút tiền với số dư hiện tại.
- Tối ưu hóa: Với $n = 10^5$, thuật toán có độ phức tạp $O(n)$ là hoàn toàn tối ưu và chạy trong thời gian cho phép. Cần lưu ý sử dụng kiểu dữ liệu long long cho các biến liên quan đến số tiền vì T, a_i lên đến 10^9 .

20.2.1 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    long long T;
    cin >> n >> T;

    int dem_buon = 0;
    for (int i = 0; i < n; i++) {
        long long a;
        cin >> a;
        if (a <= T) {
            T = T - a;
        } else {
            dem_buon++;
        }
    }

    cout << dem_buon;

    return 0;
}
```

20.3 Bài 3: Dãy con lớn nhất

Tóm tắt bài toán:

- Dữ liệu: Cho dãy a gồm n số nguyên $a_1, a_2, \dots, a_n$.
- Yêu cầu: Tìm một dãy con gồm các phần tử liên tiếp $a_i, a_{i+1}, \dots, a_j$ ($1 \leq i \leq j \leq n$) sao cho tổng các phần tử trong dãy con đó là lớn nhất.
- Kết quả: In ra giá trị tổng lớn nhất tìm được.
- Ràng buộc:
 - $n \leq 10^6$.
 - $|a_i| \leq 10^3$.

Phân tích bài toán:

- Chiến thuật ban đầu (Duyệt trâu):
 - Duyệt qua mọi cặp chỉ số (i, j) để xác định đoạn con. Với mỗi cặp, ta tính tổng các phần tử từ i đến j .
 - Độ phức tạp: $O(n^3)$.
 - Cải tiến nhẹ: Sử dụng mảng cộng dồn (prefix sum) để tính tổng đoạn con trong $O(1)$. Độ phức tạp giảm xuống $O(n^2)$.
 - Cách này chỉ phù hợp với $n \leq 500$ (Subtask 1) hoặc $n \leq 5000$ (Subtask 2).
- Tối ưu hóa (Thuật toán Kadane):
 - Ta nhận thấy rằng khi duyệt đến phần tử a_i , tổng lớn nhất của dãy con kết thúc tại a_i sẽ bằng $\max(a_i, \text{tổng lớn nhất kết thúc tại } i-1 + a_i)$.
 - Duyệt qua dãy số đúng một lần, duy trì một biến lưu tổng hiện tại ($current_sum$) và một biến lưu tổng lớn nhất toàn cục (max_sum).
 - Nếu $current_sum$ trở nên nhỏ hơn 0, ta đặt lại $current_sum = 0$ để bắt đầu một dãy con mới.
 - Độ phức tạp: $O(n)$, đáp ứng hoàn toàn giới hạn $n = 10^6$ của Subtask 3.

20.3.1 Subtask 1 & 2 ($n \leq 5000$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
```

```
vector<long long> a(n);
for (int i = 0; i < n; i++) {
    cin >> a[i];
}

long long mx = -1e18;
for (int i = 0; i < n; i++) {
    long long sum = 0;
    for (int j = i; j < n; j++) {
        sum += a[j];
        if (sum > mx) {
            mx = sum;
        }
    }
}

cout << mx;
return 0;
}
```

20.3.2 Subtask 3 ($n \leq 10^6$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;

    long long mx = -1e18;
    long long sum = 0;

    for (int i = 0; i < n; i++) {
        long long x;
        cin >> x;
        sum += x;

        if (sum > mx) {
            mx = sum;
        }

        if (sum < 0) {
            sum = 0;
        }
    }
}
```

```
cout << mx;
return 0;
}
```

20.4 Bài 4: Lát gạch

Tóm tắt bài toán:

- Dữ liệu: Cho T căn phòng hình chữ nhật kích thước $m \times n$.
- Yêu cầu: Tìm số lượng các số nguyên dương k sao cho viên gạch hình vuông cạnh k có thể lát kín nền nhà mà không cần cắt gạch.
- Điều kiện: Để lát kín nền $m \times n$ bằng gạch cạnh k mà không phải cắt, thì k phải là ước chung của cả m và n .
- Mục tiêu: Đếm số lượng ước chung của m và n .

Phân tích bài toán:

- Chiến thuật ban đầu:
 - Tìm ước chung lớn nhất của m và n , ký hiệu là $G = \gcd(m, n)$.
 - Mọi ước chung của m và n đều là ước của G .
 - Duyệt từ 1 đến G , nếu G chia hết cho i thì tăng biến đếm.
 - Đánh giá: Với G lên đến 5×10^6 , việc duyệt T lần (2×10^5) sẽ gây quá thời gian (TLE).
- Tối ưu hóa:
 - Để đếm số ước của một số G , ta chỉ cần duyệt từ 1 đến $\sqrt{G}$.
 - Với mỗi i là ước của G ($G \bmod i = 0$), ta có ước tương ứng là G/i . Nếu $i \times i \neq G$, ta tính là 2 ước, ngược lại tính là 1 ước.
 - Độ phức tạp cho mỗi căn phòng: $O(\sqrt{\gcd(m, n)})$.
 - Tổng độ phức tạp: $O(T \times \sqrt{\max(m, n)})$. Với $m, n \leq 5 \times 10^6$, $\sqrt{G} \approx 2236$. Tổng cộng khoảng 4.4×10^8 phép tính, có thể chạy kịp trong giới hạn thời gian nếu tối ưu nhập xuất.

20.4.1 Subtask 1 ($m, n \leq 100$)

```

#include <bits/stdc++.h>
using namespace std;

int gcd(int a, int b) {
    while (b != 0) {
        int r = a % b;
        a = b;
        b = r;
    }
    return a;
}

int main() {
    int t, m, n;
    cin >> t;
    while (t--) {
        cin >> m >> n;
        int g = gcd(m, n);
        int dem = 0;
        for (int i = 1; i <= g; i++) {
            if (g % i == 0) {
                dem++;
            }
        }
        cout << dem << endl;
    }
    return 0;
}

```

20.4.2 Subtask full ($T \leq 2 \times 10^5, m, n \leq 5 \times 10^6$)

```

#include <bits/stdc++.h>
using namespace std;

int gcd(int a, int b) {
    while (b != 0) {
        a %= b;
        swap(a, b);
    }
    return a;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int t;
    cin >> t;
}

```



```

while (t--) {
    int m, n;
    cin >> m >> n;
    int g = gcd(m, n);
    int dem = 0;
    for (int i = 1; i * i <= g; i++) {
        if (g % i == 0) {
            if (i * i == g) {
                dem += 1;
            } else {
                dem += 2;
            }
        }
    }
    cout << dem << "\n";
}
return 0;
}

```

20.5 Bài 5: Số đẹp

Tóm tắt bài toán:

- Định nghĩa: Một số là "số đẹp" nếu nó chia hết cho M .
- Yêu cầu: Đếm số cặp (x, y) thỏa mãn $1 \leq x < y \leq n$ và $(x + y) \pmod{M} = 0$.
- Ràng buộc: $n, M \leq 10^9$.

Phân tích bài toán:

- Chiến thuật ban đầu (Duyệt trâu):
 - Sử dụng hai vòng lặp lồng nhau duyệt mọi cặp (x, y) với $1 \leq x < y \leq n$.
 - Kiểm tra nếu $(x + y) \pmod{M} == 0$ thì tăng biến đếm.
 - Đánh giá: Độ phức tạp $O(n^2)$. Chỉ chạy được với Subtask 1 ($n \leq 5000$).
- Tối ưu hóa (Sử dụng mảng đếm số dư):
 - Ta nhận thấy $(x + y) \pmod{M} = 0 \Leftrightarrow (x \pmod{M} + y \pmod{M}) \pmod{M} = 0$.
 - Đếm số lượng các số từ $1 \dots n$ có cùng số dư khi chia cho M . Gọi $count[r]$ là số lượng số $i \in [1, n]$ sao cho $i \pmod{M} = r$.
 - Với mỗi số dư r , số dư tương ứng cần tìm để tổng chia hết cho M là $r' = (M - r) \pmod{M}$.
 - Các trường hợp đặc biệt:

* Nếu $r = 0$: Cần chọn 2 số cùng chia hết cho M . Số cách là $C_{count[0]}^2 = \frac{count[0] \times (count[0] - 1)}{2}$.

- * Nếu $r = M - r$ (tức $r = M/2$ khi M chẵn): Số cách là $C_{count[M/2]}^2 = \frac{count[M/2] \times (count[M/2] - 1)}{2}$.
- * Các trường hợp khác: Số cách là $count[r] \times count[M - r]$. Ta chỉ duyệt r từ $1 \dots \lfloor (M - 1)/2 \rfloor$.
- Đánh giá: Độ phức tạp $O(M)$. Chạy tốt với Subtask 2 và 3 ($M \leq 10^6$).
- Để tối ưu lên $M \leq 10^9$, ta không thể đếm bằng mảng $1 \dots M$. Ta cần công thức $O(1)$.
 - * Ta đặt: $n = qM + s$ ($0 \leq s < M$)
 - * Khi đó:
 - Số dư 0 xuất hiện q lần.
 - Các số dư $1 \dots s$ xuất hiện $q + 1$ lần.
 - Các số dư $s + 1 \dots M - 1$ xuất hiện q lần.
 - * Ta đếm theo các cặp số dư có tổng chia hết cho M .
 - * Với cách làm này, đáp án được tính trực tiếp:
 - Cặp dư 0: $\binom{q}{2}$
 - Các cặp $(r, M - r)$: gom theo từng cặp dư đối xứng
 - Nếu M chẵn thì còn phải tách riêng dư $M/2$
 - * **Công thức cuối:**
 - * Đặt: $P = \lfloor \frac{M-1}{2} \rfloor$
 - * Số cặp đối xứng mà cả hai dư đều nằm trong đoạn $1 \dots s$ là: $aa = \max(0, P - \max(1, M - s) + 1)$.
 - * Nếu M lẻ: $ans = \binom{q}{2} + Pq^2 + qs + aa$
 - * Nếu M chẵn:
 - * Gọi:

$$\delta = \begin{cases} 1 & \text{nếu } s \geq \frac{M}{2} \\ 0 & \text{ngược lại} \end{cases}$$
 - * thì: $ans = \binom{q}{2} + Pq^2 + q(s - \delta) + aa + \binom{q+\delta}{2}$

20.5.1 Subtask 1 ($n \leq 5000$)

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    long long n, m;
    cin >> n >> m;
    long long dem = 0;
    for (int i = 1; i < n; i++) {
        for (int j = i + 1; j <= n; j++) {
            if ((i + j) % m == 0) {
```

```

        dem++;
    }
}
cout << dem;
return 0;
}

```

20.5.2 Subtask 2 & 3 ($M \leq 10^6$)

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    long long n, m;
    cin >> n >> m;
    vector<long long> count(m, 0);
    long long q = n / m;
    int r = n % m;
    for (int i = 0; i < m; i++) count[i] = q;
    for (int i = 1; i <= r; i++) count[i]++;

    long long res = 0;
    res += count[0] * (count[0] - 1) / 2;
    for (int i = 1; i < (m + 1) / 2; i++) {
        res += count[i] * count[m - i];
    }
    if (m % 2 == 0) {
        res += count[m / 2] * (count[m / 2] - 1) / 2;
    }
    cout << res;
    return 0;
}

```

20.5.3 Subtask 4 ($M \leq 10^9$)

```

#include <bits/stdc++.h>
using namespace std;

using int64 = long long;
using i128 = __int128_t;

i128 C2(int64 x) {
    return (i128)x * (x - 1) / 2;
}

int main() {

```

```

ios::sync_with_stdio(false);
cin.tie(nullptr);

int64 n, M;
cin >> n >> M;

int64 q = n / M;
int64 s = n % M;

i128 ans = C2(q);

int64 P = (M - 1) / 2;
int64 l = max(1LL, M - s);
int64 aa = 0;
if (l <= P) aa = P - l + 1;

if (M % 2 == 1) {
    ans += (i128)P * q * q;
    ans += (i128)q * s;
    ans += aa;
} else {
    int64 delta = (s >= M / 2 ? 1LL : 0LL);

    ans += (i128)P * q * q;
    ans += (i128)q * (s - delta);
    ans += aa;
    ans += C2(q + delta);
}

cout << (long long)ans;
return 0;
}

```

21 Hải Phòng

21.1 Bài 1: Hình vành khăn

Tóm tắt bài toán:

- Dữ liệu: Hai bán kính R_1, R_2 của hai đường tròn đồng tâm tại $(0, 0)$.
- Điều kiện điểm nguyên (x, y) nằm trong hoặc trên biên đường tròn bán kính R :
 $x^2 + y^2 \leq R^2$.
- Yêu cầu: Đếm số cặp số nguyên (x, y) thỏa mãn hệ thức: $R_1^2 < x^2 + y^2 \leq R_2^2$.

Phân tích bài toán:

- Chiến thuật ban đầu (Duyệt trâu - Subtask 1):
 - * Duyệt x từ $-R_2$ đến R_2 , và y từ $-R_2$ đến R_2 .
 - * Kiểm tra điều kiện $x^2 + y^2 > R_1^2$ và $x^2 + y^2 \leq R_2^2$.
 - * Độ phức tạp: $O(R_2^2)$, chỉ chạy được với $R_2 \leq 10^3$.
- Cải tiến (Duyệt một biến - Subtask 2):
 - * Với mỗi giá trị của x cố định, ta cần tìm số lượng y nguyên sao cho $y^2 \leq R_2^2 - x^2$ và $y^2 > R_1^2 - x^2$.
 - * Số lượng y thỏa mãn $y^2 \leq K$ là $2 \cdot \lfloor \sqrt{K} \rfloor + 1$ (nếu $K \geq 0$).
 - * Gọi $f(R)$ là số điểm nguyên trong hình tròn bán kính R : $f(R) = \sum_{x=-R}^R (2 \cdot \lfloor \sqrt{R^2 - x^2} \rfloor + 1)$.
 - * Kết quả là $f(R_2) - f(R_1)$.
 - * Độ phức tạp: $O(R_2)$, chạy được với $R_2 \leq 10^6$.
- Tối ưu hóa ($O(\sqrt{R_2})$) hoặc thuật toán nâng cao - Subtask 3):
 - * Với $R_2 \leq 10^8$, độ phức tạp $O(R_2)$ vẫn có thể bị quá thời gian nếu không tối ưu. Ta có thể sử dụng tính đối xứng của 4 phần tư hoặc các thuật toán đếm điểm nguyên nâng cao hơn để giảm số lần lặp.

21.1.1 Subtask 1 ($R_2 \leq 10^3$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    long long r1, r2;
    cin >> r1 >> r2;
    long long dem = 0;
    for (long long x = -r2; x <= r2; x++) {
        for (long long y = -r2; y <= r2; y++) {
            long long kc = x * x + y * y;
            if (kc <= r2 * r2 && kc > r1 * r1) {
                dem++;
            }
        }
    }
    cout << dem;
    return 0;
}
```

21.1.2 Subtask 2 ($R_2 \leq 10^6$)

```
#include <bits/stdc++.h>
using namespace std;

long long dem_diem(long long r) {
    long long dem = 0;
    for (long long x = -r; x <= r; x++) {
        long long y_max = sqrt(r * r - x * x);
        dem += (2 * y_max + 1);
    }
    return dem;
}

int main() {
    long long r1, r2;
    cin >> r1 >> r2;
    cout << dem_diem(r2) - dem_diem(r1);
    return 0;
}
```

21.1.3 Subtask 3 (Full - Tối ưu đối xứng)

```
#include <bits/stdc++.h>
using namespace std;

long long isqrt(long long x) {
    long long r = sqrtl(x);
    while ((r + 1) * (r + 1) <= x) r++;
    while (r * r > x) r--;
    return r;
}

long long dem_diem(long long r) {
    long long res = 0;
    for (long long x = 0; x <= r; x++) {
        long long y = isqrt(r * r - x * x);
        if (x == 0) res += 2 * y + 1;
        else res += 2 * (2 * y + 1);
    }
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    long long r1, r2;
    cin >> r1 >> r2;
```

```
cout << dem_diem(r2) - dem_diem(r1);
return 0;
}
```

21.2 Bài 2: Xâu con

Tóm tắt bài toán:

- **Yêu cầu:** Cho xâu X độ dài n chỉ chứa các ký tự $\{A, U, G, C\}$. Tìm độ dài lớn nhất của một xâu con liên tiếp $[l, r]$ sao cho tồn tại ít nhất một loại ký tự xuất hiện đúng số lần bằng $\lfloor \frac{r-l+1}{2} \rfloor + 1$.
- **Input:**
 - * Dòng 1: Số nguyên dương n ($1 \leq n \leq 2 \times 10^5$).
 - * Dòng 2: Xâu X gồm n ký tự.
- **Output:** Một số nguyên duy nhất là độ dài lớn nhất tìm được.
- **Ràng buộc:**
 - * Subtask 1 (30%): $n \leq 10^2$.
 - * Subtask 2 (50%): $n \leq 10^3$.
 - * Subtask 3 (20%): $n \leq 2 \times 10^5$.

Phân tích bài toán:

- **Phân tích điều kiện:** Gọi L là độ dài xâu con, $L = r - l + 1$. Số lần xuất hiện cần thiết của một ký tự là $S = \lfloor \frac{L}{2} \rfloor + 1$.
 - * Nếu L lẻ ($L = 2k + 1$), thì $S = k + 1$. Khi đó $S > \frac{L}{2}$.
 - * Nếu L chẵn ($L = 2k$), thì $S = k + 1$. Khi đó $S > \frac{L}{2}$.

Như vậy, điều kiện bài toán tương đương với việc tìm xâu con dài nhất mà có một ký tự chiếm đa số (tổng số lần xuất hiện lớn hơn một nửa độ dài xâu).

- **Chiến thuật ban đầu ($O(n^2)$ hoặc $O(n^3)$):** Duyệt mọi cặp (l, r) đại diện cho xâu con. Với mỗi xâu con, ta đếm tần suất của 4 loại ký tự. Nếu có ký tự nào thỏa mãn số lượng bằng $\lfloor \frac{r-l+1}{2} \rfloor + 1$, ta cập nhật kết quả cực đại. Cách này phù hợp cho Subtask 1 và 2.
- **Tối ưu hóa ($O(n)$):** Xét riêng từng ký tự $c \in \{A, U, G, C\}$. Với mỗi ký tự, ta quy ước: nếu gặp c thì tính là $+1$, nếu gặp ký tự khác thì tính là -1 . Tuy nhiên, điều kiện bài toán là "đúng bằng" chứ không phải "lớn hơn hoặc bằng".

Gọi $cnt[i]$ là số lượng ký tự c trong đoạn $[1, i]$. Điều kiện bài toán cho đoạn $[l, r]$ độ dài L là:

- * Trường hợp L lẻ: $2 \times (cnt[r] - cnt[l - 1]) = L + 1 \Leftrightarrow 2 \times cnt[r] - r = 2 \times cnt[l - 1] - (l - 1) + 1$.

* Trường hợp L chẵn: $2 \times (cnt[r] - cnt[l - 1]) = L + 2 \Leftrightarrow 2 \times cnt[r] - r = 2 \times cnt[l - 1] - (l - 1) + 2$.

Đặt $V[i] = 2 \times cnt[i] - i$. Ta cần tìm i, j ($j < i$) sao cho $V[i] - V[j] = 1$ hoặc $V[i] - V[j] = 2$. Để $L = i - j$ lớn nhất, với mỗi giá trị $V[i]$, ta cần tìm vị trí j nhỏ nhất mà $V[j] = V[i] - 1$ hoặc $V[j] = V[i] - 2$.

21.2.1 Subtask 1 & 2 ($n \leq 10^3$)

```
#include <bits/stdc++.h>
using namespace std;

int n;
string s;
int a[200005][4];

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    cin >> n >> s;
    for (int i = 1; i <= n; i++) {
        for (int j = 0; j < 4; j++) a[i][j] = a[i - 1][j];
        if (s[i - 1] == 'A') a[i][0]++;
        else if (s[i - 1] == 'U') a[i][1]++;
        else if (s[i - 1] == 'G') a[i][2]++;
        else if (s[i - 1] == 'C') a[i][3]++;
    }
    int res = 0;
    for (int l = 1; l <= n; l++) {
        for (int r = l; r <= n; r++) {
            int len = r - l + 1;
            int target = len / 2 + 1;
            for (int j = 0; j < 4; j++) {
                if (a[r][j] - a[l - 1][j] == target) {
                    res = max(res, len);
                }
            }
        }
    }
    cout << res;
    return 0;
}
```

21.2.2 Subtask 3 ($n \leq 2 \times 10^5$)

```
#include <bits/stdc++.h>
using namespace std;
```



```

int n;
string s;
int first[800005];
const int offset = 400000;

int solve(char c) {
    for (int i = 0; i <= 800000; i++) first[i] = -2;
    int cur_cnt = 0;
    int ans = 0;
    first[0 + offset] = 0;
    for (int i = 1; i <= n; i++) {
        if (s[i - 1] == c) cur_cnt++;
        int val = 2 * cur_cnt - i;
        if (first[val - 1 + offset] != -2) {
            ans = max(ans, i - first[val - 1 + offset]);
        }
        if (first[val - 2 + offset] != -2) {
            ans = max(ans, i - first[val - 2 + offset]);
        }
        if (first[val + offset] == -2) {
            first[val + offset] = i;
        }
    }
    return ans;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    cin >> n >> s;
    int res = 0;
    char target[] = {'A', 'U', 'G', 'C'};
    for (int i = 0; i < 4; i++) {
        res = max(res, solve(target[i]));
    }
    cout << res;
    return 0;
}

```

21.3 Bài 3: Dãy số

Tóm tắt bài toán:

- Cho mảng a gồm n phần tử phân biệt và q truy vấn (i, j) .
- Tìm đoạn $[L, R]$ rộng nhất chứa i và j sao cho mọi phần tử trong đoạn đều nằm trong khoảng giá trị $[\min(a_i, a_j), \max(a_i, a_j)]$.

- Điều kiện cần: Bản thân mọi phần tử từ a_i đến a_j cũng phải nằm trong khoảng này. Nếu không, in ra -1 .

Phân tích bài toán:

- Chiến thuật ban đầu (Brute force):
 - * Với mỗi truy vấn (i, j) , gọi $V_{min} = \min(a_i, a_j)$ và $V_{max} = \max(a_i, a_j)$.
 - * Kiểm tra xem đoạn $[i, j]$ có hợp lệ không (tất cả $a_k \in [V_{min}, V_{max}]$). Nếu không, trả về -1 .
 - * Nếu hợp lệ, mở rộng L sang trái từ i và R sang phải từ j xa nhất có thể miễn là $a_L, a_R \in [V_{min}, V_{max}]$.
 - * Độ phức tạp: $O(q \times n)$, phù hợp cho Subtask 1, 2, 3.
- Tối ưu hóa (Subtask 4):
 - * Để kiểm tra nhanh min/max trên đoạn, ta sử dụng Sparse Table hoặc Segment Tree. Việc kiểm tra đoạn $[i, j]$ hợp lệ trở thành: $\min(a_{i...j}) == V_{min}$ và $\max(a_{i...j}) == V_{max}$.
 - * Để tìm L và R nhanh chóng, ta nhận thấy tính chất đơn điệu: nếu đoạn $[L, i]$ hợp lệ thì $[L + 1, i]$ cũng hợp lệ. Ta có thể sử dụng Binary Search (Tìm kiếm nhị phân) kết hợp với Sparse Table để tìm biên giới xa nhất.
 - * Biên bên trái L là vị trí nhỏ nhất trong $[1, i]$ sao cho $\min(a_{L...i}) \geq V_{min}$ và $\max(a_{L...i}) \leq V_{max}$.
 - * Biên bên phải R là vị trí lớn nhất trong $[j, n]$ sao cho $\min(a_{j...R}) \geq V_{min}$ và $\max(a_{j...R}) \leq V_{max}$.
 - * Độ phức tạp: $O(n \log n)$ để xây dựng bảng và $O(q \log n)$ để xử lý truy vấn.

21.3.1 Subtask 1, 2, 3 ($O(q \times n)$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, q;
    cin >> n >> q;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];

    while (q--) {
        int i, j;
        cin >> i >> j;
        int v_min = min(a[i], a[j]);
        int v_max = max(a[i], a[j]);
```

```

    bool ok = true;
    for (int k = i; k <= j; k++) {
        if (a[k] < v_min || a[k] > v_max) {
            ok = false;
            break;
        }
    }

    if (!ok) {
        cout << -1 << endl;
    } else {
        int L = i, R = j;
        while (L > 1 && a[L - 1] >= v_min && a[L - 1] <= v_max) L--;
        while (R < n && a[R + 1] >= v_min && a[R + 1] <= v_max) R++;
        cout << L << " " << R << endl;
    }
}

return 0;
}

```

21.3.2 Subtask 4 ($O((n+q)\log n)$)

```

#include <bits/stdc++.h>
using namespace std;

const int MAXN = 100005;
const int LOG = 18;
int a[MAXN];
int st_min[MAXN][LOG], st_max[MAXN][LOG];
int lg2[MAXN];

void build_st(int n) {
    lg2[1] = 0;
    for (int i = 2; i <= n; i++) lg2[i] = lg2[i / 2] + 1;
    for (int i = 1; i <= n; i++) {
        st_min[i][0] = st_max[i][0] = a[i];
    }
    for (int j = 1; j < LOG; j++) {
        for (int i = 1; i + (1 << j) - 1 <= n; i++) {
            st_min[i][j] = min(st_min[i][j - 1], st_min[i + (1 << (j - 1))][j - 1]);
            st_max[i][j] = max(st_max[i][j - 1], st_max[i + (1 << (j - 1))][j - 1]);
        }
    }
}

int query_min(int L, int R) {
    int j = lg2[R - L + 1];
    return min(st_min[L][j], st_min[R - (1 << j) + 1][j]);
}

```

```

}

int query_max(int L, int R) {
    int j = lg2[R - L + 1];
    return max(st_max[L][j], st_max[R - (1 << j) + 1][j]);
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    int n, q;
    cin >> n >> q;
    for (int i = 1; i <= n; i++) cin >> a[i];
    build_st(n);

    while (q--) {
        int i, j;
        cin >> i >> j;
        int v_min = min(a[i], a[j]);
        int v_max = max(a[i], a[j]);

        if (query_min(i, j) != v_min || query_max(i, j) != v_max) {
            cout << -1 << "\n";
            continue;
        }

        int L = i, low = 1, high = i;
        while (low <= high) {
            int mid = (low + high) / 2;
            if (query_min(mid, i) >= v_min && query_max(mid, i) <= v_max) {
                L = mid;
                high = mid - 1;
            } else {
                low = mid + 1;
            }
        }

        int R = j, low2 = j, high2 = n;
        while (low2 <= high2) {
            int mid = (low2 + high2) / 2;
            if (query_min(j, mid) >= v_min && query_max(j, mid) <= v_max) {
                R = mid;
                low2 = mid + 1;
            } else {
                high2 = mid - 1;
            }
        }

        cout << L << " " << R << "\n";
    }
}

```

```

    }
    return 0;
}

```

21.4 Bài 4: Robot di chuyển

Tóm tắt bài toán:

- Cho bảng số $m \times n$. Với mỗi ô (i, j) , ta tính trọng số $W(i, j)$ là tổng các ước của $a_{i,j}$ (ngoại trừ chính nó).
- Tìm đường đi từ $(1, 1)$ đến (m, n) chỉ bằng cách đi xuống hoặc sang phải sao cho tổng trọng số trên các ô đã đi qua là nhỏ nhất.

Phân tích bài toán:

- Chiến thuật ban đầu:
 - * Tính tổng các ước (hàm $S(a)$): Với mỗi $a_{i,j}$, ta có thể tính tổng ước bằng cách duyệt từ 1 đến $\sqrt{a_{i,j}}$. Nếu x là ước của $a_{i,j}$, thì $a_{i,j}/x$ cũng là ước.
 - * Quy hoạch động (Dynamic Programming): Gọi $dp[i][j]$ là chi phí nhỏ nhất để đi từ $(1, 1)$ đến (i, j) .
 - * Công thức truy hồi: $dp[i][j] = W(i, j) + \min(dp[i-1][j], dp[i][j-1])$.
 - * Cơ sở: $dp[1][1] = W(1, 1)$.
- Tối ưu hóa:
 - * Với $a_{i,j} \leq 10^6$, ta có thể dùng sàng số nguyên tố để tính nhanh tổng ước cho mọi giá trị trong bảng.
 - * Với $a_{i,j} \leq 10^9$, ta tính trực tiếp $S(a)$ trong $O(\sqrt{a})$.
 - * Đặc biệt ở Subtask 5, các giá trị $a_{i,j}$ rất lớn nhưng phạm vi biến thiên nhỏ. Tuy nhiên, do $m, n \leq 10^3$, ta vẫn có thể tính trực tiếp tổng ước cho từng ô hoặc dùng Map để lưu kết quả các số đã tính rồi nhằm tránh tính lại nhiều lần.

21.4.1 Subtask 1, 2, 3, 4 ($a_{i,j} \leq 10^6$)

```

#include <bits/stdc++.h>
using namespace std;

long long sum_div[1000001];

void sieve(int max_v) {
    for (int i = 1; i <= max_v; i++) {
        for (int j = 2 * i; j <= max_v; j += i) {
            sum_div[j] += i;
        }
    }
}

```

```

    }
}

int main() {
    int m, n;
    cin >> m >> n;
    vector<vector<int>> a(m + 1, vector<int>(n + 1));
    int max_val = 0;
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            cin >> a[i][j];
            max_val = max(max_val, a[i][j]);
        }
    }

    sieve(max_val);

    vector<vector<long long>> dp(m + 1, vector<long long>(n + 1));
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            long long weight = sum_div[a[i][j]];
            if (i == 1 && j == 1) dp[i][j] = weight;
            else if (i == 1) dp[i][j] = dp[i][j - 1] + weight;
            else if (j == 1) dp[i][j] = dp[i - 1][j] + weight;
            else dp[i][j] = min(dp[i - 1][j], dp[i][j - 1]) + weight;
        }
    }

    cout << dp[m][n];
    return 0;
}

```

21.4.2 Subtask 5 ($a_{i,j}$ lớn, dùng tối ưu tính tổng ước bằng sàng trên đoạn)

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int m, n;
    cin >> m >> n;

    vector<vector<int>> a(m, vector<int>(n));
    int L = 1e9, R = 0;

```

```

for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        cin >> a[i][j];
        L = min(L, a[i][j]);
        R = max(R, a[i][j]);
    }
}

int len = R - L + 1;
vector<long long> sum(len, 0);

for (int i = 0; i < len; i++) {
    if (L + i > 1) sum[i] = 1;
}

for (int d = 2; d <= R; d++) {
    int start = max(2 * d, ((L + d - 1) / d) * d);
    for (int x = start; x <= R; x += d) {
        sum[x - L] += d;
    }
}

vector<vector<long long>> dp(m, vector<long long>(n));

for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        long long w = sum[a[i][j] - L];

        if (i == 0 && j == 0) dp[i][j] = w;
        else if (i == 0) dp[i][j] = dp[i][j - 1] + w;
        else if (j == 0) dp[i][j] = dp[i - 1][j] + w;
        else dp[i][j] = min(dp[i - 1][j], dp[i][j - 1]) + w;
    }
}

cout << dp[m - 1][n - 1];
}

```

22 Hưng Yên

22.1 Bài 1

22.1.1 Subtask 1+2+3

Gọi a là chiều dài và b là chiều rộng của hình chữ nhật ($a \geq b$). Ta có nửa chu vi là $S = P/2$. Hiệu hai cạnh theo dữ liệu hệ thống cung cấp là $D = a - b$.

Áp dụng công thức tìm hai số khi biết tổng và hiệu:

- Chiều dài: $a = (S + D)/2$
- Chiều rộng: $b = (S - D)/2$
- Diện tích: $Area = a \times b$

22.1.2 Code mẫu tham khảo

```
#include<bits/stdc++.h>
using namespace std;
#define int long long

main() {
    int P , D;
    cin >> P >> D;
    int S = P / 2;

    int a = (S + D) / 2;
    int b = (S - D) / 2;

    int area = a * b;

    cout << area << "\n";
}
```

22.2 Bài 2

22.2.1 Subtask 1

Với ràng buộc $A = 1$ và $B \leq 10^6$, học sinh chỉ cần sử dụng một vòng lặp for duyệt tuyến tính từ A đến B . Tại mỗi bước, dùng câu lệnh điều kiện để kiểm tra nếu số hiện tại chia hết cho 2 hoặc 3 thì tăng biến đếm lên 1. Độ phức tạp là $O(B)$, tốn khoảng 10^6 phép toán.

22.2.2 Subtask 2+3

Với ràng buộc mở rộng lên đến $B \leq 10^{18}$ và A là một số bất kỳ.

- Gọi $Count(X)$ là hàm đếm số lượng số chia hết cho 2 hoặc 3 trong đoạn từ 1 đến X . Số lượng các số thỏa mãn trong đoạn $[A, B]$ chính là kết quả của phép bù trừ: $Count(B) - Count(A - 1)$.

22.2.3 Code mẫu tham khảo

```
#include<bits/stdc++.h>
using namespace std;
#define int long long

int count_num(int x) {
    return x / 2 + x / 3 - x / 6;
}

main() {
    int a, b;
    cin >> a >> b;

    int ans = count_num(b) - count_num(a - 1);

    cout << ans << "\n";
}
```

22.3 Bài 3

22.3.1 Subtask 1+2

Với ràng buộc $n \leq 1000$, học sinh có thể sử dụng hai vòng lặp lồng nhau. Vòng lặp ngoài duyệt các vị trí bắt đầu của xâu con từ 0 đến $n - k$, vòng lặp trong tính tổng của k ký tự liên tiếp kể từ vị trí đó. Độ phức tạp là $O(n \times k)$.

22.3.2 Subtask 3

Với ràng buộc mở rộng lên đến $n \leq 200000$. Bài toán yêu cầu tối ưu bằng kỹ thuật của sổ trượt để đạt độ phức tạp $O(n)$:

- Tính tổng của cửa sổ đầu tiên gồm k ký tự từ vị trí 0 đến $k - 1$. Lưu giá trị này làm tổng hiện tại và khởi tạo nó cho kết quả lớn nhất.
- Trượt cửa sổ dần sang phải từng bước một. Thay vì tính lại từ đầu, tổng của cửa sổ mới sẽ bằng tổng hiện tại cộng thêm giá trị của ký tự mới đi vào cửa sổ, và trừ đi giá trị của ký tự vừa bị loại ra khỏi cửa sổ ở phía sau.
- Cập nhật lại giá trị tổng lớn nhất sau mỗi lần dịch chuyển.

22.3.3 Code mẫu tham khảo

```
#include<bits/stdc++.h>
using namespace std;
```

```
#define int long long

main() {
    int n, k;
    cin >> n >> k;
    string s;
    cin >> s;

    int current_sum = 0;
    for (int i = 0; i < k; i++)
        current_sum += s[i] - '0';

    int max_sum = current_sum;

    for (int i = k; i < n; i++)
    {
        current_sum = current_sum + (s[i] - '0') - (s[i - k] - '0');
        max_sum = max(max_sum, current_sum);
    }

    cout << max_sum << "\n";
}
```

22.4 Bài 4

22.4.1 Subtask 1

Với giới hạn dữ liệu nhỏ ($T \leq 100$), học sinh có thể sử dụng hai vòng lặp lồng nhau duyệt mọi cặp cuộc họp i và j bất kỳ. Nếu phát hiện thời gian của hai cuộc họp giao nhau, lập tức kết luận không thể tham gia tất cả. Độ phức tạp là $O(n^2)$.

22.4.2 Subtask 2+3

Với tổng số lượng phần tử lên tới $T \leq 200000$. Bài toán yêu cầu tối ưu bằng thuật toán sắp xếp kết hợp duyệt:

- Ghép thời gian bắt đầu A_i và thời gian kết thúc B_i thành một cặp (pair) dữ liệu gắn liền với nhau để đại diện cho một cuộc họp.
- Sắp xếp danh sách các cuộc họp theo thứ tự tăng dần của thời gian bắt đầu.
- Duyệt qua danh sách đã sắp xếp. Một lịch trình hợp lệ nếu thời gian bắt đầu của cuộc họp hiện tại luôn lớn hơn hoặc bằng thời gian kết thúc của cuộc họp liền trước nó. Nếu tồn tại bất kỳ cuộc họp nào vi phạm điều kiện này, lập tức in ra NO.

- Nếu duyệt qua toàn bộ lịch trình mà không có sự chồng chéo nào xảy ra, in ra YES.

22.4.3 Code mẫu tham khảo

```
#include<bits/stdc++.h>
using namespace std;
#define int long long

int a[200005], b[200005];
pair<int, int> p[200005];

main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int q;
    cin >> q;

    while (q--) {
        int n;
        cin >> n;

        for (int i = 0; i < n; i++) cin >> a[i];
        for (int i = 0; i < n; i++) cin >> b[i];

        for (int i = 0; i < n; i++) {
            p[i] = {a[i], b[i]};
        }

        sort(p, p + n);

        bool ok = true;
        for (int i = 1; i < n; i++) {
            if (p[i].first < p[i - 1].second) {
                ok = false;
                break;
            }
        }

        if (ok) cout << "YES\n";
        else cout << "NO\n";
    }
}
```

22.5 Bài 5

22.5.1 Subtask 1

Với ràng buộc nhỏ $m, n \leq 10^3$, bài toán có thể giải quyết bằng cách mô phỏng trực tiếp quá trình chia kẹo. Ta sử dụng một vòng lặp chạy m lần, mỗi lần duyệt qua mảng a để tìm ra phần tử nhỏ nhất đầu tiên nằm bên trái nhất và cộng thêm 1 cho nó. Thuật toán này có độ phức tạp $O(m \times n)$.

22.5.2 Subtask 2

Với ràng buộc mảng ban đầu có tất cả phần tử bằng nhau ($a_1 = a_2 = \dots = a_n$), quá trình tìm số nhỏ nhất sẽ xoay vòng đều. Ta không cần lặp hay mô phỏng mà dùng toán học cơ bản $O(N)$. Cụ thể, khi chia thêm m viên kẹo vào n hộp đang có số lượng bằng nhau, mỗi hộp chắc chắn sẽ được nhận thêm m/n viên. Lượng kẹo còn dư lại là $m \% n$ viên sẽ được rải đều từng viên một cho các hộp từ trái sang phải.

22.5.3 Subtask 3

Với ràng buộc không giới hạn, $m \leq 10^{18}$, ta bắt buộc phải sử dụng kỹ thuật chặt nhị phân kết quả để bao quát tất cả các trường hợp:

- Chặt nhị phân đi tìm mức kẹo V giá trị nhỏ nhất của mảng sau khi chia kẹo lớn nhất có thể.
- Hàm kiểm tra tính hợp lệ: Duyệt mảng, tính tổng số kẹo cần thêm vào các hộp có lượng kẹo $< V$. Nếu tổng cần thiết $\leq m$ thì mức V là khả thi.
- Sau khi tìm được V tối ưu, nâng tất cả các hộp $< V$ lên thành V và trừ đi lượng kẹo tương ứng. Lượng kẹo còn thừa lại (rem) sẽ được phân phát ưu tiên từ trái qua phải cho các hộp đang có đúng V viên.

22.5.4 Code mẫu tham khảo

```
#include<bits/stdc++.h>
using namespace std;
#define int long long

int n, m;
int a[10005];

bool check(int v) {
    int need = 0;
    for (int i = 1; i <= n; i++) {
        if (a[i] < v) {
```

```

        need += (v - a[i]);
        if (need > m) return false;
    }
}
return need <= m;
}

main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    cin >> n >> m;
    int min_a = 2e18;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        min_a = min(min_a, a[i]);
    }

    int l = min_a, r = 2e18, v = min_a;
    while (l <= r) {
        int mid = l + (r - l) / 2;
        if (check(mid)) {
            v = mid;
            l = mid + 1;
        } else {
            r = mid - 1;
        }
    }

    int used = 0;
    for (int i = 1; i <= n; i++) {
        if (a[i] < v) {
            used += (v - a[i]);
            a[i] = v;
        }
    }

    int rem = m - used;
    for (int i = 1; i <= n && rem > 0; i++) {
        if (a[i] == v) {
            a[i]++;
            rem--;
        }
    }

    cout << v << "\n";
    for (int i = 1; i <= n; i++) {
        cout << a[i] << (i == n ? "" : " ");
    }
}

```

```

    }
    cout << "\n";
}

```

23 Khánh Hòa

23.1 Câu 1

Tóm tắt bài toán:

- Một con thỏ đang ở vị trí x_1 và cần di chuyển đến vị trí của củ cà rốt tại x_2 ($1 \leq x_1 \leq x_2 \leq 10^{12}$).
- Mỗi bước nhảy, con thỏ có thể nhảy được tối đa a đơn vị độ dài ($1 \leq a \leq 10^3$).
- Yêu cầu: Tính số bước nhảy ít nhất để con thỏ từ vị trí x_1 tới được vị trí x_2 .

Phân tích bài toán:

- Chiến thuật ban đầu:
 - * Khoảng cách cần di chuyển là $D = x_2 - x_1$.
 - * Để số bước nhảy là ít nhất, con thỏ nên thực hiện các bước nhảy có độ dài lớn nhất là a .
 - * Số bước nhảy sẽ là kết quả của phép chia D cho a . Nếu D không chia hết cho a , con thỏ cần thêm 1 bước nhảy ngắn hơn để hoàn thành quãng đường.
- Tối ưu hóa:
 - * Công thức tính nhanh số bước nhảy là $\lceil (x_2 - x_1) / a \rceil$ hoặc sử dụng phép chia nguyên trong lập trình: $(x_2 - x_1 + a - 1) / a$.
 - * Lưu ý về kiểu dữ liệu: Do x_1, x_2 có thể lên tới 10^{12} , ta bắt buộc phải sử dụng kiểu dữ liệu số nguyên 64-bit (long long trong C++) để tránh tràn số.

23.1.1 Subtask 1 (80% số điểm: $x_2 \leq 10^6$)

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    long long x1, x2, a;
    cin >> x1 >> x2 >> a;
    long long d = x2 - x1;
    long long res = d / a;
    if (d % a != 0) {
        res++;
    }
}

```

```

    }
    cout << res;
    return 0;
}

```

23.1.2 Subtask 2 (20% số điểm: Không có ràng buộc gì thêm, $x_2 \leq 10^{12}$)

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    long long x1, x2, a;
    cin >> x1 >> x2 >> a;
    long long distance = x2 - x1;
    if (distance == 0) {
        cout << 0;
    } else {
        long long ans = (distance + a - 1) / a;
        cout << ans;
    }
    return 0;
}

```

23.2 Câu 2

Tóm tắt bài toán:

- Có n thẻ bài, thẻ thứ i ghi số a_i ($1 \leq n \leq 10^5, 1 \leq a_i \leq 10^9$).
- Có m học sinh lần lượt tham gia chọn thẻ bài ($1 \leq m \leq n$).
- Mỗi học sinh chọn hai tấm thẻ bất kỳ: tấm có số nhỏ hơn được để lại bàn, tấm có số lớn hơn được mang về.
- Nếu hai tấm thẻ có số bằng nhau, học sinh mang về một tấm bất kỳ và để lại tấm kia.
- Yêu cầu: Tính giá trị lớn nhất của tổng các số trên thẻ mà m học sinh mang về được là bao nhiêu?

Phân tích bài toán:

- Chiến thuật ban đầu:
 - * Mỗi lần một học sinh chọn hai thẻ, mục tiêu là để học sinh đó mang về thẻ có giá trị lớn nhất hiện có trên bàn.

- * Để tối ưu hóa tổng, học sinh thứ nhất nên chọn thẻ lớn nhất và một thẻ khác bất kỳ (tốt nhất là thẻ nhỏ nhất để giữ lại các thẻ lớn cho những học sinh sau).
 - * Sau khi học sinh mang thẻ lớn nhất đi, thẻ nhỏ hơn vẫn còn trên bàn để kết hợp với các thẻ khác.
- Tối ưu hóa:
- * Thực chất, mỗi học sinh tham gia sẽ lấy đi đúng một thẻ từ tập hợp ban đầu, và thẻ đó luôn là thẻ lớn hơn trong cặp được chọn.
 - * Để tổng m thẻ mang về là lớn nhất, ta chỉ cần chọn m thẻ có giá trị lớn nhất trong số n thẻ ban đầu.
 - * Chiến thuật: Sắp xếp dãy a theo thứ tự giảm dần và lấy tổng của m phần tử đầu tiên.
 - * Độ phức tạp: $O(n \log n)$ do thao tác sắp xếp. Với $n = 10^5$, thuật toán này hoàn toàn đáp ứng được thời gian 1 giây/test.
 - * Kiểu dữ liệu: Vì $a_i \leq 10^9$ và $m \leq 10^5$, tổng có thể lên tới 10^{14} , nên cần sử dụng kiểu long long trong C++ để lưu trữ kết quả.

23.2.1 Subtask 1 (50% số điểm: $m = 1$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    long long max_val = 0;
    for (int i = 0; i < n; i++) {
        long long x;
        cin >> x;
        if (x > max_val) {
            max_val = x;
        }
    }
    cout << max_val;
    return 0;
}
```

23.2.2 Subtask 2 (50% số điểm: Không có ràng buộc gì thêm)

```
#include <bits/stdc++.h>
using namespace std;
```



```
int main() {
    int n, m;
    cin >> n >> m;
    vector<int> a(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }
    sort(a.begin(), a.end(), greater<int>());
    long long s = 0;
    for (int i = 0; i < m; i++) {
        s += a[i];
    }
    cout << s;
    return 0;
}
```

23.3 Câu 3

Tóm tắt bài toán:

- Cho dãy A gồm n số nguyên dương $a_1, a_2, \dots, a_n$.
- Giới hạn: $n \leq 10^6$ và $a_i \leq 10^{18}$.
- Yêu cầu: Đếm xem trong dãy A có bao nhiêu phần tử có số lượng ước nguyên dương là một số lẻ.

Phân tích bài toán:

- Chiến thuật ban đầu:
 - * Một số nguyên dương x có các ước luôn đi thành từng cặp $(d, x/d)$. Ví dụ: số 12 có các cặp ước $(1, 12), (2, 6), (3, 4)$.
 - * Số lượng ước của x chỉ có thể là số lẻ khi và chỉ khi có một cặp ước mà hai thành phần trùng nhau, tức là $d = x/d \Rightarrow x = d^2$.
 - * Vậy, bài toán quy về việc: Đếm xem trong dãy A có bao nhiêu số là số chính phương.
- Tối ưu hóa:
 - * Để kiểm tra một số a_i có phải số chính phương hay không, ta tính $k = \text{round}(\sqrt{a_i})$. Sau đó kiểm tra nếu $k \times k = a_i$ thì a_i là số chính phương.
 - * Với $a_i \leq 10^{18}$, giá trị $\sqrt{a_i}$ có thể lên tới 10^9 , vẫn nằm trong phạm vi xử lý của kiểu dữ liệu số nguyên 64-bit.
 - * Sử dụng hàm `sqrtl` trong C++ để đảm bảo độ chính xác cho số thực lớn (long double) trước khi ép kiểu về nguyên.

23.3.1 Subtask 1 (50% số điểm: $n \leq 10^3, a_i \leq 10^3$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    int cnt = 0;
    for (int i = 0; i < n; i++) {
        int a;
        cin >> a;
        int uoc = 0;
        for (int j = 1; j <= a; j++) {
            if (a % j == 0) uoc++;
        }
        if (uoc % 2 != 0) cnt++;
    }
    cout << cnt;
    return 0;
}
```

23.3.2 Subtask 2 (30% số điểm: $n \leq 10^4, a_i \leq 10^6$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    int cnt = 0;
    for (int i = 0; i < n; i++) {
        int a;
        cin >> a;
        int can = sqrt(a);
        if (can * can == a) cnt++;
    }
    cout << cnt;
    return 0;
}
```

23.3.3 Subtask 3 (20% số điểm: $n \leq 10^6, a_i \leq 10^{18}$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
```

```
ios_base::sync_with_stdio(false);
cin.tie(NULL);
int n;
cin >> n;
int cnt = 0;
for (int i = 0; i < n; i++) {
    long long a;
    cin >> a;
    long long can = round(sqrtl(a));
    if (can * can == a) cnt++;
}
cout << cnt;
return 0;
}
```

23.4 Câu 4

Tóm tắt bài toán:

- Có n chú thỏ ($1 \leq n \leq 10^6$) cùng thu hoạch cà rốt.
- Chú thỏ thứ i bắt đầu tại vị trí x_i và thực hiện m_i bước nhảy, mỗi bước nhảy dài đúng k_i đơn vị độ dài.
- Cách thu hoạch của mỗi chú thỏ:
 - * Tại vị trí bắt đầu: Thu được x_i củ.
 - * Lần nhảy thứ 1: Thu được $x_i + k_i$ củ.
 - * Lần nhảy thứ t ($2 \leq t \leq m_i$): Thu hoạch được nhiều hơn k_i củ so với lần nhảy thứ $t - 1$.
- Yêu cầu: Tính tổng số cà rốt sau khi n chú thỏ thu hoạch xong, kết quả lấy dư cho $10^9 + 7$.

Phân tích bài toán:

- Chiến thuật ban đầu:
 - * Phân tích số cà rốt chú thỏ thứ i thu hoạch được qua từng bước:
 - Bước 0: x_i
 - Bước 1: $x_i + k_i$
 - Bước 2: $(x_i + k_i) + k_i = x_i + 2k_i$
 - ...
 - Bước m_i : $x_i + m_i k_i$
 - * Đây là một cấp số cộng với $u_1 = x_i$, công sai $d = k_i$ và có $m_i + 1$ số hạng.
 - * Tổng số cà rốt của một chú thỏ là: $S_i = \frac{(m_i + 1)(2x_i + m_i k_i)}{2}$.

- * Duyệt qua từng chú thỏ và cộng dồn các S_i . Độ phức tạp $O(n)$, tuy nhiên cần xử lý phép chia cho 2 trong modulo $10^9 + 7$.
- Tối ưu hóa:
 - * Vì $10^9 + 7$ là số nguyên tố, ta có thể sử dụng nghịch đảo modulo của 2 là 500000004 để thực hiện phép chia.
 - * Công thức tính nhanh cho mỗi thỏ: $S_i = (((m_i + 1) \pmod{M}) \times ((2x_i + m_i k_i) \pmod{M}) \times 500000004) \pmod{M}$.
 - * Do x_i, m_i, k_i lên đến 10^9 , cần sử dụng kiểu long long và thực hiện phép lấy dư liên tục để tránh tràn số.

23.4.1 Subtask 1 (60% số điểm: $n, x_i, m_i, k_i \leq 10^3$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    long long total = 0;
    long long MOD = 1e9 + 7;
    for (int i = 0; i < n; i++) {
        long long x, m, k;
        cin >> x >> m >> k;
        long long sum_rabbit = 0;
        for (int t = 0; t <= m; t++) {
            sum_rabbit = (sum_rabbit + (x + t * k)) % MOD;
        }
        total = (total + sum_rabbit) % MOD;
    }
    cout << total;
    return 0;
}
```

23.4.2 Subtask 2 (40% số điểm: Không có ràng buộc gì thêm)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    int n;
    cin >> n;
    long long tot = 0;
```

```

long long MOD = 1000000007;
long long inv2 = 5000000004;
for (int i = 0; i < n; i++) {
    long long x, m, k;
    cin >> x >> m >> k;
    long long cur = (m + 1) % MOD;
    long long sum = (2 * x % MOD + m % MOD * (k % MOD) % MOD) % MOD;
    long long sum_rabbit = (cur * sum % MOD) * inv2 % MOD;
    tot = (tot + sum_rabbit) % MOD;
}
cout << tot;
return 0;
}

```

23.5 Câu 5

Tóm tắt bài toán:

- Cho dãy số gồm n phần tử ($1 \leq n \leq 10^9$) được chia thành $\frac{n}{k}$ đoạn, mỗi đoạn có k số.
- Quy luật các đoạn:
 - * Nếu đoạn thứ i là đoạn lẻ ($i = 1, 3, 5, \dots$): Các số tăng dần từ $(i - 1) \times k + 1$ đến $i \times k$.
 - * Nếu đoạn thứ i là đoạn chẵn ($i = 2, 4, 6, \dots$): Các số giảm dần từ $i \times k$ về $(i - 1) \times k + 1$.
- Yêu cầu: Tính tổng các số ở vị trí từ l đến r trong dãy số đã cho ($1 \leq l \leq r \leq n \leq 10^9$).

Phân tích bài toán:

- Chiến thuật ban đầu:
 - * Xây dựng toàn bộ dãy số theo quy luật đã cho vào một mảng.
 - * Duyệt từ vị trí l đến r để tính tổng các phần tử trong mảng đó.
 - * Hạn chế: Cách này chỉ hoạt động khi $n \leq 10^6$ hoặc 10^7 do giới hạn bộ nhớ và thời gian. Với $n = 10^9$, không thể khởi tạo mảng.
- Tối ưu hóa:
 - * Nhận xét: Tổng từ l đến r có thể tính bằng $Sum(r) - Sum(l - 1)$, với $Sum(x)$ là tổng các số từ vị trí 1 đến x .
 - * Để tính $Sum(x)$:
 - Xác định số lượng đoạn đầy đủ trước x : $num_segments = x/k$.
 - Các đoạn đầy đủ này thực chất là các hoán vị của các số từ 1 đến $num_segments \times k$. Tổng của chúng là $\frac{(num_segments \times k) \times (num_segments \times k + 1)}{2}$.
 - Tính tổng phần dư ở đoạn thứ $i = num_segments + 1$:

- Vị trí của phần tử trong đoạn này chạy từ 1 đến $rem = x \pmod k$.
- Nếu i lẻ: Các giá trị là $(i-1)k+1, (i-1)k+2, \dots, (i-1)k+rem$.
- Nếu i chẵn: Các giá trị là $ik, ik-1, \dots, ik-(rem-1)$.
- * Sử dụng công thức cấp số cộng: $Tổng = \frac{nums \times (first + last)}{2}$.
- * Độ phức tạp: $O(1)$ cho mỗi truy vấn, xử lý được $n = 10^9$.

23.5.1 Subtask 1 & 2 ($n \leq 10^6$)

```
#include <bits/stdc++.h>
using namespace std;

long long n, k, l, r;

namespace sub1 {
    bool approved() {
        return (k == 1 || k == n);
    }

    void solve() {
        cout << (r + 1) * (r - l + 1) / 2;
    }
}

namespace sub2 {
    bool approved() {
        return n <= 1000000;
    }

    void solve() {
        long long ans = 0;
        long long cnt = l % k;
        long long sub = l % k - 1;

        bool check = ((l / k + (l % k != 0)) % 2 == 0);

        for (long long i = l; i <= r; i++) {
            if (!check) {
                ans += i;
                cnt++;
            } else {
                ans += i + k - 2 * sub - 1;
                cnt++;
                sub++;
            }
        }

        if (cnt == k + 1) {
            cnt = 1;
        }
    }
}
```

```

        check = !check;
        sub = 0;
    }
}

cout << ans;
}
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> k >> l >> r;

    if (sub1::approved()) {
        sub1::solve();
        return 0;
    }

    if (sub2::approved()) {
        sub2::solve();
        return 0;
    }

    return 0;
}

```

23.5.2 Subtask 3 ($n \leq 10^9$)

```

#include <bits/stdc++.h>
using namespace std;

long long n, k, l, r;

long long sum(long long l, long long r) {
    return (l + r) * (r - l + 1) / 2;
}

long long f(long long x) {
    if (x <= 0) return 0;

    long long div;
    if (x % k == 0) div = x / k;
    else div = x / k + 1;

    if (div % 2 == 1) {
        return sum(1, x);
    }
}

```

```

    } else {
        long long y = x - (div - 1) * k;
        return sum(1, (div - 1) * k) + sum(div * k - y + 1, div * k);
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> k >> l >> r;

    cout << f(r) - f(l - 1);

    return 0;
}

```

24 Kiên Giang

24.1 Bài 1: Tính tiền taxi

Tóm tắt bài toán

- Nhập vào quãng đường di chuyển n (km) và thời gian chờ t (giờ).
- Tính toán giá cước dựa trên các mốc quãng đường lũy tiến.
- Tính toán giá dịch vụ dựa trên các mốc thời gian chờ lũy tiến.
- Kết quả là tổng của hai loại chi phí trên.

Phân tích bài toán → Chiến thuật ban đầu → Tối ưu hóa

- **Phân tích:** Bài toán yêu cầu tính tiền theo phương pháp tính từng phần (lũy tiến). Ví dụ với quãng đường $n = 2$, tiền cước sẽ gồm 1 km đầu giá 25,000 và 1 km tiếp theo giá 15,700.
- **Chiến thuật ban đầu:** Sử dụng các câu lệnh điều kiện if - else để kiểm tra n và t thuộc khoảng nào rồi nhân với đơn giá tương ứng. Tuy nhiên, lưu ý đây là bảng giá tính theo từng đoạn (như tiền điện), không phải lấy tổng n nhân với một đơn giá duy nhất.
- **Tối ưu hóa:** Để code sạch sẽ và tránh sai sót, có thể chia nhỏ việc tính toán vào hai hàm riêng biệt là `tinhGiaCuoc(n)` và `tinhGiaDichVu(t)`. Do giới hạn n và t nhỏ ($n \leq 300, t \leq 24$), ta chỉ cần sử dụng các cấu trúc rẽ nhánh cơ bản để tính tổng tích lũy qua từng bậc thang giá.

24.1.1 Code mẫu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, t;
    cin >> n >> t;

    long long giaCuoc = 0;
    if (n > 0) {
        giaCuoc += 25000;
        n--;
    }
    if (n > 0) {
        int km = min(n, 11);
        giaCuoc += (long long)km * 15700;
        n -= km;
    }
    if (n > 0) {
        int km = min(n, 13);
        giaCuoc += (long long)km * 15000;
        n -= km;
    }
    if (n > 0) {
        int km = min(n, 75);
        giaCuoc += (long long)km * 14500;
        n -= km;
    }
    if (n > 0) {
        giaCuoc += (long long)n * 13000;
    }

    long long giaDichVu = 0;
    if (t > 0 and t < 2) {
        giaDichVu += 50000;
    }
    else if (t >= 2 and t <= 5) {
        int gio = t;
        giaDichVu += (long long)gio * 45000;
        t -= gio;
    }
    else if (t >= 6 and t <= 10) {
        int gio = t;
        giaDichVu += (long long)gio * 40000;
        t -= gio;
    }
    else if (t >= 11 and t <= 15) {
        int gio = t;
```

```

        giaDichVu += (long long)gio * 35000;
        t -= gio;
    }
    else {
        giaDichVu += (long long)t * 25000;
    }

    cout << giaCuoc + giaDichVu << endl;

    return 0;
}

```

24.2 Bài 2: Bộ ba số

Tóm tắt bài toán

- Đầu vào là một số n rất lớn (đến 255 chữ số), cần xử lý dưới dạng chuỗi (string).
- Bước 1: Loại bỏ các chữ số trùng lặp trong n , chỉ giữ lại lần xuất hiện đầu tiên của mỗi chữ số theo đúng thứ tự.
- Bước 2: Từ tập hợp các chữ số duy nhất thu được, tìm tất cả các bộ ba chữ số (a, b, c) sao cho $a = b + c$ và $b > c$.

Phân tích bài toán → Chiến thuật ban đầu → Tối ưu hóa

- **Xử lý chuỗi:** Vì số k lên đến 255, ta không thể dùng kiểu số nguyên thông thường. Duyệt chuỗi n từ trái sang phải, dùng một mảng đánh dấu hoặc kiểm tra sự tồn tại để xây dựng chuỗi n_{mii} gồm các chữ số không lặp lại.
- **Tìm bộ ba:** Sau khi có chuỗi n_{mii} , số lượng chữ số tối đa chỉ còn là 10 (từ 0 đến 9).
- **Chiến thuật ban đầu:** Duyệt 3 vòng lặp lồng nhau qua các vị trí trong chuỗi mới để chọn ra a, b, c . Kiểm tra điều kiện $a = b + c$ và $b > c$. Lưu ý rằng b và c có vai trò như nhau trong tổng nhưng điều kiện $b > c$ giúp định danh bộ ba duy nhất.
- **Tối ưu hóa:** Do số lượng chữ số sau khi lọc tối đa chỉ là 10, việc dùng 3 vòng lặp $O(10^3)$ là cực kỳ nhanh. Ta có thể lưu các chữ số vào một mảng để dễ truy xuất. Để in ra đúng định dạng sample (954, 853), ta cần duyệt a là chữ số lớn nhất trước, sau đó tìm cặp b, c .

24.2.1 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    string n;
    cin >> n;

    string a_str = "";
    bool seen[256] = {false};

    for (char c : n) {
        if (!seen[(int)c]) {
            a_str += c;
            seen[(int)c] = true;
        }
    }

    cout << a_str << endl;

    vector<int> digits;
    for (char c : a_str) {
        digits.push_back(c - '0');
    }

    sort(digits.begin(), digits.end(), greater<int>());

    vector<string> kq;
    int sz = digits.size();

    for (int i = 0; i < sz - 2; i++) {
        for (int j = i + 1; j < sz - 1; j++) {
            for (int k = j + 1; k < sz; k++) {
                if (digits[j] + digits[k] == digits[i]) {
                    string s = to_string(digits[i]) + to_string(digits[j]);
                    s += to_string(digits[k]);
                    kq.push_back(s);
                }
            }
        }
    }

    if (kq.empty()) {
        cout << "Khong ton tai";
    } else {
        for (int i = 0; i < kq.size(); i++) {
            cout << kq[i] << (i == kq.size() - 1 ? "" : " ");
        }
    }
}
```

```

    }

    return 0;
}

```

24.3 Bài 3: Bội số nguyên tố

Tóm tắt bài toán

- Đếm các số $x \in [1, n]$ sao cho tồn tại ít nhất một số a_i trong tập $\{a_1, a_2, \dots, a_k\}$ mà $x : a_i$.
- Giới hạn: n rất lớn (10^{18}), k nhỏ (20).

Phân tích bài toán → Chiến thuật ban đầu → Tối ưu hóa

- **Chiến thuật ban đầu:** Duyệt qua từng số từ 1 đến n , với mỗi số kiểm tra xem nó có chia hết cho bất kỳ a_i nào không. Tuy nhiên, vì $n = 10^{18}$, cách tiếp cận này sẽ bị quá thời gian.
- **Tối ưu hóa (Nguyên lý Bao hàm - Loại trừ):**
 - * Theo nguyên lý Bao hàm - Loại trừ (Inclusion-Exclusion Principle), để đếm các phần tử chia hết cho ít nhất một số trong tập cho trước, ta tính:

$$(\text{Tổng số chia hết cho từng } a_i) - (\text{Tổng số chia hết cho tích từng cặp } a_i, a_j) + (\text{Tổng số chia hết cho tích từng bộ ba } a_i, a_j, a_m) - \dots$$
 - * Số lượng số trong $[1, n]$ chia hết cho một số V là $\lfloor n/V \rfloor$.
 - * Do $k \leq 20$, số lượng tập con của các số nguyên tố là $2^k = 2^{20} \approx 10^6$. Ta có thể sử dụng đệ quy hoặc duyệt bit để tính toán.
 - * Lưu ý: Do n, a_i lớn, khi tính tích các số nguyên tố cần kiểm tra tràn số (overflow) trước khi chia. Nếu tích vượt quá n , giá trị $\lfloor n/\text{tích} \rfloor$ sẽ bằng 0.

24.3.1 Code mẫu:

```

#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

ll n, a[25], ans = 0;
int k;

void solve(int idx, ll count, ll lcm_val) {

```

```

    if (idx == k) {
        if (count == 0) return;
        if (count % 2 == 1) ans += n / lcm_val;
        else ans -= n / lcm_val;
        return;
    }

    solve(idx + 1, count, lcm_val);

    ll next_lcm = lcm_val;
    if (n / a[idx] >= next_lcm) {
        next_lcm *= a[idx];
        solve(idx + 1, count + 1, next_lcm);
    }
}

int main() {
    cin >> n >> k;
    for (int i = 0; i < k; i++) cin >> a[i];

    solve(0, 0, 1);

    if (ans == 0) cout << -1;
    else cout << ans;

    return 0;
}

```

24.4 Bài 4: Căn hộ

Tóm tắt bài toán

- Có n khách hàng với diện tích mong muốn a_i và m căn hộ hiện có với diện tích b_j .
- Một khách hàng i có thể nhận căn hộ j nếu $|a_i - b_j| \leq k$.
- Vì ưu tiên khách hàng đăng ký sớm nhất (theo thứ tự xuất hiện trong mảng a) và mỗi căn hộ chỉ bán cho một người, ta cần tìm cách ghép tối đa số khách hàng nhận được nhà.

Phân tích bài toán → Chiến thuật ban đầu → Tối ưu hóa

- **Phân tích:** Đây là bài toán tham lam (Greedy) kết hợp với sắp xếp. Để tối ưu hóa số lượng người nhận được nhà, ta nên xét những người có nhu cầu diện tích nhỏ nhất trước và ghép với những căn hộ có diện tích nhỏ nhất phù hợp.

– **Chiến thuật ban đầu:** Duyệt qua từng khách hàng, với mỗi khách hàng lại duyệt qua danh sách căn hộ để tìm căn phù hợp. Nếu căn hộ đó chưa được bán, ta đánh dấu đã bán và tăng biến đếm. Độ phức tạp là $O(n \times m)$.

– **Tối ưu hóa:**

- * Sắp xếp mảng diện tích mong muốn a và mảng căn hộ b theo thứ tự tăng dần.
- * Sử dụng kỹ thuật hai con trỏ (2 pointers): Con trỏ i trỏ vào khách hàng, con trỏ j trỏ vào căn hộ.
- * Nếu $b[j] < a[i] - k$: Căn hộ hiện tại quá nhỏ đối với khách hàng này, tăng j để xét căn hộ lớn hơn.
- * Nếu $b[j] > a[i] + k$: Căn hộ hiện tại quá lớn đối với khách hàng này, khách hàng này không thể lấy căn hộ nào nữa (vì mảng đã sắp xếp), tăng i để xét khách hàng tiếp theo.
- * Nếu $a[i] - k \leq b[j] \leq a[i] + k$: Khách hàng lấy được căn hộ, tăng cả i, j và biến đếm.
- * Độ phức tạp sau tối ưu: $O(n \log n + m \log m)$ do thao tác sắp xếp.

24.4.1 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    long long k;
    cin >> n >> m >> k;

    vector<long long> a(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    vector<long long> b(m);
    for (int i = 0; i < m; i++) {
        cin >> b[i];
    }

    sort(a.begin(), a.end());
    sort(b.begin(), b.end());

    int i = 0, j = 0, count = 0;
    while (i < n && j < m) {
```

```

        if (b[j] < a[i] - k) {
            j++;
        } else if (b[j] > a[i] + k) {
            i++;
        } else {
            count++;
            i++;
            j++;
        }
    }

    cout << count;

    return 0;
}

```

25 Kon Tum

25.1 Bài 1: Trả tiền

Tóm tắt bài toán

- Khách mua hàng hết M , đưa vào N . Số tiền thừa cần trả là $T = N - M$.
- Có 3 loại mệnh giá: 5, 2, 1 với số lượng không hạn chế.
- Yêu cầu: Tìm tổng số tờ tiền ít nhất để có tổng mệnh giá bằng T .

Phân tích bài toán

- **Chiến thuật ban đầu:** Đây là bài toán đổi tiền (Change Making Problem). Với bộ các mệnh giá {5, 2, 1}, chúng ta có thể áp dụng chiến thuật Tham lam (Greedy).
- **Cơ sở tham lam:** Để số tờ tiền là ít nhất, ta luôn ưu tiên sử dụng mệnh giá lớn nhất trước (5), sau đó đến các mệnh giá nhỏ hơn (2 rồi đến 1).
- **Tối ưu hóa:**
 - * Số tờ mệnh giá 5: $a = T/5$ (phần nguyên).
 - * Số tiền còn lại sau khi trả tờ 5: $T' = T \pmod{5}$.
 - * Số tờ mệnh giá 2: $b = T'/2$.
 - * Số tiền còn lại sau khi trả tờ 2: $c = T' \pmod{2}$ (đây cũng chính là số tờ mệnh giá 1).
 - * Kết quả cuối cùng là: $a + b + c$.

- **Độ phức tạp:** Thuật toán đạt độ phức tạp $O(1)$, đáp ứng hoàn hảo với dữ liệu $N, M \leq 10^9$.

25.1.1 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    long long M, N;
    cin >> M >> N;
    long long T = N - M;
    long long res = 0;

    res += T / 5;
    T %= 5;

    res += T / 2;
    T %= 2;

    res += T;

    cout << res;
    return 0;
}
```

25.2 Bài 2: Chia kẹo

Tóm tắt bài toán

- Cho n cái kẹo, chia đều cho k đứa trẻ sao cho mỗi đứa nhận x cái.
- Điều kiện: $k \cdot x = n$, với $k > 1$ (không chia cho duy nhất 1 đứa).
- Yêu cầu: Tìm giá trị x lớn nhất.

Phân tích bài toán

- **Chiến thuật ban đầu:** Vì x là số kẹo mỗi đứa trẻ nhận được và k là số trẻ em, ta có x và k đều là ước của n . Để x lớn nhất thì k phải là ước nhỏ nhất của n mà lớn hơn 1.
- **Tối ưu hóa:**
 - * Ước nhỏ nhất của n (khác 1) chính là ước nguyên tố nhỏ nhất của n . Gọi ước này là p .

- * Khi đó, số kẹo lớn nhất một đứa trẻ nhận được sẽ là $x = n/p$.
 - * Để tìm p , ta chỉ cần duyệt từ 2 đến $\sqrt{n}$. Nếu không tìm thấy ước nào trong khoảng này, thì n là số nguyên tố, và khi đó $p = n$.
- **Độ phức tạp:** Thuật toán có độ phức tạp $O(\sqrt{n})$, hoàn toàn đáp ứng được giới hạn $n \leq 10^9$ trong thời gian cho phép.

25.2.1 Subtask 1 ($n \leq 10^6$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    int p = n;
    for (int i = 2; i <= n; i++) {
        if (n % i == 0) {
            p = i;
            break;
        }
    }
    cout << n / p;
    return 0;
}
```

25.2.2 Subtask 2 ($n \leq 10^9$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    long long n;
    cin >> n;
    long long p = n;
    for (long long i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            p = i;
            break;
        }
    }
    cout << n / p;
    return 0;
}
```

25.3 Bài 3: Số nguyên tố lớn nhất

Tóm tắt bài toán

- Trích xuất các cụm chữ số liên tiếp từ xâu S (loại bỏ số 0 vô nghĩa ở đầu).
- Trong các số trích xuất được, tìm số là số nguyên tố và có giá trị lớn nhất.
- Giới hạn: Xâu dài 10^5 , mỗi số không quá 9 chữ số (có thể nằm trong kiểu *int* hoặc *long long*).

Phân tích bài toán

- **Chiến thuật ban đầu:** Duyệt qua từng ký tự của xâu. Nếu gặp ký tự số, ta bắt đầu ghép thành một chuỗi con chứa toàn chữ số cho đến khi gặp ký tự chữ cái. Chuyển chuỗi con này sang số nguyên, kiểm tra tính nguyên tố và cập nhật giá trị cực đại.
- **Xử lý số 0 vô nghĩa:** Khi trích xuất một dãy số như "0065", ta cần chuyển nó thành số 65. Thư viện chuẩn hoặc cách nhân dồn ($val = val * 10 + digit$) sẽ tự động xử lý điều này.
- **Kiểm tra số nguyên tố:** Vì mỗi số tối đa 9 chữ số (khoảng 10^9), ta sử dụng thuật toán kiểm tra tính nguyên tố cơ bản với độ phức tạp $O(\sqrt{V})$ (với V là giá trị của số).
- **Tối ưu hóa:**
 - * Sử dụng biến *maxP* khởi tạo bằng -1 .
 - * Chỉ kiểm tra nguyên tố cho những số lớn hơn *maxP* hiện tại để tiết kiệm thời gian.

25.3.1 Subtask 1 ($|S| \leq 255$, số ≤ 5 chữ số)

```
#include <bits/stdc++.h>
using namespace std;

bool isPrime(int n) {
    if (n < 2) return false;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}

int main() {
```

```

string s;
cin >> s;
int maxP = -1;
int n = s.length();
for (int i = 0; i < n; i++) {
    if (isdigit(s[i])) {
        int val = 0;
        while (i < n && isdigit(s[i])) {
            val = val * 10 + (s[i] - '0');
            i++;
        }
        if (isPrime(val)) {
            if (val > maxP) maxP = val;
        }
    }
}
cout << maxP;
return 0;
}

```

25.3.2 Subtask 2 ($|S| \leq 10^5$, số ≤ 9 chữ số)

```

#include <bits/stdc++.h>
using namespace std;

bool check(long long n) {
    if (n < 2) return false;
    for (long long i = 2; i * i <= n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    string s;
    cin >> s;
    long long maxP = -1;
    int n = s.length();
    for (int i = 0; i < n; i++) {
        if (s[i] >= '0' && s[i] <= '9') {
            long long val = 0;
            while (i < n && s[i] >= '0' && s[i] <= '9') {
                val = val * 10 + (s[i] - '0');
                i++;
            }
            if (val > maxP) {

```

```

        if (check(val)) {
            maxP = val;
        }
    }
}
cout << maxP;
return 0;
}

```

25.4 Bài 4: Sơn tường

Tóm tắt bài toán

- Bức tường có độ dài M . Có N đoạn $[a_i, b_i]$ đã được sơn.
- Cần tính số đơn vị mét trong đoạn $[1, M]$ không nằm trong bất kỳ đoạn $[a_i, b_i]$ nào.
- Lưu ý: Bài toán tính trên các đơn vị mét (rời rạc).

Phân tích bài toán

- **Chiến thuật ban đầu (Dành cho M nhỏ):** Tạo một mảng đánh dấu có kích thước M . Với mỗi robot sơn đoạn $[a_i, b_i]$, ta duyệt từ a_i đến b_i và đánh dấu các vị trí này đã sơn. Cuối cùng đếm các vị trí chưa đánh dấu.
- **Tối ưu hóa (Dành cho M lớn):** Sử dụng kỹ thuật Hợp nhất các khoảng (Interval Merging).
 - * Bước 1: Lưu N đoạn sơn vào một danh sách và sắp xếp chúng theo thứ tự tăng dần của điểm đầu a_i .
 - * Bước 2: Duyệt qua danh sách đã sắp xếp để hợp nhất các đoạn chồng lấn hoặc liên tiếp nhau thành các đoạn lớn duy nhất. Ví dụ $[2, 4], [3, 6], [4, 7]$ sẽ hợp nhất thành $[2, 7]$.
 - * Bước 3: Sau khi hợp nhất, ta có các đoạn $[L_1, R_1], [L_2, R_2], \dots$ đôi một không giao nhau. Tổng số mét đã sơn là $\sum (R_i - L_i + 1)$.
 - * Bước 4: Số mét chưa sơn = M - Tổng số mét đã sơn.
- **Độ phức tạp:** $O(N \log N)$ do cần sắp xếp các đoạn, phù hợp với $N = 10^5$ và $M = 10^9$.

25.4.1 Subtask 1 & 2 ($M \leq 10^6, N \leq 10^3$)

```
#include <bits/stdc++.h>
using namespace std;

int d[1000005];

int main() {
    int m, n;
    cin >> m >> n;
    for (int i = 0; i < n; i++) {
        int a, b;
        cin >> a >> b;
        for (int j = a; j <= b; j++) {
            d[j] = 1;
        }
    }
    int cnt = 0;
    for (int i = 1; i <= m; i++) {
        if (d[i] == 0) cnt++;
    }
    cout << cnt;
    return 0;
}
```

25.4.2 Subtask 3 (Tối ưu cho $M \leq 10^9, N \leq 10^5$)

```
#include <bits/stdc++.h>
using namespace std;

struct Doan {
    int a, b;
};

bool compareDoan(Doan x, Doan y) {
    if (x.a != y.a) return x.a < y.a;
    return x.b < y.b;
}

int main() {
    long long m;
    int n;
    cin >> m >> n;
    vector<Doan> v(n);
    for (int i = 0; i < n; i++) {
        cin >> v[i].a >> v[i].b;
    }
}
```

```

sort(v.begin(), v.end(), compareDoan);

long long totalPainted = 0;
if (n > 0) {
    int currentA = v[0].a;
    int currentB = v[0].b;

    for (int i = 1; i < n; i++) {
        if (v[i].a <= currentB + 0) {
            currentB = max(currentB, v[i].b);
        } else {
            totalPainted += (currentB - currentA + 1);
            currentA = v[i].a;
            currentB = v[i].b;
        }
    }
    totalPainted += (currentB - currentA + 1);
}

cout << m - totalPainted;
return 0;
}

```

26 Lâm Đồng

26.1 Bài 1: Chia hết

Tóm tắt bài toán:

- Tìm số tự nhiên k lớn nhất thỏa mãn đồng thời hai điều kiện:
 - * $k < m$.
 - * $k \vdots n$.
- Giới hạn dữ liệu: $m, n \leq 10^6$.

Phân tích bài toán:

- Chiến thuật ban đầu (Duyệt ngược):
 - * Bắt đầu thử các giá trị k từ $m - 1$ và giảm dần về 0.
 - * Kiểm tra xem giá trị nào đầu tiên chia hết cho n thì đó là kết quả cần tìm.
 - * Độ phức tạp: $O(m/n)$, trong trường hợp xấu nhất ($n = 1$) có thể duyệt đến 10^6 bước.

– Tối ưu hóa (Toán học):

- * Ta cần tìm bội lớn nhất của n mà vẫn bé hơn m .
- * Xét số $m - 1$:
 - Nếu $(m - 1)$ chia hết cho n , kết quả là $m - 1$.
 - Nếu không, kết quả sẽ là phần nguyên của phép chia $(m - 1)$ cho n rồi nhân ngược lại với n .
- * Công thức tổng quát: $k = \lfloor \frac{m-1}{n} \rfloor \times n$.
- * Độ phức tạp: $O(1)$, thực hiện tính toán ngay lập tức.

26.1.1 Code mẫu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int m, n;
    cin >> m >> n;
    int k = ((m - 1) / n) * n;
    cout << k;
    return 0;
}
```

26.2 Bài 2: Chi phí

Tóm tắt bài toán:

- Rô bốt sản xuất d sản phẩm, mỗi sản phẩm mất c phút. Tổng thời gian hoạt động là $T_{active} = d \times c$.
- Tổng thời gian cho phép là e phút. Thời gian rô bốt ở trạng thái chờ là $T_{wait} = e - T_{active}$.
- Chi phí hoạt động mỗi phút là a , chi phí chờ mỗi phút là b .
- Yêu cầu: Tính tổng chi phí $Total = (T_{active} \times a) + (T_{wait} \times b)$.

Phân tích bài toán:

- Chiến thuật ban đầu:
 - * Tính thời gian làm việc: $work_time = d \times c$.
 - * Tính thời gian chờ: $idle_time = e - work_time$.

* Tính tổng tiền: $total = (work_time \times a) + (idle_time \times b)$.

– Tối ưu hóa & Lưu ý:

- * Các giá trị a, b, c, d, e có thể lên đến 10^9 .
- * Phép nhân $d \times c$ hoặc $work_time \times a$ có thể lên tới 10^{18} , vượt quá giới hạn của kiểu dữ liệu int (32-bit).
- * Cần sử dụng kiểu dữ liệu long long trong C++ để lưu trữ các giá trị trung gian và kết quả cuối cùng nhằm tránh tràn số.
- * Độ phức tạp thuật toán là $O(1)$.

26.2.1 Code mẫu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    long long a, b, c, d, e;
    cin >> a >> b >> c >> d >> e;

    long long work_time = d * c;
    long long idle_time = e - work_time;

    long long total_cost = (work_time * a) + (idle_time * b);

    cout << total_cost;

    return 0;
}
```

26.3 Bài 3: Mã số

Tóm tắt bài toán:

- Cho dãy n số nguyên dương $a_1, a_2, \dots, a_n$.
- Cần chọn ra hai chỉ số i, j sao cho $1 \leq i < j \leq n$.
- Ghép hai số a_i và a_j thành một số mới bằng cách viết a_i trước, a_j sau.
- Yêu cầu: Tìm số ghép được có giá trị lớn nhất.

Phân tích bài toán:

- Chiến thuật ban đầu:

- * Duyệt mọi cặp (i, j) với $i < j$, ghép chúng lại và so sánh để tìm giá trị lớn nhất.
- * Độ phức tạp: $O(n^2)$. Với $n = 10^6$, cách này sẽ bị quá thời gian (TLE).

– Tối ưu hóa:

- * Nhận xét: Để số ghép là lớn nhất, ta cần ưu tiên số a_i lớn nhất ở phía trước và số a_j lớn nhất ở phía sau. Tuy nhiên, thứ tự $i < j$ là bắt buộc.
- * Ta có thể sử dụng mảng phụ để lưu giá trị lớn nhất từ phải sang trái. Gọi $MaxSuffix[j]$ là giá trị lớn nhất trong các số từ a_j đến a_n .
- * Duyệt i từ 1 đến $n-1$: Với mỗi a_i , số tốt nhất để ghép với nó là $MaxSuffix[i+1]$.
- * Xử lý số lớn: Vì mỗi số có thể đến 10^{12} , khi ghép lại có thể lên đến 24 chữ số. Ta không thể dùng kiểu long long (64-bit, tối đa khoảng 19 chữ số). Phải sử dụng kiểu dữ liệu chuỗi (string) để ghép và so sánh.
- * So sánh chuỗi số: Hai chuỗi đại diện cho số, chuỗi nào dài hơn thì lớn hơn; nếu độ dài bằng nhau thì so sánh theo thứ tự từ điển.

26.3.1 Subtask ($n \leq 10^3$ - Duyệt $O(n^2)$)

```
#include <bits/stdc++.h>
using namespace std;

bool isGreater(string s1, string s2) {
    if (s1.length() > s2.length()) return true;
    if (s1.length() < s2.length()) return false;
    return s1 > s2;
}

int main() {
    int n;
    cin >> n;
    vector<string> a(n);
    for (int i = 0; i < n; i++) cin >> a[i];

    string res = "";
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            string combined = a[i] + a[j];
            if (res == "" || isGreater(combined, res)) {
                res = combined;
            }
        }
    }
}
```

```

    cout << res;
    return 0;
}

```

26.3.2 Code tối ưu ($n \leq 10^6$ - Độ phức tạp $O(n)$)

```

#include <bits/stdc++.h>
using namespace std;

bool isGreater(string s1, string s2) {
    if (s1.length() > s2.length()) return true;
    if (s1.length() < s2.length()) return false;
    return s1 > s2;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;
    vector<string> a(n);
    for (int i = 0; i < n; i++) cin >> a[i];

    vector<string> max_suffix(n);
    max_suffix[n - 1] = a[n - 1];
    for (int i = n - 2; i >= 0; i--) {
        if (isGreater(a[i], max_suffix[i + 1])) {
            max_suffix[i] = a[i];
        } else {
            max_suffix[i] = max_suffix[i + 1];
        }
    }

    string best = "";
    for (int i = 0; i < n - 1; i++) {
        string current = a[i] + max_suffix[i + 1];
        if (best == "" || isGreater(current, best)) {
            best = current;
        }
    }

    cout << best;
    return 0;
}

```

26.4 Bài 4: Rô bốt

Tóm tắt bài toán:

- Mỗi rô bốt i tương ứng với một đoạn thẳng trên trục số từ $X_i - L_i$ đến $X_i + L_i$.
- Hai rô bốt va chạm nếu phạm vi hoạt động của chúng giao nhau.
- Yêu cầu: Chọn ra tập hợp các đoạn thẳng sao cho không có hai đoạn nào giao nhau và số lượng đoạn thẳng chọn được là lớn nhất.

Phân tích bài toán:

- Chiến thuật ban đầu:
 - * Đây là bài toán kinh điển: "Lập lịch hội trường" hoặc "Chọn tập các đoạn thẳng không giao nhau tối đại".
 - * Mỗi rô bốt được coi là một công việc có thời gian bắt đầu $S_i = X_i - L_i$ và thời gian kết thúc $E_i = X_i + L_i$.
- Tối ưu hóa (Thuật toán tham lam):
 - * Bước 1: Sắp xếp tất cả các rô bốt theo thứ tự tăng dần của điểm kết thúc E_i . Nếu hai rô bốt có cùng điểm kết thúc, có thể sắp xếp tùy ý (hoặc theo điểm bắt đầu).
 - * Bước 2: Duyệt qua danh sách đã sắp xếp. Luôn ưu tiên chọn rô bốt kết thúc sớm nhất để dành nhiều không gian nhất cho các rô bốt phía sau.
 - * Bước 3: Khi xét rô bốt tiếp theo, nếu điểm bắt đầu của nó không nhỏ hơn điểm kết thúc của rô bốt vừa chọn trước đó ($S_{curr} \geq E_{prev}$), ta chọn nó và cập nhật điểm kết thúc mới.
 - * Độ phức tạp: $O(n \log n)$ do thao tác sắp xếp. Với $n = 10^6$, thuật toán này hoàn toàn đáp ứng được giới hạn thời gian 1 giây.

26.4.1 Code mẫu

```
#include <bits/stdc++.h>
using namespace std;

struct Robot {
    long long start, end;
};

bool compareRobots(Robot a, Robot b) {
    return a.end < b.end;
```

```

}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;

    vector<Robot> robots(n);
    for (int i = 0; i < n; i++) {
        long long x, l;
        cin >> x >> l;
        robots[i].start = x - l;
        robots[i].end = x + l;
    }

    sort(robots.begin(), robots.end(), compareRobots);

    int count = 0;
    long long last_end = -2000000000000000000LL;

    for (int i = 0; i < n; i++) {
        if (robots[i].start >= last_end) {
            count++;
            last_end = robots[i].end;
        }
    }

    cout << count;

    return 0;
}

```

27 Lạng Sơn

27.1 Bài 1: TÍNH TỔNG (SUM)

Tóm tắt đề bài: Cho số nguyên dương n ($0 < n < 10^8$). Tính tổng tất cả các số lẻ nằm trong đoạn từ 1 đến n . Dữ liệu đọc từ tệp SUM.INP và ghi ra tệp SUM.OUT.

Phân tích thuật toán:

- Cách tiếp cận ngây thơ là dùng một vòng lặp chạy từ 1 đến n , mỗi bước cộng thêm 2 (duyệt qua các số lẻ). Thuật toán này có độ phức tạp $O(n)$, tuy nhiên với $n = 10^8$, nó có thể mất khoảng 0.1 đến 0.2 giây.

- Để tối ưu tuyệt đối về $O(1)$, ta sử dụng tính chất Toán học: **Tổng của k số lẻ đầu tiên luôn bằng k^2 .**
- Số lượng các số lẻ từ 1 đến n được xác định bằng công thức: $k = \lfloor \frac{n+1}{2} \rfloor$.
- Do đó, tổng cần tìm là: $S = k \times k$.
- **Lưu ý dữ liệu:** Với $n = 10^8$, ta có $k = 5 \cdot 10^7$. Suy ra tổng lớn nhất có thể đạt là $k^2 = 2.5 \cdot 10^{15}$. Giá trị này vượt xa giới hạn của số nguyên int 32-bit ($2 \cdot 10^9$), nên bắt buộc phải ép kiểu về số nguyên 64-bit (long long).

27.1.1 Code mẫu (Toán học tối ưu $O(1)$)

```
#include <bits/stdc++.h>
#define int long long

using namespace std;

int n;
main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);

    cin >> n;

    int k = (n + 1) / 2;
    int ans = k * k;

    cout << ans << "\n";

    return 0;
}
```

27.2 Bài 2: ƯỚC SỐ (UOCSO)

Tóm tắt đề bài: Cho số nguyên dương n ($0 < n < 10^8$). Yêu cầu:

1. Đếm tổng số lượng các ước nguyên dương của n .
2. Tìm ước nguyên tố lớn nhất của n .

Dữ liệu đọc từ tệp UOCSO.INP và ghi ra tệp UOCSO.OUT.

Phân tích thuật toán: Với giới hạn $n < 10^8$, ta có thể giải quyết bài toán với độ phức tạp $O(\sqrt{n})$ bằng cách duyệt qua các giá trị từ 1 đến $\sqrt{n}$, cụ thể tối đa chỉ khoảng 10^4 phép lặp, hoàn toàn tối ưu và chạy trong tích tắc.

- **Đếm số lượng ước:** Duyệt i từ 1 đến $\sqrt{n}$. Nếu $n \pmod i == 0$, ta tìm được 2 ước là i và n/i . Cần cẩn thận trường hợp $i \times i = n$ (số chính phương) thì chỉ đếm 1 lần để tránh trùng lặp.
- **Tìm ước nguyên tố lớn nhất:** Sử dụng thuật toán phân tích ra thừa số nguyên tố. Khởi tạo một biến $m = n$. Duyệt i từ 2 đến $\sqrt{m}$. Nếu m chia hết cho i , ta lưu i là ước nguyên tố lớn nhất hiện tại, đồng thời chia m cho i liên tục cho đến khi không còn chia hết (để loại bỏ hoàn toàn các bội của số nguyên tố này). Sau khi vòng lặp kết thúc, nếu $m > 1$ thì bản thân m còn lại chính là một số nguyên tố và cũng là ước nguyên tố lớn nhất cần tìm.

27.2.1 Code mẫu

```
#include<bits/stdc++.h>
#define int long long
using namespace std;

int n, m, dem, mx = -1;

main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    cin >> n;

    for (int i = 1; i * i <= n; i++) {
        if (n % i == 0) {
            dem++;
            if (i * i != n) dem++;
        }
    }

    m = n;

    for (int i = 2; i * i <= m; i++) {
        if (m % i == 0) {
            mx = i;
            while (m % i == 0) m /= i;
        }
    }

    if (m > 1) mx = m;

    cout << dem << '\n' << mx;
}
```

27.3 Bài 3: QUAY THƯỜNG (THUONG)

Tóm tắt đề bài: Siêu thị có n mặt hàng, mỗi mặt hàng i có mã từ 1 đến n với giá tiền tương ứng là A_i . Có m khách hàng, mỗi người bốc ngẫu nhiên một mã dự thưởng B_j ($1 \leq B_j \leq n$). Minh bốc trúng mã k . Yêu cầu:

1. Tìm số tiền thưởng Minh nhận được (chính là A_k).
2. Tìm tổng số tiền lớn nhất mà siêu thị phải trả cho **một loại mã** dự thưởng (bằng số người bốc trúng mã đó $\times$ giá tiền của mã đó).

Phân tích thuật toán:

- **Yêu cầu 1:** Chỉ cần in ra A_k .
- **Yêu cầu 2:** Ta cần đếm số lần xuất hiện của từng mã dự thưởng. Vì mã dự thưởng có giá trị từ 1 đến n và $n \leq 10^6$, ta có thể sử dụng một mảng đếm tần số (mảng phân phối) cnt có kích thước $n + 1$.
- Duyệt qua danh sách mã dự thưởng B của m khách hàng: nếu khách hàng bốc trúng mã x , ta tăng cnt[x] lên 1.
- Sau khi đếm xong, ta duyệt qua tất cả các mã từ 1 đến n . Tổng tiền phải trả cho mã i sẽ là cnt[i] $\times$ A_i . Ta cập nhật giá trị lớn nhất trong quá trình duyệt.
- **Đánh giá độ phức tạp:** Thời gian duyệt mảng là $O(n + m)$. Với $n, m < 10^6$, thuật toán sẽ chạy cực kỳ nhanh (chỉ mất khoảng 0.05 giây). Bộ nhớ tiêu tốn $O(n)$ cho các mảng A và cnt, hoàn toàn thỏa mãn giới hạn bộ nhớ.
- **Lưu ý kiểu dữ liệu:** Tổng tiền lớn nhất cho một loại mã có thể đạt $m \times \max(A_i) \approx 10^6 \times 10^6 = 10^{12}$, vượt qua giới hạn của int 32-bit. Do đó, việc ép kiểu toàn bộ qua long long là bắt buộc.

27.3.1 Code mẫu (Tối ưu $O(n + m)$)

```
#include<bits/stdc++.h>
#define int long long
using namespace std;

const int N = 1e6 + 5;

int n, m, k, b, mx;
int a[N], cnt[N];

main() {
    ios::sync_with_stdio(0);
```

```

cin.tie(0);

cin >> n >> m >> k;

for (int i = 1; i <= n; i++) cin >> a[i];

for (int i = 1; i <= m; i++) {
    cin >> b;
    cnt[b]++;
}

for (int i = 1; i <= n; i++) {
    mx = max(mx, cnt[i] * a[i]);
}

cout << a[k] << '\n' << mx;
}

```

28 Lào Cai

28.1 Bài 1: CHIA HẾT CHO 3 HOẶC 5

Tóm tắt đề bài: Cho số nguyên dương N ($1 \leq N \leq 10^{12}$). Yêu cầu đếm số lượng các số nguyên dương $\leq N$ chia hết cho 3 hoặc chia hết cho 5. Có T truy vấn ($1 \leq T \leq 10^5$).

Phân tích thuật toán:

- **Tiếp cận ngây thơ (Subtask 1):** Với mỗi truy vấn N , sử dụng vòng lặp từ 1 đến N để kiểm tra điều kiện chia hết cho 3 hoặc 5. Độ phức tạp là $O(T \times N)$. Tuy nhiên, với $T = 10^5$ và $N = 10^{12}$, cách này chắc chắn sẽ dẫn đến quá thời gian (Time Limit Exceeded).
- **Tiếp cận tối ưu $O(1)$ (Subtask 2 - Full Test):** Áp dụng nguyên lý bù trừ (Inclusion-Exclusion Principle).
 - * Số lượng các số chia hết cho 3 trong khoảng từ 1 đến N là: $\lfloor \frac{N}{3} \rfloor$.
 - * Số lượng các số chia hết cho 5 trong khoảng từ 1 đến N là: $\lfloor \frac{N}{5} \rfloor$.
 - * Tuy nhiên, các số chia hết cho cả 3 và 5 (tức là chia hết cho BCNN(3, 5) = 15) đã bị đếm hai lần (một lần trong nhóm chia hết cho 3 và một lần trong nhóm chia hết cho 5). Ta cần trừ đi phần giao này.
 - * Số lượng các số chia hết cho 15 trong khoảng từ 1 đến N là: $\lfloor \frac{N}{15} \rfloor$.

- **Công thức tổng quát:**

$$\text{Kết quả} = \left\lfloor \frac{N}{3} \right\rfloor + \left\lfloor \frac{N}{5} \right\rfloor - \left\lfloor \frac{N}{15} \right\rfloor$$

- Mỗi truy vấn được giải quyết bằng các phép chia số học cơ bản trong thời gian $O(1)$. Tổng thời gian cho T truy vấn là $O(T)$, dễ dàng vượt qua thời gian quy định.
- Vì $N \leq 10^{12}$, kết quả lớn nhất cũng ở mức $\approx 10^{12}$, nằm gọn trong giới hạn của số nguyên 64-bit (long long).

28.1.1 Code mẫu C++

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int t, n;

main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    cin >> t;
    while (t--) {
        cin >> n;
        int cnt3 = n / 3;
        int cnt5 = n / 5;
        int cnt15 = n / 15;

        int ans = cnt3 + cnt5 - cnt15;
        cout << ans << '\n';
    }
}
```

28.2 Bài 2: KÝ TỰ GIỐNG NHAU

Tóm tắt đề bài: Cho hai chuỗi x và y có cùng độ dài $L \leq 10^5$. Có Q truy vấn, mỗi truy vấn thuộc một trong hai loại:

- Loại 1 ($u = 1, v$): Kiểm tra xem ký tự ở vị trí v của chuỗi x có giống ký tự ở vị trí v của chuỗi y hay không.
- Loại 2 ($u = 2, v$): Kiểm tra xem ký tự ở vị trí v của chuỗi x có giống ký tự ở vị trí ánh xạ của v trong chuỗi y hay không. Vị trí ánh xạ được tính bằng: $L - v + 1$.

—

28.2.1 Subtask 1: $Q = 1$

Phân tích: Với Subtask này, số lượng truy vấn luôn bằng 1. Ta chỉ cần đọc hai xâu x, y , đọc số Q , sau đó đọc luôn một cặp u, v và đưa ra câu trả lời. Thuật toán có độ phức tạp $O(L)$ để đọc xâu và $O(1)$ để xử lý truy vấn. Nhớ chuyển đổi vị trí từ 1-based (đề bài) sang 0-based (C++).

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

string x, y;
int q, u, v, L;

main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    cin >> x >> y >> q >> u >> v;
    L = x.length();

    if (u == 1) {
        if (x[v - 1] == y[v - 1]) cout << "YES\n";
        else cout << "NO\n";
    } else if (u == 2) {
        if (x[v - 1] == y[L - v]) cout << "YES\n";
        else cout << "NO\n";
    }
}
```

28.2.2 Subtask 2: $Q \leq 10^5$ (Full Test)

Phân tích: Số lượng truy vấn có thể lên tới 10^5 . Ta sử dụng vòng lặp while (q-) để xử lý lần lượt từng truy vấn. Việc truy xuất ký tự bằng chỉ số mảng mất thời gian $O(1)$, do đó tổng thời gian xử lý toàn bộ truy vấn chỉ là $O(Q)$, cực kỳ tối ưu và nhẹ nhàng vượt qua giới hạn thời gian.

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

string x, y;
int q, L, u, v;

main() {
    ios_base::sync_with_stdio(false);
```

```

cin.tie(NULL);

cin >> x >> y >> q;
L = x.length();

while (q--) {
    cin >> u >> v;
    if (u == 1) {
        if (x[v - 1] == y[v - 1]) cout << "YES\n";
        else cout << "NO\n";
    } else if (u == 2) {
        if (x[v - 1] == y[L - v]) cout << "YES\n";
        else cout << "NO\n";
    }
}
}

```

28.3 Bài 3: TỔNG NHỎ NHẤT

Tóm tắt đề bài: Tìm hai số tự nhiên a, b có tổng nhỏ nhất sao cho $\gcd(a, b) = c$ và $\text{lcm}(a, b) = d$. Nếu không tồn tại cặp số nào thỏa mãn, in ra -1 . Giới hạn: $1 \leq c, d \leq 10^9$.

28.3.1 Phân tích Toán học

Ta có tính chất cơ bản của Ước chung lớn nhất (ƯCLN) và Bội chung nhỏ nhất (BCNN):

- Vì $\gcd(a, b) = c$, ta có thể đặt $a = c \cdot x$ và $b = c \cdot y$, với điều kiện $\gcd(x, y) = 1$.
- Mối liên hệ giữa ƯCLN và BCNN: $a \cdot b = \gcd(a, b) \cdot \text{lcm}(a, b)$.
- Thay vào ta được: $(c \cdot x) \cdot (c \cdot y) = c \cdot d \implies x \cdot y = \frac{d}{c}$.

Từ đây ta rút ra các kết luận quan trọng:

1. Nếu d không chia hết cho c , chắc chắn không tồn tại cặp (a, b) nào thỏa mãn. Kết quả là -1 .
2. Nếu d chia hết cho c , gọi $k = \frac{d}{c}$. Bài toán đưa về việc tìm hai số x, y sao cho:
 - $x \cdot y = k$
 - $\gcd(x, y) = 1$
 - $x + y$ đạt giá trị nhỏ nhất.

Để tổng $x + y$ nhỏ nhất khi tích $x \cdot y = k$ không đổi, hai số x và y phải có giá trị càng gần nhau càng tốt. Do đó, ta sẽ duyệt x từ $\lfloor \sqrt{k} \rfloor$ giảm dần về 1. Cặp (x, y) đầu tiên thỏa mãn $\gcd(x, y) = 1$ chính là cặp cho tổng nhỏ nhất.

28.3.2 Subtask 1: $c \times d \leq 10^9$

Phân tích: Giới hạn $c \times d \leq 10^9$ đồng nghĩa với việc $k = \frac{d}{c}$ rất nhỏ (tối đa 10^9). Việc duyệt tìm x, y chỉ mất tối đa $\sqrt{10^9} \approx 31622$ phép toán, hoàn toàn có thể chạy mượt mà. Tổng $a + b = c \cdot (x + y)$ cũng nằm trong giới hạn số nguyên thông thường.

28.3.3 Subtask 2: $c \times d \leq 10^{18}$ (Full Test)

Phân tích: Với giới hạn $1 \leq c, d \leq 10^9$, thì giá trị lớn nhất của $k = \frac{d}{c}$ vẫn chỉ là 10^9 . Thuật toán duyệt từ $\lfloor \sqrt{k} \rfloor$ về 1 vẫn có độ phức tạp là $O(\sqrt{k})$, tức là tối đa khoảng $3 \cdot 10^4$ vòng lặp.

Tuy nhiên, tổng $a + b$ có thể đạt đến $c \cdot (x + y) \approx 10^9 \cdot (1 + 10^9) \approx 10^{18}$. Do đó, bắt buộc phải sử dụng kiểu dữ liệu số nguyên 64-bit để tránh tràn số. Cấu trúc mã nguồn dưới đây đã bao trọn giải pháp tối ưu $O(\sqrt{d/c})$ cho mọi Subtask.

```
#include<bits/stdc++.h>
#define int long long
using namespace std;

int c, d, k, best = -1;

main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    cin >> c >> d;

    if (d % c != 0) {
        cout << -1;
        return 0;
    }

    k = d / c;

    for (int x = sqrt(k); x >= 1; x--) {
        if (k % x == 0) {
            int y = k / x;
            if (__gcd(x, y) == 1) {
                best = c * (x + y);
                break;
            }
        }
    }

    cout << best;
```

}

28.4 Bài 4: MUA HÀNG TIẾT KIỆM

Tóm tắt đề bài: Công ty KFC cần mua tổng cộng M đùi gà. Có N nhà cung cấp, mỗi nhà cung cấp i có tối đa a_i đùi gà với đơn giá b_i trên mỗi đơn vị. Mục tiêu là tìm phương án mua hàng sao cho tổng chi phí là nhỏ nhất.

28.4.1 Subtask 1: $N \leq 10^3$

Phân tích: Với số lượng nhà cung cấp nhỏ (10^3), ta có thể sử dụng thuật toán tham lam (Greedy). Ý tưởng chủ đạo là luôn ưu tiên mua hàng từ nhà cung cấp có giá rẻ nhất trước, sau đó mới đến các nhà cung cấp có giá cao hơn cho đến khi đủ số lượng M .

Các bước thực hiện:

1. Lưu thông tin N nhà cung cấp vào một mảng các cấu trúc (struct) hoặc cặp (pair).
2. Sắp xếp mảng này theo tiêu chí giá bán (b_i) tăng dần.
3. Duyệt qua từng nhà cung cấp trong mảng đã sắp xếp:
 - Nếu số lượng cần mua M vẫn lớn hơn khả năng cung cấp a_i : Mua hết a_i cái và trừ M đi a_i .
 - Nếu M nhỏ hơn a_i : Chỉ mua đúng M cái và kết thúc.

Code mẫu Subtask 1:

```
#include<bits/stdc++.h>
#define int long long
using namespace std;

const int N = 100005;

struct node{
    int a, b;
} v[N];

int n, m, ans;

bool cmp(node x, node y) {
    return x.b < y.b;
}
```

```
main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    cin >> n >> m;

    for (int i = 0; i < n; i++) cin >> v[i].a >> v[i].b;

    sort(v, v + n, cmp);

    for (int i = 0; i < n; i++) {
        if (m <= 0) break;

        int cnt = min(m, v[i].a);
        ans += cnt * v[i].b;
        m -= cnt;
    }

    cout << ans;
}
```

28.4.2 Subtask 2: $N \leq 10^5$

Phân tích: Thuật toán tham lam kết hợp sắp xếp ở Subtask 1 có độ phức tạp là $O(N \log N)$. Với $N = 10^5$, giá trị $N \log N$ xấp xỉ 1.7×10^6 phép tính, hoàn toàn đáp ứng tốt giới hạn thời gian 1 giây của hầu hết các hệ thống chấm bài. Điểm cần lưu ý duy nhất là tổng tiền có thể vượt quá giới hạn của kiểu int 32-bit ($10^9 \times 10^9 = 10^{18}$), do đó việc sử dụng long long là bắt buộc.

Code mẫu Subtask 2:

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

const int N = 100005;

struct node{
    int sl, gia;
} a[N];

int n, m, ans;

bool cmp(node x, node y) {
    return x.gia < y.gia;
}
```

```
main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    cin >> n >> m;

    for (int i = 0; i < n; i++) cin >> a[i].sl >> a[i].gia;

    sort(a, a + n, cmp);

    for (int i = 0; i < n; i++) {
        if (m > a[i].sl) {
            ans += a[i].sl * a[i].gia;
            m -= a[i].sl;
        } else {
            ans += m * a[i].gia;
            break;
        }
    }

    cout << ans;
}
```

28.5 Bài 5: LUYỆN TẬP

Tóm tắt đề bài: Có n học sinh với kỹ năng lập trình ban đầu a_i . Có m bài tập với độ khó b_j . Học sinh chỉ giải được bài tập nếu kỹ năng $\geq$ độ khó bài tập. Sau khi giải, kỹ năng tăng thêm một lượng bằng đúng độ khó đó. Tìm kỹ năng tối đa của mỗi học sinh.

28.5.1 Subtask 1: $n, m \leq 10^4$

Phân tích: Với giới hạn này, ta có thể xử lý từng học sinh một cách độc lập. Đối với mỗi học sinh, để đạt kỹ năng cao nhất, ta nên cho học sinh đó giải các bài tập từ dễ đến khó (ưu tiên các bài có độ khó nhỏ để tích lũy kỹ năng trước).

- Bước 1: Sắp xếp mảng bài tập b tăng dần.
- Bước 2: Với mỗi học sinh, dùng vòng lặp duyệt qua mảng bài tập đã sắp xếp. Nếu kỹ năng hiện tại $\geq b[j]$, cộng dồn $b[j]$ vào kỹ năng và tiếp tục. Nếu không giải được bài $b[j]$, chắc chắn các bài sau cũng không giải được (vì b đã sắp xếp).

Độ phức tạp: $O(n \times m)$, xấp xỉ 10^8 phép tính, có thể chạy trong giới hạn 1-2 giây.

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int n, m;
int a[200005], b[200005];

main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    cin >> n >> m;
    for (int i = 0; i < n; i++) cin >> a[i];
    for (int i = 0; i < m; i++) cin >> b[i];

    sort(b, b + m);

    for (int i = 0; i < n; i++) {
        int curr = a[i];
        for (int j = 0; j < m; j++) {
            if (curr >= b[j]) curr += b[j];
            else break;
        }
        cout << curr << " ";
    }
}
```

28.5.2 Subtask 2: $n, m \leq 10^5$

Phân tích: Thuật toán $O(n \times m)$ sẽ bị quá thời gian. Ta cần nhận xét: Nếu học sinh A có kỹ năng ban đầu nhỏ hơn học sinh B, thì số lượng bài tập học sinh A giải được chắc chắn là tập con của những bài học sinh B giải được.

- Bước 1: Sắp xếp mảng bài tập b tăng dần.
- Bước 2: Sắp xếp các học sinh theo kỹ năng tăng dần (vẫn giữ lại chỉ số cũ để in kết quả).
- Bước 3: Sử dụng kỹ thuật hai con trỏ. Khi chuyển từ học sinh có kỹ năng thấp sang học sinh có kỹ năng cao hơn, ta không cần duyệt lại các bài tập cũ mà chỉ cần kiểm tra xem học sinh mới có giải thêm được các bài tập tiếp theo hay không.
- Duy trì biến `sum_solved` để lưu tổng độ khó các bài tập mà nhóm học sinh trước đó đã giải được.

Độ phức tạp: $O(n \log n + m \log m)$, đảm bảo vượt qua mọi bộ test.


```
#include <bits/stdc++.h>
#define int long long
using namespace std;

struct Student {
    int skill, id;
};

int n, m;
Student st[200005];
int b[200005], res[200005];

bool cmp(Student x, Student y) {
    return x.skill < y.skill;
}

main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    cin >> n >> m;
    for (int i = 0; i < n; i++) {
        cin >> st[i].skill;
        st[i].id = i;
    }
    for (int i = 0; i < m; i++) cin >> b[i];

    sort(b, b + m);
    sort(st, st + n, cmp);

    int p = 0, sum_val = 0;
    for (int i = 0; i < n; i++) {
        int cur = st[i].skill + sum_val;
        while (p < m && cur >= b[p]) {
            cur += b[p];
            sum_val += b[p];
            p++;
        }
        res[st[i].id] = cur;
    }

    for (int i = 0; i < n; i++) {
        cout << res[i] << " ";
    }
}
```

29 Nam Định

29.1 Bài 1. BẰNG NHAU

Tóm tắt đề bài:

Cho 4 số nguyên dương a, b, c, d lần lượt là khối lượng của 4 quả cam. An cần chia 4 quả cam này cho hai người em sao cho tổng khối lượng cam mỗi người nhận được là bằng nhau. Yêu cầu kiểm tra xem có phương án chia nào thỏa mãn hay không.

Giới hạn: $0 < a, b, c, d \leq 100$.

29.1.1 Phân tích và Thuật toán

Vì số lượng quả cam rất ít (chỉ có 4 quả), ta có thể liệt kê tất cả các trường hợp chia cam cho hai người. Gọi $S = a + b + c + d$ là tổng khối lượng của cả 4 quả. Để chia bằng nhau, mỗi người phải nhận được tổng khối lượng là $S/2$. Điều này chỉ xảy ra khi S là số chẵn và tồn tại một nhóm quả có tổng bằng $S/2$.

Các tập con có thể có của một người (người còn lại sẽ lấy các quả còn lại):

- Trường hợp 1 người lấy 1 quả, 1 người lấy 3 quả:

$$* a = b + c + d$$

$$* b = a + c + d$$

$$* c = a + b + d$$

$$* d = a + b + c$$

- Trường hợp mỗi người lấy 2 quả:

$$* a + b = c + d$$

$$* a + c = b + d$$

$$* a + d = b + c$$

Nếu bất kỳ điều kiện nào ở trên thỏa mãn, kết quả là CÓ, ngược lại là KHÔNG.

Độ phức tạp: $O(1)$.

29.1.2 Code mẫu (C++):

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int a, b, c, d;
```

```
main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    cin >> a >> b >> c >> d;
    int sum = a + b + c + d;
    if (sum % 2 != 0) {
        cout << "KHONG";
        return 0;
    }
    int target = sum / 2;
    if (a == target || b == target || c == target || d == target ||
        a + b == target || a + c == target || a + d == target) {
        cout << "CO";
    } else {
        cout << "KHONG";
    }
}
```

29.2 Bài 2. KHÁC NHAU

Tóm tắt đề bài:

Hai số nguyên dương x và y được gọi là "khác nhau" nếu thỏa mãn:

1. $x \neq y$.
2. Không tồn tại hai số nguyên $a, b > 1$ phân biệt mà cả x và y đều chia hết cho cả a và b .

Nói cách khác, x và y "khác nhau" khi chúng có **không quá một ước chung lớn hơn 1**.

Yêu cầu: Cho x và n , liệt kê các số $y \in [1, n]$ sao cho x và y là "khác nhau".

Giới hạn: $1 \leq x, n \leq 1000$.

29.2.1 Phân tích thuật toán

Điều kiện "không tồn tại hai số nguyên $a, b > 1$ là ước chung của x và y " có thể được hiểu thông qua số lượng ước của số $G = \gcd(x, y)$:

- Nếu $\gcd(x, y) = 1$: x và y không có ước chung nào lớn hơn 1. Thỏa mãn.

- Nếu $\gcd(x, y) = G > 1$: Các ước chung của x và y chính là các ước của G . Để không tồn tại hai số $a, b > 1$ phân biệt, thì G chỉ được phép có duy nhất một ước lớn hơn 1. Điều này đồng nghĩa với việc G **phải là một số nguyên tố hoặc lũy thừa của một số nguyên tố sao cho chỉ có đúng 1 ước nguyên tố?**

Định chính: Nếu G có từ 2 ước trở lên lớn hơn 1 (ví dụ $G = 4$ có ước $\{2, 4\}$, $G = 6$ có ước $\{2, 3, 6\}$), ta cần kiểm tra kỹ: - Nếu $G = 4$, các ước lớn hơn 1 là $\{2, 4\}$. Tồn tại $a = 2, b = 4 \Rightarrow$ Không thỏa mãn. - Nếu $G = p$ (nguyên tố), chỉ có 1 ước > 1 là $p \Rightarrow$ Thỏa mãn. - Nếu $G = 1$, không có ước $> 1 \Rightarrow$ Thỏa mãn.

Vậy điều kiện "khác nhau" tương đương với: 1. $x \neq y$. 2. $G = \gcd(x, y)$ phải là một số nguyên tố hoặc $G = 1$.

Độ phức tạp: $O(n \cdot \log(\min(x, n)))$.

29.2.2 Code mẫu (C++):

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int x, n;
int res[1005];

bool nt(int k) {
    if (k < 2) return 0;
    for (int i = 2; i * i <= k; i++) {
        if (k % i == 0) return 0;
    }
    return 1;
}

int main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);

    cin >> x >> n;
    int cnt = 0;
    for (int y = 1; y <= n; y++) {
        if (x == y) continue;
        int g = __gcd(x, y);
        if (g == 1 || nt(g)) {
            res[cnt++] = y;
        }
    }
    cout << cnt << "\n";
    for (int i = 0; i < cnt; i++) {
```

```

        cout << res[i] << " ";
    }
}

```

29.3 Bài 3. CẮT DÂY

Tóm tắt đề bài:

An có một sợi dây độ dài n . An thực hiện k lần cắt dây, mỗi lần chọn đoạn dài nhất để cắt:

- Nếu độ dài chẵn $2u$, cắt thành u và u .
- Nếu độ dài lẻ $2u + 1$, cắt thành u và $u + 1$.

Yêu cầu: Sau k lần cắt, hãy cho biết độ dài đoạn dây dài nhất và số lượng đoạn dây đó.

Giới hạn: $2 \leq n \leq 10^{18}$; $1 \leq k \leq n - 1$.

29.3.1 Subtask 1: $n, k \leq 10^4$

Với giới hạn này, ta có thể mô phỏng trực tiếp quá trình cắt dây bằng một cấu trúc dữ liệu luôn cho phép lấy ra phần tử lớn nhất, ví dụ như hàng đợi ưu tiên (priority_queue).

Mỗi bước, ta lấy độ dài lớn nhất L , cắt thành $L_1 = \lfloor L/2 \rfloor$ và $L_2 = L - L_1$, sau đó đẩy L_1, L_2 ngược lại vào hàng đợi. Sau k lần, phần tử đầu hàng đợi chính là độ dài lớn nhất, và ta đếm xem có bao nhiêu phần tử bằng nó.

Độ phức tạp: $O(k \log k)$.

29.3.2 Subtask 2: $n \leq 10^{18}$

Với n rất lớn, ta không thể mô phỏng từng lần cắt. Tuy nhiên, quan sát quy tắc cắt $\lfloor L/2 \rfloor$ và $\lceil L/2 \rceil$, ta thấy rằng ở mỗi "tầng" của quá trình cắt, các đoạn dây chỉ có tối đa 2 loại độ dài khác nhau.

Ví dụ $n = 100$:

- Tầng 0: {100} (1 đoạn)
- Tầng 1: {50, 50} (2 đoạn)

- Tầng 2: {25,25,25,25} (4 đoạn)
- Tầng 3: {12,13,12,13,12,13,12,13} (8 đoạn, gồm hai loại độ dài 12 và 13)

Ta sử dụng một cấu trúc `std::map<long long, long long, greater<long long>>` để lưu trữ: `map[độ dài] = số lượng`. Vì luôn có tối đa 2 loại độ dài tại mỗi thời điểm tác động, số lượng phần tử trong map sẽ rất nhỏ (khoảng $2\log n$).

Thuật toán: 1. Khởi tạo `map[n] = 1`. 2. Lặp cho đến khi tổng số lần cắt đã thực hiện đạt k : - Lấy độ dài lớn nhất L và số lượng cnt từ map. - Nếu số lần cắt còn lại ($k - \text{đã cắt}$) nhỏ hơn cnt : - Độ dài lớn nhất vẫn là L , nhưng số lượng giảm đi $k - \text{đã cắt}$. - Các đoạn mới sinh ra từ việc cắt này có độ dài nhỏ hơn L , nên không ảnh hưởng kết quả. - Thoát vòng lặp. - Nếu $(k - \text{đã cắt}) \geq cnt$: - Thực hiện cắt toàn bộ cnt đoạn độ dài L . - Cập nhật số lượng cho hai đoạn mới: $L_1 = L/2$ và $L_2 = L - L_1$. - Xóa L khỏi map. - Nếu độ dài mới sinh ra ≤ 1 và vẫn còn lượt cắt, ta cần cẩn thận (nhưng đề bài cho $k \leq n - 1$ nên luôn cắt được).

Độ phức tạp: $O(\log n \cdot \log(\log n))$.

29.3.3 Code mẫu (C++):

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

map<int, int, greater<int>>> mp;
int n, k, cuts;

main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    cin >> n >> k;
    mp[n] = 1;
    cuts = 0;

    while (cuts < k) {
        int L = mp.begin()->first;
        int sl = mp.begin()->second;
        mp.erase(mp.begin());

        if (cuts + sl > k) {
            cout << L << " " << sl - (k - cuts);
            return 0;
        }
    }
```

```

        cuts += sl;
        mp[L / 2] += sl;
        mp[L - L / 2] += sl;
    }

    cout << mp.begin()->first << " " << mp.begin()->second;
}

```

29.4 Bài 4. CHÍNH PHƯƠNG

Tóm tắt đề bài: Tìm số tập hợp $A = \{a_1, a_2, \dots, a_k\}$ gồm k số tự nhiên khác nhau có tổng bằng n , sao cho tổng của bất kỳ $k - 1$ phần tử nào cũng là một số chính phương.

Giới hạn: $2 \leq n \leq 10000, 2 \leq k \leq 10$.

29.4.1 Phân tích thuật toán

Gọi $S = \sum_{i=1}^k a_i = n$. Theo yêu cầu đề bài, với mọi $i \in \{1, \dots, k\}$, ta có:

$$S - a_i = x_i^2 \implies a_i = n - x_i^2$$

Vì a_i là các số tự nhiên khác nhau, ta suy ra:

1. $a_i > 0 \implies x_i^2 < n$ (Các phần tử khác nhau và tổng của $k - 1$ số là chính phương).
2. $\sum_{i=1}^k a_i = n \iff \sum_{i=1}^k (n - x_i^2) = n \iff \sum_{i=1}^k x_i^2 = (k - 1)n$.

Mô hình bài toán: Bài toán trở thành tìm số cách chọn k số chính phương **phân biệt** x_i^2 từ tập các số chính phương nhỏ hơn n , sao cho tổng của chúng bằng $(k - 1)n$.

Phương pháp Quy hoạch động: Gọi $dp[j][s]$ là số cách chọn j số chính phương phân biệt để có tổng bằng s .

- **Trạng thái chuyển:** Với mỗi số chính phương $x^2 < n$, ta cập nhật:

$$dp[j][s] = dp[j][s] + dp[j - 1][s - x^2]$$

- **Thứ tự duyệt:** Duyệt các số chính phương trước, sau đó duyệt ngược j và s để đảm bảo mỗi số chính phương chỉ được chọn một lần (giống bài toán Cái túi - Knapsack 0/1).

Độ phức tạp: $O(\sqrt{n} \cdot k \cdot (k \cdot n))$. Với $n = 10000, k = 10$, tổng trạng thái cần tính toán là khoảng $100 \cdot 10 \cdot 90000$, vẫn nằm trong giới hạn cho phép.

29.4.2 Code mẫu (C++):

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int n, k;
int dp[11][100005];
int a[105];

main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL); cout.tie(0);

    cin >> n >> k;
    int target = (k - 1) * n;
    int cnt = 0;
    for (int i = 0; i * i < n; i++) {
        a[cnt++] = i * i;
    }
    dp[0][0] = 1;
    for (int i = 0; i < cnt; i++) {
        int v = a[i];
        for (int j = k; j >= 1; j--) {
            for (int s = target; s >= v; s--) {
                dp[j][s] += dp[j - 1][s - v];
            }
        }
    }
    cout << dp[k][target];
}
```

29.5 Bài 5. CHỤP ẢNH

Tóm tắt đề bài:

Cho N_1 học sinh nam và N_2 học sinh nữ. Cần xếp tất cả học sinh thành một hàng ngang sao cho nam và nữ đứng xen kẽ nhau (không có hai học sinh cùng giới tính đứng cạnh nhau). Hai bức ảnh khác nhau nếu có ít nhất một vị trí có học sinh khác nhau. Đếm số cách xếp và in ra kết quả theo modulo $10^9 + 7$.

Giới hạn: $1 \leq N_1, N_2 \leq 100$.

29.5.1 Phân tích thuật toán

Để nam và nữ đứng xen kẽ nhau, hiệu số lượng giữa nam và nữ không được vượt quá 1. Cụ thể:

1. Nếu $|N_1 - N_2| > 1$: Không có cách xếp nào thỏa mãn. Kết quả là 0.
2. Nếu $N_1 = N_2 + 1$: Hàng phải bắt đầu và kết thúc bằng nam (Nam - Nữ - Nam - ... - Nam).
3. Nếu $N_2 = N_1 + 1$: Hàng phải bắt đầu và kết thúc bằng nữ (Nữ - Nam - Nữ - ... - Nữ).
4. Nếu $N_1 = N_2$: Có 2 kịch bản: bắt đầu bằng Nam hoặc bắt đầu bằng Nữ.

Cách tính số hoán vị: Giả sử ta đã xác định được các vị trí cho Nam và các vị trí cho Nữ. - Có $N_1!$ cách xếp N_1 học sinh nam vào các vị trí dành cho nam. - Có $N_2!$ cách xếp N_2 học sinh nữ vào các vị trí dành cho nữ. - Tổng số cách cho một kịch bản vị trí là: $N_1! \times N_2!$.

Công thức tổng quát: - Nếu $N_1 = N_2 + 1$ hoặc $N_2 = N_1 + 1$: Kết quả = $(N_1! \times N_2!) \pmod{10^9 + 7}$. - Nếu $N_1 = N_2$: Kết quả = $(2 \times N_1! \times N_2!) \pmod{10^9 + 7}$. - Các trường hợp khác: Kết quả = 0.

Độ phức tạp: $O(N_1 + N_2)$ để tính giai thừa.

29.5.2 Code mẫu (C++):

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

const int MOD = 1e9 + 7;
int n1, n2;

int giathua(int n) {
    int res = 1;
    for (int i = 1; i <= n; i++)
        res = (res * i) % MOD;
    return res;
}

signed main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);
```

```

cin >> n1 >> n2;
if (abs(n1 - n2) > 1) {
    cout << 0;
    return 0;
}
int ans = (giathua(n1) * giathua(n2)) % MOD;
if (n1 == n2) {
    ans = (ans * 2) % MOD;
}
cout << ans;
}

```

Giải thích ví dụ $N_1 = 3, N_2 = 2$: - Chênh lệch $|3 - 2| = 1$, thỏa mãn.

- Cấu trúc hàng: Nam - Nữ - Nam - Nữ - Nam.

- Số cách xếp Nam: $3! = 3 \times 2 \times 1 = 6$.

- Số cách xếp Nữ: $2! = 2 \times 1 = 2$.

- Tổng: $6 \times 2 = 12$. Kết quả: 12.

30 Nghệ An

30.1 Bài 1: SỐ THỨ VỊ

Tóm tắt đề bài: Một số nguyên dương X được gọi là số thứ vị nếu hai chữ số tận cùng của X (tức là $X \pmod{100}$) chia hết cho số k cho trước. Yêu cầu đếm số lượng số thứ vị nằm trong đoạn $[a, b]$. **Giới hạn:** $T \leq 10^5$, $k \leq 99$, $10^2 \leq a \leq b \leq 10^{18}$.

30.1.1 Subtask 1: $T = 1, 10^2 \leq a \leq b \leq 10^6$

Phân tích: Với giới hạn truy vấn $T = 1$ và b nhỏ, khoảng cách giữa a và b tối đa là 10^6 . Ta có thể sử dụng vòng lặp duyệt qua từng số X trong đoạn $[a, b]$, lấy phần dư $X \pmod{100}$ rồi kiểm tra xem có chia hết cho k hay không.

```

#include <bits/stdc++.h>
#define int long long
using namespace std;

int T, k;
main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);

    cin >> T >> k;
    while (T--) {

```

```

    int a, b;
    cin >> a >> b;
    int ans = 0;
    for (int i = a; i <= b; i++) {
        if ((i % 100) % k == 0) {
            ans++;
        }
    }
    cout << ans << "\n";
}

```

30.1.2 Subtask 2: $T \leq 10^5, 10^2 \leq a \leq b \leq 10^6$

Phân tích: Số lượng truy vấn T lớn, nếu dùng vòng lặp cho mỗi truy vấn sẽ dẫn đến Time Limit Exceeded (TLE). Tuy nhiên $b \leq 10^6$, ta có thể sử dụng mảng cộng dồn (Prefix Sum). Gọi $P[i]$ là số lượng số thú vị từ 0 đến i . Kết quả cho mỗi truy vấn $[a, b]$ sẽ là $P[b] - P[a - 1]$. Mỗi truy vấn phản hồi trong $O(1)$.

```

#include<bits/stdc++.h>
#define int long long
using namespace std;

const int N = 1000005;

int t, k, a, b;
int p[N];

main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    cin >> t >> k;

    p[0] = 1;

    for (int i = 1; i <= 1000000; i++) {
        p[i] = p[i - 1];
        if ((i % 100) % k == 0) p[i]++;
    }

    while (t--) {
        cin >> a >> b;
        cout << p[b] - p[a - 1] << '\n';
    }
}

```

30.1.3 Subtask 3: $T \leq 10^5, 10^2 \leq a \leq b \leq 10^{18}$

Phân tích thuật toán tối ưu $O(1)$: Với $b \leq 10^{18}$, mảng cộng dồn không thể đáp ứng do vượt quá giới hạn bộ nhớ và thời gian khởi tạo. Ta cần thiết lập công thức Toán học để tính trực tiếp số lượng số thú vị trong đoạn $[0, X]$, gọi là hàm $f(X)$.

- Hai chữ số tận cùng lặp lại theo chu kỳ 100 số (từ đuôi 00 đến 99).
- Số lượng số chia hết cho k trong một chu kỳ 100 số này là: $C = \lfloor 99/k \rfloor + 1$.
- Số X chứa trọn vẹn $q = \lfloor X/100 \rfloor$ chu kỳ.
- Đoạn đuôi dư ra từ 00 đến $X \pmod{100}$ chứa $\lfloor (X \pmod{100})/k \rfloor + 1$ số chia hết cho k .
- Công thức: $f(X) = q \times C + \lfloor (X \pmod{100})/k \rfloor + 1$.
- Đáp án truy vấn đoạn $[a, b]$: $f(b) - f(a - 1)$.

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int T, k;

int f(int x, int k) {
    if (x < 0) return 0;
    int C = 99 / k + 1;
    int q = x / 100;
    int r = x % 100;
    return q * C + (r / k) + 1;
}

main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);

    cin >> T >> k;

    while (T--) {
        int a, b;
        cin >> a >> b;
        cout << f(b, k) - f(a - 1, k) << "\n";
    }
}
```

30.2 Bài 2: HỘP QUÀ MAY MẮN

Tóm tắt đề bài: Cho xâu S gồm các chữ số có độ dài $|S| \leq 10^5$. Cần đếm số lượng xâu con của S tạo thành một số (chấp nhận chữ số 0 ở đầu) thỏa mãn 2 điều kiện:

1. Chia hết cho 97.
2. Tổng các chữ số chia hết cho 3.

Phân tích Toán học:

- Một số có tổng các chữ số chia hết cho 3 khi và chỉ khi bản thân số đó chia hết cho 3.
- Do đó, số cần tìm phải chia hết cho cả 97 và 3.
- Vì $\gcd(97, 3) = 1$, số đó phải chia hết cho $97 \times 3 = 291$.
- Bài toán được thu gọn thành: Đếm số lượng xâu con của S mà giá trị của nó tạo thành số chia hết cho 291.

30.2.1 Subtask 1 & 2: $|S| \leq 10^3$

Phân tích: Với độ dài xâu nhỏ hơn hoặc bằng 1000, ta có thể sử dụng 2 vòng lặp lồng nhau. Cố định vị trí bắt đầu i , duyệt vị trí kết thúc j từ i đến $|S| - 1$. Ta duy trì giá trị phần dư của xâu con $S[i \dots j]$ khi chia cho 291. Nếu phần dư bằng 0, ta tăng biến đếm.

Độ phức tạp: $O(|S|^2)$.

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

string s;

main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    cin >> s;
    int n = s.length();
    int ans = 0;
    for (int i = 0; i < n; i++) {
        int cur = 0;
        for (int j = i; j < n; j++) {
            cur = (cur * 10 + (s[j] - '0')) % 291;
            if (cur == 0) ans++;
        }
    }
}
```

```

    }
}
cout << ans;
}

```

30.2.2 Subtask 3: $|S| \leq 10^5$

Phân tích: Độ phức tạp $O(|S|^2)$ sẽ gây ra Time Limit Exceeded (TLE) với $|S| = 10^5$. Ta cần giải thuật $O(|S|)$ bằng Quy hoạch động (Dynamic Programming).

- Ta dùng mảng dp kích thước 291, trong đó dp[r] lưu số lượng xâu con **kết thúc tại vị trí hiện tại** có số dư là r khi chia cho 291.
- Khi xét đến chữ số $d = S[i]$, một xâu con mới có thể được tạo ra bằng 2 cách:
 1. Nối chữ số d vào sau các xâu con kết thúc ở vị trí $i - 1$. Số dư mới sẽ là $(r \times 10 + d) \pmod{291}$.
 2. Chữ số d tự đứng một mình tạo thành xâu con có chiều dài 1, số dư là $d \pmod{291}$.
- Tổng số xâu con thỏa mãn được cộng dồn bằng số lượng xâu con có phần dư là 0 tại mỗi bước.
- Ta dùng mảng cuộn next_dp để tối ưu không gian bộ nhớ.

Độ phức tạp: Thời gian $O(|S| \times 291)$, bộ nhớ $O(291)$ tương đương $O(1)$. Chạy cực kỳ nhanh với giới hạn 2 giây.

```

#include<bits/stdc++.h>
#define int long long
using namespace std;

const int M = 305;

string s;
int n, ans;
int dp[M], nxt[M];

main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    cin >> s;
    n = s.size();

    for (int i = 0; i < n; i++) {
        memset(nxt, 0, sizeof(nxt));
    }
}

```

```

    int d = s[i] - '0';

    for (int r = 0; r < 291; r++) {
        if (dp[r]) nxt[(r * 10 + d) % 291] += dp[r];
    }

    nxt[d % 291]++;

    ans += nxt[0];

    for (int r = 0; r < 291; r++) dp[r] = nxt[r];
}

cout << ans;
}

```

30.3 Bài 3: QUY HOẠCH THÀNH PHỐ

Tóm tắt đề bài: Cho một xâu S biểu diễn màu sơn hiện tại của các ngôi nhà trên trục đường, mỗi màu là một kí tự từ 'a' đến 'z'. Chi phí để đổi từ màu x sang màu y là $|x - y|$ (khoảng cách mã ASCII). Cho trước giới hạn chi phí k , yêu cầu tìm ra đoạn con dài nhất gồm các ngôi nhà có cùng màu sơn sau khi thực hiện chuyển đổi màu mà tổng chi phí không vượt quá k .

Phân tích chung:

- Đích đến của đoạn con có thể là bất kỳ màu nào từ 'a' đến 'z' (tổng cộng 26 màu). Do số lượng màu đích rất nhỏ, ta có thể duyệt qua toàn bộ 26 màu sơn khả dĩ này.
- Với một màu đích c cố định, chi phí để đổi màu của ngôi nhà thứ i thành c là $|S[i] - c|$. Tổng chi phí trên một đoạn $[L, R]$ là $\sum_{i=L}^R |S[i] - c|$. Bài toán đưa về việc tìm đoạn $[L, R]$ dài nhất thỏa mãn tổng chi phí này $\leq k$.

30.3.1 Subtask 1: $|S| \leq 10^3$

Phân tích: Với giới hạn độ dài xâu nhỏ, ta có thể cố định màu đích c , sau đó sử dụng hai vòng lặp lồng nhau để duyệt mọi đoạn con $[i, j]$. Từ vị trí bắt đầu i , ta mở rộng vị trí kết thúc j sang phải, cộng dồn chi phí. Nếu chi phí vượt quá k , ta dừng việc mở rộng từ i và chuyển sang duyệt từ vị trí $i + 1$. Độ phức tạp cho cách làm này là $O(26 \times |S|^2)$, hoàn toàn vượt qua giới hạn của Subtask 1.

```

#include <bits/stdc++.h>
#define int long long

```

```
using namespace std;

int k, n, ans, cost;
string s;

main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    cin >> k >> s;

    n = s.size();

    for (char c = 'a'; c <= 'z'; c++) {
        for (int i = 0; i < n; i++) {
            cost = 0;

            for (int j = i; j < n; j++) {
                cost += abs(s[j] - c);

                if (cost <= k) ans = max(ans, j - i + 1);
                else break;
            }
        }
    }

    cout << ans;
}
```

30.3.2 Subtask 2: $|S| \leq 5 \times 10^5$ (Full Test)

Phân tích tối ưu: Để vượt qua giới hạn 5×10^5 , thuật toán $O(|S|^2)$ là không đủ. Ta cần một phương pháp $O(|S|)$ cho mỗi màu đích c .

- Vì chi phí thay đổi màu luôn ≥ 0 , tổng chi phí trên một đoạn sẽ tăng đơn điệu khi đoạn đó được mở rộng.
- Ta sử dụng kỹ thuật **Hai con trỏ (Sliding Window)**: Duy trì một cửa sổ trượt từ vị trí L đến R .
- Mở rộng cửa sổ bằng cách tiến con trỏ R sang phải và cộng thêm chi phí của ngôi nhà R .
- Nếu tổng chi phí trong cửa sổ $> k$, ta phải thu hẹp cửa sổ từ bên trái bằng cách tiến con trỏ L sang phải và trừ đi chi phí của ngôi nhà L cho đến khi tổng chi phí $\leq k$.

- Cập nhật độ dài lớn nhất sau mỗi lần điều chỉnh cửa sổ hợp lệ.

Độ phức tạp tổng thời gian sẽ giảm xuống chỉ còn $O(26 \times |S|)$, giải quyết xuất sắc trọn vẹn toàn bộ các test của đề bài.

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int k, n, ans;
string s;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);

    cin >> k >> s;
    n = s.length();
    ans = 0;

    for (char c = 'a'; c <= 'z'; c++) {
        int L = 0;
        int cur = 0;
        for (int R = 0; R < n; R++) {
            cur += abs(s[R] - c);
            while (cur > k && L <= R) {
                cur -= abs(s[L] - c);
                L++;
            }
            ans = max(ans, R - L + 1);
        }
    }
    cout << ans;
}
```

30.4 Bài 4: ĐẦU TƯ TÀI CHÍNH

Tóm tắt đề bài: Cho chuỗi n ngày với các chỉ số thị trường $a_1, a_2, \dots, a_n$. Cần chọn ra đúng k ngày theo đúng thứ tự thời gian ($1 \leq i_1 < i_2 < \dots < i_k \leq n$) để nhân với trọng số tương ứng $w_1, w_2, \dots, w_k$. Yêu cầu tìm tổng lợi nhuận $S = \sum_{j=1}^k w_j \times a_{i_j}$ lớn nhất có thể và đếm số cách chọn để đạt được lợi nhuận lớn nhất đó (kết quả lấy dư cho $10^9 + 7$).

Giới hạn chung: $n \leq 10^5, k \leq 100, k \leq n; a_i \leq 10^9; w_i \leq 10^6$. Lợi nhuận lớn nhất có thể lên tới $100 \times 10^9 \times 10^6 = 10^{17}$, do đó bắt buộc phải sử dụng kiểu dữ liệu số nguyên 64-bit (long long).

30.4.1 Subtask 1: $n \leq 10^2, k = 1$

Phân tích: Do $k = 1$, ta chỉ có duy nhất một trọng số w_1 và cần chọn một ngày a_i sao cho $w_1 \times a_i$ là lớn nhất. Ta duyệt qua tất cả n ngày, duy trì giá trị lớn nhất đạt được. Nếu gặp giá trị lớn hơn mức tối đa hiện tại, ta cập nhật lại và khởi tạo lại số cách đếm là 1. Nếu gặp giá trị bằng mức tối đa, ta cộng dồn thêm số cách đếm.

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

const int MOD = 1e9 + 7;
int n, k, max_val, cnt;
int a[200005], w[200005];

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);

    cin >> n >> k;
    for (int i = 0; i < n; i++) cin >> a[i];
    for (int i = 0; i < k; i++) cin >> w[i];

    max_val = -2e18;
    cnt = 0;

    for (int i = 0; i < n; i++) {
        int val = a[i] * w[0];
        if (val > max_val) {
            max_val = val;
            cnt = 1;
        } else if (val == max_val) {
            cnt = (cnt + 1) % MOD;
        }
    }

    cout << max_val << "\n" << cnt;
}
```

30.4.2 Subtask 2 & 3: $n \leq 10^5, k \leq 100$

Phân tích Quy hoạch động (Dynamic Programming): Với k lên đến 100, bài toán trở thành một bài Quy hoạch động điển hình. Ta duyệt qua từng ngày i từ 1 đến n . Tại mỗi bước, trạng thái cập nhật phụ thuộc vào các giá trị đã tính trước đó. Để tối ưu bộ nhớ $O(k)$ thay vì $O(n \times k)$, ta sử dụng mảng 1 chiều.

- Gọi $dp[j]$ là lợi nhuận lớn nhất khi chọn đúng j ngày trong số các ngày đã xét.
- Gọi $ways[j]$ là số cách để đạt được lợi nhuận $dp[j]$.
- Khởi tạo: $dp[0] = 0$, $ways[0] = 1$. Các $dp[j]$ khác ($j \geq 1$) mang giá trị $-\infty$ và $ways[j] = 0$.
- Khi xét đến chỉ số a_i tại ngày thứ i , ta duyệt j ngược từ k về 1. Nếu ta quyết định chọn a_i làm phần tử thứ j , giá trị thu được sẽ là: $pick_val = dp[j-1] + w_{j-1} \times a_i$.
- Quy tắc cập nhật:
 1. Nếu $pick_val > dp[j]$, ta tìm được cấu hình tốt hơn $\Rightarrow dp[j] = pick_val$, $ways[j] = ways[j-1]$.
 2. Nếu $pick_val == dp[j]$, ta tìm thêm được các cách tương đương $\Rightarrow ways[j] = (ways[j] + ways[j-1]) \pmod{10^9 + 7}$.

Thời gian thực thi $O(n \times k) \approx 10^7$ phép tính, chạy cực kỳ an toàn và vượt qua toàn bộ Subtask trong giới hạn 1 giây.

```
#include<bits/stdc++.h>
#define int long long
using namespace std;

const int MOD = 1e9 + 7;
const int INF = 2e18;
const int N = 100005;

int n, k;
int a[N], w[N], dp[N], ways[N];

main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    cin >> n >> k;

    for (int i = 0; i < n; i++) cin >> a[i];
    for (int i = 0; i < k; i++) cin >> w[i];

    for (int i = 0; i <= k; i++) {
        dp[i] = -INF;
        ways[i] = 0;
    }

    dp[0] = 0;
    ways[0] = 1;
```

```

for (int i = 0; i < n; i++) {
    for (int j = k; j >= 1; j--) {
        if (dp[j - 1] != -INF) {
            int val = dp[j - 1] + w[j - 1] * a[i];

            if (val > dp[j]) {
                dp[j] = val;
                ways[j] = ways[j - 1];
            } else if (val == dp[j]) {
                ways[j] = (ways[j] + ways[j - 1]) % MOD;
            }
        }
    }
}

cout << dp[k] << '\n' << ways[k];
}

```

31 Chuyên ĐH Vinh

31.1 Bài 1: TAM GIÁC VUÔNG

31.1.1 Subtask 1

Với N nhỏ, học sinh có thể sử dụng ba vòng lặp lồng nhau để duyệt qua tất cả các bộ ba số (a, b, c) thỏa mãn $1 \leq a \leq b < c \leq N$. Trong vòng lặp trong cùng, chỉ cần kiểm tra điều kiện Pytago $a^2 + b^2 = c^2$. Độ phức tạp thuật toán là $O(N^3)$, hoàn toàn chạy nhanh với giới hạn nhỏ.

31.1.2 Subtask 2

Với $N \leq 10^4$, thuật toán $O(N^3)$ sẽ dẫn đến quá thời gian vì số phép toán lên tới 10^{12} . Ta cần tối ưu độ phức tạp xuống $O(N^2)$:

- Chỉ sử dụng hai vòng lặp duyệt độ dài hai cạnh góc vuông: cạnh a chạy từ 1 đến N , và cạnh b chạy từ a đến N (để đảm bảo $a \leq b$).
- Thay vì duyệt cạnh huyền c , ta tính trực tiếp bình phương cạnh huyền $c^2 = a^2 + b^2$.
- Nếu $c^2 > N^2$, ta có thể dừng ngay vòng lặp của b vì b càng tăng thì c^2 càng lớn và chắc chắn vượt quá giới hạn N .
- Dùng hàm sqrt để tính c . Nếu $c \times c == c^2$ tức c^2 là một số chính phương, ta tìm được một bộ ba thỏa mãn, lập tức tăng biến đếm lên 1.

31.1.3 Code mẫu tham khảo

```
#include<bits/stdc++.h>
using namespace std;
#define int long long

main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    freopen("TRIA.INP", "r", stdin);
    freopen("TRIA.OUT", "w", stdout);

    int n;
    cin >> n;

    int ans = 0;
    int n2 = n * n;

    for (int a = 1; a < n; a++) {
        for (int b = a; b < n; b++) {
            int c2 = a * a + b * b;

            if (c2 > n2) break;

            int c = sqrt(c2);
            if (c * c == c2) {
                ans++;
            }
        }
    }

    cout << ans << "\n";
}
```

31.2 Bài 2: XẤU TƯƠNG TỰ

31.2.1 Subtask 1+2+3

Với giới hạn chiều dài xâu tối đa là 20000 ký tự, tổng số phép toán cho mọi subtask đều rất nhỏ so với giới hạn 1 giây của hệ thống. Khóa giải quyết bài này nằm ở việc xử lý chuỗi và đếm tần suất chữ cái. Thuật toán tối ưu $O(|st1| + |st2|)$:

- Đọc dữ liệu vào 2 xâu (lưu ý dùng lệnh getline vì xâu có thể chứa dấu cách).
- Khởi tạo hai mảng đánh dấu để ghi nhận sự xuất hiện của các chữ cái trong từng xâu.

- Duyệt qua từng ký tự của xâu. Nếu là chữ cái dùng hàm `isalpha`, ta chuyển nó về in thường dùng hàm `tolower` và đánh dấu vào mảng.
- Duyệt 26 chữ cái từ 'a' đến 'z':
 - * Nếu chữ cái được đánh dấu ở cả 2 xâu, tăng biến x lên 1.
 - * Nếu chữ cái được đánh dấu ở ít nhất 1 trong 2 xâu, tăng biến y lên 1.
- Kết quả bài toán là phần nguyên của $100 \times x/y$.

31.2.2 Code mẫu tham khảo

```
#include<bits/stdc++.h>
using namespace std;
#define int long long

bool mark1[256], mark2[256];

main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    freopen("STRING.INP", "r", stdin);
    freopen("STRING.OUT", "w", stdout);

    string s1, s2;
    getline(cin, s1);
    getline(cin, s2);

    for (char c : s1) {
        if (isalpha(c)) {
            mark1[tolower(c)] = true;
        }
    }

    for (char c : s2) {
        if (isalpha(c)) {
            mark2[tolower(c)] = true;
        }
    }

    int x = 0, y = 0;
    for (char c = 'a'; c <= 'z'; c++) {
        if (mark1[c] && mark2[c]) x++;
        if (mark1[c] || mark2[c]) y++;
    }

    if (y == 0) {
```

```

        cout << 0 << "\n";
    } else {
        cout << (100 * x) / y << "\n";
    }
}

```

31.3 Bài 3: QUÀ CỨU TRỢ

31.3.1 Subtask 1

Với giới hạn $n, m \leq 10^4$, một cách tiếp cận ngây thơ là duyệt qua mảng nhu cầu của từng gia đình, sau đó dùng một vòng lặp khác duyệt qua danh sách các gói quà để tìm gói quà đầu tiên thỏa mãn điều kiện và chưa được sử dụng. Cách làm này có độ phức tạp $O(n \times m)$.

31.3.2 Subtask 2+3

Với $n, m \leq 2 \times 10^5$, thuật toán $O(n \times m)$ chắc chắn sẽ bị quá thời gian. Bài toán này cần được tối ưu bằng thuật toán sắp xếp kết hợp kỹ thuật Hai con trỏ trên mảng đã sắp xếp với độ phức tạp $O(N \log N + M \log M)$:

- Sắp xếp cả mảng nhu cầu của gia đình (A) và mảng giá trị gói quà (B) theo thứ tự tăng dần. Tư duy tham lam ở đây là: ưu tiên phát những gói quà có giá trị nhỏ nhất thỏa mãn cho những gia đình có nhu cầu nhỏ nhất, để dành các gói quà lớn cho các gia đình có nhu cầu lớn ở phía sau.
- Sử dụng hai con trỏ i và j lần lượt trỏ vào phần tử đầu tiên của mảng A và mảng B .
- Tại mỗi bước, kiểm tra độ chênh lệch giữa a_i và b_j :
 - * Nếu $|a_i - b_j| \leq k$: Gia đình i nhận được quà j . Tăng cả i và j lên 1 bước, đồng thời tăng biến kết quả (số gia đình nhận quà).
 - * Nếu $b_j < a_i - k$: Gói quà hiện tại quá nhỏ so với nhu cầu của gia đình i , ta cần gói quà có giá trị lớn hơn nên chỉ tăng con trỏ j .
 - * Nếu $b_j > a_i + k$: Gói quà hiện tại quá lớn so với gia đình i , trong khi các gói quà phía sau còn lớn hơn nữa. Do đó gia đình i chắc chắn không thể nhận được gói quà nào, ta bỏ qua gia đình này bằng cách tăng con trỏ i .
- Lặp lại quá trình cho đến khi một trong hai con trỏ duyệt hết mảng của mình.

31.3.3 Code mẫu tham khảo

```
#include<bits/stdc++.h>
using namespace std;
#define int long long

int a[200005];
int b[200005];

main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    freopen("PAIR.INP", "r", stdin);
    freopen("PAIR.OUT", "w", stdout);

    int n, m, k;
    cin >> n >> m >> k;

    for (int i = 1; i <= n; i++) cin >> a[i];
    for (int i = 1; i <= m; i++) cin >> b[i];

    sort(a + 1, a + n + 1);
    sort(b + 1, b + m + 1);

    int i = 1, j = 1;
    int ans = 0;

    while (i <= n && j <= m) {
        if (abs(a[i] - b[j]) <= k) {
            ans++;
            i++;
            j++;
        } else if (b[j] < a[i] - k) {
            j++;
        } else {
            i++;
        }
    }

    cout << ans << "\n";
}
```


31.4 Bài 4: VÙNG SÁNG ẢNH

31.4.1 Subtask 1

Với $n, m \leq 10^3$ và $k \leq 5$, giới hạn khá nhỏ. Ta có thể mô phỏng "trâu bò" hoàn toàn.

- Xét từng ô (i, j) làm tâm của một vùng sáng ($k + 1 \leq i \leq n - k$ và $k + 1 \leq j \leq m - k$).
- Với mỗi tâm (i, j) , ta dùng 2 vòng lặp lồng nhau duyệt qua các ô trong vùng $(2k + 1) \times (2k + 1)$ xung quanh nó. Vừa kiểm tra xem ô tâm có phải là giá trị lớn nhất ngất hay không, vừa cộng dồn để tính tổng mức sáng của cả vùng.
- Độ phức tạp thuật toán là $O(n \times m \times (2k + 1)^2)$.

31.4.2 Subtask 2

Với $n, m \leq 3 \times 10^3$ và $k \leq 10$, cách tính tổng trực tiếp $O(k^2)$ ở subtask 1 sẽ tốn rất nhiều thời gian. Ta cần tối ưu thao tác này về $O(1)$:

- Sử dụng mảng cộng dồn 2 chiều. Gọi $S[i][j]$ là tổng các điểm ảnh trong hình chữ nhật từ ô $(1, 1)$ đến ô (i, j) . Tổng của một vùng sáng tâm (i, j) bán kính k sẽ được tính cực kỳ nhanh bằng công thức: $Sum = S[i + k][j + k] - S[i - k - 1][j + k] - S[i + k][j - k - 1] + S[i - k - 1][j - k - 1]$.
- Việc kiểm tra ô lớn nhất vẫn dùng 2 vòng lặp, nhưng ngay khi thấy một ô xung quanh lớn hơn hoặc bằng ô tâm, lập tức dùng break để dừng việc kiểm tra. Nhờ vậy, số vòng lặp thực tế tiệm cận $O(n \times m)$.

31.4.3 Subtask 3

Với giới hạn $n, m \leq 5 \times 10^3$ và $k \leq 20$:

- Để ổn định độ phức tạp ở mức tuyệt đối $O(n \times m)$, ta dùng cấu trúc hàng đợi hai đầu deque để trượt theo 2 pha:
 - * *Pha 1 (Trượt ngang)*: Duyệt từng hàng, dùng Deque tìm giá trị lớn nhất trong cửa sổ ngang độ dài $2k + 1$. Lưu kết quả vào mảng trung gian `max_row`.
 - * *Pha 2 (Trượt dọc)*: Duyệt từng cột của `max_row`, tiếp tục dùng Deque trượt dọc với cửa sổ $2k + 1$. Giá trị thu được `W[i][j]` chính là mức sáng cao nhất của toàn bộ vùng $(2k + 1) \times (2k + 1)$ có tâm tại (i, j) .

31.4.4 Code mẫu tham khảo

```
#include<bits/stdc++.h>
using namespace std;

int a[5005][5005];
int max_row[5005][5005];
int w[5005][5005];
long long s[5005][5005];

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    freopen("IMAGE.INP", "r", stdin);
    freopen("IMAGE.OUT", "w", stdout);

    int n, m, k;
    cin >> n >> m >> k;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            cin >> a[i][j];
            s[i][j] = s[i - 1][j] + s[i][j - 1] - s[i - 1][j - 1] + a[i][j];
        }
    }

    for (int i = 1; i <= n; i++) {
        deque<int> dq;
        for (int j = 1; j <= m; j++) {
            while (!dq.empty() && dq.front() < j - 2 * k) dq.pop_front();
            while (!dq.empty() && a[i][dq.back()] <= a[i][j]) dq.pop_back();
            dq.push_back(j);
            if (j >= 2 * k + 1) max_row[i][j - k] = a[i][dq.front()];
        }
    }

    for (int j = k + 1; j <= m - k; j++) {
        deque<int> dq;
        for (int i = 1; i <= n; i++) {
            while (!dq.empty() && dq.front() < i - 2 * k) dq.pop_front();
            while (!dq.empty() && max_row[dq.back()][j] <= max_row[i][j]) dq.pop_back();
            dq.push_back(i);
            if (i >= 2 * k + 1) w[i - k][j] = max_row[dq.front()][j];
        }
    }

    long long ans = 0;
```

```

for (int i = k + 1; i <= n - k; i++) {
    for (int j = k + 1; j <= m - k; j++) {
        if (a[i][j] == w[i][j]) {
            bool ok = true;
            for (int r = i - k; r <= i + k; r++) {
                for (int c = j - k; c <= j + k; c++) {
                    if (r == i && c == j) continue;
                    if (a[r][c] >= a[i][j]) {
                        ok = false;
                        break;
                    }
                }
            }
            if (!ok) break;
        }

        if (ok) {
            long long sum = s[i + k][j + k] - s[i - k - 1][j + k] - s[i + k][j - k - 1] + s[i - k - 1][j - k - 1];
            if (sum > ans) ans = sum;
        }
    }
}

cout << ans << "\n";
}

```

32 Ninh Bình

32.1 Bài 1

32.1.1 Subtask 1

Với ràng buộc $n \leq 10^2$, số lượng cặp (i, j) cần xét là khoảng $\frac{100 \times 99}{2} \approx 5000$ cặp. Học sinh có thể dùng hai vòng lặp lồng nhau duyệt mọi cặp, tính tổng và viết một hàm kiểm tra số nguyên tố cơ bản bằng vòng lặp $O(\sqrt{X})$ cho từng tổng. Thuật toán hoạt động cực kỳ nhẹ nhàng và dễ dàng qua giới hạn thời gian.

32.1.2 Subtask 2

Với ràng buộc $n \leq 5 \times 10^3$, số lượng cặp cần xét lên tới hơn 1.2×10^7 cặp. Nếu mỗi cặp đều dùng hàm kiểm tra $O(\sqrt{X})$ thì tổng số phép toán sẽ rất lớn, dẫn đến quá thời gian. Bài toán yêu cầu tối ưu bằng kỹ thuật sàng nguyên tố:

- Nhận thấy giá trị tối đa của một phần tử là 10^6 , nên tổng lớn nhất của hai phần tử bất kỳ chỉ là 2×10^6 .
- Sử dụng sàng nguyên tố để đánh dấu trước tính nguyên tố của tất cả các số từ 1 đến 2×10^6 chỉ trong một lần duy nhất lúc bắt đầu chương trình.
- Sau khi sàng, việc kiểm tra tổng $a_i + a_j$ có phải số nguyên tố hay không chỉ mất độ phức tạp $O(1)$ thông qua việc tra cứu mảng. Tổng độ phức tạp giảm xuống còn $O(Max_Sum + n^2)$.

32.1.3 Code mẫu tham khảo

```
#include<bits/stdc++.h>
using namespace std;
#define int long long

const int MAX_VAL = 2000005;
bool p[MAX_VAL];
int a[5005];

void sieve() {
    fill(p, p + MAX_VAL, true);
    p[0] = p[1] = false;
    for (int i = 2; i * i < MAX_VAL; i++) {
        if (p[i]) {
            for (int j = i * i; j < MAX_VAL; j += i) {
                p[j] = false;
            }
        }
    }
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    sieve();

    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) cin >> a[i];

    int ans = 0;
    for (int i = 1; i < n; i++) {
        for (int j = i + 1; j <= n; j++) {
            if (p[a[i] + a[j]]) ans++;
        }
    }
}
```

```

    }

    cout << ans << "\n";
}

```

32.2 Bài 2

32.2.1 Subtask 1

Với ràng buộc $n, m, p, q < 10$, số lượng của mỗi nguyên tố chỉ là một chữ số. Học sinh có thể duyệt xâu bằng một vòng lặp for đơn giản. Mỗi khi gặp một chữ cái (C, H, O, N), ta chỉ cần đọc ký tự ngay phía sau nó, chuyển từ ký tự sang số nguyên và nhân với khối lượng nguyên tử tương ứng để cộng dồn vào tổng.

32.2.2 Subtask 2+3

Với ràng buộc số lượng có thể lên đến hàng nghìn, các chỉ số xuất hiện trong công thức hóa học có thể là số có nhiều chữ số. Thuật toán cần tổng quát hóa quá trình bóc tách số liệu:

- Duyệt qua từng ký tự của xâu. Khi gặp một chữ cái, ta xác định khối lượng nguyên tử tương ứng (C là 12, H là 1, O là 16, N là 14).
- Dùng một vòng lặp while để đọc liên tiếp các ký tự chữ số ngay sau chữ cái đó. Ghép các chữ số này lại thành một số nguyên hoàn chỉnh bằng công thức: số hiện tại nhân 10 cộng chữ số mới.
- Nhân khối lượng nguyên tử với số lượng vừa tách được và cộng vào tổng khối lượng. Độ phức tạp của thuật toán là $O(|S|)$, duyệt qua xâu đúng một lần nên chạy cực kỳ mượt mà.

32.2.3 Code mẫu tham khảo

```

#include<bits/stdc++.h>
using namespace std;
#define int long long

main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    string s;
    cin >> s;
}

```

```

long long ans = 0;
int n = s.length();
int i = 0;

while (i < n) {
    int w = 0;
    if (s[i] == 'C') w = 12;
    else if (s[i] == 'H') w = 1;
    else if (s[i] == 'O') w = 16;
    else if (s[i] == 'N') w = 14;

    i++;
    long long cnt = 0;

    while (i < n && isdigit(s[i])) {
        cnt = cnt * 10 + (s[i] - '0');
        i++;
    }

    if (cnt == 0) cnt = 1;

    ans += w * cnt;
}

cout << ans << "\n";
}

```

32.3 Bài 3

32.3.1 Subtask 1+2

Với ràng buộc $n \leq 5 \times 10^3$, bài toán có thể giải quyết bằng cách sử dụng hai vòng lặp lồng nhau. Vòng lặp ngoài cố định vị trí bắt đầu i của đoạn, vòng lặp trong duyệt vị trí kết thúc j từ i đến n . Tại mỗi bước di chuyển j , ta duy trì một biến lưu tổng đan dấu hiện tại: nếu độ dài đoạn đang xét là số lẻ thì cộng thêm a_j , ngược lại nếu là số chẵn thì trừ đi a_j . Cập nhật kết quả lớn nhất sau mỗi bước. Độ phức tạp thuật toán là $O(n^2)$.

32.3.2 Subtask 3

Với ràng buộc $n \leq 10^6$, thuật toán $O(n^2)$ sẽ bị quá thời gian. Bài toán cần được tối ưu bằng quy hoạch động với độ phức tạp $O(n)$:

- Gọi $dp1$ là tổng đan dấu lớn nhất của các đoạn con kết thúc tại phần tử hiện tại và mang dấu cộng.

- Gọi $dp2$ là tổng đơn dấu lớn nhất của các đoạn con kết thúc tại phần tử hiện tại và mang dấu trừ.
- Khi xét đến phần tử a_i , để đoạn con kết thúc bằng dấu cộng, ta có thể lấy riêng một mình a_i , hoặc ghép a_i vào đoạn liền trước kết thúc bằng dấu trừ. Do đó $dp1$ mới bằng giá trị lớn nhất giữa a_i và $a_i + dp2$.
- Để đoạn con kết thúc bằng dấu trừ, ta bắt buộc phải ghép $-a_i$ vào đoạn liền trước kết thúc bằng dấu cộng. Do đó $dp2$ mới bằng $-a_i + dp1$.
- Khởi tạo tại phần tử đầu tiên: $dp1 = a_1$ và $dp2$ là một số âm vô cùng vì chưa thể kết thúc bằng dấu trừ. Cập nhật kết quả lớn nhất trong quá trình duyệt.

32.3.3 Code mẫu tham khảo

```
#include<bits/stdc++.h>
using namespace std;
#define int long long

int a[1000005];

main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) cin >> a[i];

    int dp1 = a[1];
    int dp2 = -1e18;
    int ans = a[1];

    for (int i = 2; i <= n; i++) {
        int n_dp1 = max(a[i], a[i] + dp2);
        int n_dp2 = dp1 - a[i];

        dp1 = n_dp1;
        dp2 = n_dp2;

        ans = max({ans, dp1, dp2});
    }

    cout << ans << "\n";
}
```

32.4 Bài 4

32.4.1 Subtask 1+2

Một số được gọi là số cực đại khi và chỉ khi không thể hoán đổi các chữ số để tạo thành số lớn hơn. Điều này đồng nghĩa với việc các chữ số của nó phải được sắp xếp theo thứ tự giảm dần hoặc bằng nhau từ trái qua phải ($d_1 \geq d_2 \geq \dots \geq d_k$).

Với ràng buộc $b - a \leq 10^6$, ta có thể duyệt tuyến tính bằng một vòng lặp từ a đến b . Tại mỗi số, ta tách các chữ số của nó ra để kiểm tra tính giảm dần. Thuật toán tốn khoảng 10^6 phép toán, hoàn toàn vượt qua giới hạn thời gian một cách dễ dàng.

32.4.2 Subtask 3

Với ràng buộc mở rộng lên đến $b \leq 10^{15}$, cách duyệt trâu bò sẽ dẫn đến quá thời gian. Bài toán cần được tối ưu bằng kỹ thuật quy hoạch động chữ số:

- Gọi $Solve(X)$ là hàm đếm số lượng số cực đại trong đoạn từ 1 đến X . Số lượng các số thỏa mãn trong đoạn $[a, b]$ chính là kết quả của phép bù trừ: $Solve(b) - Solve(a - 1)$.
- Trạng thái quy hoạch động bao gồm: vị trí chữ số hiện tại, chữ số liền trước đó, trạng thái đã nhỏ hơn mốc giới hạn hay chưa, và trạng thái số đã bắt đầu hay chưa để xử lý các chữ số 0 vô nghĩa ở đầu.
- Tại mỗi bước, ta thử đặt một chữ số từ 0 đến 9. Điều kiện bắt buộc là chữ số mới được thêm vào phải nhỏ hơn hoặc bằng chữ số liền trước nó.
- Kết quả được lưu trữ lại giúp độ phức tạp giảm xuống chỉ còn $O(18 \times 10 \times 2 \times 2)$, chạy gần như tức thời.

32.4.3 Code mẫu tham khảo

```
#include<bits/stdc++.h>
using namespace std;
#define int long long

int f[20][10][2][2];
string s;

int dp(int i, int last, int less, int start) {
    if (i == s.length()) return start;
    if (f[i][last][less][start] != -1) return f[i][last][less][start];

    int res = 0;
```



```

    int limit = less ? 9 : (s[i] - '0');

    for (int d = 0; d <= limit; d++) {
        if (!start) {
            if (d == 0) res += dp(i + 1, 9, less | (d < limit), 0);
            else res += dp(i + 1, d, less | (d < limit), 1);
        } else {
            if (d <= last) res += dp(i + 1, d, less | (d < limit), 1);
        }
    }
    return f[i][last][less][start] = res;
}

int solve(int x) {
    if (x == 0) return 0;
    s = to_string(x);
    memset(f, -1, sizeof(f));
    return dp(0, 9, 0, 0);
}

main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int a, b;
    cin >> a >> b;

    int ans = solve(b) - solve(a - 1);
    cout << ans << "\n";
}

```

33 Phú Thọ

33.1 Bài 1: Tính nhanh

Tóm tắt bài toán:

- **Đầu vào:** Đoạn $[a, b]$ và hai số n, m .
- **Đầu ra:** Giá trị của biểu thức: $count_divisible(a, b, n) - count_divisible(a, b, m)$.
- **Điều kiện:** Các số đều có thể lên tới 2×10^9 , đòi hỏi thuật toán tối ưu hơn việc duyệt thông thường.

Phân tích bài toán:

- Chiến thuật ban đầu:

- * Sử dụng vòng lặp for chạy từ a đến b .
- * Kiểm tra từng số i : nếu $i \pmod n == 0$ thì tăng biến đếm S_n , nếu $i \pmod m == 0$ thì tăng S_m .
- * **Nhược điểm:** Độ phức tạp $O(b-a)$ sẽ bị quá thời gian (TLE) khi $b = 2 \times 10^9$.

- Tối ưu hóa:

- * Áp dụng công thức toán học: Số lượng số chia hết cho k trong đoạn $[1, X]$ là $\lfloor \frac{X}{k} \rfloor$.
- * Số lượng số chia hết cho k trong đoạn $[a, b]$ là: $F(b, k) - F(a-1, k) = \lfloor \frac{b}{k} \rfloor - \lfloor \frac{a-1}{k} \rfloor$.
- * **Ưu điểm:** Độ phức tạp $O(1)$, tính toán trực tiếp kết quả bất kể khoảng cách giữa a và b .

33.1.1 Subtask 1 ($a, b \leq 10^6$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    long long a, b, n, m;
    cin >> a >> b >> n >> m;
    long long sn = 0, sm = 0;
    for (long long i = a; i <= b; i++) {
        if (i % n == 0) sn++;
        if (i % m == 0) sm++;
    }
    cout << sn - sm;
    return 0;
}
```

33.1.2 Subtask 2 (Full điểm - $a, b \leq 2 \times 10^9$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    long long a, b, n, m;
    cin >> a >> b >> n >> m;
    long long sn = (b / n) - ((a - 1) / n);
    long long sm = (b / m) - ((a - 1) / m);
    cout << sn - sm;
    return 0;
}
```

33.2 Bài 2: Trò chơi

Tóm tắt bài toán:

- Cho dãy số A có n phần tử.
- Thực hiện q truy vấn, mỗi truy vấn yêu cầu tính tổng số lượng ước của các số từ chỉ số l đến r .
- Gọi $d(x)$ là số lượng ước của số nguyên dương x . Yêu cầu tính: $\sum_{i=l}^r d(a_i)$.

Phân tích bài toán:

- **Chiến thuật ban đầu (Vét cạn):** Với mỗi truy vấn (l, r) , ta duyệt qua từng phần tử a_i ($l \leq i \leq r$). Với mỗi a_i , ta đếm số lượng ước bằng cách duyệt từ 1 đến $\sqrt{a_i}$. Độ phức tạp cho mỗi truy vấn là $O((r-l+1) \times \sqrt{\max(a_i)})$. Tổng độ phức tạp $O(q \times n \times \sqrt{\max(a_i)})$, chỉ đạt được **Subtask 1** và **Subtask 2**.
- **Tối ưu hóa 1 (Tiền xử lý số lượng ước):** Nhận thấy $a_i \leq 10^6$, ta có thể dùng thuật toán sàng số nguyên tố hoặc sàng ước số để tính trước mảng $d[v]$ là số lượng ước của mọi số v từ 1 đến 10^6 . Cách nhanh nhất là dùng sàng: duyệt i từ 1 đến 10^6 , với mỗi i , duyệt các bội j của i và tăng $d[j]$ lên 1 đơn vị.
- **Tối ưu hóa 2 (Mảng cộng dồn):** Sau khi có số lượng ước của từng a_i , bài toán trở thành tính tổng trên đoạn $[l, r]$. Ta xây dựng mảng cộng dồn S , trong đó $S[i] = S[i-1] + d(a_i)$. Khi đó, kết quả mỗi truy vấn chỉ tốn $O(1)$ bằng công thức: $S[r] - S[l-1]$.
- **Độ phức tạp cuối cùng:** $O(V \log V + n + q)$ với $V = \max(a_i) = 10^6$. Đây là giải pháp tối ưu cho **Subtask 3**.

33.2.1 Subtask 1 & 2 ($n \times q \leq 10^4$)

```
#include <bits/stdc++.h>
using namespace std;

int count_divisors(int n) {
    int cnt = 0;
    for (int i = 1; i * i <= n; i++) {
        if (n % i == 0) {
            if (i * i == n) cnt++;
            else cnt += 2;
        }
    }
    return cnt;
}
```

```

int main() {
    int n, q;
    cin >> n >> q;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];
    while (q--) {
        int l, r;
        long long sum = 0;
        cin >> l >> r;
        for (int i = l; i <= r; i++) {
            sum += count_divisors(a[i]);
        }
        cout << sum << endl;
    }
    return 0;
}

```

33.2.2 Subtask 3

```

#include <bits/stdc++.h>
using namespace std;

const int MAXA = 1000001;
int d[MAXA];
long long s[1000001];

void sieve() {
    for (int i = 1; i < MAXA; i++) {
        for (int j = i; j < MAXA; j += i) {
            d[j]++;
        }
    }
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    sieve();
    int n, q;
    cin >> n >> q;
    for (int i = 1; i <= n; i++) {
        int x;
        cin >> x;
        s[i] = s[i - 1] + d[x];
    }
    while (q--) {
        int l, r;

```

```

        cin >> l >> r;
        cout << s[r] - s[l - 1] << "\n";
    }
    return 0;
}

```

33.3 Bài 3: Xuất bản

Tóm tắt bài toán:

- Cho dãy A gồm n số nguyên dương.
- Yêu cầu chia dãy thành k đoạn liên tiếp sao cho giá trị cực đại của tổng các phần tử trong mỗi đoạn là nhỏ nhất có thể.
- Đây là bài toán tìm giá trị "Minimax" kinh điển trên dãy số.

Phân tích bài toán:

- **Chiến thuật ban đầu (Quy hoạch động):** Gọi $dp[i][j]$ là giá trị tối ưu khi chia i chương đầu tiên thành j tập. Công thức: $dp[i][j] = \min_{p < i} \{\max(dp[p][j-1], \sum_{x=p+1}^i a_x)\}$. Độ phức tạp $O(n^2 \times k)$, chỉ đáp ứng được các **Subtask 1** và **Subtask 2**.
- **Tối ưu hóa (Chặt nhị phân kết quả):** Nhận thấy nếu một giá trị X có thể là số trang tối đa của một tập, thì mọi giá trị lớn hơn X cũng có thể thỏa mãn (nhưng không tối ưu bằng). Ta thực hiện chặt nhị phân trên miền giá trị của kết quả (từ $\max(a_i)$ đến $\sum a_i$).
- **Hàm kiểm tra (Check):** Với một giá trị mid cố định, ta kiểm tra xem có thể chia dãy thành tối đa k tập mà mỗi tập không vượt quá mid trang hay không bằng thuật toán tham lam: Duyệt từ trái sang phải, gom nhiều chương nhất có thể vào một tập cho đến khi nếu thêm chương tiếp theo sẽ vượt quá mid .
- **Độ phức tạp cuối cùng:** $O(n \cdot \log(\sum a_i))$. Với $n = 10^5$ và $a_i = 10^9$, $\log(\sum a_i) \approx \log(10^{14}) \approx 47$, thuật toán hoàn toàn chạy tốt trong thời gian cho phép cho **Subtask 3** và **Subtask 4**.

33.3.1 Subtask 1 & 2 - Quy hoạch động

```

#include <bits/stdc++.h>
using namespace std;

long long a[105];

```

```

long long s[105];
long long dp[105][105];

int main() {
    int n, k;
    cin >> n >> k;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        s[i] = s[i - 1] + a[i];
    }
    for (int i = 1; i <= n; i++) dp[i][1] = s[i];
    for (int j = 2; j <= k; j++) {
        for (int i = 1; i <= n; i++) {
            dp[i][j] = 2e18;
            for (int p = 1; p < i; p++) {
                dp[i][j] = min(dp[i][j], max(dp[p][j - 1], s[i] - s[p]));
            }
        }
    }
    cout << dp[n][k];
    return 0;
}

```

33.3.2 Subtask 3 & 4 - Chặt nhị phân

```

#include <bits/stdc++.h>
using namespace std;

int n, k;
long long a[100005];

bool check(long long lim) {
    int cnt = 1;
    long long sum = 0;
    for (int i = 1; i <= n; i++) {
        if (a[i] > lim) return false;
        if (sum + a[i] <= lim) {
            sum += a[i];
        } else {
            cnt++;
            sum = a[i];
        }
    }
    return cnt <= k;
}

int main() {
    ios_base::sync_with_stdio(false);

```

```

cin.tie(NULL);
cin >> n >> k;
long long low = 0, high = 0;
for (int i = 1; i <= n; i++) {
    cin >> a[i];
    low = max(low, a[i]);
    high += a[i];
}
long long ans = high;
while (low <= high) {
    long long mid = low + (high - low) / 2;
    if (check(mid)) {
        ans = mid;
        high = mid - 1;
    } else {
        low = mid + 1;
    }
}
cout << ans;
return 0;
}

```

33.4 Bài 4: Chia kẹo

Tóm tắt bài toán:

- Đã có sẵn X cái kẹo. Cần chọn thêm một số hộp kẹo từ N hộp sao cho tổng số kẹo thêm vào là $Y - X$.
- Gọi $T = Y - X$ là số kẹo cần lấy thêm từ các hộp.
- Mục tiêu: Chọn được nhiều hộp kẹo nhất sao cho tổng a_i của các hộp được chọn đúng bằng T .
- Đây là bài toán cái túi (Knapsack) biến thể: Tìm số lượng vật phẩm lớn nhất để đạt được tổng khối lượng chính xác.

Phân tích bài toán:

- **Chiến thuật ban đầu (Đệ quy/Quay lui):** Thử mọi tập con của N hộp kẹo để xem tập nào có tổng bằng T và có số phần tử lớn nhất. Độ phức tạp $O(2^N)$, chỉ dùng cho N nhỏ (Subtask 1).
- **Tối ưu hóa (Quy hoạch động):** Gọi $dp[j]$ là số hộp kẹo tối đa để có tổng bằng j .

- Với mỗi hộp kẹo a_i , ta cập nhật: $dp[j] = \max(dp[j], dp[j - a_i] + 1)$.
- **Lưu ý quan trọng:** Vì N rất lớn (10^5) nhưng giá trị T lại nhỏ ($\leq 10^3$), nếu chạy $O(N \times T)$ thì vẫn đạt yêu cầu nhưng có thể tối ưu hơn bằng cách nhóm các hộp kẹo cùng số lượng. Tuy nhiên, với $T \leq 10^3$, thuật toán $O(N \times T)$ là đủ để vượt qua các Subtask.
- Nếu $X = Y$, ta không cần chọn thêm hộp nào, kết quả là 0 (Subtask 3).

33.4.1 Subtask 1 ($N \leq 20$)

```
#include <bits/stdc++.h>
using namespace std;

int n, x, y, t, res = -1;
int a[25];

void back(int i, int cur_sum, int cnt) {
    if (cur_sum == t) {
        res = max(res, cnt);
        return;
    }
    if (i > n || cur_sum > t) return;
    back(i + 1, cur_sum + a[i], cnt + 1);
    back(i + 1, cur_sum, cnt);
}

int main() {
    cin >> n >> x >> y;
    t = y - x;
    for (int i = 1; i <= n; i++) cin >> a[i];
    if (t < 0) cout << -1;
    else {
        back(1, 0, 0);
        cout << res;
    }
    return 0;
}
```

33.4.2 Subtask 2 → 4

```
#include <bits/stdc++.h>
using namespace std;

int dp[1005];

int main() {
```



```

int n, x, y;
cin >> n >> x >> y;
int t = y - x;

if (t < 0) {
    cout << -1;
    return 0;
}

for (int j = 1; j <= t; j++) dp[j] = -1e9;
dp[0] = 0;

for (int i = 0; i < n; i++) {
    int val;
    cin >> val;
    if (val > t) continue;
    for (int j = t; j >= val; j--) {
        if (dp[j - val] != -1e9) {
            dp[j] = max(dp[j], dp[j - val] + 1);
        }
    }
}

if (dp[t] < 0) cout << -1;
else cout << dp[t];

return 0;
}

```

34 Quảng Bình

34.1 Bài 1: Tính tổng

Tóm tắt bài toán:

- Tìm tất cả các số x sao cho $1 \leq x \leq n$ và $x \vdots 7$.
- Tính và in ra tổng của các số x này.
- Ví dụ: Với $n = 15$, các số chia hết cho 7 là $\{7, 14\}$. Tổng là $7 + 14 = 21$.

Phân tích bài toán:

- **Chiến thuật ban đầu (Duyệt tuyến tính):**

* Duyệt một vòng lặp i từ 1 đến n .

- * Kiểm tra nếu $i \pmod{7} = 0$ thì cộng i vào biến tổng.
- * Độ phức tạp: $O(n)$. Với $n = 10^6$, cách này hoàn toàn khả thi.

– Tối ưu hóa (Cấp số cộng):

- * Các số chia hết cho 7 lập thành một cấp số cộng: $7, 14, 21, \dots, 7k$ với $7k \leq n$.
- * Số các số hạng là $k = \lfloor n/7 \rfloor$.
- * Tổng của dãy là: $S = \frac{k \times (7 + 7k)}{2}$.
- * Độ phức tạp: $O(1)$. Đây là cách tối ưu nhất.

34.1.1 Cách 1: duyệt vòng lặp (Dễ hiểu)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    long long n;
    cin >> n;
    long long sum = 0;
    for (int i = 1; i <= n; i++) {
        if (i % 7 == 0) {
            sum += i;
        }
    }
    cout << sum;
    return 0;
}
```

34.1.2 Cách 2: sử dụng công thức Toán học (Tối ưu)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    long long n;
    cin >> n;
    long long k = n / 7;
    long long first = 7;
    long long last = k * 7;
    long long sum = k * (first + last) / 2;
    cout << sum;
    return 0;
}
```

34.2 Bài 2: Đếm ký tự

Tóm tắt bài toán:

- Cho một xâu văn bản S có thể chứa khoảng trắng.
- Cho một ký tự mục tiêu ch .
- Cần đếm xem ký tự ch xuất hiện bao nhiêu lần trong xâu S .
- Lưu ý: Ký tự hoa và ký tự thường được phân biệt khác nhau (ví dụ: 'h' khác 'H').

Phân tích bài toán:

– Chiến thuật ban đầu:

- * Sử dụng một biến đếm $cnt = 0$.
- * Duyệt qua từng ký tự trong xâu S từ đầu đến cuối.
- * Nếu ký tự đang xét trùng với ch , tăng cnt lên 1.

– Tối ưu hóa:

- * Vì xâu S có thể chứa khoảng trắng, ta không nên dùng $cin \gg S$ vì nó sẽ dừng lại ở khoảng trắng đầu tiên. Thay vào đó, hãy sử dụng `getline(cin, S)`.
- * Độ phức tạp thuật toán là $O(|S|)$, với $|S| = 10^6$, thuật toán chạy cực nhanh trong thời gian cho phép.

34.2.1 Code mẫu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    string s;
    char ch;
    getline(cin, s);
    cin >> ch;

    int cnt = 0;
    for (int i = 0; i < s.length(); i++) {
        if (s[i] == ch) {
            cnt++;
        }
    }
}
```

```
cout << cnt;

return 0;
}
```

34.3 Bài 3: Đoạn đường đẹp nhất

Tóm tắt bài toán:

- Cho dãy A gồm n phần tử và một số nguyên k .
- Cần tìm một đoạn con liên tiếp $a_i, a_{i+1}, \dots, a_j$ sao cho: $\frac{\sum_{m=i}^j a_m}{j-i+1} = k$.
- Mục tiêu: Tìm độ dài đoạn con $(j-i+1)$ lớn nhất thỏa mãn điều kiện trên.

Phân tích bài toán:

- Chiến thuật ban đầu (Vét cạn $O(n^2)$):

- * Duyệt qua mọi cặp (i, j) với $1 \leq i \leq j \leq n$.
- * Tính tổng các phần tử từ i đến j . Để tính nhanh, ta sử dụng mảng cộng dồn $P[i] = a_1 + \dots + a_i$. Tổng đoạn $[i, j]$ là $P[j] - P[i-1]$.
- * Điều kiện trung bình bằng k tương đương với: $P[j] - P[i-1] = k \times (j-i+1)$.
- * Với $n = 10^4$, độ phức tạp $O(n^2)$ thực hiện được khoảng 10^8 phép tính, vẫn có thể chạy kịp trong giới hạn thời gian thông thường (1 giây).

- Tối ưu hóa ($O(n)$ hoặc $O(n \log n)$):

- * Biến đổi điều kiện: $\sum_{m=i}^j a_m = k(j-i+1) \Leftrightarrow \sum_{m=i}^j (a_m - k) = 0$.
- * Đặt một dãy mới B với $b_i = a_i - k$. Bài toán trở thành tìm đoạn con liên tiếp dài nhất của dãy B có tổng bằng 0.
- * Sử dụng mảng cộng dồn S cho dãy B và dùng một cấu trúc dữ liệu Map (hoặc mảng băm) để lưu vị trí xuất hiện đầu tiên của mỗi giá trị $S[x]$. Nếu gặp lại cùng một giá trị tổng, độ dài đoạn là hiệu hai chỉ số.

34.3.1 Subtask 1 ($n \leq 10^3$ - Duyệt $O(n^2)$ cơ bản)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    long long k;
```

```

cin >> n >> k;
vector<long long> a(n + 1);
vector<long long> p(n + 1, 0);
for (int i = 1; i <= n; i++) {
    cin >> a[i];
    p[i] = p[i - 1] + a[i];
}
int max_len = 0;
for (int i = 1; i <= n; i++) {
    for (int j = i; j <= n; j++) {
        long long sum = p[j] - p[i - 1];
        long long len = j - i + 1;
        if (sum == k * len) {
            if (len > max_len) {
                max_len = len;
            }
        }
    }
}
cout << max_len;
return 0;
}

```

34.3.2 Subtask 2 ($10^3 < n \leq 10^4$ - Tối ưu hóa hoặc dùng $O(n^2)$ nhanh)

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    long long k;
    cin >> n >> k;
    vector<long long> a(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }

    long long current_sum = 0;
    int max_len = 0;
    map<long long, int> first_pos;
    first_pos[0] = 0;

    for (int i = 1; i <= n; i++) {
        current_sum += (a[i] - k);
        if (first_pos.count(current_sum)) {
            int len = i - first_pos[current_sum];
            if (len > max_len) {
                max_len = len;
            }
        }
    }
}

```

```

    }
    } else {
        first_pos[current_sum] = i;
    }
}

cout << max_len;
return 0;
}

```

34.4 Bài 4: Robot tìm đường đi

Tóm tắt bài toán:

- Robot đi từ ô $(1,1)$ đến ô (i,j) sao cho $i \times j = k$.
- Quy tắc di chuyển: Chỉ được sang phải hoặc xuống dưới.
- Số ô đi qua (không tính ô đầu) khi đến ô (i,j) là: $(i-1) + (j-1)$.
- Yêu cầu: Tìm cặp (i,j) sao cho $i \times j = k$ và $(i+j-2)$ là nhỏ nhất.

Phân tích bài toán:

- **Chiến thuật ban đầu:** Duyệt mọi cặp số (i,j) sao cho $i \times j = k$. Với mỗi i là ước của k , ta có $j = k/i$. Khi đó số bước đi là $f(i) = i + \frac{k}{i} - 2$.
- **Tối ưu hóa:**
 - * Ta cần tối thiểu hóa hàm $f(i) = i + \frac{k}{i} - 2$. Theo bất đẳng thức Cauchy (AM-GM), tổng của hai số có tích không đổi ($i \times j = k$) đạt giá trị nhỏ nhất khi hai số đó gần nhau nhất (tức là i và j gần $\sqrt{k}$ nhất).
 - * Do đó, ta chỉ cần duyệt i từ 1 đến $\sqrt{k}$. Nếu $k \div i$, ta tính $j = k/i$ và cập nhật giá trị nhỏ nhất của $(i+j-2)$.
 - * Với $k = 10^{10}$, $\sqrt{k} = 10^5$. Vòng lặp chạy 10^5 lần hoàn toàn đáp ứng thời gian cho Subtask cuối.

34.4.1 Subtask 1 & 2 ($k \leq 10^4$)

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    long long k;
    cin >> k;

```

```

long long min_step = k - 1;
for (long long i = 1; i <= k; i++) {
    if (k % i == 0) {
        long long j = k / i;
        if (i + j - 2 < min_step) {
            min_step = i + j - 2;
        }
    }
}
cout << min_step;
return 0;
}

```

34.4.2 Subtask 3 ($k \leq 10^{10}$ - Tối ưu $\sqrt{k}$)

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    long long k;
    cin >> k;
    long long min_step = 2e10;
    for (long long i = 1; i * i <= k; i++) {
        if (k % i == 0) {
            long long j = k / i;
            min_step = min(min_step, i + j - 2);
        }
    }
    cout << min_step;
    return 0;
}

```

35 Quảng Nam

35.1 Bài 1: Bàn học sinh

Tóm tắt bài toán:

- Cho số học sinh hai phòng là a, b và sức chứa của 2 loại bàn là x, y .
- Mỗi phòng chỉ được chọn duy nhất 1 trong 2 loại bàn (x hoặc y) sao cho số học sinh chia hết cho sức chứa của loại bàn đó.
- Tìm cách chọn loại bàn cho phòng 1A và loại bàn cho phòng 1B sao cho tổng số bàn là nhỏ nhất.

Phân tích bài toán:**– Chiến thuật ban đầu:**

- * Xét phòng 1A: Kiểm tra xem a chia hết cho x không, nếu có thì số bàn là a/x . Kiểm tra xem a chia hết cho y không, nếu có thì số bàn là a/y . Chọn số bàn ít nhất cho phòng 1A từ các cách khả thi.
- * Làm tương tự với phòng 1B để tìm số bàn ít nhất.
- * Tổng số bàn cần tìm là: (Số bàn ít nhất phòng 1A)+(Số bàn ít nhất phòng 1B).

– Tối ưu hóa:

- * Vì mỗi phòng chỉ có 2 lựa chọn và các giá trị a, b, x, y đều rất nhỏ ($a, b \leq 50, x, y \leq 4$), ta có thể sử dụng các cấu trúc rẽ nhánh (if-else) hoặc vòng lặp đơn giản.
- * Để đảm bảo tìm "ít nhất", trong trường hợp một phòng chia hết cho cả x và y , ta so sánh giá trị a/x và a/y .

35.1.1 Subtask 1 ($x = y$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int a, b, x, y;
    cin >> a >> b >> x >> y;
    cout << (a / x) + (b / x);
    return 0;
}
```

35.1.2 Subtask 2 ($x = 2, y = 1$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int a, b, x, y;
    cin >> a >> b >> x >> y;
    int res = 0;
    if (a % 2 == 0) res += a / 2;
    else res += a;

    if (b % 2 == 0) res += b / 2;
    else res += b;
}
```



```

    cout << res;
    return 0;
}

```

35.1.3 Subtask 3 (Full - Không ràng buộc)

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    int a, b, x, y;
    cin >> a >> b >> x >> y;

    int minA = 1e9;
    if (a % x == 0) minA = min(minA, a / x);
    if (a % y == 0) minA = min(minA, a / y);

    int minB = 1e9;
    if (b % x == 0) minB = min(minB, b / x);
    if (b % y == 0) minB = min(minB, b / y);

    cout << minA + minB;
    return 0;
}

```

35.2 Bài 2: Mật thư

Tóm tắt bài toán:

- Trích xuất tất cả các chữ cái viết hoa từ xâu S theo thứ tự để tạo thành xâu tiền tố X .
- Tìm tất cả các cụm chữ số liên tiếp trong S , coi mỗi cụm là một số nguyên, rồi tính tổng của chúng.
- Kết quả mật thư R là sự kết hợp của X và tổng vừa tính được (nếu tổng tồn tại).

Phân tích bài toán:

- **Chiến thuật ban đầu:** Duyệt qua từng ký tự của xâu S . Nếu là chữ hoa thì cộng vào xâu kết quả. Nếu là chữ số, ta cần một biến phụ để "gom" các chữ số liên tiếp lại thành một số nguyên hoàn chỉnh trước khi cộng vào tổng.
- **Xử lý số nguyên:** Duyệt xâu, khi gặp một ký tự số, ta bắt đầu đọc tiếp các ký tự sau đó cho đến khi gặp một ký tự không phải số hoặc hết xâu. Chuyển cụm ký tự số này thành số nguyên và cộng vào biến sum.

– Tối ưu hóa:

- * Sử dụng biến kiểu long long cho tổng để tránh tràn số vì n lên tới 10^5 .
- * Độ phức tạp thuật toán là $O(n)$, duyệt xâu một lần duy nhất, phù hợp với mọi subtask.

35.2.1 Subtask 1 (S chỉ gồm chữ cái)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    string s;
    cin >> n >> s;
    for (int i = 0; i < s.length(); i++) {
        if (s[i] >= 'A' && s[i] <= 'Z') {
            cout << s[i];
        }
    }
    return 0;
}
```

35.2.2 Subtask 2 (S có số 1 chữ số, n nhỏ)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    string s;
    cin >> n >> s;
    string x = "";
    long long sum = 0;
    bool hasNumber = false;
    for (int i = 0; i < s.length(); i++) {
        if (s[i] >= 'A' && s[i] <= 'Z') x += s[i];
        else if (s[i] >= '0' && s[i] <= '9') {
            sum += (s[i] - '0');
            hasNumber = true;
        }
    }
    cout << x;
    if (hasNumber) cout << sum;
    return 0;
}
```

35.2.3 Subtask 3 (Full - Số nguyên nhiều chữ số, n lớn)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    string s;
    cin >> n >> s;
    string x = "";
    long long totalSum = 0;
    bool hasNumber = false;

    for (int i = 0; i < n; i++) {
        if (s[i] >= 'A' && s[i] <= 'Z') {
            x += s[i];
        } else if (s[i] >= '0' && s[i] <= '9') {
            hasNumber = true;
            long long currentNum = 0;
            while (i < n && s[i] >= '0' && s[i] <= '9') {
                currentNum = currentNum * 10 + (s[i] - '0');
                i++;
            }
            totalSum += currentNum;
            i--;
        }
    }

    cout << x;
    if (hasNumber) cout << totalSum;

    return 0;
}
```

35.3 Bài 3: Khai thác gỗ

Tóm tắt bài toán:

- Cho một dãy gồm N số nguyên dương v_j là khối lượng của từng cây gỗ.
- Có P truy vấn, mỗi truy vấn cung cấp một giá trị ngưỡng k_i .
- Với mỗi k_i , cần tính tổng các giá trị v_j thỏa mãn $v_j \geq k_i$.

Phân tích bài toán:

- **Chiến thuật ban đầu (Duyệt trực tiếp):** Với mỗi phương án k_i , ta duyệt qua toàn bộ N cây gố. Nếu $v_j \geq k_i$ thì cộng v_j vào tổng. Độ phức tạp $O(P \times N)$. Với $N, P = 10^6, 10^5$, cách này sẽ bị quá thời gian (TLE).
- **Tối ưu hóa (Sử dụng mảng cộng dồn/Tiền xử lý):**
 - * Do khối lượng gố v_j và ngưỡng k_i không quá 10^6 , ta có thể đếm số lần xuất hiện của từng mức khối lượng.
 - * Gọi $cnt[x]$ là số lượng cây có khối lượng đúng bằng x . Khi đó, tổng khối lượng của các cây có khối lượng x là $S[x] = x \times cnt[x]$.
 - * Ta cần tính tổng $S[x]$ với x chạy từ k_i đến 10^6 . Để thực hiện nhanh, ta sử dụng mảng cộng dồn ngược.
 - * Đặt $Total[x] = \sum_{t=x}^{10^6} S[t]$. Ta có công thức truy hồi: $Total[x] = Total[x+1] + S[x]$.
 - * Lúc này, với mỗi truy vấn k_i , kết quả chỉ là $Total[k_i]$ với độ phức tạp $O(1)$.
- **Cách tiếp cận khác (Sắp xếp + Prefix Sum):** Sắp xếp dãy v tăng dần, sau đó dùng chắt nhị phân để tìm vị trí đầu tiên $\geq k_i$ và tính tổng đoạn còn lại bằng mảng cộng dồn.

35.3.1 Subtask 1 & 2 (Duyệt đơn giản)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, p;
    cin >> n;
    vector<int> v(n);
    for (int i = 0; i < n; i++) cin >> v[i];
    cin >> p;
    while (p--) {
        int k;
        cin >> k;
        long long sum = 0;
        for (int i = 0; i < n; i++) {
            if (v[i] >= k) sum += v[i];
        }
        cout << sum << endl;
    }
    return 0;
}
```

35.3.2 Subtask 3 (Full - Tối ưu bằng mảng cộng dồn ngược)

```
#include <bits/stdc++.h>
using namespace std;

long long s[1000005];
long long total[1000005];

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n, p;
    cin >> n;
    for (int i = 0; i < n; i++) {
        int v_val;
        cin >> v_val;
        s[v_val] += v_val;
    }

    for (int i = 1000000; i >= 1; i--) {
        total[i] = total[i + 1] + s[i];
    }

    cin >> p;
    for (int i = 0; i < p; i++) {
        int k;
        cin >> k;
        if (k > 1000000) cout << 0 << "\n";
        else cout << total[k] << "\n";
    }

    return 0;
}
```

35.4 Bài 4: Số đẹp

Tóm tắt bài toán:

- Cho số nguyên dương N . Cần kiểm tra xem $N!$ (tích từ 1 đến N) có chia hết cho $S = \frac{N(N+1)}{2}$ (tổng từ 1 đến N) hay không.
- Vì N có thể lên đến 10^9 , ta không thể tính trực tiếp giá trị của $N!$ và S .

Phân tích bài toán:

- **Phân tích biểu thức:** Ta cần kiểm tra xem $1 \times 2 \times \dots \times N$ có chia hết cho $\frac{N(N+1)}{2}$ hay không.

- Điều này tương đương với việc kiểm tra xem $2 \times (1 \times 2 \times \dots \times N)$ có chia hết cho $N(N+1)$ hay không.
- Vì trong tích $N!$ đã có sẵn thừa số N , bài toán quy về kiểm tra xem $2 \times (N-1)!$ có chia hết cho $(N+1)$ hay không.
- **Chiến thuật ban đầu ($N \leq 15$):** Tính trực tiếp giai thừa và tổng để kiểm tra.
- **Tối ưu hóa (N lớn):**
 - * Nếu $N+1$ là hợp số: Hầu hết các hợp số $N+1$ sẽ là ước của $2 \times (N-1)!$ (ngoại trừ trường hợp đặc biệt $N+1=4$).
 - * Nếu $N+1$ là số nguyên tố: Theo định lý Wilson, $(N)! \equiv -1 \pmod{N+1}$. Điều này dẫn đến việc $N!$ thường không chia hết cho $N+1$ trừ khi $N+1$ nhỏ.
 - * Thực tế, ta chỉ cần kiểm tra xem $N+1$ có phải là số nguyên tố hay không. Nếu $N+1$ là số nguyên tố lớn hơn 2, thì $N!$ sẽ không chia hết cho $N+1$, dẫn đến không chia hết cho S .
 - * Trường hợp đặc biệt: $N=1$ (đẹp), $N=2$ (không đẹp). Với $N>2$, N đẹp khi và chỉ khi $N+1$ là hợp số và $N+1 \neq 4$.

35.4.1 Subtask 1 ($N \leq 15$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int t;
    cin >> t;
    while (t--) {
        long long n;
        cin >> n;
        long long tich = 1;
        for (int i = 1; i <= n; i++) tich *= i;
        long long tong = n * (n + 1) / 2;
        if (tich % tong == 0) cout << "YES" << endl;
        else cout << "NO" << endl;
    }
    return 0;
}
```

35.4.2 Subtask 2 & 3 (Full $N \leq 10^9$)

```
#include <bits/stdc++.h>
using namespace std;
```

```
bool la_so_nguyen_to(long long k) {
    if (k < 2) return false;
    for (long long i = 2; i * i <= k; i++) {
        if (k % i == 0) return false;
    }
    return true;
}

int main() {
    int t;
    cin >> t;
    while (t--) {
        long long n;
        cin >> n;
        if (n == 1) {
            cout << "YES" << endl;
            continue;
        }
        long long k = n + 1;
        if (n == 1) {
            cout << "YES" << endl;
        } else if (la_so_nguyen_to(k)) {
            cout << "NO" << endl;
        } else {
            cout << "YES" << endl;
        }
    }
    return 0;
}
```

36 Quảng Ngãi

36.1 Bài 1: Số chính phương

Tóm tắt bài toán:

- Cho số nguyên dương $n \leq 10^9$.
- Tính tổng các chữ số của n , gọi là S .
- Kiểm tra xem S có phải là số chính phương hay không.
- In ra trạng thái kiểm tra (1 hoặc 0) và giá trị S .

Phân tích bài toán:

- Chiến thuật ban đầu: * Sử dụng vòng lặp while ($n > 0$) kết hợp phép chia lấy dư $\% 10$ và chia nguyên $/ 10$ để tách từng chữ số và cộng dồn vào biến tổng S .
 - * Vì $n \leq 10^9$, số lượng chữ số tối đa là 10 (trong trường hợp $n = 1,000,000,000$), tổng S lớn nhất có thể đạt được là khi $n = 999,999,999 \Rightarrow S = 9 \times 9 = 81$.
- Tối ưu hóa:
 - * Với giá trị S rất nhỏ ($S \leq 81$), ta có thể kiểm tra số chính phương bằng cách khai căn bậc hai $\text{sqrt}(S)$ rồi bình phương ngược lại để so sánh, hoặc dùng một mảng hằng số các số chính phương nhỏ hơn 81 để tra cứu nhanh.
 - * Do giới hạn n nhỏ, thuật toán lặp tách chữ số đạt độ phức tạp $O(\log_{10} n)$, hoàn toàn đáp ứng thời gian thực thi.

36.1.1 Code mẫu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    long long n;
    cin >> n;

    int s = 0;
    long long temp = n;
    while (temp > 0) {
        s += temp % 10;
        temp /= 10;
    }

    int can = sqrt(s);
    if (can * can == s) {
        cout << 1 << " " << s;
    } else {
        cout << 0 << " " << s;
    }

    return 0;
}
```

36.2 Bài 2: Diện tích

Tóm tắt bài toán:

- Cho các thông số của hình thang cân: đáy nhỏ a , đáy lớn b và cạnh bên c .

- Tính chiều cao h của hình thang dựa trên các cạnh đã cho.
- Tính bán kính bồn hoa $r = \frac{h}{3}$.
- Kết quả cần tìm là hiệu số giữa diện tích hình thang S_1 và diện tích hình tròn S_2 .
- Sử dụng hằng số $\pi = 3.14$ và làm tròn đến 2 chữ số thập phân.

Phân tích bài toán:

- Chiến thuật ban đầu:

- * Để tính diện tích hình thang $S_1 = \frac{(a+b) \times h}{2}$, ta cần tìm h .
- * Xét tam giác vuông ADH với H là chân đường cao hạ từ A xuống DC . Đoạn DH được tính bằng: $DH = \frac{b-a}{2}$.
- * Áp dụng định lý Pytago trong tam giác ADH : $h = \sqrt{c^2 - DH^2}$.

- Tối ưu hóa:

- * Bài toán chỉ yêu cầu tính toán công thức đơn giản với độ phức tạp $O(1)$.
- * Cần lưu ý sử dụng kiểu dữ liệu số thực (double) để đảm bảo độ chính xác và dùng hàm làm tròn của ngôn ngữ lập trình.

36.2.1 Code mẫu

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    double a, b, c;
    cin >> a >> b >> c;

    double dh = (b - a) / 2.0;
    double h = sqrt(c * c - dh * dh);

    double s_thang = ((a + b) * h) / 2.0;

    double r = h / 3.0;
    double s_tron = 3.14 * r * r;

    double ket_qua = s_thang - s_tron;

    cout << fixed << setprecision(2) << ket_qua << endl;

    return 0;
}
```

36.3 Bài 3: Người may mắn

Tóm tắt bài toán:

- Cho dãy gồm n số nguyên $a_1, a_2, \dots, a_n$.
- Với mỗi phần tử a_i , ta cần đếm số lần xuất hiện của giá trị a_i trong toàn bộ dãy (gọi số lần này là cnt_i).
- Kiểm tra xem cnt_i có phải là số nguyên tố hay không.
- Nếu cnt_i là số nguyên tố thì in ra 1, ngược lại in ra 0.

Phân tích bài toán:

- **Chiến thuật ban đầu (Dành cho Subtask 1):**
 - * Duyệt qua từng phần tử a_i . Với mỗi a_i , ta dùng một vòng lặp phụ để đếm xem có bao nhiêu phần tử $a_j = a_i$ trong dãy.
 - * Sau khi có số lần xuất hiện, ta viết hàm kiểm tra số nguyên tố cơ bản.
 - * Độ phức tạp: $O(n^2)$, chỉ phù hợp với $n \leq 10^3$.
- **Tối ưu hóa (Dành cho Subtask 2):**
 - * Do n lên đến 10^6 và a_i lên đến 10^9 , ta không thể dùng mảng đếm trực tiếp hay duyệt $O(n^2)$.
 - * Sử dụng cấu trúc dữ liệu `std::map` trong C++ để đếm số lần xuất hiện của từng giá trị trong $O(n \log n)$. Tuy nhiên, với $n = 10^6$, `std::unordered_map` sẽ có tốc độ thực thi nhanh hơn (trung bình $O(n)$).
 - * Số lần xuất hiện tối đa của một giá trị là n (10^6). Ta có thể dùng thuật toán Sàng Eratosthenes để đánh dấu các số nguyên tố từ 1 đến n trong $O(n \log \log n)$.
 - * Bước cuối: Duyệt lại dãy một lần nữa, lấy số lần xuất hiện từ map và tra cứu trong mảng sàng nguyên tố để in kết quả.

36.3.1 Subtask 1 ($O(n^2)$)

```
#include <bits/stdc++.h>
using namespace std;

bool isPrime(int k) {
    if (k < 2) return false;
    for (int i = 2; i * i <= k; i++) {
        if (k % i == 0) return false;
    }
    return true;
}
```

```

    }
    return true;
}

int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; i++) cin >> a[i];

    for (int i = 0; i < n; i++) {
        int cnt = 0;
        for (int j = 0; j < n; j++) {
            if (a[i] == a[j]) cnt++;
        }
        if (isPrime(cnt)) cout << 1 << endl;
        else cout << 0 << endl;
    }
    return 0;
}

```

36.3.2 Subtask 2 (Tối ưu $O(n)$ với unordered_map và Sàng Eratosthenes)

```

#include <bits/stdc++.h>
using namespace std;

bool prime[1000001];

void sieve(int n) {
    for (int i = 2; i <= n; i++) prime[i] = true;
    for (int i = 2; i * i <= n; i++) {
        if (prime[i]) {
            for (int j = i * i; j <= n; j += i)
                prime[j] = false;
        }
    }
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;
    vector<int> a(n);
    unordered_map<int, int> counts;

    for (int i = 0; i < n; i++) {

```

```

        cin >> a[i];
        counts[a[i]]++;
    }

    sieve(n);

    for (int i = 0; i < n; i++) {
        int cnt = counts[a[i]];
        if (prime[cnt]) cout << 1 << "\n";
        else cout << 0 << "\n";
    }

    return 0;
}

```

36.4 Bài 4: Hộp quà

Tóm tắt bài toán:

- Cho dãy n số nguyên phân biệt a_i .
- Với mỗi truy vấn (x, y, z) , ta luôn xét tập hợp S gồm y món quà có giá trị nhỏ nhất trong n món quà ban đầu.
- Nếu $x = 0$: Tính tổng z món quà nhỏ nhất trong tập S .
- Nếu $x = 1$: Tính tổng z món quà lớn nhất trong tập S .
- Do $z \leq y$ và tập S là y phần tử nhỏ nhất của dãy gốc, nên:
 - * $x = 0$: Thực chất là tính tổng z phần tử nhỏ nhất của dãy gốc.
 - * $x = 1$: Thực chất là tính tổng các phần tử từ vị trí $y - z + 1$ đến y sau khi dãy gốc đã được sắp xếp tăng dần.

Phân tích bài toán:

- Chiến thuật ban đầu:

- * Sắp xếp dãy a tăng dần: $a_1 < a_2 < \dots < a_n$.
- * Với mỗi truy vấn:
 - Nếu $x = 0$: Lấy tổng z phần tử đầu tiên: $\sum_{i=1}^z a_i$.
 - Nếu $x = 1$: Trong y phần tử đầu tiên, z phần tử lớn nhất sẽ bắt đầu từ chỉ số $y - z + 1$ đến y . Vậy tổng là: $\sum_{i=y-z+1}^y a_i$.
- * Độ phức tạp: $O(Q \times n)$ nếu mỗi lần đều lặp để tính tổng.

– Tối ưu hóa:

- * Sử dụng kỹ thuật Mảng cộng dồn (Prefix Sum) để tính tổng đoạn trong $O(1)$.
- * Gọi $P[i] = a_1 + a_2 + \dots + a_i$. Khi đó tổng đoạn $[L, R]$ là $P[R] - P[L - 1]$.
- * Với $x = 0$: Kết quả là $P[z]$.
- * Với $x = 1$: Kết quả là $P[y] - P[y - z]$.
- * Tổng độ phức tạp: $O(n \log n)$ cho sắp xếp và $O(n + Q)$ cho tính toán. Phù hợp hoàn toàn với $n, Q \leq 10^5$.

36.4.1 Subtask 1 (Sử dụng mảng cộng dồn cơ bản)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, q;
    cin >> n >> q;
    vector<long long> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];

    sort(a.begin() + 1, a.end());

    vector<long long> p(n + 1, 0);
    for (int i = 1; i <= n; i++) p[i] = p[i - 1] + a[i];

    while (q--) {
        int x, y, z;
        cin >> x >> y >> z;
        if (x == 0) {
            cout << p[z] << endl;
        } else {
            cout << p[y] - p[y - z] << endl;
        }
    }
    return 0;
}
```

36.4.2 Subtask 2 (Tương tự Subtask 1 nhưng tối ưu tốc độ nhập xuất)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
```

```

int n, q;
cin >> n >> q;
vector<long long> a(n + 1);
for (int i = 1; i <= n; i++) {
    cin >> a[i];
}

sort(a.begin() + 1, a.end());

vector<long long> p(n + 1, 0);
for (int i = 1; i <= n; i++) {
    p[i] = p[i - 1] + a[i];
}

for (int i = 0; i < q; i++) {
    int x, y, z;
    cin >> x >> y >> z;
    if (x == 0) {
        cout << p[z] << "\n";
    } else {
        cout << p[y] - p[y - z] << "\n";
    }
}

return 0;
}

```

37 Quảng Ninh

37.1 Bài 1: Ước số

Tóm tắt:

- Cho số nguyên dương n .
- Tìm số nguyên dương a lớn nhất sao cho n chia hết cho a và $a < n$.

Phân tích bài toán:

- Chiến thuật ban đầu:

- * Để tìm ước lớn nhất nhưng nhỏ hơn n , ta có thể duyệt ngược từ $a = n - 1$ về 1.
- * Số đầu tiên mà n chia hết cho a chính là đáp án cần tìm.

- * Chiến thuật này có độ phức tạp $O(n)$, với $n \leq 10^5$ thì hoàn toàn có thể đáp ứng được thời gian.

– Tối ưu hóa:

- * Ta biết rằng nếu a là ước lớn nhất của n ($a < n$), thì $k = n/a$ phải là ước nhỏ nhất của n mà lớn hơn 1.
- * Ước nhỏ nhất lớn hơn 1 của một số chính là ước nguyên tố nhỏ nhất của số đó.
- * Thay vì duyệt từ $n-1$, ta có thể tìm số nguyên k nhỏ nhất sao cho n chia hết cho k ($k \geq 2$). Khi đó, đáp án sẽ là n/k .
- * Độ phức tạp tối ưu: $O(\sqrt{n})$.

37.1.1 Code theo chiến thuật duyệt ngược ($O(n)$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    for (int i = n - 1; i >= 1; i--) {
        if (n % i == 0) {
            cout << i;
            break;
        }
    }
    return 0;
}
```

37.1.2 Code theo chiến thuật tối ưu ($O(\sqrt{n})$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    int k = n;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            k = i;
            break;
        }
    }
}
```

```

if (k == n) cout << 1;
else cout << n / k;
return 0;
}

```

37.2 Bài 2: Tổng dãy

Tóm tắt bài toán:

- Cho dãy a có n phần tử.
- Cần tìm số lượng các đoạn con $[i, j]$ (với $i < j$) sao cho tổng các phần tử trong đoạn đó là số chẵn.

Phân tích bài toán:

– Chiến thuật ban đầu (Dành cho Subtask 1 và 2):

- * Sử dụng hai vòng lặp lồng nhau để duyệt mọi cặp (i, j) .
- * Với mỗi cặp, tính tổng từ a_i đến a_j . Nếu tổng chẵn thì tăng biến đếm.
- * Độ phức tạp: $O(n^3)$ hoặc $O(n^2)$ nếu dùng mảng cộng dồn. Phù hợp với $n \leq 5 \times 10^3$.

– Tối ưu hóa (Dành cho Subtask 3):

- * Gọi $S[k]$ là tổng cộng dồn của k phần tử đầu tiên ($S[k] = a_1 + a_2 + \dots + a_k$).
- * Tổng đoạn $[i, j]$ là $S[j] - S[i - 1]$. Để hiệu này chẵn, $S[j]$ và $S[i - 1]$ phải cùng tính chẵn lẻ (cùng chẵn hoặc cùng lẻ).
- * Ta chỉ cần quan tâm đến tính chẵn lẻ của $S[k]$ (lấy $S[k] \pmod{2}$).
- * Đếm số lượng tổng cộng dồn là số chẵn ($cnt0$) và số lượng tổng cộng dồn là số lẻ ($cnt1$) trong các vị trí từ 0 đến n .
- * Số cặp $(i - 1, j)$ cùng tính chẵn lẻ là: $\frac{cnt0 \times (cnt0 - 1)}{2} + \frac{cnt1 \times (cnt1 - 1)}{2}$.
- * **Lưu ý quan trọng:** Đề bài yêu cầu $i < j$, điều này tương đương với $i - 1 < j$. Công thức tổ hợp trên đã bao gồm tất cả các cặp có chỉ số khác nhau. Tuy nhiên, cần trừ đi các trường hợp có độ dài đoạn bằng 1 (tức là $i = j$) vì đề yêu cầu $i < j$.
- * Độ phức tạp: $O(n)$.

37.2.1 Subtask 1 & 2 ($O(n^2)$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<long long> a(n + 1);
    vector<long long> s(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        s[i] = s[i - 1] + a[i];
    }

    long long count = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = i + 1; j <= n; j++) {
            if ((s[j] - s[i - 1]) % 2 == 0) {
                count++;
            }
        }
    }

    cout << count;
    return 0;
}
```

37.2.2 Subtask 3 ($O(n)$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    long long cnt[2] = {1, 0};
    long long sum = 0;
    long long tot = 0;
    long long dem = 0;

    for (int i = 1; i <= n; i++) {
        long long x;
        cin >> x;
        if (x % 2 == 0) dem++;
        sum += x;
        int par = sum % 2;
        tot += cnt[par];
    }
```

```

        cnt[par]++;
    }

    cout << tot - dem;
    return 0;
}

```

37.3 Bài 3: Trung vị

Tóm tắt bài toán:

- Tìm dãy con liên tiếp $[i, j]$ có độ dài $L = j - i + 1 \geq k$.
- Trong các dãy con đó, tìm giá trị trung vị (phần tử thứ $\lfloor \frac{L+1}{2} \rfloor$ sau khi sắp xếp) lớn nhất có thể đạt được.

Phân tích bài toán:

- **Chiến thuật ban đầu:** Duyệt qua tất cả các cặp (i, j) sao cho $j - i + 1 \geq k$. Với mỗi dãy con, ta copy ra một mảng tạm, sắp xếp lại và lấy phần tử ở vị trí trung vị. Độ phức tạp sẽ là $O(n^3 \log n)$ hoặc $O(n^3)$, chỉ phù hợp với n rất nhỏ (Subtask 3).
- **Tối ưu hóa (Chặt nhị phân kết quả):**
 - * Nhận xét: Nếu một giá trị x có thể là trung vị, thì mọi giá trị nhỏ hơn x cũng có thể là trung vị. Điều này cho phép ta chặt nhị phân trên giá trị của trung vị trong đoạn $[1, n]$.
 - * Để kiểm tra xem giá trị M có thể là trung vị của một đoạn con độ dài ít nhất k hay không:
 - Thay các phần tử $a_i \geq M$ bằng $+1$ và các phần tử $a_i < M$ bằng -1 .
 - Một đoạn con có trung vị $\geq M$ khi và chỉ khi số lượng phần tử $\geq M$ nhiều hơn số lượng phần tử $< M$. Tức là tổng các giá trị $+1$ và -1 trong đoạn đó phải > 0 (đối với L lẻ) hoặc ≥ 0 (tùy quy ước vị trí trung vị). Cụ thể với định nghĩa vị trí $\lfloor \frac{L+1}{2} \rfloor$, điều kiện là: $count(\geq M) \geq \lfloor \frac{L+1}{2} \rfloor$.
 - Điều kiện này tương đương với tổng đoạn con sau khi biến đổi phải > 0 .
 - * Bài toán trở thành: Tìm đoạn con độ dài $\geq k$ có tổng lớn nhất. Nếu tổng này > 0 , thì M có thể là trung vị.
 - * Ta sử dụng mảng cộng dồn P và duy trì $min_P[i] = \min(P[0], \dots, P[i])$. Tổng lớn nhất của đoạn con kết thúc tại j có độ dài ít nhất k là $P[j] - min_P[j-k]$.
- **Độ phức tạp:** $O(n \log n)$.

37.3.1 Subtask 3 ($O(n^3)$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, k;
    cin >> n >> k;
    vector<int> a(n);
    for (int i = 0; i < n; i++) cin >> a[i];

    int res = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i + k - 1; j < n; j++) {
            vector<int> sub;
            for (int l = i; l <= j; l++) sub.push_back(a[l]);
            sort(sub.begin(), sub.end());
            int mid = sub[(sub.size() + 1) / 2 - 1];
            res = max(res, mid);
        }
    }
    cout << res;
    return 0;
}
```

37.3.2 Subtask 4 ($O(n \log n)$)

```
#include <bits/stdc++.h>
using namespace std;

int n, k;
int a[100005];
int b[100005];
int p[100005];
int min_p[100005];

bool check(int m) {
    for (int i = 1; i <= n; i++) {
        if (a[i] >= m) b[i] = 1;
        else b[i] = -1;
    }

    for (int i = 1; i <= n; i++) p[i] = p[i - 1] + b[i];

    min_p[0] = 0;
    for (int i = 1; i <= n; i++) min_p[i] = min(min_p[i - 1], p[i]);

    for (int j = k; j <= n; j++) {
```

```

        if (p[j] - min_p[j - k] > 0) return true;
    }
    return false;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    cin >> n >> k;
    for (int i = 1; i <= n; i++) cin >> a[i];

    int low = 1, high = n, ans = 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (check(mid)) {
            ans = mid;
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    cout << ans;
    return 0;
}

```

37.4 Bài 4: Dãy con tăng

Tóm tắt bài toán:

- Cho hai dãy a và b chỉ gồm các giá trị 0 và 1.
- Tìm độ dài lớn nhất của một dãy con vừa xuất hiện trong a , vừa xuất hiện trong b , đồng thời dãy con đó phải là dãy không giảm (các số 0 luôn đứng trước các số 1).

Phân tích bài toán:

- Chiến thuật ban đầu (Quy hoạch động):

- * Một dãy con không giảm chỉ gồm các số 0 và 1 sẽ có dạng: một chuỗi số 0 liên tiếp theo sau bởi một chuỗi số 1 liên tiếp.
- * Gọi độ dài dãy con chung không giảm là $L = count(0) + count(1)$.
- * Ta có thể thử mọi điểm chia: giả sử dãy con chung kết thúc phần số 0 tại vị trí i trong dãy a và j trong dãy b . Tuy nhiên, cách này rất phức tạp.

- Tối ưu hóa:

- * Để tối đa hóa độ dài, ta cần lấy nhiều số 0 chung nhất có thể, sau đó lấy nhiều số 1 chung nhất có thể.
- * Gọi $cnt0_a[i]$ là số lượng số 0 trong a từ đầu đến vị trí i , $cnt0_b[j]$ tương tự cho b . Số lượng số 0 chung tối đa nếu ta xét đến $a[i]$ và $b[j]$ là $\min(cnt0_a[i], cnt0_b[j])$.
- * Tuy nhiên, quy luật của dãy con (subsequence) đơn giản hơn: số lượng số 0 chung tối đa của hai dãy chính là $\min(\text{tổng số 0 trong } a, \text{tổng số 0 trong } b)$. Gọi giá trị này là $max0$. Tương tự, gọi số lượng số 1 chung tối đa là $max1 = \min(\text{tổng số 1 trong } a, \text{tổng số 1 trong } b)$.
- * **Nhận xét quan trọng:** Vì dãy con không giảm chỉ gồm 0 rồi đến 1, ta có thể chọn một vị trí p trong dãy a và vị trí q trong dãy b để làm "điểm chuyển giao". Trước đó ta lấy toàn bộ số 0 chung, sau đó lấy toàn bộ số 1 chung.
- * Thực tế, với dãy chỉ có 0 và 1, độ dài lớn nhất đơn giản là: $\min(\text{số lượng 0 của } a, \text{số lượng 0 của } b) + \min(\text{số lượng 1 của } a, \text{số lượng 1 của } b)$ là sai, vì các số 0 phải xuất hiện TRƯỚC các số 1 ở cả hai dãy.
- * **Thuật toán chuẩn:** Duyệt mọi vị trí i trong dãy a để làm điểm phân tách. Tại mỗi i , ta đếm số lượng 0 chung của $a[1 \dots i]$ và $b[1 \dots m]$, cộng với số lượng 1 chung của $a[i + 1 \dots n]$ và $b[1 \dots m]$. Nhưng cách này vẫn chưa tối ưu.
- * Cách giải đúng nhất: Gọi $f0(i, j)$ là số lượng số 0 chung dài nhất kết thúc tại $a[i], b[j]$ và $f1(i, j)$ tương tự cho số 1. Vì chỉ có hai giá trị, ta có thể tính:
 - $N0 = \min(\text{tổng số 0 trong } a, \text{tổng số 0 trong } b)$
 - $N1 = \min(\text{tổng số 1 trong } a, \text{tổng số 1 trong } b)$
- * Nhưng vì phải đảm bảo thứ tự, ta duyệt điểm chia i trong dãy a và điểm chia j trong dãy b . Cách tối ưu nhất cho $N = 2 \times 10^5$ là dùng mảng cộng dồn. Kết quả là $\max_{i,j} (\min(pref0_a[i], pref0_b[j]) + \min(suff1_a[i+1], suff1_b[j+1]))$.

37.4.1 Subtask 3 ($O(n \times m)$ hoặc $O(n^2)$)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];
    cin >> m;
    vector<int> b(m + 1);
    for (int i = 1; i <= m; i++) cin >> b[i];
```

```

vector<int> pre0a(n + 1, 0), sufla(n + 2, 0);
for (int i = 1; i <= n; i++) pre0a[i] = pre0a[i - 1] + (a[i] == 0);
for (int i = n; i >= 1; i--) sufla[i] = sufla[i + 1] + (a[i] == 1);

vector<int> pre0b(m + 1, 0), suflb(m + 2, 0);
for (int i = 1; i <= m; i++) pre0b[i] = pre0b[i - 1] + (b[i] == 0);
for (int i = m; i >= 1; i--) suflb[i] = suflb[i + 1] + (b[i] == 1);

int ans = 0;
for (int i = 0; i <= n; i++) {
    for (int j = 0; j <= m; j++) {
        int c0 = min(pre0a[i], pre0b[j]);
        int c1 = min(sufla[i + 1], suflb[j + 1]);
        ans = max(ans, c0 + c1);
    }
}
cout << ans;
return 0;
}

```

37.4.2 Subtask 4 ($O(n + m)$)

- Với $N = 2 \times 10^5$, ta nhận thấy với một i cố định, ta cần tìm j để $\min(pre0_a[i], pre0_b[j]) + \min(suf1_a[i + 1], suf1_b[j + 1])$ đạt cực đại.
- Ta có thể sử dụng hai con trỏ hoặc nhận xét về tính đơn điệu để tìm trong $O(1)$ hoặc $O(\log m)$.

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n; cin >> n;
    vector<int> a(n + 1), sufla(n + 2, 0), pos0a;

    pos0a.push_back(0);

    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        if (a[i] == 0) pos0a.push_back(i);
    }

    for (int i = n; i >= 1; i--) {
        sufla[i] = sufla[i + 1] + (a[i] == 1);
    }
}

```

```

    }

    int m; cin >> m;
    vector<int> b(m + 1), suf1b(m + 2, 0), pos0b;

    pos0b.push_back(0);

    for (int i = 1; i <= m; i++) {
        cin >> b[i];
        if (b[i] == 0) pos0b.push_back(i);
    }

    for (int i = m; i >= 1; i--) {
        suf1b[i] = suf1b[i + 1] + (b[i] == 1);
    }

    int max0 = min((int)pos0a.size() - 1, (int)pos0b.size() - 1);
    int ans = 0;

    for (int k = 0; k <= max0; k++) {
        int posA = pos0a[k];
        int posB = pos0b[k];

        int onesA = suf1a[posA + 1];
        int onesB = suf1b[posB + 1];

        int t = min(onesA, onesB);

        ans = max(ans, k + t);
    }

    cout << ans;
    return 0;
}

```

38 Quảng Trị

38.1 Bài 1: BƠM NƯỚC

Phân tích thuật toán: Bài toán yêu cầu tìm số lần bơm ít nhất để mực nước từ X đạt tối thiểu Y , với mỗi lần bơm tăng K đơn vị. Gọi lượng nước cần bơm thêm là $D = Y - X$. Số lần bơm cần thiết chính là phép chia lấy phần nguyên trần (ceiling) của D/K .

38.1.1 Subtask 1: $X, Y \leq 10^8, K \leq 10^5$

Phân tích: Với giới hạn nhỏ, ta có thể dùng vòng lặp `while` để mô phỏng quá trình bơm. Chẳng nào $X < Y$, ta tiếp tục cộng thêm K vào X và tăng biến đếm số lần bơm. Trong trường hợp xấu nhất ($D = 10^8, K = 1$), vòng lặp chạy 10^8 lần, vẫn kịp thời gian giới hạn 1 giây.

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int x, y, k;
int ans = 0;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> x >> y;
    cin >> k;

    while (x < y) {
        x += k;
        ans++;
    }

    cout << ans << "\n";
}
```

38.1.2 Subtask 2: $X, Y \leq 10^{12}, K \leq 10^4$

Phân tích: Với $Y \leq 10^{12}$, vòng lặp mô phỏng sẽ chạy tối đa 10^{12} lần, dẫn tới quá thời gian (TLE). Ta phải dùng công thức toán học $O(1)$. Số lần bơm tối thiểu là $\lceil \frac{Y-X}{K} \rceil$. Trong lập trình số nguyên, công thức tính nhanh phần nguyên trần của phép chia A/B (với $A, B > 0$) là: $(A + B - 1)/B$. Áp dụng vào bài toán: $\text{ans} = (y - x + k - 1) / k$.

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int x, y, k;
int ans;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
```



```

cin >> x >> y;
cin >> k;

if (y <= x) {
    ans = 0;
} else {
    ans = (y - x + k - 1) / k;
}

cout << ans << "\n";
}

```

38.2 Bài 2: TAM GIÁC

Phân tích thuật toán: Dựa vào bất đẳng thức tam giác, ba số nguyên dương a, b, c lập thành độ dài ba cạnh của một tam giác khi và chỉ khi cạnh thứ ba c thỏa mãn điều kiện:

$$|a - b| < c < a + b$$

Vì c là số nguyên, nên giá trị nhỏ nhất của c là $|a - b| + 1$ và giá trị lớn nhất của c là $a + b - 1$.

38.2.1 Subtask 1: $a, b \leq 10^5$

Phân tích: Với giới hạn $a, b \leq 10^5$, khoảng giá trị lớn nhất của c cỡ 2×10^5 . Ta hoàn toàn có thể dùng một vòng lặp chạy từ giá trị nhỏ nhất đến giá trị lớn nhất của c để đếm số lượng. Thuật toán này có độ phức tạp $O(\max(a, b))$, chạy rất an toàn trong thời gian cho phép.

```

#include <bits/stdc++.h>
#define int long long
using namespace std;

int a, b;
int ans = 0;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> a >> b;

    for(int i = abs(a - b) + 1; i < a + b; i++) {

```

```

        ans++;
    }

    cout << ans << "\n";
}

```

38.2.2 Subtask 2: $a, b \leq 10^9$ (Full Test)

Phân tích: Với $a, b \leq 10^9$, vòng lặp sẽ gây ra lỗi Quá thời gian (TLE). Ta phải áp dụng công thức Toán học để tính trực tiếp số lượng c trong $O(1)$. Số lượng các số nguyên c trong khoảng $(|a - b|, a + b)$ là:

$$\text{Số lượng} = (a + b - 1) - (|a - b| + 1) + 1 = a + b - |a - b| - 1$$

Nếu giả sử $a \leq b$, ta có $|a - b| = b - a$. Khi đó:

$$\text{Số lượng} = a + b - (b - a) - 1 = 2a - 1$$

Tổng quát với mọi a, b , số lượng các giá trị c thỏa mãn luôn luôn là:

$$2 \times \min(a, b) - 1$$

```

#include <bits/stdc++.h>
#define int long long
using namespace std;

int a, b;
int ans;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> a >> b;

    ans = 2 * min(a, b) - 1;

    cout << ans << "\n";
}

```

38.3 Bài 3: CHIA KẸO

Phân tích thuật toán: Với Subtask 1 và 2, các cách tiếp cận duyệt đến $\sqrt{G}$ hoặc phân tích thừa số nguyên tố bằng vòng lặp thông thường là đủ để vượt qua. Tuy nhiên, ở

Subtask 3 với giới hạn $A, B \leq 10^{18}$, độ phức tạp $O(\sqrt{G})$ tương đương với 10^9 phép tính, hoàn toàn có nguy cơ bị TLE nếu hệ thống giới hạn thời gian chặt chẽ (vd: 1 giây).

Để đạt được tốc độ nhanh nhất, ta sử dụng tổ hợp thuật toán nâng cao:

1. **Kiểm tra tính nguyên tố Miller-Rabin:** Thuật toán xác suất cho phép kiểm tra một số $N \leq 10^{18}$ có phải là số nguyên tố hay không trong thời gian $O(\log N)$.
2. **Thuật toán phân tích Pollard's Rho:** Một thuật toán phân tích số nguyên lớn thành các thừa số với độ phức tạp kỳ vọng cực nhỏ $O(N^{1/4})$. Nếu kết hợp với Miller-Rabin, quá trình phân tích sẽ dừng lại ngay khi phần tử đang xét là số nguyên tố.
3. **Sinh tập ước bằng Đệ quy (Backtracking):** Sau khi có được các thừa số nguyên tố và số mũ tương ứng, ta duyệt đệ quy để tổ hợp ra tất cả các ước số của G , lưu vào mảng và sắp xếp.

Với cách làm này, thời gian chạy cho 10^{18} gần như không đáng kể (vài mili-giây). Do phải nhân hai số cỡ 10^{18} có thể gây tràn kiểu long long, ta sẽ sử dụng kiểu `__int128_t` để xử lý các phép nhân module.

38.3.1 Code Full

```
#include<bits/stdc++.h>
#define int long long
using namespace std;

int a, b, g;
int rf[100], nr;
int p[100], c[100], np;
int d[105000], nd;

int mul(int x, int y, int m){
    return (__int128_t)x * y % m;
}

int pw(int b, int e, int m){
    int r = 1;
    b %= m;

    while(e){
        if(e & 1) r = mul(r, b, m);
        b = mul(b, b, m);
        e >>= 1;
    }

    return r;
}
```

```

}

bool mr(int n){
    if(n < 2) return 0;
    if(n == 2 || n == 3 || n == 5 || n == 7) return 1;
    if(n % 2 == 0 || n % 3 == 0 || n % 5 == 0 || n % 7 == 0) return 0;

    int x = n - 1, s = 0;

    while(x % 2 == 0){
        x /= 2;
        s++;
    }

    int bs[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37};

    for(int a : bs){
        if(n <= a) break;

        int y = pw(a, x, n);

        if(y == 1 || y == n - 1) continue;

        bool ok = 0;

        for(int i = 1; i < s; i++){
            y = mul(y, y, n);

            if(y == n - 1){
                ok = 1;
                break;
            }
        }

        if(!ok) return 0;
    }

    return 1;
}

int pr(int n){
    if(n % 2 == 0) return 2;
    if(mr(n)) return n;

    int x = 2, y = 2, g = 1, k = 1;

    while(g == 1){
        x = (mul(x, x, n) + k) % n;
    }
}

```

```

        y = (mul(y, y, n) + k) % n;
        y = (mul(y, y, n) + k) % n;

        g = __gcd(abs(x - y), n);

        if(g == n){
            x = rand() % (n - 2) + 2;
            y = x;
            k = rand() % (n - 1) + 1;
            g = 1;
        }
    }

    return g;
}

void fac(int n){
    if(n == 1) return;

    if(mr(n)){
        rf[nr++] = n;
        return;
    }

    int x = pr(n);

    fac(x);
    fac(n / x);
}

void gen(int i, int v){
    if(i == np){
        d[++nd] = v;
        return;
    }

    int t = 1;

    for(int j = 0; j <= c[i]; j++){
        gen(i + 1, v * t);

        if(j < c[i]) t *= p[i];
    }
}

main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
}

```

```

srand(time(0));

cin >> a >> b;

g = __gcd(a, b);

fac(g);

sort(rf, rf + nr);

for(int i = 0; i < nr; i++){
    if(i == 0 || rf[i] != rf[i - 1]){
        p[np] = rf[i];
        c[np] = 1;
        np++;
    }
    else{
        c[np - 1]++;
    }
}

gen(0, 1);

sort(d + 1, d + nd + 1);

for(int i = 1; i <= nd; i++){
    cout << d[i] << " " << a / d[i] << " " << b / d[i] << '\n';
}
}

```

38.4 Bài 4: SỐ CAMERAS

Phân tích thuật toán: Bài toán yêu cầu tính số lượng camera giám sát tại mỗi khu vực. Với mỗi camera đặt tại vị trí c_i , nó sẽ giám sát một đoạn các khu vực từ $\max(1, c_i - r)$ đến $\min(n, c_i + r)$. Bản chất của bài toán là cập nhật tăng giá trị trên một đoạn $[L, R]$ liên tiếp và sau đó truy vấn giá trị của từng phần tử.

38.4.1 Subtask 1, 2 & 3: $n, q \leq 7000$ hoặc $r = 0$

Phân tích: Với các subtask nhỏ, ta có thể áp dụng thuật toán mô phỏng trực tiếp (Duyệt trâu). Với mỗi camera nhập vào, ta dùng một vòng lặp for chạy từ giới hạn trái L đến giới hạn phải R và tăng số đếm của các khu vực đó lên 1. Tuy nhiên, cách

làm này có độ phức tạp là $O(q \times r)$. Nếu q và r đều lớn cỡ 10^5 , số phép tính sẽ lên tới 10^{10} , gây ra lỗi Quá thời gian (TLE) ở Subtask 4.

38.4.2 Subtask 4: Không có ràng buộc gì thêm ($n, q \leq 10^5$)

Phân tích thuật toán tối ưu $O(n + q)$: Để khắc phục vấn đề TLE, ta sử dụng kỹ thuật **Mảng hiệu số (Difference Array)**. Mảng hiệu số `diff` hoạt động dựa trên nguyên lý: để cộng thêm V vào tất cả các phần tử trong đoạn $[L, R]$, ta chỉ cần thực hiện 2 thao tác:

- `diff[L] += V` (Đánh dấu điểm bắt đầu tăng).
- `diff[R + 1] -= V` (Đánh dấu điểm kết thúc sự tăng).

Trong bài toán này, $V = 1$. Sau khi đã xử lý xong q thao tác đánh dấu điểm nút cho tất cả các camera ($O(q)$), ta tiến hành cộng dồn (Prefix Sum) trên mảng `diff` từ 1 đến n . Giá trị cộng dồn tại vị trí i chính là tổng số camera giám sát khu vực i ($O(n)$).

Tổng thời gian chạy là $O(n + q)$, cực kỳ nhanh và tối ưu tuyệt đối cho giới hạn 10^5 .

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int n, q, r;
int diff_arr[100005];
int c;
int cur_sum = 0;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n >> q >> r;

    for (int i = 1; i <= q; i++) {
        cin >> c;

        int left_idx = max(1LL, c - r);
        int right_idx = min(n, c + r);

        diff_arr[left_idx]++;
        diff_arr[right_idx + 1]--;
    }

    for (int i = 1; i <= n; i++) {
```

```

        cur_sum += diff_arr[i];
        cout << cur_sum << " ";
    }
}

```

39 Sốc Trắng

39.1 Bài 1: PHẦN THƯỞNG

Phân tích thuật toán: Bài toán yêu cầu tính tổng số tiền thưởng mà An nhận được từ nhà trường, mẹ và bố.

- Số tiền nhận được từ nhà trường là X .
- Số tiền nhận được từ mẹ là Y .
- Số tiền nhận được từ bố bằng số tiền của mẹ, tức là Y .

Vậy tổng số tiền thưởng mà An nhận được sẽ được tính bằng công thức: $X + Y + Y$ hoặc $X + 2 \times Y$. Với giới hạn $1 \leq X, Y \leq 1000$, phép tính rất nhỏ và hoàn toàn nằm trong giới hạn lưu trữ số nguyên cơ bản, độ phức tạp thuật toán là $O(1)$.

39.1.1 Code mẫu

```

#include <bits/stdc++.h>
#define int long long
using namespace std;

int x, y;
int ans;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> x >> y;

    ans = x + y + y;

    cout << ans << "\n";
}

```


39.2 Bài 2: BÁNH PÍA

Phân tích thuật toán: Bài toán yêu cầu tính tổng số tiền khi mua N bánh Pía loại K . Ta chỉ cần sử dụng cấu trúc rẽ nhánh (if - else hoặc switch - case) để xác định đơn giá của loại bánh K tương ứng:

- Nếu $K = 1$, giá là 70000.
- Nếu $K = 2$, giá là 60000.
- Nếu $K = 3$, giá là 50000.

Sau khi có đơn giá, tổng số tiền cần trả là $N \times$ đơn giá. Do giới hạn $N \leq 1000$ nên tổng số tiền tối đa là 70,000,000, hoàn toàn nằm trong giới hạn của kiểu số nguyên thông thường. Tuy nhiên để an toàn và đồng nhất, ta vẫn sử dụng kiểu long long. Độ phức tạp thuật toán là $O(1)$.

39.2.1 Code mẫu

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int n, k;
int price;
int ans;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n >> k;

    if (k == 1) {
        price = 70000;
    } else if (k == 2) {
        price = 60000;
    } else if (k == 3) {
        price = 50000;
    }

    ans = n * price;

    cout << ans << "\n";
}
```

39.3 Bài 3: MUA SÁCH

Phân tích thuật toán: Bài toán yêu cầu tìm một đoạn con liên tiếp ngắn nhất có tổng các phần tử lớn hơn hoặc bằng X . Vì tất cả các phần tử A_i đều là số nguyên dương ($A_i \geq 1$), tổng của một đoạn con sẽ luôn tăng lên khi ta mở rộng đoạn đó. Điều này cho phép ta áp dụng các kỹ thuật tối ưu thay vì phải duyệt mọi đoạn con.

39.3.1 Subtask 1, 2 & 3: $N \leq 1000$ hoặc mảng đặc biệt

Phân tích: Với giới hạn $N \leq 1000$, ta có thể duyệt qua tất cả các điểm bắt đầu i . Từ mỗi điểm i , ta dùng một vòng lặp j chạy tiếp để cộng dồn vào tổng. Ngay khi tổng $\geq X$, ta cập nhật kết quả là đoạn ngắn nhất và dừng vòng lặp j (vì nếu mở rộng thêm thì độ dài chỉ tăng lên, không thể tối ưu hơn). Thuật toán này có độ phức tạp $O(N^2)$, đủ để vượt qua Subtask 1, 2 và 3.

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int n, x;
int a[100005];
int ans = 1e15;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n >> x;
    for(int i = 1; i <= n; i++) {
        cin >> a[i];
    }

    for(int i = 1; i <= n; i++) {
        int sum = 0;
        for(int j = i; j <= n; j++) {
            sum += a[j];
            if(sum >= x) {
                ans = min(ans, j - i + 1);
                break;
            }
        }
    }

    if(ans == 1e15) {
        cout << 0 << "\n";
    }
}
```

```

    } else {
        cout << ans << "\n";
    }
}

```

39.3.2 Subtask 4: Không có ràng buộc gì thêm ($N \leq 10^5$)

Phân tích: Với $N \leq 10^5$, thuật toán $O(N^2)$ sẽ bị quá thời gian. Ta cần áp dụng kỹ thuật **Hai con trỏ (Sliding Window)** để giảm độ phức tạp xuống $O(N)$.

- Ta duy trì một "cửa sổ" từ vị trí l đến vị trí r .
- Lần lượt tăng r từ 1 đến N để mở rộng cửa sổ sang phải, đồng thời cộng giá trị $a[r]$ vào biến sum .
- Nếu $sum \geq X$, ta đã tìm được một đoạn thỏa mãn. Ta cập nhật kết quả nhỏ nhất, sau đó thử **thu hẹp cửa sổ từ bên trái** (tăng l và trừ $a[l]$ khỏi sum) để xem có thể tìm được đoạn nào ngắn hơn mà vẫn thỏa mãn tổng $\geq X$ hay không. Quá trình thu hẹp lặp lại cho đến khi $sum < X$.

```

#include <bits/stdc++.h>
#define int long long
using namespace std;

int n, x;
int a[100005];
int ans = 1e15;
int sum = 0;
int l = 1;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n >> x;
    for(int i = 1; i <= n; i++) {
        cin >> a[i];
    }

    for(int r = 1; r <= n; r++) {
        sum += a[r];

        while(sum >= x) {
            ans = min(ans, r - l + 1);
            sum -= a[l];
        }
    }
}

```

```

        l++;
    }
}

if(ans == 1e15) {
    cout << 0 << "\n";
} else {
    cout << ans << "\n";
}
}

```

39.4 Bài 4: ROBOT

Phân tích thuật toán: Bài toán yêu cầu giải mã một xâu ký tự nén theo thuật toán RLE (Run-length encoding). Thuật toán chung để giải quyết cho mọi subtask là duyệt qua từng ký tự của xâu Y :

- Sử dụng một biến cnt để lưu số lần lặp, khởi tạo bằng 0.
- Nếu ký tự đang xét là một chữ số (từ '0' đến '9'), ta ghép nó vào biến cnt bằng công thức: $cnt = cnt * 10 + (Y[i] - '0')$. Điều này giúp ta xử lý được cả các số có nhiều chữ số (Subtask 4).
- Nếu ký tự đang xét là một chữ cái ('E', 'S', 'W', 'N'):
 - * Nếu $cnt == 0$, nghĩa là chữ cái này đứng một mình không có số phía trước (ví dụ 'W'), ta gán $cnt = 1$.
 - * In chữ cái đó ra màn hình cnt lần.
 - * Sau khi in xong, bắt buộc phải reset $cnt = 0$ để chuẩn bị tính toán cho cụm ký tự tiếp theo.

Thuật toán này chỉ duyệt qua xâu Y đúng một lần và trực tiếp in ra xâu giải mã X mà không cần lưu lại toàn bộ 10^5 ký tự vào bộ nhớ, giúp tiết kiệm không gian và đạt thời gian chạy tối ưu $O(|Y| + |X|)$.

39.4.1 Code mẫu cho toàn bộ Subtasks

```

#include <bits/stdc++.h>
#define int long long
using namespace std;

string y;

```

```

int cnt = 0;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> y;

    for (int i = 0; i < y.length(); i++) {
        if (y[i] >= '0' && y[i] <= '9') {
            cnt = cnt * 10 + (y[i] - '0');
        } else {
            if (cnt == 0) {
                cnt = 1;
            }
            for (int j = 0; j < cnt; j++) {
                cout << y[i];
            }
            cnt = 0;
        }
    }

    cout << "\n";
}

```

40 Tây Ninh

40.1 Bài 1: Counting

40.1.1 Lời giải:

Sử dụng vòng for bước nhảy 5 để duyệt và in ra.

40.1.2 Code mẫu:

```

#include <bits/stdc++.h>
using namespace std;

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    int n;
    cin >> n;
    for (int i = 0; i <= n; i += 5) cout << i << "\n";
    return 0;
}

```

40.2 Bài 2: Min3

40.2.1 Lời giải:

Sắp xếp lại dãy số theo thứ tự tăng dần và in ra 3 phần tử đầu tiên của dãy.

40.2.2 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

const int N = 1e3 + 5;
int a[N];

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) cin >> a[i];
    sort(a + 1, a + n + 1);
    cout << a[1] << " " << a[2] << " " << a[3] << "\n";
    return 0;
}
```

40.3 Bài 3: Primes

40.3.1 Subtask 1, 2, 3: $1 \leq n \leq 10^4, 1 \leq A_i \leq 10^6$

Duyệt qua từng phần tử trong dãy. Với mỗi phần tử, kiểm tra xem phần tử đó có phải là số nguyên tố hay không.

Độ phức tạp: $O(N \times \sqrt{\max A})$.

40.3.2 Subtask 4: $1 \leq n \leq 10^6, 1 \leq A_i \leq 10^6$

Nếu sử dụng hàm kiểm tra số nguyên tố với độ phức tạp $O(\sqrt{n})$ để kiểm tra từng phần tử có phải số nguyên tố hay không thì sẽ bị quá thời gian.

Ở đây, ta thấy, $1 \leq A_i \leq 10^6$, nên có thể dùng sàng nguyên tố để đánh dấu xem một số có là số nguyên tố hay không.

Độ phức tạp: $O(N \log(\log N))$.

40.3.3 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

const int N = 1e6 + 5;
int a[N], isPrime[N];

void sieve() {
    for (int i = 2; i <= 1000000; i++) isPrime[i] = 1;
    for (int i = 2; i <= 1000; i++)
        if (isPrime[i])
            for (int j = i * i; j <= 1000000; j += i) isPrime[j] = 0;
}

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) cin >> a[i];
    sieve();
    int ans = 0;
    for (int i = 1; i <= n; i++)
        if (isPrime[a[i]]) ans++;
    cout << ans << "\n";
    return 0;
}
```

40.4 Bài 4: Pair

40.4.1 Lời giải:

Để dễ dàng cho việc tính toán, sắp xếp dãy a theo thứ tự tăng dần.

Với mỗi b_i ($1 \leq i \leq m$), duyệt qua từng phần tử a_j ($1 \leq j \leq n$, tìm phần tử a_k thỏa mãn $a_k - a_j \geq b_i$ hay $a_k \geq b_i + a_j$. Khi đó, các phần tử x trong đoạn $[k, n]$ đều thỏa mãn $a_x - a_j \geq b_i$. Ta có thể dùng kỹ thuật hai con trỏ hoặc tìm kiếm nhị phân để tìm. Tuy nhiên, thuật toán tìm kiếm nhị phân chỉ làm được đến subtask 3 của bài này.

40.4.2 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

const int N = 5e3 + 5;
```

```

int a[N],b[N];

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    int n,m;
    cin >> n >> m;
    for (int i = 1; i <= n; i++) cin >> a[i];
    for (int i = 1; i <= m; i++) cin >> b[i];
    sort(a + 1,a + n + 1);
    for (int i = 1; i <= m; i++) {
        int ans = 0;
        int k = 1;
        for (int j = 1; j <= n; j++) {
            while (k <= n && a[k] < b[i] + a[j]) k++;
            ans += n - k + 1;
        }
        cout << ans << "\n";
    }
}

```

41 Thái Nguyên

41.1 Bài 1: CẶP SỐ NGUYÊN TỔ CÙNG NHAU

Tóm tắt đề bài: Cho số nguyên dương n . Yêu cầu tìm số lượng các số nguyên dương $x < n$ sao cho $\gcd(x, n) = 1$. Đây chính là định nghĩa của hàm Phi Euler, ký hiệu là $\phi(n)$.
Giới hạn: $n \leq 2 \times 10^6$.

Phân tích thuật toán:

- **Subtask 1** ($n \leq 2 \times 10^3$): Với n nhỏ, ta có thể duyệt trực tiếp mọi số x từ 1 đến $n-1$, sau đó sử dụng thuật toán Euclid để tính $\gcd(x, n)$. Nếu $\gcd(x, n) = 1$ thì tăng biến đếm. Độ phức tạp $O(n \log n)$.
- **Subtask 2** ($n \leq 2 \times 10^6$): Cách duyệt trực tiếp sẽ tốn nhiều thời gian nếu có nhiều bộ test hoặc n lớn. Ta sử dụng công thức hàm Phi Euler dựa trên các thừa số nguyên tố của n :

$$\phi(n) = n \cdot \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

Trong đó p là các ước nguyên tố phân biệt của n .

- **Các bước thực hiện:**

1. Khởi tạo $\text{ans} = n$.

2. Duyệt các số i từ 2 đến $\sqrt{n}$ để tìm các ước nguyên tố của n .
3. Nếu $n \pmod i = 0$:
 - * Cập nhật $\text{ans} = \text{ans} - \text{ans} / i$.
 - * Chia n cho i liên tục để loại bỏ hoàn toàn ước nguyên tố này.
4. Sau vòng lặp, nếu $n > 1$ thì n còn lại là một ước nguyên tố cuối cùng, cập nhật $\text{ans} = \text{ans} - \text{ans} / n$.

Độ phức tạp: $O(\sqrt{n})$. Với $n = 2 \times 10^6$, $\sqrt{n} \approx 1414$, thuật toán chạy cực nhanh.

41.1.1 Code mẫu (Không comment)

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int n, temp, ans;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n;

    temp = n;
    ans = n;

    for (int i = 2; i * i <= temp; i++) {
        if (temp % i == 0) {
            while (temp % i == 0) {
                temp /= i;
            }
            ans -= ans / i;
        }
    }

    if (temp > 1) {
        ans -= ans / temp;
    }

    cout << ans;
}
```

41.2 Bài 2: TÌM XÂU KÍ TỰ

Tóm tắt đề bài: Đếm số lần xuất hiện của bất kỳ hoán vị nào của xâu A ($|A| \leq 100$) bên trong xâu B ($|B| \leq 10000$). Hai xâu là hoán vị của nhau nếu chúng có cùng độ dài và số lượng các ký tự tương ứng hoàn toàn giống nhau.

41.2.1 Subtask 1: Độ dài xâu A bằng 1

Phân tích: Khi $|A| = 1$, xâu A chỉ chứa duy nhất một ký tự. Hoán vị của một ký tự chính là chính nó. Bài toán trở thành đếm xem ký tự $A[0]$ xuất hiện bao nhiêu lần trong xâu B . Ta chỉ cần một vòng lặp đơn duyệt qua B .

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

string A, B;
int ans;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> A >> B;
    char target = A[0];
    ans = 0;

    for (int i = 0; i < B.length(); i++) {
        if (B[i] == target) ans++;
    }

    cout << ans;
}
```

41.2.2 Subtask 2: Xâu A chỉ chứa các ký tự giống nhau

Phân tích: Ví dụ $A = "bbb"$. Một hoán vị của A cũng chỉ có thể là $"bbb"$. Ta cần tìm tất cả các đoạn con liên tiếp trong B có độ dài $|A|$ mà mọi ký tự đều là ký tự của A . Ta có thể sử dụng kỹ thuật kiểm tra từng đoạn con.

```
#include <bits/stdc++.h>
#define int long long
```

```
using namespace std;

string A, B;
int n, m, ans = 0;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);

    cin >> A >> B;
    n = A.length();
    m = B.length();
    char target = A[0];

    for (int i = 0; i <= m - n; i++) {
        bool ok = true;
        for (int j = i; j < i + n; j++) {
            if (B[j] != target) {
                ok = false;
                break;
            }
        }
        if (ok) ans++;
    }
    cout << ans << "\n";
}
```

41.2.3 Subtask 3: Không có ràng buộc thêm (Full Test)

Phân tích tối ưu (Cửa sổ trượt): Để kiểm tra một đoạn con có là hoán vị của A hay không, ta so sánh bảng tần số ký tự. Thay vì mỗi lần đều đếm lại từ đầu, ta có thể duy trì bảng tần số của cửa sổ độ dài $|A|$ trên xâu B . Khi cửa sổ trượt sang phải, ta bỏ ký tự bên trái và thêm ký tự bên phải vào bảng đếm.

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

string A, B;
int cntA[256], cntB[256];
int n, m, ans;

bool check() {
    for (int i = 0; i < 256; i++) {
        if (cntA[i] != cntB[i]) return 0;
    }
}
```

```

    return 1;
}

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> A >> B;
    n = A.length();
    m = B.length();

    if (n > m) {
        cout << 0;
        return 0;
    }

    for (int i = 0; i < n; i++) {
        cntA[A[i]]++;
        cntB[B[i]]++;
    }

    ans = 0;
    if (check()) ans++;

    for (int i = n; i < m; i++) {
        cntB[B[i]]++;
        cntB[B[i - n]]--;
        if (check()) ans++;
    }

    cout << ans;
}

```

41.3 Bài 3: TRÒ CHƠI DÂY SỐ

Phân tích Toán học: Giá trị của a_i sau khi biến đổi liên tục bằng tổng các chữ số của nó cho đến khi còn 1 chữ số chính là **Căn số** (Digital Root). Công thức toán học của nó là:

- Nếu $i \pmod 9 \neq 0$ thì $a_i = i \pmod 9$.
- Nếu $i \pmod 9 = 0$ thì $a_i = 9$.
- Công thức tổng quát: $a_i = (i - 1) \pmod 9 + 1$.

Dãy số sau biến đổi sẽ lặp lại chu kỳ 9 số: 1, 2, 3, 4, 5, 6, 7, 8, 9, 1, 2, 3, ...

41.3.1 Subtask 1: $1 \leq l \leq r \leq 9$

Phân tích: Trong phạm vi này, giá trị a_i chính bằng i . Ta chỉ cần tính tổng các số tự nhiên từ l đến r .

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int Q, L, R;

main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);

    cin >> Q;
    while (Q--) {
        cin >> L >> R;
        int sum = 0;
        for (int i = L; i <= R; i++) sum += i;
        cout << sum << '\n';
    }
}
```

41.3.2 Subtask 2: $r - l \leq 10^3$

Phân tích: Với khoảng cách $r - l$ nhỏ, ta có thể duyệt qua từng số trong đoạn $[L, R]$. Với mỗi số i , ta tính a_i theo công thức căn số và cộng dồn vào kết quả.

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int Q, L, R;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);
    cin >> Q;
    while (Q--) {
        cin >> L >> R;
        int res = 0;
        for (int i = L; i <= R; i++) {
            res += (i - 1) % 9 + 1;
        }
        cout << res << '\n';
    }
}
```

}

41.3.3 Subtask 3: $1 \leq l \leq r \leq 10^{18}$ (Full Test)

Phân tích: Sử dụng tính chất chu kỳ của dãy số. Tổng của một chu kỳ đầy đủ (từ 1 đến 9) là $1+2+3+4+5+6+7+8+9=45$. Để tính tổng từ L đến R , ta tính $F(R)-F(L-1)$ với $F(X)$ là tổng các phần tử của dãy từ 1 đến X .

- Số chu kỳ đầy đủ trong $F(X)$: $k = X/9$.
- Số phần tử dư ra: $r = X \pmod 9$.
- Công thức: $F(X) = k \times 45 + \frac{r(r+1)}{2}$.

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int Q, L, R;

int get(int n) {
    if (n == 0) return 0;
    int k = n / 9;
    int r = n % 9;
    return k * 45 + (r * (r + 1)) / 2;
}

main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    cin >> Q;
    while (Q--) {
        cin >> L >> R;
        cout << get(R) - get(L - 1) << '\n';
    }
}
```

42 Thanh Hóa

42.1 Bài 1: THỐNG KÊ

Phân tích thuật toán: Bài toán yêu cầu đếm số lượng mã hiệu hợp lệ trong đoạn $[A, B]$. Một mã hiệu bị coi là lỗi (không hợp lệ) nếu nó có số lượng ước dương là một số lẻ.

Theo tính chất Toán học cốt lõi, một số nguyên dương có số lượng ước là số lẻ khi và chỉ khi số đó là một **số chính phương** (bình phương của một số nguyên). Ví dụ: Số 9 có các ước là 1, 3, 9 (3 ước - lẻ), số 16 có các ước là 1, 2, 4, 8, 16 (5 ước - lẻ). Các số không phải chính phương luôn có ước xuất hiện theo từng cặp, nên số lượng ước luôn chẵn.

Do đó, bài toán tìm số lượng mã hiệu hợp lệ đưa về việc đếm xem có bao nhiêu số **không phải** là số chính phương trong đoạn $[A, B]$.

- Tổng số các mã hiệu trong đoạn $[A, B]$ là: $B - A + 1$.
- Số lượng các số chính phương nằm trong khoảng từ 1 đến X chính là phần nguyên của căn bậc hai của X , tức là $\lfloor \sqrt{X} \rfloor$.
- Số lượng số chính phương trong đoạn $[A, B]$ được tính bằng: $\lfloor \sqrt{B} \rfloor - \lfloor \sqrt{A-1} \rfloor$.
- Vậy, kết quả cuối cùng là:

$$\text{Kết quả} = (B - A + 1) - (\lfloor \sqrt{B} \rfloor - \lfloor \sqrt{A-1} \rfloor)$$

42.1.1 Subtask 1 & 2: $A \leq B \leq 10^6$

Phân tích: Với các subtask nhỏ, ta có thể duyệt vòng lặp từ A đến B , sau đó viết một hàm đếm số lượng ước của mỗi số i để kiểm tra chẵn lẻ, hoặc sử dụng mảng đếm (Sieve) giống như sàng nguyên tố với độ phức tạp $O(B \log B)$. Dù cách này qua được test nhỏ, nhưng việc triển khai mất nhiều thời gian code, tiêu tốn bộ nhớ và hoàn toàn bất lực trước giới hạn lớn hơn.

42.1.2 Subtask 3: $A \leq B \leq 10^{18}$ (Full Test)

Phân tích: Với giới hạn khổng lồ 10^{18} , ta bắt buộc phải sử dụng công thức toán học $O(1)$ đã phân tích ở trên.

Tuy nhiên có một "cái bẫy" rất nguy hiểm ở đây: Hàm `sqrt()` mặc định của C++ xử lý trên kiểu số thực `double` và chỉ đảm bảo độ chính xác tuyệt đối cho các số nguyên đến khoảng 9×10^{15} . Với các số tiệm cận 10^{18} , do sai số dấu phẩy động, `sqrt()` có thể làm tròn sai giá trị căn bậc hai dẫn đến đếm sai số lượng số chính phương.

Để giải quyết triệt để vấn đề sai số này, ta sẽ viết một hàm tính căn bậc hai trên số nguyên: dùng `sqrt()` để lấy kết quả xấp xỉ ban đầu, sau đó dùng các phép nhân số nguyên và vòng lặp `while` để hiệu chỉnh kết quả tăng/giảm chính xác tuyệt đối.

42.1.3 Code tối ưu

```
#include<bits/stdc++.h>
#define int long long
using namespace std;

int a, b;
int t, x, ans;

int sq(int n){
    if(n == 0) return 0;

    int k = sqrt(n);

    while((k + 1) * (k + 1) <= n)
        k++;
    while(k * k > n)
        k--;

    return k;
}

main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> a >> b;
    t = b - a + 1;
    x = sq(b) - sq(a - 1);
    ans = t - x;
    cout << ans;
}
```

42.2 Bài 2: THÁP ĐẦY ĐỦ

Nhắc lại quy luật chung: Để tạo thành một tháp đầy đủ từ a đến b mà không thể chèn thêm đĩa, tỉ số giữa 2 đĩa liên tiếp phải là một **số nguyên tố**. Do đó, gọi $N = b/a$. Nếu b không chia hết cho a , in ra -1 . Ngược lại, bài toán trở thành:

- Tìm tất cả các thừa số nguyên tố của N .
- Chiều cao $h =$ số lượng thừa số $+ 1$.
- Để trọng số w nhỏ nhất, ta sắp xếp các thừa số nguyên tố theo thứ tự tăng dần rồi nhân dần vào a để tính tổng các đĩa.

42.2.1 Subtask 1: $a < b \leq 4a \leq 10^{18}$

Phân tích: Từ giả thiết $b \leq 4a$, suy ra $N = b/a \leq 4$. Vì $a < b$ nên N chỉ có thể nhận các giá trị 2, 3, 4. Với số lượng trường hợp quá ít, ta có thể dùng if - else xử lý trực tiếp luôn mà không cần vòng lặp phân tích thừa số.

- $N = 2$: Thừa số là {2}. Tháp là $(a, 2a)$.
- $N = 3$: Thừa số là {3}. Tháp là $(a, 3a)$.
- $N = 4$: Thừa số là {2, 2}. Tháp là $(a, 2a, 4a)$.

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int a, b, n;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> a >> b;

    if (b % a != 0) {
        cout << -1 << '\n';
    } else {
        n = b / a;
        if (n == 2 || n == 3) {
            cout << 2 << " " << a + b << '\n';
        } else if (n == 4) {
            cout << 3 << " " << a + a * 2 + b << '\n';
        }
    }
}
```

42.2.2 Subtask 2: $1 \leq a < b \leq 10^5$

Phân tích: Với $b \leq 10^5$, suy ra $N \leq 10^5$. Ở mức giới hạn này, ta có thể dùng vòng lặp chạy từ 2 đến hẳn N để chia dần và lấy thừa số nguyên tố. Độ phức tạp lớn nhất trong trường hợp N là số nguyên tố là $O(N)$, khoảng 10^5 phép tính, hoàn toàn chạy mượt. Do vòng lặp tự động chạy từ nhỏ đến lớn nên mảng thừa số sinh ra đã được sắp xếp sẵn.

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int a, b, n, p[100], k = 0;
int h, w, c;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> a >> b;

    if (b % a != 0) {
        cout << -1 << '\n';
    } else {
        n = b / a;

        for (int i = 2; i <= n; i++) {
            while (n % i == 0) {
                k++;
                p[k] = i;
                n /= i;
            }
        }

        h = k + 1;
        w = a;
        c = a;

        for (int i = 1; i <= k; i++) {
            c *= p[i];
            w += c;
        }

        cout << h << " " << w << '\n';
    }
}
```

42.2.3 Subtask 3: $1 \leq a < b \leq 10^{12}$

Phân tích: Giới hạn $b \leq 10^{12}$ nên vòng lặp $O(N)$ ở Subtask 2 sẽ bị TLE. Ta tối ưu hóa vòng lặp phân tích thừa số nguyên tố xuống $O(\sqrt{N})$. Ta chỉ duyệt i đến $\sqrt{N}$. Nếu sau vòng lặp mà $N > 1$ thì bản thân phần còn lại đó chính là một số nguyên tố lớn. Do có thể thừa số cuối cùng này bị đẩy vào sau, ta cần gọi hàm sort để mảng p luôn theo

thứ tự tăng dần nhằm tối ưu trọng số.

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int a, b, n, p[100], k = 0;
int h, w, c;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> a >> b;

    if (b % a != 0) {
        cout << -1 << '\n';
    } else {
        n = b / a;

        for (int i = 2; i * i <= n; i++) {
            while (n % i == 0) {
                k++;
                p[k] = i;
                n /= i;
            }
        }

        if (n > 1) {
            k++;
            p[k] = n;
        }

        sort(p + 1, p + k + 1);

        h = k + 1;
        w = a;
        c = a;

        for (int i = 1; i <= k; i++) {
            c *= p[i];
            w += c;
        }

        cout << h << " " << w << '\n';
    }
}
```

42.3 Bài 3: TÍNH CĂN

Phân tích thuật toán chung: Một số nguyên dương được gọi là một số lập phương (có căn bậc ba là số nguyên) khi và chỉ khi trong dạng phân tích thừa số nguyên tố của nó, tất cả các số mũ đều là bội của 3. Giả sử $N!$ được phân tích thành các thừa số nguyên tố:

$$N! = p_1^{e_1} \times p_2^{e_2} \times \dots \times p_k^{e_k}$$

Để tìm ước số lập phương lớn nhất của $N!$, ta cần chọn số mũ lớn nhất là bội của 3 và không vượt quá e_i . Số mũ đó chính là $3 \times \lfloor e_i/3 \rfloor$.

Để tính nhanh số mũ e_i của một số nguyên tố p trong $N!$, ta áp dụng **Công thức Legendre**:

$$e_i = \left\lfloor \frac{N}{p} \right\rfloor + \left\lfloor \frac{N}{p^2} \right\rfloor + \left\lfloor \frac{N}{p^3} \right\rfloor + \dots$$

Bài toán yêu cầu in ra kết quả theo modulo $10^9 + 7$, ta sẽ dùng thuật toán lũy thừa nhị phân (Binary Exponentiation) để tính trong quá trình nhân dồn kết quả.

42.3.1 Subtask 1: $1 < N \leq 20$

Phân tích: Với $N \leq 20$, số lượng các số nguyên tố rất ít. Ta có thể khai báo trực tiếp mảng các số nguyên tố nhỏ hơn hoặc bằng 20 là $\{2, 3, 5, 7, 11, 13, 17, 19\}$ và duyệt qua chúng. Với mỗi truy vấn, ta tính bậc của các số nguyên tố này trong $N!$ và nhân vào đáp án. Thuật toán vô cùng nhẹ và không cần sàng nguyên tố.

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

const int MOD = 1e9 + 7;
int t, n;
int p[8] = {2, 3, 5, 7, 11, 13, 17, 19};
int sz = 8;

int pw(int a, int b) {
    int r = 1;
    a %= MOD;
    while (b > 0) {
        if (b % 2 == 1) r = (r * a) % MOD;
        a = (a * a) % MOD;
        b /= 2;
    }
    return r;
}
```

```

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> t;
    while (t--) {
        cin >> n;
        int res = 1;
        for (int i = 0; i < sz; i++) {
            if (p[i] > n) break;
            int tmp = n;
            int e = 0;
            while (tmp > 0) {
                e += tmp / p[i];
                tmp /= p[i];
            }
            int u = (e / 3) * 3;
            if (u > 0) {
                res = (res * pw(p[i], u)) % MOD;
            }
        }
        cout << res << '\n';
    }
}

```

42.3.2 Subtask 2: $N \leq 10^5$

Phân tích: Giới hạn $N \leq 10^5$, ta có rất nhiều testcase ($T \leq 10$). Để tối ưu hóa, ta sử dụng **Sàng nguyên tố (Sieve of Eratosthenes)** để tìm trước một lần tất cả các số nguyên tố từ 1 đến 10^5 và lưu vào một mảng toàn cục. Khi trả lời mỗi truy vấn, ta chỉ việc lặp qua danh sách số nguyên tố đã sàng, áp dụng công thức Legendre tính số mũ, điều chỉnh về bội của 3 và nhân với kết quả. Vòng lặp dừng sớm khi $p > N$. Độ phức tạp mỗi truy vấn chỉ còn rất nhỏ, đảm bảo vượt qua giới hạn thời gian dễ dàng.

```

#include <bits/stdc++.h>
#define int long long
using namespace std;

const int MOD = 1e9 + 7;
int t, n, k = 0;
bool is_p[100005];
int pr[100005];

int pw(int a, int b) {
    int r = 1;
    a %= MOD;

```

```

    while (b > 0) {
        if (b % 2 == 1) r = (r * a) % MOD;
        a = (a * a) % MOD;
        b /= 2;
    }
    return r;
}

void sangnt() {
    for (int i = 2; i <= 100000; i++)
        is_p[i] = 1;

    for (int i = 2; i * i <= 100000; i++)
        if (is_p[i])
            for (int j = i * i; j <= 100000; j += i)
                is_p[j] = 0;

    for (int i = 2; i <= 100000; i++) {
        if (is_p[i]) {
            k++;
            pr[k] = i;
        }
    }
}

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    sangnt();

    cin >> t;
    while (t--) {
        cin >> n;
        int res = 1;
        for (int i = 1; i <= k; i++) {
            if (pr[i] > n) break;
            int tmp = n, e = 0;
            while (tmp > 0) {
                e += tmp / pr[i];
                tmp /= pr[i];
            }
            int u = (e / 3) * 3;
            if (u > 0) {
                res = (res * pw(pr[i], u)) % MOD;
            }
        }
    }
}

```

```
    cout << res << '\n';
}
}
```

42.4 Bài 4: TẦN SỐ

Phân tích thuật toán chung: Một xâu con X được gọi là có tần số xuất hiện cao nếu tồn tại một kí tự c trong X có số lần xuất hiện lớn hơn tổng số lần xuất hiện của tất cả các kí tự còn lại. Gọi $\text{cnt}(c, X)$ là số lần xuất hiện của c trong X . Điều kiện trên tương đương với:

$$\text{cnt}(c, X) > |X| - \text{cnt}(c, X) \iff 2 \times \text{cnt}(c, X) > |X|$$

Thay vì tìm một xâu con rồi mới đếm tần số, ta có thể đảo ngược bài toán: Cố định trước kí tự c (từ 'a' đến 'z'), sau đó đi tìm xâu con dài nhất thỏa mãn kí tự c chiếm đa số. Kết quả của bài toán sẽ là độ dài lớn nhất đạt được trong 26 lần kiểm tra.

42.4.1 Subtask 1 & 2: Độ dài $S \leq 2 \cdot 10^3$

Phân tích: Với giới hạn $N \leq 2000$, ta có thể duyệt qua mọi cặp điểm đầu l và điểm cuối r của xâu con. Duyệt l từ 1 đến N , với mỗi l , ta tịnh tiến r từ l đến N . Ta dùng một mảng đếm tần số của 26 chữ cái và cập nhật động khi r tăng lên. Tại mỗi bước, ta tìm kí tự xuất hiện nhiều nhất, nếu số lần xuất hiện của nó $\times 2$ lớn hơn độ dài xâu hiện tại ($r - l + 1$), ta cập nhật kết quả. Độ phức tạp là $O(26 \cdot N^2)$, hoàn toàn vượt qua thời gian giới hạn cho Subtask 1 và 2.

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

string s;
int n;
int res = 0;
int cnt[30];

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> s;
    n = s.length();
    s = " " + s;

    for (int l = 1; l <= n; l++) {
```

```

    for (int i = 0; i < 26; i++) cnt[i] = 0;

    int mx = 0;
    for (int r = 1; r <= n; r++) {
        int c = s[r] - 'a';
        cnt[c]++;
        mx = max(mx, cnt[c]);

        if (mx * 2 > r - l + 1) {
            res = max(res, r - l + 1);
        }
    }

    cout << res << "\n";
}

```

42.4.2 Subtask 3: Độ dài $S \leq 2 \cdot 10^5$ (Full Test)

Phân tích: Giới hạn $N \leq 200000$ đòi hỏi thuật toán $O(N \log N)$ hoặc $O(N)$. Áp dụng tư tưởng cố định kí tự c : Với mỗi c , ta tạo một mảng giá trị A sao cho $A[i] = 1$ nếu $S[i] = c$, và $A[i] = -1$ nếu $S[i] \neq c$. Bài toán quy về việc tìm một mảng con liên tiếp có tổng các phần tử lớn hơn 0 và độ dài dài nhất.

Sử dụng mảng cộng dồn P (Prefix Sum): $P[i] = A[1] + A[2] + \dots + A[i]$. Tổng của đoạn từ $l + 1$ đến r là $P[r] - P[l]$. Điều kiện tổng > 0 tương đương với $P[r] > P[l]$ với $l < r$. Ta cần tìm $\max(r - l)$.

Thuật toán tối ưu $O(N)$:

- Ta dùng một **ngăn xếp (Stack)** lưu lại các chỉ số i sao cho giá trị $P[i]$ giảm dần ngặt. (Vì nếu $P[x] \geq P[y]$ với $x < y$, thì x luôn là mốc trái tốt hơn y vì vừa có giá trị nhỏ hơn/bằng, vừa nằm xa hơn về bên trái).
- Sau khi xây xong ngăn xếp từ trái sang phải, ta duyệt mốc phải j lùi từ N về 1.
- Chừng nào $P[j] > P[\text{stack.top()}]$, ta đã tìm được một đoạn hợp lệ. Ta cập nhật chiều dài, sau đó **pop (xóa) phần tử đó khỏi ngăn xếp**. Lý do xóa rất an toàn: vì j đang duyệt lùi, các mốc j sau này sẽ nhỏ hơn, nên khoảng cách sinh ra chắc chắn sẽ ngắn hơn, ta không cần giữ mốc l này lại nữa.

Thời gian cho mỗi chữ cái là $O(N)$, duyệt 26 chữ cái mất $O(26 \cdot N)$, tối ưu tuyệt đối.

```

#include <bits/stdc++.h>
#define int long long

```



```

using namespace std;

string s;
int n, res = 0;
int p[200005];
int st[200005];
int top;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> s;
    n = s.length();
    s = " " + s;

    for (char c = 'a'; c <= 'z'; c++) {
        p[0] = 0;
        top = 0;
        st[++top] = 0;

        for (int i = 1; i <= n; i++) {
            if (s[i] == c) {
                p[i] = p[i - 1] + 1;
            } else {
                p[i] = p[i - 1] - 1;
            }

            if (p[i] < p[st[top]]) {
                st[++top] = i;
            }
        }

        for (int j = n; j >= 1; j--) {
            while (top > 0 && p[j] > p[st[top]]) {
                res = max(res, j - st[top]);
                top--;
            }
        }
    }

    cout << res;
}

```

43 Thừa Thiên Huế - ĐH Khoa Học

43.1 Bài 1: Kim tự tháp

43.1.1 Tóm tắt đề bài

Kim tự tháp được ghép theo cách sau: tầng trên cùng có 1 khối, tầng thứ 2 có $1 + 2 = 3$ khối, tầng thứ 3 có $1 + 2 + 3 = 6$ khối, cứ như vậy cho các tầng ở dưới.

Yêu cầu: Tính số khối hộp lập phương với chiều cao h ($1 \leq h \leq 200$) cho trước.

43.1.2 Lời giải

- Vì giới hạn của h rất bé, ta có thể dùng 2 vòng for lồng nhau để tính tổng đáp án.

Cài đặt:

43.1.3 Code mẫu tham khảo:

```
#include<bits/stdc++.h>

using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);

    int h;
    cin >> h;
    int ans = 0;
    for(int i = 1; i <= h; ++i) {
        for(int j = 1; j <= i; ++j) ans += j;
    }

    cout << ans << '\n';

    return 0;
}
```

43.2 Bài 2: Đếm số từ

43.2.1 Tóm tắt đề bài

Từ là một dãy các ký tự liên tiếp nhau không chứa ký tự khoảng trắng trong xâu ký tự, mỗi từ cách nhau ít nhất một khoảng trắng.

Yêu cầu: Đếm số từ trong một xâu ký tự cho trước.

43.2.2 Lời giải

- Vì mỗi từ chỉ cần cách nhau ít nhất 1 khoảng trắng, ta có thể liên tục nhập vào xâu trong C++ và đếm.

Cài đặt:

43.2.3 Code mẫu tham khảo:

```
#include<bits/stdc++.h>

using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);

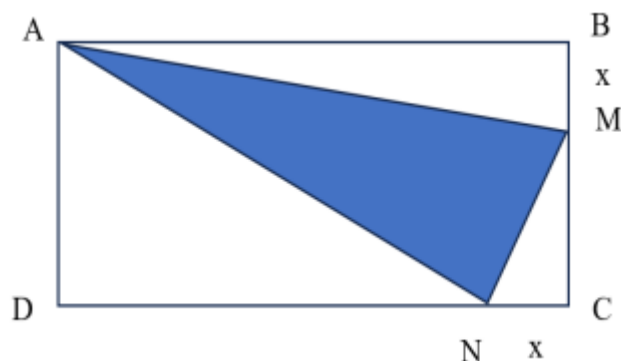
    string s;
    int ans = 0;
    while(cin >> s) ++ans;
    cout << ans << '\n';

    return 0;
}
```

43.3 Bài 3: Diện tích tam giác

43.3.1 Tóm tắt đề bài

Cho một hình chữ nhật $ABCD$, cạnh $AB = a$, cạnh $BC = b$, với a, b là các số nguyên dương có giá trị trong khoảng $[1, 120]$. Một điểm M chạy lưu động trên cạnh BC với $BM = x$, một điểm N chạy trên cạnh CD với $CN = x$, trong đó x là số nguyên dương có giá trị trong khoảng $[0, b]$.



Yêu cầu: Tính giá trị lớn nhất và giá trị nhỏ nhất của diện tích tam giác AMN khi M, N di chuyển trên cạnh BC và CD tương ứng.

43.3.2 Lời giải

– Để ý rằng, để yêu cầu x là số nguyên, vì thế ta chỉ cần duyệt từ 0 tới b để xác định vị trí của x , vấn đề còn lại là tính diện tích tam giác AMN .

– Ta có:

$$* BM = x \Rightarrow MC = b - x$$

$$* CN = x \Rightarrow DN = a - x$$

$$* AD = b$$

$$* S_{ABCD} = a \cdot b$$

$$* S_{ABM} = \frac{1}{2} \cdot AB \cdot BM = \frac{1}{2}ax$$

$$* S_{MCN} = \frac{1}{2} \cdot MC \cdot CN = \frac{1}{2}(b-x)x = \frac{bx - x^2}{2}$$

$$* S_{ADN} = \frac{1}{2} \cdot AD \cdot DN = \frac{1}{2}b(a-x) = \frac{ab - bx}{2}$$

– Vì thế:

$$S_{AMN} = ab - \left(\frac{ax}{2} + \frac{bx - x^2}{2} + \frac{ab - bx}{2} \right)$$

$$S_{AMN} = \frac{2ab - (ax + bx - x^2 + ab - bx)}{2}$$

$$S_{AMN} = \frac{x^2 - ax + ab}{2}$$

– Từ đây, ta chỉ cần áp dụng công thức diện tích trên và duyệt qua từng x nguyên để tính, có thể không dùng số thực để tránh sai số.

Cài đặt:

43.3.3 Code mẫu tham khảo:

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);

    int a, b;
```

```

cin >> a >> b;

int maxS = -1;
int minS = 2e9;
int xMax = 0, xMin = 0;

for (int x = 0; x <= b; ++x) {
    int curS = x * x - a * x + a * b;

    if (curS > maxS) {
        maxS = curS;
        xMax = x;
    }

    if (curS < minS) {
        minS = curS;
        xMin = x;
    }
}

cout << (maxS / 2) << (maxS % 2 ? ".5" : "") << '\n';
cout << xMax << '\n';
cout << (minS / 2) << (minS % 2 ? ".5" : "") << '\n';
cout << xMin << '\n';

return 0;
}

```

43.4 Bài 4: Mã hoá thẻ

43.4.1 Tóm tắt đề bài

Yêu cầu: Liệt kê tất cả các dãy nhị phân độ dài n (theo thứ tự từ điển) và in ra số lượng dãy nhị phân có độ dài n .

43.4.2 Lời giải

- Vì giới hạn n bé ($1 \leq n \leq 17$), ta có thể dùng đệ quy để lần lượt sinh ra các dãy nhị phân độ dài n , đồng thời lưu biến đếm trong lúc sinh dãy nhị phân để in ra sau cùng.

Cài đặt:

43.4.3 Code mẫu tham khảo:

```

#include<bits/stdc++.h>

using namespace std;

int n;
int a[17];
int cnt = 0;

void Try(int x) {
    if (x == n) {
        for(int i = 0; i < n; ++i) cout << a[i];
        cout << " ";
        ++cnt;
        return;
    }
    a[x] = 0;
    Try(x + 1);
    a[x] = 1;
    Try(x + 1);
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);

    cin >> n;

    Try(0);

    cout << '\n';
    cout << cnt;

    return 0;
}

```

44 Tiền Giang

44.1 Bài 1: Tính tổng:

44.1.1 Subtask 1: $1 \leq n \leq 10^6$

Sử dụng vòng lặp for để tính tổng.

44.1.2 Subtask 2: $10^6 < n \leq 10^9$

Nhận thấy:

- Nếu n là số lẻ, thì S chính là tổng của dãy cấp số cộng có số hạng đầu là 1 và công sai là 2. Khi đó, S được tính bởi công thức:

$$S = \frac{1}{2} \times (n + 1) \left(\frac{n-1}{2} + 1 \right)$$

- Nếu n là số chẵn, thì S chính là tổng của dãy cấp số cộng có số hạng đầu là 2 và công sai là 2. Khi đó, S được tính bởi công thức:

$$S = \frac{1}{2} \times (n + 2) \left(\frac{n-2}{2} + 1 \right)$$

44.1.3 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    long long n;
    cin >> n;
    if (n % 2 == 0) cout << (n + 2) * ((n - 2) / 2 + 1) / 2 << "\n";
    else cout << (n + 1) * ((n - 1) / 2 + 1) / 2 << "\n";
    return 0;
}
```

44.2 Bài 2: Số Niven cân bằng

44.2.1 Lời giải:

Vì $n \leq 10^6$ nên có thể duyệt ngược từ n trở về 10, sử dụng thêm một hàm để kiểm tra xem số hiện tại có thỏa mãn điều kiện là số Niven cân bằng hay không.

44.2.2 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

bool check(int n) {
    int x = n, cntOdd = 0, cntEven = 0, sum = 0;
    while (x) {
        int digit = x % 10;
        if (digit % 2 == 0) cntEven++;
        else cntOdd++;
        sum += digit;
        x /= 10;
    }
    return cntOdd == cntEven;
}
```

```

    }
    if (n % sum == 0 && cntOdd == cntEven) return true;
    else return false;
}

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    int n;
    cin >> n;
    while (!check(n)) n--;
    cout << n << "\n";
    return 0;
}

```

44.3 Bài 3: Tìm số lớn nhất

44.3.1 Subtask 1: $1 \leq n, Q \leq 10^3$

Với mỗi truy vấn (u, v, M) , duyệt từ vị trí u đến v trong mảng, cập nhật giá trị cho từng vị trí. Sau khi thực hiện xong Q truy vấn, tìm giá trị lớn nhất trong mảng.

44.3.2 Subtask 2: $10^3 < n, Q \leq 10^6$

Vì chỉ cần lấy đáp án sau khi thực hiện Q truy vấn, nên có thể sử dụng mảng hiệu để cập nhật nhanh giá trị cho từng vị trí.

Đặt $diff_i = a_i - a_{i-1}$, với $1 \leq i \leq n$.

Với mỗi truy vấn (u, v, M) , ta thấy:

- Các phần tử trong đoạn $[u, v]$ đều cùng tăng lên M đơn vị, nên chênh lệch giữa hai phần tử liên tiếp trong đoạn này sẽ không đổi, nghĩa là $diff_{u+1}, \dots, diff_v$ không đổi.
- Chênh lệch giữa a_u và a_{u-1} tăng lên M , nên $diff_u$ sẽ tăng lên M đơn vị.
- Chênh lệch giữa a_v và a_{v+1} sẽ giảm đi M đơn vị, nên $diff_{v+1}$ sẽ giảm đi M đơn vị.

Cuối cùng, sau Q truy vấn, ta được $a_i = \sum_{j=1}^i diff_j$.

44.3.3 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

#define int long long
const int N = 1e6 + 5;
const int inf = 1e18;
int diff[N];

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    int n, q;
    cin >> n >> q;
    for (int i = 1; i <= q; i++) {
        int u, v, m;
        cin >> u >> v >> m;
        diff[u] += m;
        diff[v + 1] -= m;
    }
    int ans = -inf;
    for (int i = 1; i <= n; i++) {
        diff[i] += diff[i - 1];
        ans = max(ans, diff[i]);
    }
    cout << ans << "\n";
    return 0;
}
```

44.4 Bài 4: Từ đa dạng

44.4.1 Lời giải:

Sử dụng `std::stringstream` để tách các từ ra. Với mỗi từ, có thể sử dụng mảng đánh dấu hoặc đếm phân phối để xác định số lượng kí tự phân biệt xuất hiện trong từ đang xét.

44.4.2 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

int cnt[30];

signed main() {
```

```

ios_base::sync_with_stdio(0);
cin.tie(0); cout.tie(0);
string s;
getline(cin, s);
stringstream ss(s);
string word, res;
int ans = 0;
while (ss >> word) {
    for (int i = 0; i < 26; i++) cnt[i] = 0;
    int num = 0;
    for (int i = 0; i < (int)word.size(); i++) {
        cnt[word[i] - 'a']++;
        if (cnt[word[i] - 'a'] == 1) num++;
    }
    if (num > ans) ans = num, res = word;
}
cout << res << "\n";
return 0;
}

```

44.5 Bài 5: Miền nguyên tố

Phân tích thuật toán: Giả sử trong dãy số có num số nguyên tố. Để di chuyển num số nguyên tố này thành một đoạn liên tiếp $[i, i + num - 1]$ với i là một vị trí bất kỳ $1 \leq i \leq n - num + 1$, số thao tác đổi chỗ hai phần tử bất kỳ cần dùng sẽ là:

- Đặt X là số lượng phần tử không là số nguyên tố trong đoạn $[i, i + num - 1]$.
- Ta có thể đổi chỗ X phần tử này với X số nguyên tố không nằm trong đoạn $[i, i + num - 1]$.
- Vậy số thao tác đổi chỗ sẽ là X .

44.5.1 Subtask 1: $n \leq 10^3, A_i \leq 10^3$

Ở subtask này, với mỗi vị trí i ($1 \leq i \leq n - num + 1$, num là số lượng số nguyên tố có trong dãy). Sử dụng thêm một vòng for để xác định số lượng phần tử không là số nguyên tố trong đoạn $[i, i + num - 1]$.

Độ phức tạp: $O(n^2 \times \sqrt{n})$, nếu sử dụng thuật toán kiểm tra số nguyên tố trong $O(\sqrt{n})$.

44.5.2 Subtask 2: $n \leq 10^4, A_i \leq 10^5$

Ở subtask này, để xác định nhanh số lượng phần tử không là số nguyên tố trong đoạn $[i, i + num - 1]$ nhanh, ta sử dụng tổng tiền tố.

Đặt pre_i là số lượng phần tử là số nguyên tố trong đoạn $[1, i]$. Khi đó, số lượng phần tử là số nguyên tố trong đoạn $[l, r]$ là $pre_r - pre_{l-1}$.

Vậy khi xét đến vị trí i , số lượng phần tử không là số nguyên tố trong đoạn $[i, i + num - 1]$ là $num - (pre_{i+num-1} - pre_{i-1})$.

Độ phức tạp: $O(n \times \sqrt{n})$

44.5.3 Subtask 2: $n \leq 10^5, A_i \leq 10^9$

Ở subtask này, vẫn sử dụng tổng tiền tố để xác định nhanh số lượng phần tử không là số nguyên tố trong đoạn $[i, i + num - 1]$. Tuy nhiên, nếu sử dụng thuật toán có độ phức tạp $O(\sqrt{n})$ để kiểm tra số nguyên tố thì sẽ bị quá thời gian. Ta cải tiến bằng thuật toán kiểm tra số nguyên tố bước nhảy 6 có độ phức tạp $O(\frac{\sqrt{n}}{6})$.

Độ phức tạp: $O(n \times \frac{\sqrt{n}}{6})$

44.5.4 Code mẫu:

```
#include <bits/stdc++.h>
using namespace std;

const int N = 1e5 + 5;
int pre[N];

bool isPrime(int n) {
    if (n == 2 || n == 3) return true;
    if (n < 2 || n % 2 == 0 || n % 3 == 0) return false;
    for (int i = 5; i * i <= n; i += 6)
        if (n % i == 0 || n % (i + 2) == 0) return false;
    return true;
}

signed main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) {
        int x;
        cin >> x;
        pre[i] = pre[i - 1];
        if (isPrime(x)) pre[i]++;
    }
    int num = pre[n], ans = n;
    for (int i = 1; i <= n - num + 1; i++)
        ans = min(ans, num - (pre[i + num - 1] - pre[i - 1]));
}
```

```

    cout << ans << "\n";
    return 0;
}

```

45 Tuyên Quang

45.1 Bài 1: THU DỌN ĐỒ CHƠI

Phân tích thuật toán: Đây là bài toán cơ bản về phép chia lấy thương và phép chia lấy dư:

- Số hộp màu xanh = $N/20$ (phép chia nguyên).
- Số hộp màu đỏ = $M/30$ (phép chia nguyên).
- Số bi xanh còn dư = $N \pmod{20}$.
- Số bi đỏ còn dư = $M \pmod{30}$.
- Tổng số bi còn lại trong hộp vàng = $(N \pmod{20}) + (M \pmod{30})$.

Lưu ý về dữ liệu: Giới hạn $N, M \leq 10^{18}$ vượt quá khả năng lưu trữ của kiểu số nguyên 32-bit (int). Do đó, bắt buộc phải sử dụng kiểu số nguyên 64-bit (long long trong C++).

45.1.1 Code mẫu

```

#include <bits/stdc++.h>
#define int long long
using namespace std;

int n, m, blue_boxes, red_boxes, blue_rem, red_rem, total_rem;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n >> m;

    blue_boxes = n / 20;
    blue_rem = n % 20;

    red_boxes = m / 30;
    red_rem = m % 30;
}

```

```
total_rem = blue_rem + red_rem;

cout << blue_boxes << " " << red_boxes << "\n";
cout << total_rem;
}
```

Giải thích Sample:

- Với $N = 24$: $24/20 = 1$ (hộp xanh), dư $24 \pmod{20} = 4$.
- Với $M = 95$: $95/30 = 3$ (hộp đỏ), dư $95 \pmod{30} = 5$.
- Tổng dư: $4 + 5 = 9$.

45.2 Bài 2: SIÊU LẼ

Tóm tắt đề bài: Một số N là số siêu lẻ nếu N lẻ và tổng các chữ số của N cũng là số lẻ. Cho M truy vấn ($M \leq 10^6$), mỗi truy vấn đếm số lượng số siêu lẻ trong đoạn $[a, b]$ với $a, b \leq 10^6$.

45.2.1 Subtask 1: $M = 1$

Phân tích: Với chỉ một truy vấn duy nhất, ta có thể sử dụng vòng lặp duyệt trực tiếp từ a đến b . Tại mỗi bước, ta kiểm tra hai điều kiện: số đó có lẻ không và tổng chữ số có lẻ không.

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int M, A, B, cnt;

bool check(int n) {
    if (n % 2 == 0) return 0;
    int s = 0;
    int tmp = n;
    while (tmp > 0) {
        s += tmp % 10;
        tmp /= 10;
    }
    return (s % 2 != 0);
}

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
```

```
cin >> M >> A >> B;
cnt = 0;
for (int i = A; i <= B; i++) {
    if (check(i)) cnt++;
}
cout << cnt;
}
```

45.2.2 Subtask 2: $M \leq 10^6, a, b \leq 10^6$

Phân tích: Khi số lượng truy vấn M lên tới 10^6 , việc duyệt cho từng truy vấn sẽ gây quá tải thời gian ($O(M \times N)$). Ta cần sử dụng kỹ thuật **Mảng cộng dồn (Prefix Sum)**.

- Ta chuẩn bị một mảng pref lưu trữ số lượng số siêu lẻ từ 1 đến i .
- Số lượng số siêu lẻ trong đoạn $[a, b]$ sẽ được tính nhanh bằng công thức: $\text{pref}[b] - \text{pref}[a-1]$.

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int M, A, B;
int pre[1000005];

bool check(int n) {
    if (n % 2 == 0) return 0;
    int s = 0;
    int tmp = n;
    while (tmp > 0) {
        s += tmp % 10;
        tmp /= 10;
    }
    return (s % 2 != 0);
}

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    for (int i = 1; i <= 1000000; i++) {
        pre[i] = pre[i - 1];
        if (check(i))
            pre[i]++;
    }
}
```

```
cin >> M;
while (M--) {
    cin >> A >> B;
    cout << pre[B] - pre[A - 1] << '\n';
}
}
```

45.3 Bài 3: NGUYÊN TỐ

Tóm tắt đề bài: Cho dãy A gồm N số. Với mỗi phần tử A_i ($i \geq 2$), ta cần kiểm tra xem có tồn tại một chỉ số $j < i$ nào đó sao cho $A_i + S(A_j)$ là một số nguyên tố hay không (với $S(x)$ là tổng các chữ số của x). Đếm số lượng A_i thỏa mãn.

45.3.1 Phân tích thuật toán

- **Điều kiện kiểm tra:** Đối với A_i , thay vì duyệt lại mọi $j < i$ (mất $O(N^2)$), ta chỉ cần biết tập hợp các giá trị $S(A_j)$ đã xuất hiện trước đó.
- **Giới hạn giá trị:** $A_i \leq 10^6$, tổng chữ số lớn nhất của A_j là $S(999,999) = 54$. Vậy giá trị lớn nhất của tổng $A_i + S(A_j)$ xấp xỉ 1,000,054.
- **Sàng nguyên tố:** Ta cần sàng các số nguyên tố lên đến 1,000,060 để kiểm tra nhanh trong $O(1)$.
- **Tối ưu hóa:** Ta sử dụng một mảng đánh dấu `has_sum[v]` để biết giá trị tổng chữ số v đã từng xuất hiện ở các phần tử trước đó hay chưa. Với mỗi A_i , ta chỉ cần duyệt v từ 1 đến 54. Nếu `has_sum[v] == true` và $A_i + v$ là số nguyên tố, thì A_i thỏa mãn.

45.3.2 Subtask 1: $N \leq 10^3, A_i \leq 1000$

Phân tích: Với giới hạn này, ta có thể sử dụng hai vòng lặp lồng nhau $O(N^2)$ để kiểm tra từng cặp (i, j) mà không cần tối ưu tập hợp tổng chữ số.

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

int n, a[1005], cnt;

bool nt(int x) {
```

```

    if (x < 2) return 0;
    for (int i = 2; i * i <= x; i++) {
        if (x % i == 0) return 0;
    }
    return 1;
}

int get_s(int x) {
    int res = 0;
    while (x > 0) {
        res += x % 10;
        x /= 10;
    }
    return res;
}

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n;
    for (int i = 1; i <= n; i++) cin >> a[i];

    cnt = 0;
    for (int i = 2; i <= n; i++) {
        for (int j = 1; j < i; j++) {
            if (nt(a[i] + get_s(a[j]))) {
                cnt++;
                break;
            }
        }
    }
    cout << cnt;
}

```

45.3.3 Subtask 2: $N \leq 10^6, A_i \leq 10^6$ (Full Test)

Phân tích: Ta sử dụng Sàng Eratosthenes và mảng đánh dấu các tổng chữ số đã xuất hiện để đưa độ phức tạp về $O(N \times 54)$.

```

#include <bits/stdc++.h>
#define int long long
using namespace std;

bool nt[1000101], dd[60];
int a[1000005];

```



```
int n, ans;

void sangnt() {
    for(int i = 0; i <= 1000100; i++)
        nt[i] = 1;
    nt[0] = nt[1] = 0;
    for (int i = 2; i * i <= 1000100; i++)
        if (nt[i])
            for (int j = i * i; j <= 1000100; j += i)
                nt[j] = 0;
}

int get(int x) {
    int res = 0;
    while (x > 0) {
        res += x % 10;
        x /= 10;
    }
    return res;
}

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    sangnt();

    cin >> n;
    for (int i = 1; i <= n; i++) cin >> a[i];

    ans = 0;
    dd[get(a[1])] = 1;

    for (int i = 2; i <= n; i++) {
        bool ok = 0;
        for (int v = 1; v <= 54; v++) {
            if (dd[v]) {
                if (nt[a[i] + v]) {
                    ok = 1;
                    break;
                }
            }
        }
        if (ok) ans++;
        dd[get(a[i])] = 1;
    }

    cout << ans;
```

}

45.4 Bài 4: ĐOẠN CON CHUNG

Tóm tắt đề bài: Tìm hai đoạn con có đúng H phần tử chung. Gọi S_1 là tổng của H phần tử chung, S_2 là tổng của các phần tử còn lại thuộc hai đoạn con. Cần tìm giá trị nhỏ nhất của $S_1 - S_2$.

Như đã chứng minh, để $S_1 - S_2$ nhỏ nhất, phần mở rộng về hai phía của đoạn chung (tạo nên S_2) phải đạt tổng lớn nhất có thể.

—

45.4.1 Subtask 1 & 2: $N \leq 1000$

Phân tích: Với $N \leq 1000$, ta có thể duyệt qua mọi vị trí bắt đầu i của đoạn chung độ dài H . Với mỗi vị trí i , ta dùng vòng lặp để tính trực tiếp:

- Tổng S_1 của đoạn chung từ i đến $i + H - 1$.
- Tổng lớn nhất của phần mở rộng bên trái (kết thúc tại $i - 1$) bằng cách duyệt lùi từ $i - 1$ về 1.
- Tổng lớn nhất của phần mở rộng bên phải (bắt đầu tại $i + H$) bằng cách duyệt tiến từ $i + H$ đến N .

Độ phức tạp của cách này là $O(N^2)$, hoàn toàn vượt qua được giới hạn thời gian cho Subtask 2.

```
#include<bits/stdc++.h>
#define int long long
using namespace std;

int n, h;
int a[1000005];
int ans = 4e18;

main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n >> h;

    for(int i = 1; i <= n; i++){
        cin >> a[i];
    }
```

```

for(int i = 1; i <= n - h + 1; i++){
    int s1 = 0;

    for(int j = i; j < i + h; j++){
        s1 += a[j];
    }

    int l = 0, cur = 0;

    for(int j = i - 1; j >= 1; j--){
        cur += a[j];
        l = max(l, cur);
    }

    int r = 0;
    cur = 0;

    for(int j = i + h; j <= n; j++){
        cur += a[j];
        r = max(r, cur);
    }

    ans = min(ans, s1 - l - r);
}

cout << ans;
}

```

45.4.2 Subtask 3: $N \leq 10^6$

Phân tích: Để đạt điểm tối đa, ta cần giảm độ phức tạp xuống $O(N)$ bằng quy hoạch động và mảng cộng dồn.

- Dùng mảng cộng dồn $P[i]$ để lấy tổng S_1 trong $O(1)$.
- Xây dựng mảng $F_1[i]$ lưu tổng lớn nhất của đoạn con kết thúc tại i .
- Xây dựng mảng $F_2[i]$ lưu tổng lớn nhất của đoạn con bắt đầu tại i .
- Duyệt i từ 1 đến $N - H + 1$, kết quả là giá trị nhỏ nhất của $(S_1 - F_1[i - 1] - F_2[i + H])$.

```

#include <bits/stdc++.h>
#define int long long

using namespace std;

```

```

int n, h, a[1000005], p[1000005], f1[1000005], f2[1000005], ans = 4e18;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n >> h;

    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        p[i] = p[i - 1] + a[i];
    }

    f1[0] = 0;
    for (int i = 1; i <= n; i++)
        f1[i] = max(0LL, f1[i - 1] + a[i]);

    f2[n + 1] = 0;
    for (int i = n; i >= 1; i--)
        f2[i] = max(0LL, f2[i + 1] + a[i]);

    for (int i = 1; i <= n - h + 1; i++) {
        int s1 = p[i + h - 1] - p[i - 1];
        int s2 = f1[i - 1] + f2[i + h];
        ans = min(ans, s1 - s2);
    }

    cout << ans;
}

```

46 Vĩnh Phúc

46.1 Bài 1: QUÀ [GIFT]

Phân tích thuật toán: Mỗi phần quà yêu cầu đúng 1 bó hoa, 1 hộp bút và 1 quyển vở. Để chuẩn bị đủ k phần quà, số lượng tối thiểu cần có cho mỗi loại đồ vật chính là k .

- Nếu số lượng hiện có của một món đồ nhỏ hơn k , ta cần mua thêm một lượng bằng $k - \text{số lượng hiện có}$.
- Nếu số lượng hiện có đã lớn hơn hoặc bằng k , ta không cần mua thêm món đồ đó (cần thêm 0 món).

- Tổng số món đồ cần chuẩn bị thêm chính là tổng của số lượng cần mua thêm ở mỗi loại.

Công thức tính:

$$\text{Total} = \max(0, k - a) + \max(0, k - b) + \max(0, k - c)$$

46.1.1 Code mẫu

```
#include <bits/stdc++.h>
#define int long long

using namespace std;

int a, b, c, k;
int ans = 0;

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> a >> b >> c >> k;

    if (k > a)
        ans += (k - a);

    if (k > b)
        ans += (k - b);

    if (k > c)
        ans += (k - c);

    cout << ans << '\n';

    return 0;
}
```

46.2 Bài 2: TÀU ĐIỆN [METRO]

Phân tích thuật toán: Chuyến tàu đầu tiên (chuyến số 1) đến ga tại thời điểm T_0 . Các chuyến tiếp theo đến vào các thời điểm $T_0 + D, T_0 + 2D, \dots$ Tổng quát, chuyến tàu thứ k ($k \geq 1$) sẽ đến ga tại thời điểm:

$$T_k = T_0 + (k - 1) \times D$$

Hành khách đến ga tại thời điểm s_i . Ta chia làm 2 trường hợp:

- **Trường hợp 1:** Nếu $s_i \leq T_0$, hành khách đến trước hoặc đúng lúc chuyển tàu đầu tiên tới. Chắc chắn hành khách này sẽ lên chuyển số 1.
- **Trường hợp 2:** Nếu $s_i > T_0$, hành khách đã lỡ chuyển đầu tiên và phải đi chuyển thứ k ($k > 1$) sao cho $T_k \geq s_i$. Ta có bất phương trình:

$$T_0 + (k - 1) \times D \geq s_i$$

$$\Rightarrow k - 1 \geq \frac{s_i - T_0}{D}$$

Vì k là số nguyên, ta cần tìm giá trị nguyên nhỏ nhất thỏa mãn, tức là lấy phần nguyên trần (ceiling) của phép chia:

$$k - 1 = \left\lceil \frac{s_i - T_0}{D} \right\rceil$$

Trong lập trình với số nguyên, phép chia lấy phần trần của $\frac{A}{B}$ có thể viết gọn bằng công thức: $(A + B - 1) / B$. Do đó, $k = \frac{s_i - T_0 + D - 1}{D} + 1$.

Độ phức tạp: Vì ta có thể tính trực tiếp kết quả cho mỗi hành khách trong thời gian $O(1)$, tổng thời gian để xử lý N hành khách chỉ là $O(N)$. Thuật toán này vô cùng tối ưu và dư sức vượt qua giới hạn $N \leq 10^4$.

46.2.1 Code mẫu

```
#include <bits/stdc++.h>
#define int long long

using namespace std;

int n, d, t0;
int s[10005];

main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);

    cin >> n >> d >> t0;

    for (int i = 0; i < n; i++)
        cin >> s[i];

    for (int i = 0; i < n; i++) {
        if (s[i] <= t0) {
            cout << 1;
        } else {
            int k = (s[i] - t0 + d - 1) / d + 1;
```

```

        cout << k;
    }

    if (i < n - 1) {
        cout << " ";
    }
}
}

```

46.3 Bài 3: TRÒ CHƠI [GAME]

Phân tích thuật toán: Đây là một bài toán kinh điển về chiến thuật **Tham lam (Greedy)**. Để Bờm thắng được nhiều ván nhất, với mỗi món đồ chơi của Cuội, Bờm nên dùng món đồ chơi có giá trị **nhỏ nhất có thể** nhưng vẫn **đủ lớn hơn** đồ chơi của Cuội. Những món đồ chơi quá nhỏ, không thể thắng được bất kỳ món nào của Cuội thì đành chấp nhận bỏ qua (dùng để "thí" cho những món đồ chơi lớn nhất của Cuội).

Các bước thực hiện chung:

1. Đánh dấu các đồ chơi Bờm đã sở hữu.
2. Gom các đồ chơi còn lại (từ 1 đến $2N$) vào danh sách đồ chơi của Cuội.
3. Sắp xếp đồ chơi của Bờm và Cuội theo thứ tự tăng dần. (Vì Cuội lấy các giá trị còn lại duyệt từ 1 đến $2N$ nên danh sách của Cuội tự động tăng dần, ta chỉ cần sắp xếp danh sách của Bờm).
4. Dùng kỹ thuật để bắt cặp sao cho số lượng cặp (Bờm > Cuội) là lớn nhất.

46.3.1 Subtask 1: $N \leq 100$

Phân tích: Với N nhỏ, ta có thể dùng hai vòng lặp lồng nhau. Duyệt qua từng đồ chơi của Cuội, sau đó tìm trong danh sách đồ chơi của Bờm xem có món nào lớn hơn và chưa được sử dụng hay không. Nếu có, ta lấy món nhỏ nhất thỏa mãn, đánh dấu là đã dùng và tăng số ván thắng. Độ phức tạp $O(N^2)$.

```

#include<bits/stdc++.h>
#define int long long
using namespace std;

int n;
int a[105], b[105];
bool ok[205], used[105];
int ans;

main(){

```

```

ios_base::sync_with_stdio(0);
cin.tie(0); cout.tie(0);

cin >> n;

for(int i = 0; i < n; i++){
    cin >> a[i];
    ok[a[i]] = 1;
}

int m = 0;

for(int i = 1; i <= 2 * n; i++){
    if(!ok[i]){
        b[m++] = i;
    }
}

sort(a, a + n);

for(int i = 0; i < n; i++){
    for(int j = 0; j < n; j++){
        if(!used[j] && a[j] > b[i]){
            used[j] = 1;
            ans++;
            break;
        }
    }
}

cout << ans;
}

```

46.3.2 Subtask 2: $N \leq 50000$ (Full Test)

Phân tích tối ưu $O(N \log N)$: Với $N = 50000$, $O(N^2)$ sẽ bị Quá thời gian (TLE). Ta áp dụng kỹ thuật **Hai con trỏ (Two Pointers)**:

- Dùng con trỏ i duyệt đồ chơi của Bờm, con trỏ j duyệt đồ chơi của Cuội.
- Nếu đồ chơi của Bờm $a[i]$ lớn hơn đồ chơi của Cuội $b[j]$, Bờm thắng ván này. Ta tăng cả i và j lên để xét cặp tiếp theo, đồng thời tăng biến đếm wins.
- Nếu $a[i] \leq b[j]$, món đồ chơi hiện tại của Bờm không thể thắng Cuội. Ta chỉ tăng i để tìm món đồ chơi lớn hơn của Bờm, giữ nguyên j .


```
#include<bits/stdc++.h>
#define int long long
using namespace std;

int n;
int a[50005], b[50005];
bool ok[100005];
int ans;

main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n;

    for(int i = 1; i <= n; i++){
        cin >> a[i];
        ok[a[i]] = 1;
    }

    int m = 0;

    for(int i = 1; i <= 2 * n; i++){
        if(!ok[i]){
            b[++m] = i;
        }
    }

    sort(a + 1, a + n + 1);

    int i = 1, j = 1;

    while(i <= n && j <= n){
        if(a[i] > b[j]){
            ans++;
            i++;
            j++;
        }
        else{
            i++;
        }
    }

    cout << ans;
}
```

46.4 Bài 4: CHỌN [SELECT]

Phân tích thuật toán: Đây là một bài toán tối ưu hóa kinh điển có thể giải quyết bằng kỹ thuật **Tham lam có hoàn tác (Greedy with Undo)** hoặc còn được biết đến qua khái niệm Hàng đợi ưu tiên kết hợp Danh sách liên kết kép.

Yêu cầu của bài toán là ở lượt thứ i , cần chọn đúng i phần tử không kề nhau để có tổng lớn nhất.

- Ban đầu, ta đưa tất cả các giá trị vào một Hàng đợi ưu tiên (Max Heap) để luôn lấy ra được phần tử lớn nhất.
- Khi lấy một phần tử A_u ra khỏi hàng đợi để cộng vào tổng điểm, ta không được phép chọn hai phần tử kề cạnh nó là A_L và A_R .
- Tuy nhiên, chiến thuật tham lam đơn thuần có thể sai. Đôi khi việc không chọn A_u mà thay vào đó chọn cả A_L và A_R lại mang lại tổng lớn hơn. Để xử lý việc này, mỗi khi chọn A_u , ta "gộp" A_L, A_u, A_R thành một phần tử mới thay thế tại vị trí u với giá trị là: $V_{mii} = A_L + A_R - A_u$.
- Ý nghĩa của V_{mii} : Nếu trong các lượt tiếp theo, hàng đợi ưu tiên lại chọn V_{mii} , bản chất là ta đang "hoàn tác" việc chọn A_u và chuyển sang chọn A_L và A_R . Sự chênh lệch điểm số khi đó tăng đúng bằng $(A_L + A_R - A_u)$.
- Sử dụng hai mảng L và R đóng vai trò như một danh sách liên kết kép để nhanh chóng tìm được các phần tử kề trái và kề phải hiện tại của một vị trí, bỏ qua các phần tử đã bị "gộp".
- Để xử lý các phần tử ở hai đầu mút (không có kề trái hoặc kề phải), ta thêm hai lính canh ở vị trí 0 và $n + 1$ với giá trị âm vô cùng $(-\infty)$.

Độ phức tạp: Với $n \leq 200000$, ta thực hiện $k \approx \frac{n}{2}$ vòng lặp. Mỗi vòng lặp thao tác với Priority Queue mất $O(\log n)$. Tổng thời gian là $O(n \log n)$, hoàn toàn tối ưu và vượt qua mọi Subtask.

46.4.1 Code mẫu

```
#include<bits/stdc++.h>
#define int long long
using namespace std;

const int inf = 1e15;

int n, k;
```

```
int a[200005], l[200005], r[200005];
bool del[200005];

priority_queue<pair<int,int>> q;

main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    cin >> n;

    k = (n + 1) / 2;

    a[0] = -inf;
    a[n + 1] = -inf;

    l[0] = 0;
    r[0] = 1;

    l[n + 1] = n;
    r[n + 1] = n + 1;

    for(int i = 1; i <= n; i++){
        cin >> a[i];
        l[i] = i - 1;
        r[i] = i + 1;
        q.push({a[i], i});
    }

    int ans = 0;

    for(int i = 1; i <= k; i++){
        while(del[q.top().second]){
            q.pop();
        }

        int v = q.top().first;
        int u = q.top().second;
        q.pop();

        ans += v;

        cout << ans << '\n';

        int x = l[u];
        int y = r[u];

        del[x] = 1;
```

```
del[y] = 1;

int nv = a[x] + a[y] - a[u];

if(nv < -inf) nv = -inf;

a[u] = nv;

l[u] = l[x];
r[u] = r[y];

r[l[u]] = u;
l[r[u]] = u;

q.push({a[u], u});
}

return 0;
}
```

47 Tổng biên tập

Nguyễn Nhâm Tấn

48 Ban tạo bài

Nguyễn Nhâm Tấn

Võ Trần Ngọc Vy - votranngocvy

Trần Đức Khải - tdkhai

Nguyễn Thái Hưng - hungeazy

Phạm Thanh Sang - sangph2612

49 Ban viết đề + solution

Bùi Tuấn Anh - icebear_meow

Võ Tấn Bảo - enxiayy

Nguyễn Thái Hưng - hungeazy

Trần Kim Nhật - knhatdev

Nguyễn Thanh Minh - 3M17

Võ Trần Ngọc Vy - votranngocvy